

Address		Contents		
101	0010	0010	101	2201
102	0001	0010	102	1202
103	0001	0010	103	1203
104	0011	0010	104	3204
201	0000	0000	201	0002
202	0000	0000	202	0003
203	0000	0000	203	0004
204	0000	0000	204	0000

(a) Binary program

Address	Contents
101	2201
102	1202
103	1203
104	3204
201	0002
202	0003
203	0004
204	0000

(b) Hexadecimal program

Address	Instruction	
101	LDA	201
102	ADD	202
103	ADD	203
104	STA	204
201	DAT	2
202	DAT	3
203	DAT	4
204	DAT	0

(c) Symbolic program

Label	Operation	Operand
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

(d) Assembly program

Figure 13.13 Computation of the Formula $N = I + J + K$

fields, separated by spaces. The first field contains the address of a location. For an instruction, the second field contains the three-letter symbol for the opcode. If it is a memory-referencing instruction, then a third field contains the address. To store arbitrary data in a location, we invent a *pseudoinstruction* with the symbol DAT. This is merely an indication that the third field on the line contains a hexadecimal number to be stored in the location specified in the first field.

For this type of input we need a slightly more complex program. The program accepts each line of input, generates a binary number based on the second and third (if present) fields, and stores it in the location specified by the first field.

The use of a symbolic program makes life much easier but is still awkward. In particular, we must give an absolute address for each word. This means that the program and data can be loaded into only one place in memory, and we must know that place ahead of time. Worse, suppose we wish to change the program some day by adding or deleting a line. This will change the addresses of all subsequent words.

A much better system, and one commonly used, is to use symbolic addresses. This is illustrated in Figure 10.11d. Each line still consists of three fields. The first field is still for the address, but a symbol is used instead of an absolute numerical address. Some lines have no address, implying that the address of that line is one more than the address of the previous line. For memory-reference instructions, the third field also contains a symbolic address.

With this last refinement, we have an *assembly language*. Programs written in assembly language (assembly programs) are translated into machine language by an *assembler*. This program must not only do the symbolic translation discussed earlier but also assign some form of memory addresses to symbolic addresses.

The development of assembly language was a major milestone in the evolution of computer technology. It was the first step to the high-level languages in use today. Although few programmers use assembly language, virtually all machines provide one. They are used, if at all, for systems programs such as compilers and I/O routines.

Appendix B provides a more detailed examination of assembly language.

13.6 RECOMMENDED READING

The references cited in Chapter 12 are equally applicable to the material of this chapter. [BLAA97] contains a detailed discussion of instruction formats and addressing modes. In addition, the reader may wish to consult [FLYN85] for a discussion and analysis of instruction set design issues, particularly those relating to formats.

BLAA97 Blaauw, G., and Brooks, F. *Computer Architecture: Concepts and Evolution*. Reading, MA: Addison-Wesley, 1997.

FLYN85 Flynn, M.; Johnson, J.; and Wakefield, S. "On Instruction Sets and Their Formats." *IEEE Transactions on Computers*, March 1985.

13.7 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

autoindexing base-register addressing direct addressing displacement addressing effective address	immediate addressing indexing indirect addressing instruction format postindexing	preindexing register addressing register indirect addressing relative addressing word
---	---	---

Review Questions

- 13.1** Briefly define immediate addressing.
- 13.2** Briefly define **direct addressing**.
- 13.3** Briefly define indirect addressing.
- 13.4** Briefly define register addressing.
- 13.5** Briefly define register indirect addressing.
- 13.6** Briefly define displacement addressing.
- 13.7** Briefly define relative addressing.
- 13.8** What is the advantage of autoindexing?
- 13.9** What is the difference between postindexing and preindexing?
- 13.10** What facts go into determining the use of the addressing bits of an instruction?
- 13.11** What are the advantages and disadvantages of using a variable-length instruction format?

Problems

- 13.1** Given the following memory values and a one-address machine with an accumulator, what values do the following instructions load into the accumulator?
- Word 20 contains 40.
 - Word 30 contains 50.
 - Word 40 contains 60.
 - Word 50 contains 70.
- a. LOAD IMMEDIATE 20
 - b. LOAD DIRECT 20
 - c. LOAD INDIRECT 20
 - d. LOAD IMMEDIATE 30
 - e. LOAD DIRECT 30
 - f. LOAD INDIRECT 30
- 13.2** Let the address stored in the program counter be designated by the symbol X1. The instruction stored in X1 has an address part (operand reference) X2. The operand needed to execute the instruction is stored in the memory word with address X3. An index register contains the value X4. What is the relationship between these various quantities if the addressing mode of the instruction is (a) direct; (b) indirect; (c) PC relative; (d) indexed?
- 13.3** An address field in an instruction contains decimal value 14. Where is the corresponding operand located for
- a. immediate addressing?
 - b. direct addressing?
 - c. indirect addressing?
 - d. register addressing?
 - e. register indirect addressing?
- 13.4** Consider a 16-bit processor in which the following appears in main memory, starting at location 200:

200	Load to AC	Mode
201	500	
202	Next instruction	

The first part of the first word indicates that this instruction loads a value into an accumulator. The Mode field specifies an addressing mode and, if appropriate, indicates a source register; assume that when used, the source register is R1, which has a value of 400. There is also a base register that contains the value 100. The value of 500 in location 201 may be part of the address calculation. Assume that location 399 contains the value 999, location 400 contains the value 1000, and so on. Determine the effective address and the operand to be loaded for the following address modes:

a. Direct	d. PC relative	g. Register indirect
b. Immediate	e. Displacement	h. Autoindexing with increment, using R1
c. Indirect	f. Register	

- 13.5** A PC-relative mode branch instruction is 3 bytes long. The address of the instruction, in decimal, is 256028. Determine the branch target address if the signed displacement in the instruction is -31 .
- 13.6** A PC-relative mode branch instruction is stored in memory at address 620_{10} . The branch is made to location 530_{10} . The address field in the instruction is 10 bits long. What is the binary value in the instruction?

- 13.7** How many times does the processor need to refer to memory when it fetches and executes an indirect-address-mode instruction if the instruction is (a) a computation requiring a single operand; (b) a branch?
- 13.8** The IBM 370 does not provide indirect addressing. Assume that the address of an operand is in main memory. How would you access the operand?
- 13.9** In [COOK82], the author proposes that the PC-relative addressing modes be eliminated in favor of other modes, such as the use of a stack. What is the disadvantage of this proposal?
- 13.10** The x86 includes the following instruction:

IMUL op1, op2, immediate

This instruction multiplies op2, which may be either register or memory, by the immediate operand value, and places the result in op1, which must be a register. There is no other three-operand instruction of this sort in the instruction set. What is the possible use of such an instruction? (*Hint: Consider indexing.*)

- 13.11** Consider a processor that includes a base with indexing addressing mode. Suppose an instruction is encountered that employs this addressing mode and specifies a displacement of 1970, in decimal. Currently the base and index register contain the decimal numbers 48,022 and 8, respectively. What is the address of the operand?
- 13.12** Define: $EA = (X) +$ is the effective address equal to the contents of location X, with X incremented by one word length after the effective address is calculated; $EA = -(X)$ is the effective address equal to the contents of location X, with X decremented by one word length before the effective address is calculated; $EA = (X) -$ is the effective address equal to the contents of location X, with X decremented by one word length after the effective address is calculated. Consider the following instructions, each in the format (Operation Source Operand, Destination Operand), with the result of the operation placed in the destination operand.
- OP X, (X)
 - OP (X), (X) +
 - OP (X) +, (X)
 - OP - (X), (X)
 - OP - (X), (X) +
 - OP (X) +, (X) +
 - OP (X) -, (X)

Using X as the stack pointer, which of these instructions can pop the top two elements from the stack, perform the designated operation (e.g., ADD source to destination and store in destination), and push the result back on the stack? For each such instruction, does the stack grow toward memory location 0 or in the opposite direction?

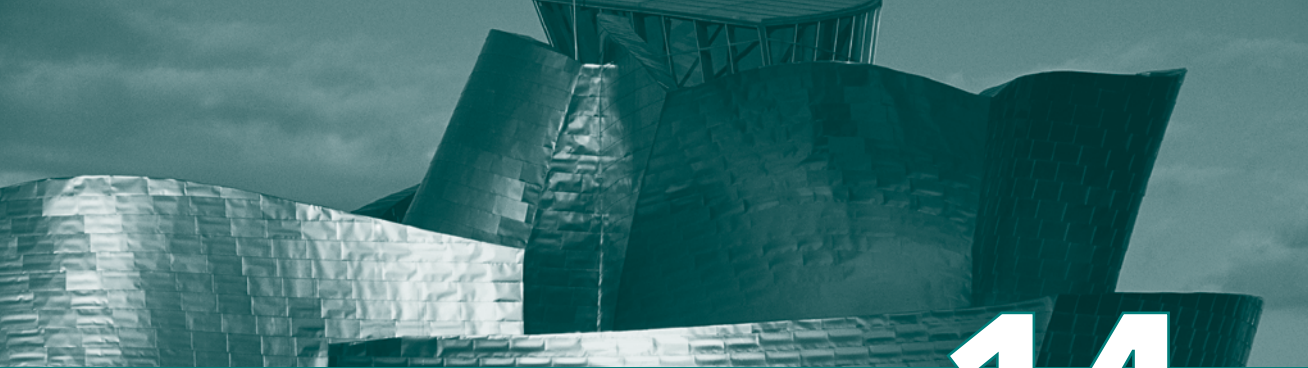
- 13.13** Assume a stack-oriented processor that includes the stack operations PUSH and POP. Arithmetic operations automatically involve the top one or two stack elements. Begin with an empty stack. What stack elements remain after the following instructions are executed?
- ```
PUSH 4
PUSH 7
PUSH 8
ADD
PUSH 10
SUB
MUL
```
- 13.14** Justify the assertion that a 32-bit instruction is probably much less than twice as useful as a 16-bit instruction.
- 13.15** Why was IBM's decision to move from 36 bits to 32 bits per word wrenching, and to whom?

- 13.16** Assume an instruction set that uses a fixed 16-bit instruction length. Operand specifiers are 6 bits in length. There are  $K$  two-operand instructions and  $L$  zero-operand instructions. What is the maximum number of one-operand instructions that can be supported?
- 13.17** Design a variable-length opcode to allow all of the following to be encoded in a 36-bit instruction:
- instructions with two 15-bit addresses and one 3-bit register number
  - instructions with one 15-bit address and one 3-bit register number
  - instructions with no addresses or registers
- 13.18** Consider the results of Problem 10.6. Assume that  $M$  is a 16-bit memory address and that  $X$ ,  $Y$ , and  $Z$  are either 16-bit addresses or 4-bit register numbers. The one-address machine uses an accumulator, and the two- and three-address machines have 16 registers and instructions operating on all combinations of memory locations and registers. Assuming 8-bit opcodes and instruction lengths that are multiples of 4 bits, how many bits does each machine need to compute  $X$ ?
- 13.19** Is there any possible justification for an instruction with two opcodes?
- 13.20** The 16-bit Zilog Z8001 has the following general instruction format:

|      |    |        |    |    |    |     |   |           |   |   |   |           |   |   |   |
|------|----|--------|----|----|----|-----|---|-----------|---|---|---|-----------|---|---|---|
| 15   | 14 | 13     | 12 | 11 | 10 | 9   | 8 | 7         | 6 | 5 | 4 | 3         | 2 | 1 | 0 |
| Mode |    | Opcode |    |    |    | w/b |   | Operand 2 |   |   |   | Operand 1 |   |   |   |

The *mode* field specifies how to locate the operands from the *operand* fields. The *w/b* field is used in certain instructions to specify whether the operands are bytes or 16-bit words. The *operand 1* field may (depending on the *mode field* contents) specify one of 16 general-purpose registers. The *operand 2* field may specify any general-purpose registers except register 0. When the *operand 2* field is all zeros, each of the original opcodes takes on a new meaning.

- How many opcodes are provided on the Z8001?
- Suggest an efficient way to provide more opcodes and indicate the trade-off involved.



# CHAPTER 14

## PROCESSOR STRUCTURE AND FUNCTION

### 14.1 Processor Organization

### 14.2 Register Organization

- User-Visible Registers

- Control and Status Registers

- Example Microprocessor Register Organizations

### 14.3 Instruction Cycle

- The Indirect Cycle

- Data Flow

### 14.4 Instruction Pipelining

- Pipelining Strategy

- Pipeline Performance

- Pipeline Hazards

- Dealing with Branches

- Intel 80486 Pipelining

### 14.5 The x86 Processor Family

- Register Organization

- Interrupt Processing

### 14.6 The Arm Processor

- Processor Organization

- Processor Modes

- Register Organization

- Interrupt Processing

### 14.7 Recommended Reading

### 14.8 Key Terms, Review Questions, and Problems

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- ◆ Distinguish between user-visible and control/status registers, and discuss the purposes of registers in each category.
- ◆ Summarize the instruction cycle.
- ◆ Discuss the principle behind instruction pipelining and how it works in practice.
- ◆ Compare and contrast the various forms of pipeline hazards.
- ◆ Present an overview of the x86 processor structure.
- ◆ Present an overview of the ARM processor structure.

This chapter discusses aspects of the processor not yet covered in Part Three and sets the stage for the discussion of RISC and superscalar architecture in Chapters 15 and 16.

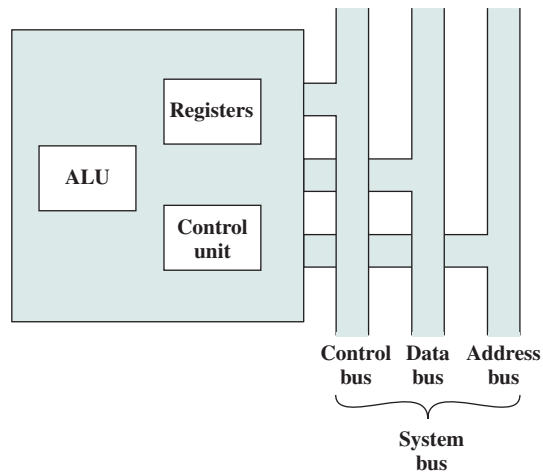
We begin with a summary of processor organization. Registers, which form the internal memory of the processor, are then analyzed. We are then in a position to return to the discussion (begun in Section 3.2) of the instruction cycle. A description of the instruction cycle and a common technique known as instruction pipelining complete our description. The chapter concludes with an examination of some aspects of the x86 and ARM organizations.

## 14.1 PROCESSOR ORGANIZATION

To understand the organization of the processor, let us consider the requirements placed on the processor, the things that it must do:

- **Fetch instruction:** The processor reads an instruction from memory (register, cache, main memory).
- **Interpret instruction:** The instruction is decoded to determine what action is required.
- **Fetch data:** The execution of an instruction may require reading data from memory or an I/O module.
- **Process data:** The execution of an instruction may require performing some arithmetic or logical operation on data.
- **Write data:** The results of an execution may require writing data to memory or an I/O module.

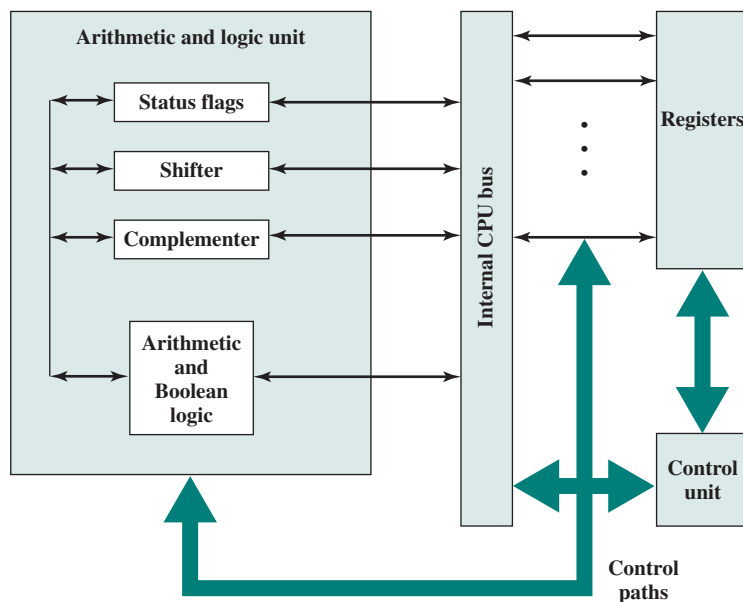
To do these things, it should be clear that the processor needs to store some data temporarily. It must remember the location of the last instruction so that it can know where to get the next instruction. It needs to store instructions and data temporarily while an instruction is being executed. In other words, the processor needs a small internal memory.



**Figure 14.1** The CPU with the System Bus

Figure 14.1 is a simplified view of a processor, indicating its connection to the rest of the system via the system bus. A similar interface would be needed for any of the interconnection structures described in Chapter 3. The reader will recall that the major components of the processor are an *arithmetic and logic unit* (ALU) and a *control unit* (CU). The ALU does the actual computation or processing of data. The control unit controls the movement of data and instructions into and out of the processor and controls the operation of the ALU. In addition, the figure shows a minimal internal memory, consisting of a set of storage locations, called *registers*.

Figure 14.2 is a slightly more detailed view of the processor. The data transfer and logic control paths are indicated, including an element labeled *internal*



**Figure 14.2** Internal Structure of the CPU



*processor bus.* This element is needed to transfer data between the various registers and the ALU because the ALU in fact operates only on data in the internal processor memory. The figure also shows typical basic elements of the ALU. Note the similarity between the internal structure of the computer as a whole and the internal structure of the processor. In both cases, there is a small collection of major elements (computer: processor, I/O, memory; processor: control unit, ALU, registers) connected by data paths.

## 14.2 REGISTER ORGANIZATION

As we discussed in Chapter 4, a computer system employs a memory hierarchy. At higher levels of the hierarchy, memory is faster, smaller, and more expensive (per bit). Within the processor, there is a set of registers that function as a level of memory above main memory and cache in the hierarchy. The registers in the processor perform two roles:

- **User-visible registers:** Enable the machine- or assembly language programmer to minimize main memory references by optimizing use of registers.
- **Control and status registers:** Used by the control unit to control the operation of the processor and by privileged, operating system programs to control the execution of programs.

There is not a clean separation of registers into these two categories. For example, on some machines the program counter is user visible (e.g., x86), but on many it is not. For purposes of the following discussion, however, we will use these categories.

### User-Visible Registers

A user-visible register is one that may be referenced by means of the machine language that the processor executes. We can characterize these in the following categories:

- General purpose
- Data
- Address
- Condition codes

**General-purpose registers** can be assigned to a variety of functions by the programmer. Sometimes their use within the instruction set is orthogonal to the operation. That is, any general-purpose register can contain the operand for any opcode. This provides true general-purpose register use. Often, however, there are restrictions. For example, there may be dedicated registers for floating-point and stack operations.

In some cases, general-purpose registers can be used for addressing functions (e.g., register indirect, displacement). In other cases, there is a partial or clean separation between data registers and address registers. **Data registers** may be used only to hold data and cannot be employed in the calculation of an operand address.

**Address registers** may themselves be somewhat general purpose, or they may be devoted to a particular addressing mode. Examples include the following:

- **Segment pointers:** In a machine with segmented addressing (see Section 8.3), a segment register holds the address of the base of the segment. There may be multiple registers: for example, one for the operating system and one for the current process.
- **Index registers:** These are used for indexed addressing and may be autoindexed.
- **Stack pointer:** If there is user-visible stack addressing, then typically there is a dedicated register that points to the top of the stack. This allows implicit addressing; that is, push, pop, and other stack instructions need not contain an explicit stack operand.

There are several design issues to be addressed here. An important issue is whether to use completely general-purpose registers or to specialize their use. We have already touched on this issue in the preceding chapter because it affects instruction set design. With the use of specialized registers, it can generally be implicit in the opcode which type of register a certain operand specifier refers to. The operand specifier must only identify one of a set of specialized registers rather than one out of all the registers, thus saving bits. On the other hand, this specialization limits the programmer's flexibility.

Another design issue is the number of registers, either general purpose or data plus address, to be provided. Again, this affects instruction set design because more registers require more operand specifier bits. As we previously discussed, somewhere between 8 and 32 registers appears optimum [LUND77]. Fewer registers result in more memory references; more registers do not noticeably reduce memory references (e.g., see [WILL90]). However, a new approach, which finds advantage in the use of hundreds of registers, is exhibited in some RISC systems and is discussed in Chapter 15.

Finally, there is the issue of register length. Registers that must hold addresses obviously must be at least long enough to hold the largest address. Data registers should be able to hold values of most data types. Some machines allow two contiguous registers to be used as one for holding double-length values.

A final category of registers, which is at least partially visible to the user, holds **condition codes** (also referred to as *flags*). Condition codes are bits set by the processor hardware as the result of operations. For example, an arithmetic operation may produce a positive, negative, zero, or overflow result. In addition to the result itself being stored in a register or memory, a condition code is also set. The code may subsequently be tested as part of a conditional branch operation.

Condition code bits are collected into one or more registers. Usually, they form part of a control register. Generally, machine instructions allow these bits to be read by implicit reference, but the programmer cannot alter them.

Many processors, including those based on the IA-64 architecture and the MIPS processors, do not use condition codes at all. Rather, conditional branch instructions specify a comparison to be made and act on the result of the comparison, without storing a condition code. Table 14.1, based on [DERO87], lists key advantages and disadvantages of condition codes.

**Table 14.1** Condition Codes

| Advantages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Disadvantages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"> <li>1. Because condition codes are set by normal arithmetic and data movement instructions, they should reduce the number of COMPARE and TEST instructions needed.</li> <li>2. Conditional instructions, such as BRANCH are simplified relative to composite instructions, such as TEST AND BRANCH.</li> <li>3. Condition codes facilitate multiway branches. For example, a TEST instruction can be followed by two branches, one on less than or equal to zero and one on greater than zero.</li> <li>4. Condition codes can be saved on the stack during subroutine calls along with other register information.</li> </ol> | <ol style="list-style-type: none"> <li>1. Condition codes add complexity, both to the hardware and software. Condition code bits are often modified in different ways by different instructions, making life more difficult for both the microprogrammer and compiler writer.</li> <li>2. Condition codes are irregular; they are typically not part of the main data path, so they require extra hardware connections.</li> <li>3. Often condition code machines must add special non-condition-code instructions for special situations anyway, such as bit checking, loop control, and atomic semaphore operations.</li> <li>4. In a pipelined implementation, condition codes require special synchronization to avoid conflicts.</li> </ol> |

In some machines, a subroutine call will result in the automatic saving of all user-visible registers, to be restored on return. The processor performs the saving and restoring as part of the execution of call and return instructions. This allows each subroutine to use the user-visible registers independently. On other machines, it is the responsibility of the programmer to save the contents of the relevant user-visible registers prior to a subroutine call, by including instructions for this purpose in the program.

### Control and Status Registers

There are a variety of processor registers that are employed to control the operation of the processor. Most of these, on most machines, are not visible to the user. Some of them may be visible to machine instructions executed in a control or operating system mode.

Of course, different machines will have different register organizations and use different terminology. We list here a reasonably complete list of register types, with a brief description.

Four registers are essential to instruction execution:

- **Program counter (PC):** Contains the address of an instruction to be fetched.
- **Instruction register (IR):** Contains the instruction most recently fetched.
- **Memory address register (MAR):** Contains the address of a location in memory.
- **Memory buffer register (MBR):** Contains a word of data to be written to memory or the word most recently read.

Not all processors have internal registers designated as MAR and MBR, but some equivalent buffering mechanism is needed whereby the bits to be transferred

to the system bus are staged and the bits to be read from the data bus are temporarily stored.

Typically, the processor updates the PC after each instruction fetch so that the PC always points to the next instruction to be executed. A branch or skip instruction will also modify the contents of the PC. The fetched instruction is loaded into an IR, where the opcode and operand specifiers are analyzed. Data are exchanged with memory using the MAR and MBR. In a bus-organized system, the MAR connects directly to the address bus, and the MBR connects directly to the data bus. User-visible registers, in turn, exchange data with the MBR.

The four registers just mentioned are used for the movement of data between the processor and memory. Within the processor, data must be presented to the ALU for processing. The ALU may have direct access to the MBR and user-visible registers. Alternatively, there may be additional buffering registers at the boundary to the ALU; these registers serve as input and output registers for the ALU and exchange data with the MBR and user-visible registers.

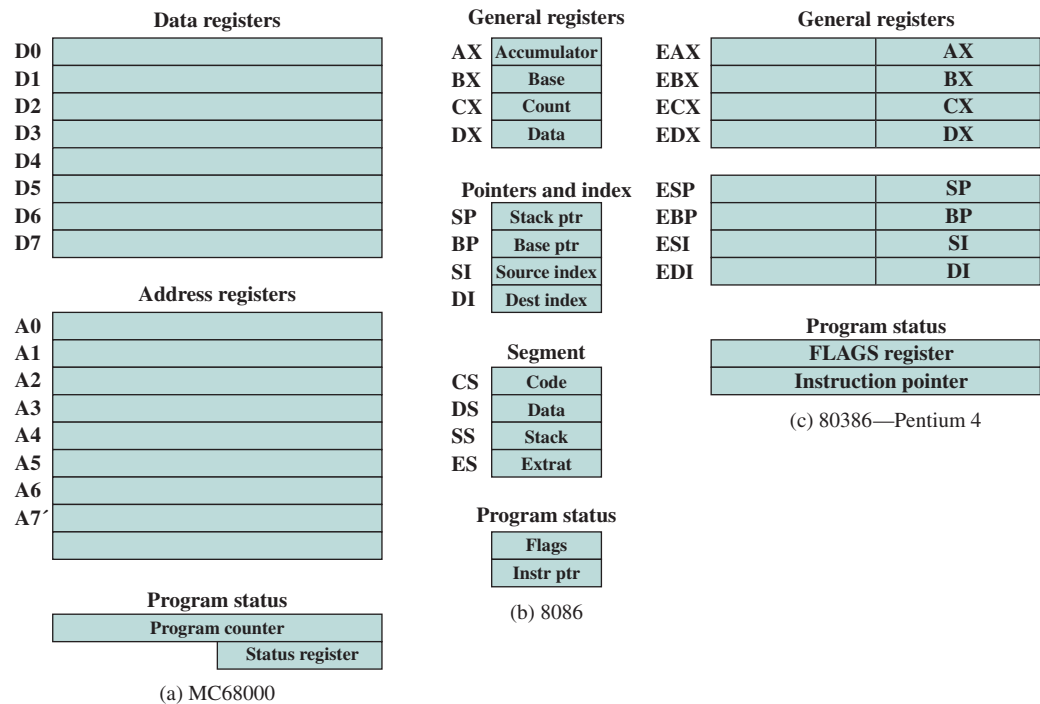
Many processor designs include a register or set of registers, often known as the *program status word* (PSW), that contain status information. The PSW typically contains condition codes plus other status information. Common fields or flags include the following:

- **Sign:** Contains the sign bit of the result of the last arithmetic operation.
- **Zero:** Set when the result is 0.
- **Carry:** Set if an operation resulted in a carry (addition) into or borrow (subtraction) out of a high-order bit. Used for multiword arithmetic operations.
- **Equal:** Set if a logical compare result is equality.
- **Overflow:** Used to indicate arithmetic overflow.
- **Interrupt Enable/Disable:** Used to enable or disable interrupts.
- **Supervisor:** Indicates whether the processor is executing in supervisor or user mode. Certain privileged instructions can be executed only in supervisor mode, and certain areas of memory can be accessed only in supervisor mode.

A number of other registers related to status and control might be found in a particular processor design. There may be a pointer to a block of memory containing additional status information (e.g., process control blocks). In machines using vectored interrupts, an interrupt vector register may be provided. If a stack is used to implement certain functions (e.g., subroutine call), then a system stack pointer is needed. A page table pointer is used with a virtual memory system. Finally, registers may be used in the control of I/O operations.

A number of factors go into the design of the control and status register organization. One key issue is operating system support. Certain types of control information are of specific utility to the operating system. If the processor designer has a functional understanding of the operating system to be used, then the register organization can to some extent be tailored to the operating system.

Another key design decision is the allocation of control information between registers and memory. It is common to dedicate the first (lowest) few hundred or



**Figure 14.3** Example Microprocessor Register Organizations

thousand words of memory for control purposes. The designer must decide how much control information should be in registers and how much in memory. The usual trade-off of cost versus speed arises.

### Example Microprocessor Register Organizations

It is instructive to examine and compare the register organization of comparable systems. In this section, we look at two 16-bit microprocessors that were designed at about the same time: the Motorola MC68000 [STRI79] and the Intel 8086 [MORS78]. Figures 14.3a and b depict the register organization of each; purely internal registers, such as a memory address register, are not shown.

The MC68000 partitions its 32-bit registers into eight data registers and nine address registers. The eight data registers are used primarily for data manipulation and are also used in addressing as index registers. The width of the registers allows 8-, 16-, and 32-bit data operations, determined by opcode. The address registers contain 32-bit (no segmentation) addresses; two of these registers are also used as stack pointers, one for users and one for the operating system, depending on the current execution mode. Both registers are numbered 7, because only one can be used at a time. The MC68000 also includes a 32-bit program counter and a 16-bit status register.

The Motorola team wanted a very regular instruction set, with no special-purpose registers. A concern for code efficiency led them to divide the registers into

two functional components, saving one bit on each register specifier. This seems a reasonable compromise between complete generality and code compaction.

The Intel 8086 takes a different approach to register organization. Every register is special purpose, although some registers are also usable as general purpose. The 8086 contains four 16-bit data registers that are addressable on a byte or 16-bit basis, and four 16-bit pointer and index registers. The data registers can be used as general purpose in some instructions. In others, the registers are used implicitly. For example, a multiply instruction always uses the accumulator. The four pointer registers are also used implicitly in a number of operations; each contains a segment offset. There are also four 16-bit segment registers. Three of the four segment registers are used in a dedicated, implicit fashion, to point to the segment of the current instruction (useful for branch instructions), a segment containing data, and a segment containing a stack, respectively. These dedicated and implicit uses provide for compact encoding at the cost of reduced flexibility. The 8086 also includes an instruction pointer and a set of 1-bit status and control flags.

The point of this comparison should be clear. There is no universally accepted philosophy concerning the best way to organize processor registers [TOON81]. As with overall instruction set design and so many other processor design issues, it is still a matter of judgment and taste.

A second instructive point concerning register organization design is illustrated in Figure 14.3c. This figure shows the user-visible register organization for the Intel 80386 [ELAY85], which is a 32-bit microprocessor designed as an extension of the 8086.<sup>1</sup> The 80386 uses 32-bit registers. However, to provide upward compatibility for programs written on the earlier machine, the 80386 retains the original register organization embedded in the new organization. Given this design constraint, the architects of the 32-bit processors had limited flexibility in designing the register organization.

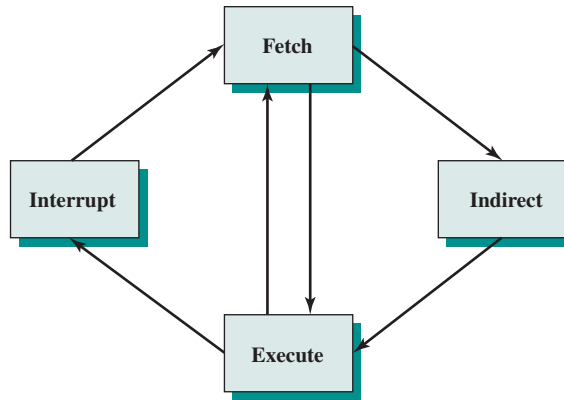
## 14.3 INSTRUCTION CYCLE

In Section 3.2, we described the processor's instruction cycle (Figure 3.9). To recall, an instruction cycle includes the following stages:

- **Fetch:** Read the next instruction from memory into the processor.
- **Execute:** Interpret the opcode and perform the indicated operation.
- **Interrupt:** If interrupts are enabled and an interrupt has occurred, save the current process state and service the interrupt.

We are now in a position to elaborate somewhat on the instruction cycle. First, we must introduce one additional stage, known as the indirect cycle.

<sup>1</sup>Because the MC68000 already uses 32-bit registers, the MC68020 [MACD84], which is a full 32-bit architecture, uses the same register organization.



**Figure 14.4** The Instruction Cycle

### The Indirect Cycle

We have seen, in Chapter 13, that the execution of an instruction may involve one or more operands in memory, each of which requires a memory access. Further, if indirect addressing is used, then additional memory accesses are required.

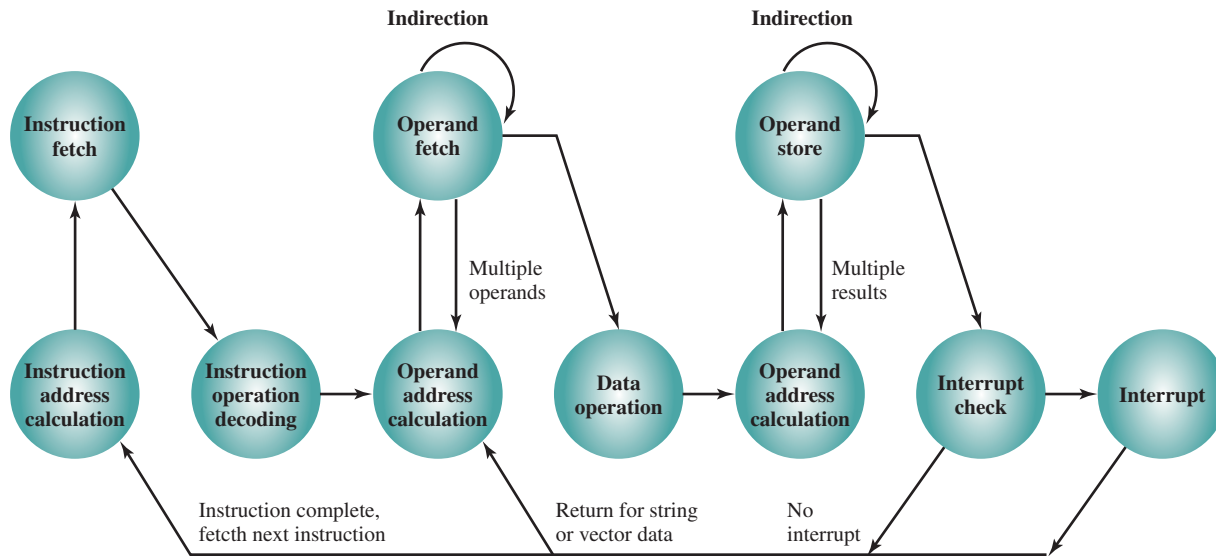
We can think of the fetching of indirect addresses as one more instruction stages. The result is shown in Figure 14.4. The main line of activity consists of alternating instruction fetch and instruction execution activities. After an instruction is fetched, it is examined to determine if any indirect addressing is involved. If so, the required operands are fetched using indirect addressing. Following execution, an interrupt may be processed before the next instruction fetch.

Another way to view this process is shown in Figure 14.5, which is a revised version of Figure 3.12. This illustrates more correctly the nature of the instruction cycle. Once an instruction is fetched, its operand specifiers must be identified. Each input operand in memory is then fetched, and this process may require indirect addressing. Register-based operands need not be fetched. Once the opcode is executed, a similar process may be needed to store the result in main memory.

### Data Flow

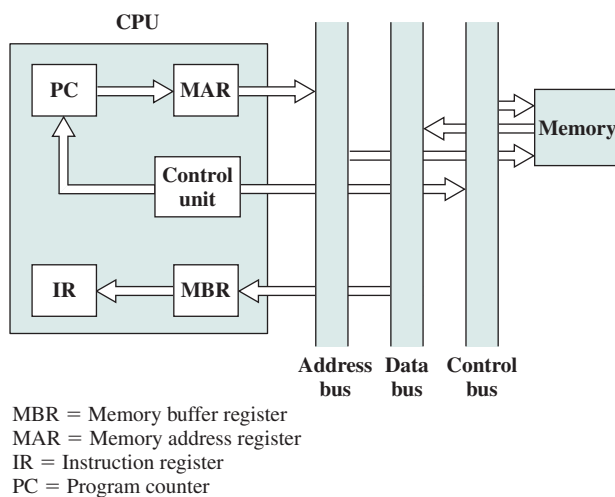
The exact sequence of events during an instruction cycle depends on the design of the processor. We can, however, indicate in general terms what must happen. Let us assume that a processor that employs a memory address register (MAR), a memory buffer register (MBR), a program counter (PC), and an instruction register (IR).

During the *fetch cycle*, an instruction is read from memory. Figure 14.6 shows the flow of data during this cycle. The PC contains the address of the next instruction to be fetched. This address is moved to the MAR and placed on the address bus. The control unit requests a memory read, and the result is placed on the data bus and copied into the MBR and then moved to the IR. Meanwhile, the PC is incremented by 1, preparatory for the next fetch.



**Figure 14.5** Instruction Cycle State Diagram



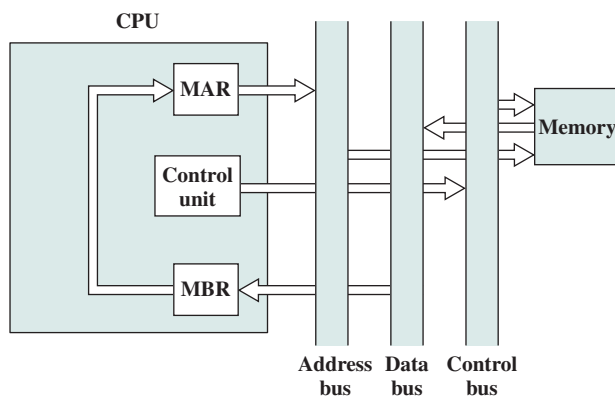


**Figure 14.6** Data Flow, Fetch Cycle

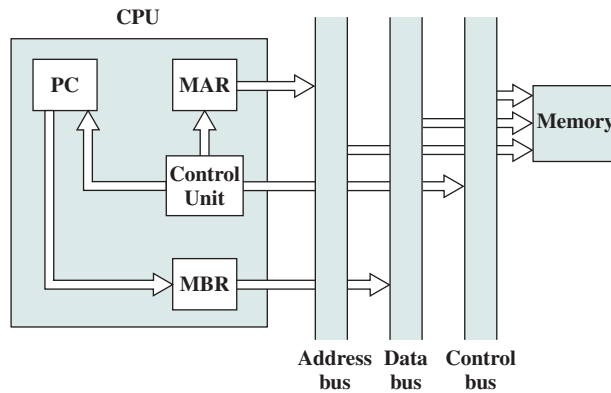
Once the fetch cycle is over, the control unit examines the contents of the IR to determine if it contains an operand specifier using indirect addressing. If so, an *indirect cycle* is performed. As shown in Figure 14.7, this is a simple cycle. The right-most  $N$  bits of the MBR, which contain the address reference, are transferred to the MAR. Then the control unit requests a memory read, to get the desired address of the operand into the MBR.

The fetch and indirect cycles are simple and predictable. The *execute cycle* takes many forms; the form depends on which of the various machine instructions is in the IR. This cycle may involve transferring data among registers, read or write from memory or I/O, and/or the invocation of the ALU.

Like the fetch and indirect cycles, the *interrupt cycle* is simple and predictable (Figure 14.8). The current contents of the PC must be saved so that the processor can resume normal activity after the interrupt. Thus, the contents of the PC are



**Figure 14.7** Data Flow, Indirect Cycle



**Figure 14.8** Data Flow, Interrupt Cycle

transferred to the MBR to be written into memory. The special memory location reserved for this purpose is loaded into the MAR from the control unit. It might, for example, be a stack pointer. The PC is loaded with the address of the interrupt routine. As a result, the next instruction cycle will begin by fetching the appropriate instruction.

## 14.4 INSTRUCTION PIPELINING

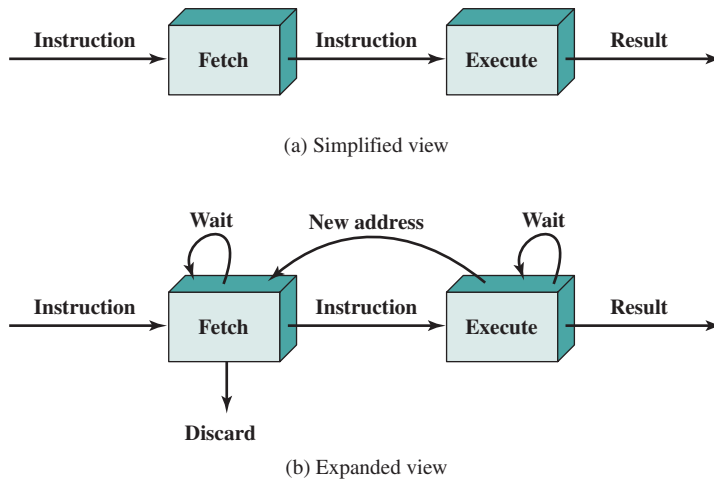
As computer systems evolve, greater performance can be achieved by taking advantage of improvements in technology, such as faster circuitry. In addition, organizational enhancements to the processor can improve performance. We have already seen some examples of this, such as the use of multiple registers rather than a single accumulator, and the use of a cache memory. Another organizational approach, which is quite common, is instruction pipelining.

### Pipelining Strategy

Instruction pipelining is similar to the use of an assembly line in a manufacturing plant. An assembly line takes advantage of the fact that a product goes through various stages of production. By laying the production process out in an assembly line, products at various stages can be worked on simultaneously. This process is also referred to as *pipelining*, because, as in a pipeline, new inputs are accepted at one end before previously accepted inputs appear as outputs at the other end.

To apply this concept to instruction execution, we must recognize that, in fact, an instruction has a number of stages. Figures 14.5, for example, breaks the instruction cycle up into 10 tasks, which occur in sequence. Clearly, there should be some opportunity for pipelining.

As a simple approach, consider subdividing instruction processing into two stages: fetch instruction and execute instruction. There are times during the execution of an instruction when main memory is not being accessed. This time could be used to fetch the next instruction in parallel with the execution of the current



**Figure 14.9** Two-Stage Instruction Pipeline

one. Figure 14.9a depicts this approach. The pipeline has two independent stages. The first stage fetches an instruction and buffers it. When the second stage is free, the first stage passes it the buffered instruction. While the second stage is executing the instruction, the first stage takes advantage of any unused memory cycles to fetch and buffer the next instruction. This is called instruction prefetch or *fetch overlap*. Note that this approach, which involves instruction buffering, requires more registers. In general, pipelining requires registers to store data between stages.

It should be clear that this process will speed up instruction execution. If the fetch and execute stages were of equal duration, the instruction cycle time would be halved. However, if we look more closely at this pipeline (Figure 14.9b), we will see that this doubling of execution rate is unlikely for two reasons:

1. The execution time will generally be longer than the fetch time. Execution will involve reading and storing operands and the performance of some operation. Thus, the fetch stage may have to wait for some time before it can empty its buffer.
2. A conditional branch instruction makes the address of the next instruction to be fetched unknown. Thus, the fetch stage must wait until it receives the next instruction address from the execute stage. The execute stage may then have to wait while the next instruction is fetched.

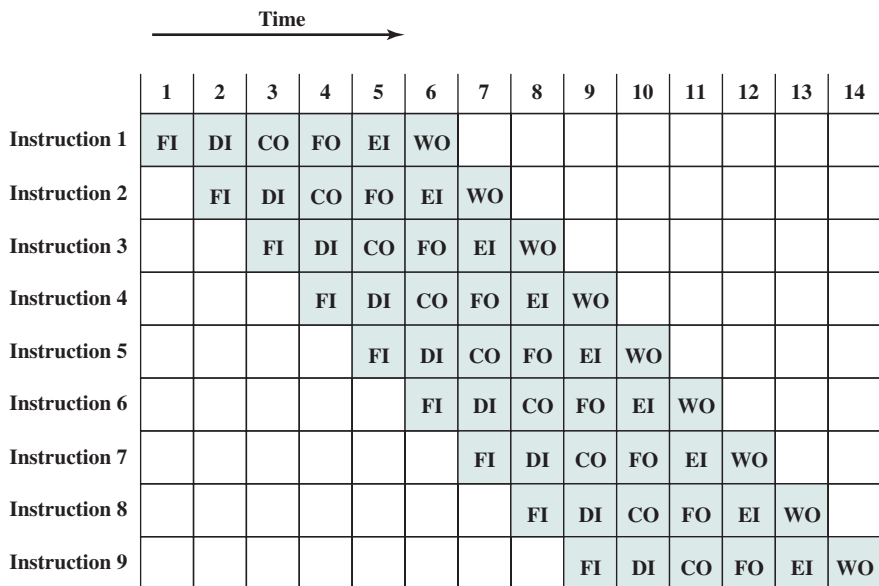
Guessing can reduce the time loss from the second reason. A simple rule is the following: When a conditional branch instruction is passed on from the fetch to the execute stage, the fetch stage fetches the next instruction in memory after the branch instruction. Then, if the branch is not taken, no time is lost. If the branch is taken, the fetched instruction must be discarded and a new instruction fetched.

While these factors reduce the potential effectiveness of the two-stage pipeline, some speedup occurs. To gain further speedup, the pipeline must have more stages. Let us consider the following decomposition of the instruction processing.

- **Fetch instruction (FI):** Read the next expected instruction into a buffer.
- **Decode instruction (DI):** Determine the opcode and the operand specifiers.
- **Calculate operands (CO):** Calculate the effective address of each source operand. This may involve displacement, register indirect, indirect, or other forms of address calculation.
- **Fetch operands (FO):** Fetch each operand from memory. Operands in registers need not be fetched.
- **Execute instruction (EI):** Perform the indicated operation and store the result, if any, in the specified destination operand location.
- **Write operand (WO):** Store the result in memory.

With this decomposition, the various stages will be of more nearly equal duration. For the sake of illustration, let us assume equal duration. Using this assumption, Figure 14.10 shows that a six-stage pipeline can reduce the execution time for 9 instructions from 54 time units to 14 time units.

Several comments are in order: The diagram assumes that each instruction goes through all six stages of the pipeline. This will not always be the case. For example, a load instruction does not need the WO stage. However, to simplify the pipeline hardware, the timing is set up assuming that each instruction requires all six stages. Also, the diagram assumes that all of the stages can be performed in parallel. In particular, it is assumed that there are no memory conflicts. For example, the FI, FO, and WO stages involve a memory access. The diagram implies that all these accesses can occur simultaneously. Most memory systems will not permit that. However, the desired value may be in cache, or the FO or WO stage may be null. Thus, much of the time, memory conflicts will not slow down the pipeline.



**Figure 14.10** Timing Diagram for Instruction Pipeline Operation

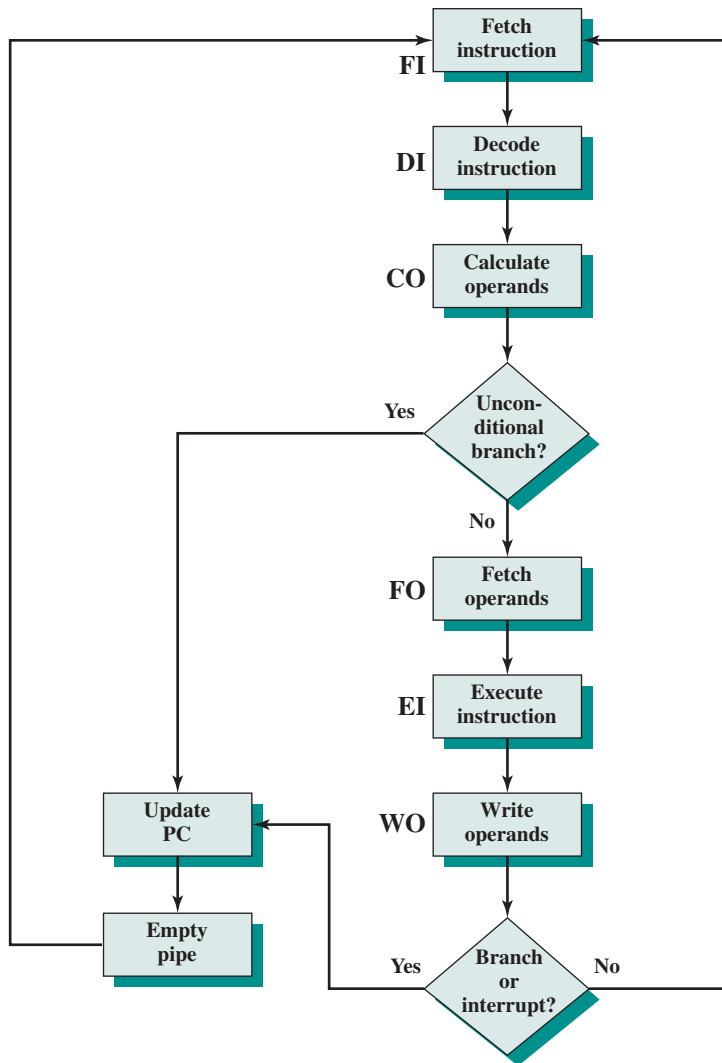
|                | Time → |    |    |    |    |    |    | ← Branch penalty |    |    |    |    |    |    |
|----------------|--------|----|----|----|----|----|----|------------------|----|----|----|----|----|----|
|                | 1      | 2  | 3  | 4  | 5  | 6  | 7  | 8                | 9  | 10 | 11 | 12 | 13 | 14 |
| Instruction 1  | FI     | DI | CO | FO | EI | WO |    |                  |    |    |    |    |    |    |
| Instruction 2  |        | FI | DI | CO | FO | EI | WO |                  |    |    |    |    |    |    |
| Instruction 3  |        |    | FI | DI | CO | FO | EI | WO               |    |    |    |    |    |    |
| Instruction 4  |        |    |    | FI | DI | CO | FO |                  |    |    |    |    |    |    |
| Instruction 5  |        |    |    |    | FI | DI | CO |                  |    |    |    |    |    |    |
| Instruction 6  |        |    |    |    |    | FI | DI |                  |    |    |    |    |    |    |
| Instruction 7  |        |    |    |    |    |    | FI |                  |    |    |    |    |    |    |
| Instruction 15 |        |    |    |    |    |    |    | FI               | DI | CO | FO | EI | WO |    |
| Instruction 16 |        |    |    |    |    |    |    |                  | FI | DI | CO | FO | EI | WO |

**Figure 14.11** The Effect of a Conditional Branch on Instruction Pipeline Operation

Several other factors serve to limit the performance enhancement. If the six stages are not of equal duration, there will be some waiting involved at various pipeline stages, as discussed before for the two-stage pipeline. Another difficulty is the conditional branch instruction, which can invalidate several instruction fetches. A similar unpredictable event is an interrupt. Figure 14.11 illustrates the effects of the conditional branch, using the same program as Figure 14.10. Assume that instruction 3 is a conditional branch to instruction 15. Until the instruction is executed, there is no way of knowing which instruction will come next. The pipeline, in this example, simply loads the next instruction in sequence (instruction 4) and proceeds. In Figure 14.10, the branch is not taken, and we get the full performance benefit of the enhancement. In Figure 14.11, the branch is taken. This is not determined until the end of time unit 7. At this point, the pipeline must be cleared of instructions that are not useful. During time unit 8, instruction 15 enters the pipeline. No instructions complete during time units 9 through 12; this is the performance penalty incurred because we could not anticipate the branch. Figure 14.12 indicates the logic needed for pipelining to account for branches and interrupts.

Other problems arise that did not appear in our simple two-stage organization. The CO stage may depend on the contents of a register that could be altered by a previous instruction that is still in the pipeline. Other such register and memory conflicts could occur. The system must contain logic to account for this type of conflict.

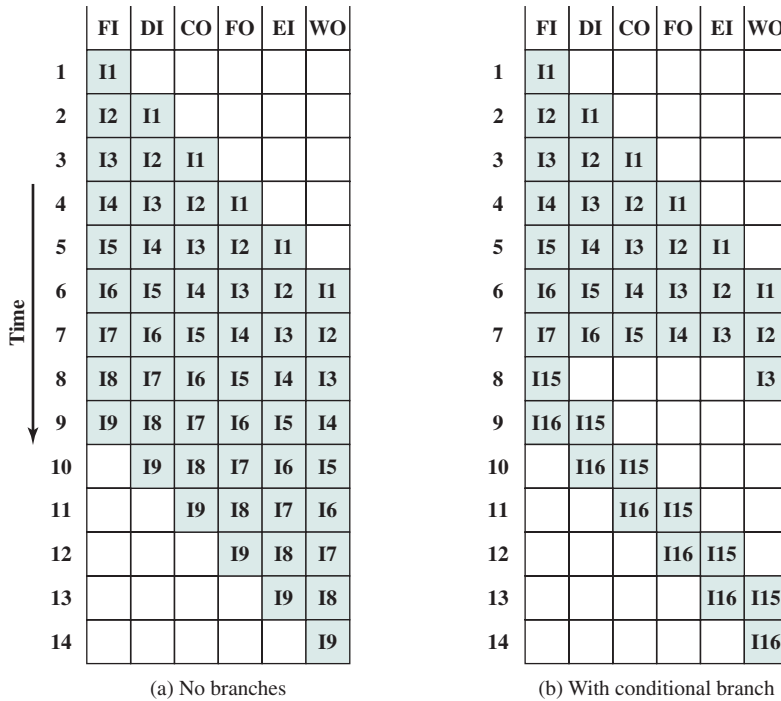
To clarify pipeline operation, it might be useful to look at an alternative depiction. Figures 14.10 and 14.11 show the progression of time horizontally across the figures, with each row showing the progress of an individual instruction. Figure 14.13 shows same sequence of events, with time progressing vertically down the figure,



**Figure 14.12** Six-Stage CPU Instruction Pipeline

and each row showing the state of the pipeline at a given point in time. In Figure 14.13a (which corresponds to Figure 14.10), the pipeline is full at time 6, with 6 different instructions in various stages of execution, and remains full through time 9; we assume that instruction I9 is the last instruction to be executed. In Figure 14.13b, (which corresponds to Figure 14.11), the pipeline is full at times 6 and 7. At time 7, instruction 3 is in the execute stage and executes a branch to instruction 15. At this point, instructions I4 through I7 are flushed from the pipeline, so that at time 8, only two instructions are in the pipeline, I3 and I15.

From the preceding discussion, it might appear that the greater the number of stages in the pipeline, the faster the execution rate. Some of the IBM S/360 designers



**Figure 14.13** An Alternative Pipeline Depiction

pointed out two factors that frustrate this seemingly simple pattern for high-performance design [ANDE67a], and they remain elements that designer must still consider:

1. At each stage of the pipeline, there is some overhead involved in moving data from buffer to buffer and in performing various preparation and delivery functions. This overhead can appreciably lengthen the total execution time of a single instruction. This is significant when sequential instructions are logically dependent, either through heavy use of branching or through memory access dependencies.
2. The amount of control logic required to handle memory and register dependencies and to optimize the use of the pipeline increases enormously with the number of stages. This can lead to a situation where the logic controlling the gating between stages is more complex than the stages being controlled.

Another consideration is latching delay: It takes time for pipeline buffers to operate and this adds to instruction cycle time.

Instruction pipelining is a powerful technique for enhancing performance but requires careful design to achieve optimum results with reasonable complexity.

### Pipeline Performance

In this subsection, we develop some simple measures of pipeline performance and relative speedup (based on a discussion in [HWAN93]). The cycle time  $\tau$  of an **instruction pipeline** is the time needed to advance a set of instructions one stage

through the pipeline; each column in Figures 14.10 and 14.11 represents one cycle time. The cycle time can be determined as

$$\tau = \max_i [\tau_i] + d = \tau_m + d \quad 1 \leq i \leq k$$

where

$\tau_i$  = time delay of the circuitry in the  $i$ th stage of the pipeline

$\tau_m$  = maximum stage delay (delay through stage which experiences the largest delay)

$k$  = number of stages in the instruction pipeline

$d$  = time delay of a latch, needed to advance signals and data from one stage to the next

In general, the time delay  $d$  is equivalent to a clock pulse and  $\tau_m \gg d$ . Now suppose that  $n$  instructions are processed, with no branches. Let  $T_{k,n}$  be the total time required for a pipeline with  $k$  stages to execute  $n$  instructions. Then

$$T_{k,n} = [k + (n - 1)]\tau \quad (14.1)$$

A total of  $k$  cycles are required to complete the execution of the first instruction, and the remaining  $n - 1$  instructions require  $n - 1$  cycles.<sup>2</sup> This equation is easily verified from Figures 14.10. The ninth instruction completes at time cycle 14:

$$14 = [6 + (9 - 1)]$$

Now consider a processor with equivalent functions but no pipeline, and assume that the instruction cycle time is  $k\tau$ . The speedup factor for the instruction pipeline compared to execution without the pipeline is defined as

$$S_k = \frac{T_{1,n}}{T_{k,n}} = \frac{nk\tau}{[k + (n - 1)]\tau} = \frac{nk}{k + (n - 1)} \quad (14.2)$$

Figure 14.14a plots the speedup factor as a function of the number of instructions that are executed without a branch. As might be expected, at the limit ( $n \rightarrow \infty$ ), we have a  $k$ -fold speedup. Figure 14.14b shows the speedup factor as a function of the number of stages in the instruction pipeline.<sup>3</sup> In this case, the speedup factor approaches the number of instructions that can be fed into the pipeline without branches. Thus, the larger the number of pipeline stages, the greater the potential for speedup. However, as a practical matter, the potential gains of additional pipeline stages are countered by increases in cost, delays between stages, and the fact that branches will be encountered requiring the flushing of the pipeline.

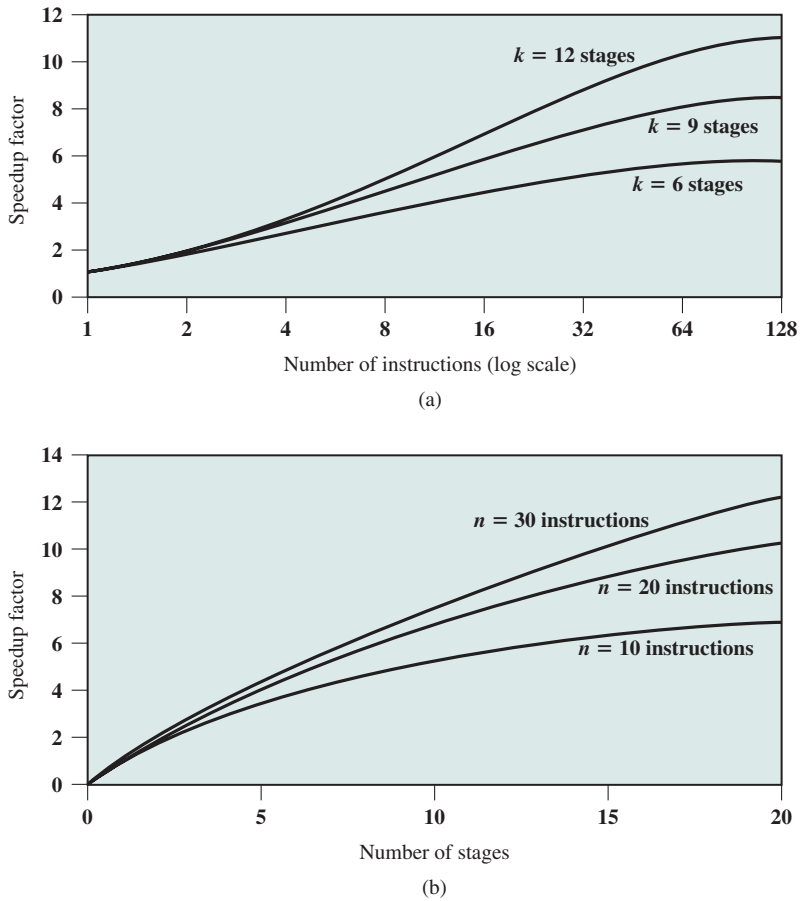
## Pipeline Hazards

In the previous subsection, we mentioned some of the situations that can result in less than optimal pipeline performance. In this subsection, we examine this issue in

<sup>2</sup>We are being a bit sloppy here. The cycle time will only equal the maximum value of  $\tau$  when all the stages are full. At the beginning, the cycle time may be less for the first one or few cycles.

<sup>3</sup>Note that the  $x$ -axis is logarithmic in Figure 14.14a and linear in Figure 14.14b.





**Figure 14.14** Speedup Factors with Instruction Pipelining

a more systematic way. Chapter 16 revisits this issue, in more detail, after we have introduced the complexities found in superscalar pipeline organizations.

A **pipeline hazard** occurs when the pipeline, or some portion of the pipeline, must stall because conditions do not permit continued execution. Such a pipeline stall is also referred to as a *pipeline bubble*. There are three types of hazards: resource, data, and control.

**RESOURCE HAZARDS** A resource hazard occurs when two (or more) instructions that are already in the pipeline need the same resource. The result is that the instructions must be executed in serial rather than parallel for a portion of the pipeline. A resource hazard is sometime referred to as a *structural hazard*.

Let us consider a simple example of a resource hazard. Assume a simplified five-stage pipeline, in which each stage takes one clock cycle. Figure 14.15a shows the ideal case, in which a new instruction enters the pipeline each clock cycle. Now assume that main memory has a single port and that all instruction fetches and data reads and writes must be performed one at a time. Further, ignore the cache. In this

|             |    | Clock cycle |    |    |    |    |    |    |    |   |
|-------------|----|-------------|----|----|----|----|----|----|----|---|
|             |    | 1           | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9 |
| Instruction | I1 | FI          | DI | FO | EI | WO |    |    |    |   |
|             | I2 |             | FI | DI | FO | EI | WO |    |    |   |
|             | I3 |             |    | FI | DI | FO | EI | WO |    |   |
|             | I4 |             |    |    | FI | DI | FO | EI | WO |   |

(a) Five-stage pipeline, ideal case

|             |    | Clock cycle |    |      |    |    |    |    |    |    |
|-------------|----|-------------|----|------|----|----|----|----|----|----|
|             |    | 1           | 2  | 3    | 4  | 5  | 6  | 7  | 8  | 9  |
| Instruction | I1 | FI          | DI | FO   | EI | WO |    |    |    |    |
|             | I2 |             | FI | DI   | FO | EI | WO |    |    |    |
|             | I3 |             |    | Idle | FI | DI | FO | EI | WO |    |
|             | I4 |             |    |      |    | FI | DI | FO | EI | WO |

(b) I1 source operand in memory

**Figure 14.15** Example of Resource Hazard

case, an operand read to or write from memory cannot be performed in parallel with an instruction fetch. This is illustrated in Figure 14.15b, which assumes that the source operand for instruction I1 is in memory, rather than a register. Therefore, the fetch instruction stage of the pipeline must idle for one cycle before beginning the instruction fetch for instruction I3. The figure assumes that all other operands are in registers.

Another example of a resource conflict is a situation in which multiple instructions are ready to enter the execute instruction phase and there is a single ALU. One solution to such resource hazards is to increase available resources, such as having multiple ports into main memory and multiple ALU units.



### Reservation Table Analyzer

One approach to analyzing resource conflicts and aiding in the design of pipelines is the reservation table. We examine reservation tables in Appendix I.

**DATA HAZARDS** A data hazard occurs when there is a conflict in the access of an operand location. In general terms, we can state the hazard in this form: Two instructions in a program are to be executed in sequence and both access a particular memory or register operand. If the two instructions are executed in strict sequence,

no problem occurs. However, if the instructions are executed in a pipeline, then it is possible for the operand value to be updated in such a way as to produce a different result than would occur with strict sequential execution. In other words, the program produces an incorrect result because of the use of pipelining.

As an example, consider the following x86 machine instruction sequence:

```
ADD EAX, EBX /* EAX = EAX + EBX
SUB ECX, EAX /* ECX = ECX - EAX
```

The first instruction adds the contents of the 32-bit registers EAX and EBX and stores the result in EAX. The second instruction subtracts the contents of EAX from ECX and stores the result in ECX. Figure 14.16 shows the pipeline behavior. The ADD instruction does not update register EAX until the end of stage 5, which occurs at clock cycle 5. But the SUB instruction needs that value at the beginning of its stage 2, which occurs at clock cycle 4. To maintain correct operation, the pipeline must stall for two clocks cycles. Thus, in the absence of special hardware and specific avoidance algorithms, such a data hazard results in inefficient pipeline usage.

There are three types of data hazards;

- **Read after write (RAW), or true dependency:** An instruction modifies a register or memory location and a succeeding instruction reads the data in that memory or register location. A hazard occurs if the read takes place before the write operation is complete.
- **Write after read (WAR), or antidependency:** An instruction reads a register or memory location and a succeeding instruction writes to the location. A hazard occurs if the write operation completes before the read operation takes place.
- **Write after write (WAW), or output dependency:** Two instructions both write to the same location. A hazard occurs if the write operations take place in the reverse order of the intended sequence.

The example of Figure 14.16 is a RAW hazard. The other two hazards are best discussed in the context of superscalar organization, discussed in Chapter 16.

**CONTROL HAZARDS** A control hazard, also known as a *branch hazard*, occurs when the pipeline makes the wrong decision on a branch prediction and therefore brings instructions into the pipeline that must subsequently be discarded. We discuss approaches to dealing with control hazards next.

|              |  |             |    |    |      |    |    |    |    |    |    |
|--------------|--|-------------|----|----|------|----|----|----|----|----|----|
|              |  | Clock cycle |    |    |      |    |    |    |    |    |    |
|              |  | 1           | 2  | 3  | 4    | 5  | 6  | 7  | 8  | 9  | 10 |
| ADD EAX, EBX |  | FI          | DI | FO | EI   | WO |    |    |    |    |    |
|              |  |             |    |    |      |    |    |    |    |    |    |
| SUB ECX, EAX |  |             | FI | DI | Idle |    | FO | EI | WO |    |    |
|              |  |             |    |    |      |    |    |    |    |    |    |
| 13           |  |             |    | FI |      |    | DI | FO | EI | WO |    |
| 14           |  |             |    |    |      |    | FI | DI | FO | EI | WO |

**Figure 14.16** Example of Data Hazard

## Dealing with Branches

One of the major problems in designing an instruction pipeline is assuring a steady flow of instructions to the initial stages of the pipeline. The primary impediment, as we have seen, is the conditional branch instruction. Until the instruction is actually executed, it is impossible to determine whether the branch will be taken or not.

A variety of approaches have been taken for dealing with conditional branches:

- Multiple streams
- Prefetch branch target
- Loop buffer
- Branch prediction
- Delayed branch

**MULTIPLE STREAMS** A simple pipeline suffers a penalty for a branch instruction because it must choose one of two instructions to fetch next and may make the wrong choice. A brute-force approach is to replicate the initial portions of the pipeline and allow the pipeline to fetch both instructions, making use of two streams. There are two problems with this approach:

- With multiple pipelines there are contention delays for access to the registers and to memory.
- Additional branch instructions may enter the pipeline (either stream) before the original branch decision is resolved. Each such instruction needs an additional stream.

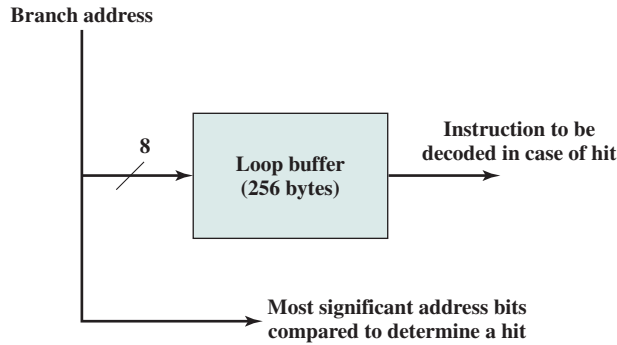
Despite these drawbacks, this strategy can improve performance. Examples of machines with two or more pipeline streams are the IBM 370/168 and the IBM 3033.

**PREFETCH BRANCH TARGET** When a conditional branch is recognized, the target of the branch is prefetched, in addition to the instruction following the branch. This target is then saved until the branch instruction is executed. If the branch is taken, the target has already been prefetched.

The IBM 360/91 uses this approach.

**LOOP BUFFER** A loop buffer is a small, very-high-speed memory maintained by the instruction fetch stage of the pipeline and containing the  $n$  most recently fetched instructions, in sequence. If a branch is to be taken, the hardware first checks whether the branch target is within the buffer. If so, the next instruction is fetched from the buffer. The loop buffer has three benefits:

1. With the use of prefetching, the loop buffer will contain some instruction sequentially ahead of the current instruction fetch address. Thus, instructions fetched in sequence will be available without the usual memory access time.
2. If a branch occurs to a target just a few locations ahead of the address of the branch instruction, the target will already be in the buffer. This is useful for the rather common occurrence of IF-THEN and IF-THEN-ELSE sequences.



**Figure 14.17** Loop Buffer

3. This strategy is particularly well suited to dealing with loops, or iterations; hence the name *loop buffer*. If the loop buffer is large enough to contain all the instructions in a loop, then those instructions need to be fetched from memory only once, for the first iteration. For subsequent iterations, all the needed instructions are already in the buffer.

The loop buffer is similar in principle to a cache dedicated to instructions. The differences are that the loop buffer only retains instructions in sequence and is much smaller in size and hence lower in cost.

Figure 14.17 gives an example of a loop buffer. If the buffer contains 256 bytes, and byte addressing is used, then the least significant 8 bits are used to index the buffer. The remaining most significant bits are checked to determine if the branch target lies within the environment captured by the buffer.

Among the machines using a loop buffer are some of the CDC machines (Star-100, 6600, 7600) and the CRAY-1. A specialized form of loop buffer is available on the Motorola 68010, for executing a three-instruction loop involving the DBcc (decrement and branch on condition) instruction (see Problem 14.14). A three-word buffer is maintained, and the processor executes these instructions repeatedly until the loop condition is satisfied.



**Branch Prediction Simulator**  
**Branch Target Buffer**

**BRANCH PREDICTION** Various techniques can be used to predict whether a branch will be taken. Among the more common are the following:

- Predict never taken
- Predict always taken
- Predict by opcode
- Taken/not taken switch
- Branch history table

The first three approaches are static: they do not depend on the execution history up to the time of the conditional branch instruction. The latter two approaches are dynamic: They depend on the execution history.

The first two approaches are the simplest. These either always assume that the branch will not be taken and continue to fetch instructions in sequence, or they always assume that the branch will be taken and always fetch from the branch target. The predict-never-taken approach is the most popular of all the branch prediction methods.

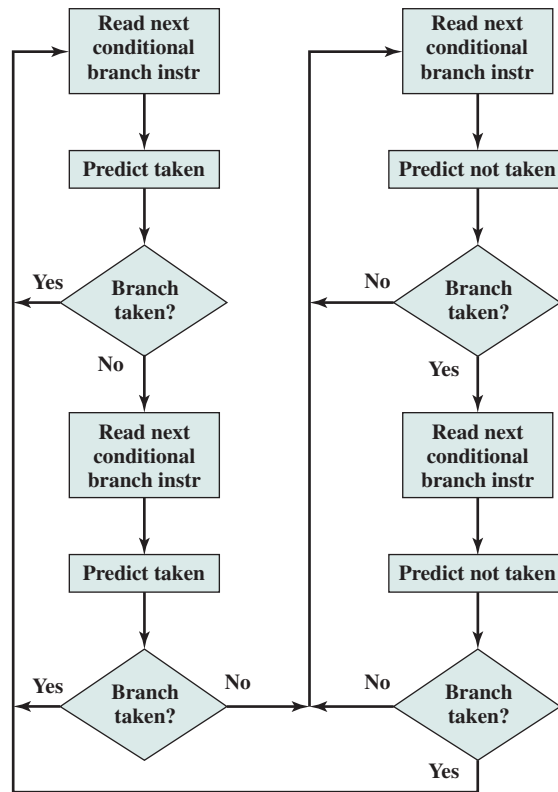
Studies analyzing program behavior have shown that conditional branches are taken more than 50% of the time [LILJ88], and so if the cost of prefetching from either path is the same, then always prefetching from the branch target address should give better performance than always prefetching from the sequential path. However, in a paged machine, prefetching the branch target is more likely to cause a page fault than prefetching the next instruction in sequence, and so this performance penalty should be taken into account. An avoidance mechanism may be employed to reduce this penalty.

The final static approach makes the decision based on the opcode of the branch instruction. The processor assumes that the branch will be taken for certain branch opcodes and not for others. [LILJ88] reports success rates of greater than 75% with this strategy.

Dynamic branch strategies attempt to improve the accuracy of prediction by recording the history of conditional branch instructions in a program. For example, one or more bits can be associated with each conditional branch instruction that reflect the recent history of the instruction. These bits are referred to as a taken/not taken switch that directs the processor to make a particular decision the next time the instruction is encountered. Typically, these history bits are not associated with the instruction in main memory. Rather, they are kept in temporary high-speed storage. One possibility is to associate these bits with any conditional branch instruction that is in a cache. When the instruction is replaced in the cache, its history is lost. Another possibility is to maintain a small table for recently executed branch instructions with one or more history bits in each entry. The processor could access the table associatively, like a cache, or by using the low-order bits of the branch instruction's address.

With a single bit, all that can be recorded is whether the last execution of this instruction resulted in a branch or not. A shortcoming of using a single bit appears in the case of a conditional branch instruction that is almost always taken, such as a loop instruction. With only one bit of history, an error in prediction will occur twice for each use of the loop: once on entering the loop, and once on exiting.

If two bits are used, they can be used to record the result of the last two instances of the execution of the associated instruction, or to record a state in some other fashion. Figure 14.18 shows a typical approach (see Problem 14.13 for other possibilities). Assume that the algorithm starts at the upper-left-hand corner of the flowchart. As long as each succeeding conditional branch instruction that is encountered is taken, the decision process predicts that the next branch will be taken. If a single prediction is wrong, the algorithm continues to predict that the next branch is taken. Only if two successive branches are not taken does the algorithm shift to the right-hand side of the flowchart. Subsequently, the algorithm



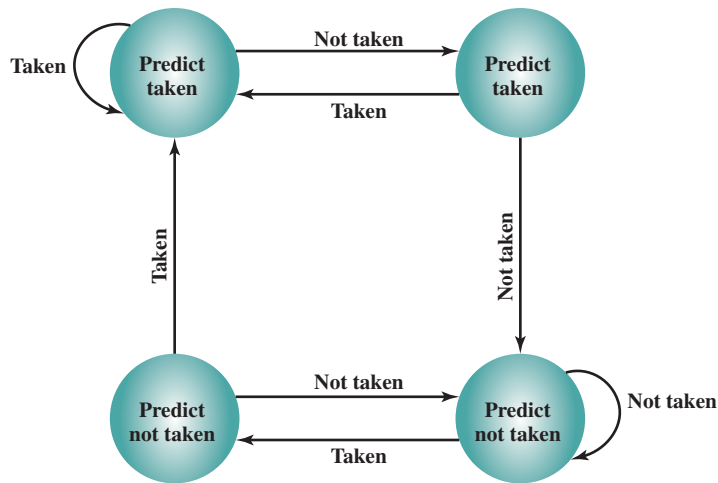
**Figure 14.18** Branch Prediction Flowchart

will predict that branches are not taken until two branches in a row are taken. Thus, the algorithm requires two consecutive wrong predictions to change the prediction decision.

The decision process can be represented more compactly by a finite-state machine, shown in Figure 14.19. The finite-state machine representation is commonly used in the literature.

The use of history bits, as just described, has one drawback: If the decision is made to take the branch, the target instruction cannot be fetched until the target address, which is an operand in the conditional branch instruction, is decoded. Greater efficiency could be achieved if the instruction fetch could be initiated as soon as the branch decision is made. For this purpose, more information must be saved, in what is known as a branch target buffer, or a branch history table.

The branch history table is a small cache memory associated with the instruction fetch stage of the pipeline. Each entry in the table consists of three elements: the address of a branch instruction, some number of history bits that record the state of use of that instruction, and information about the target instruction. In most proposals and implementations, this third field contains the address of the target instruction. Another possibility is for the third field to actually contain the target instruction. The trade-off is clear: Storing the target address yields a smaller



**Figure 14.19** Branch Prediction State Diagram

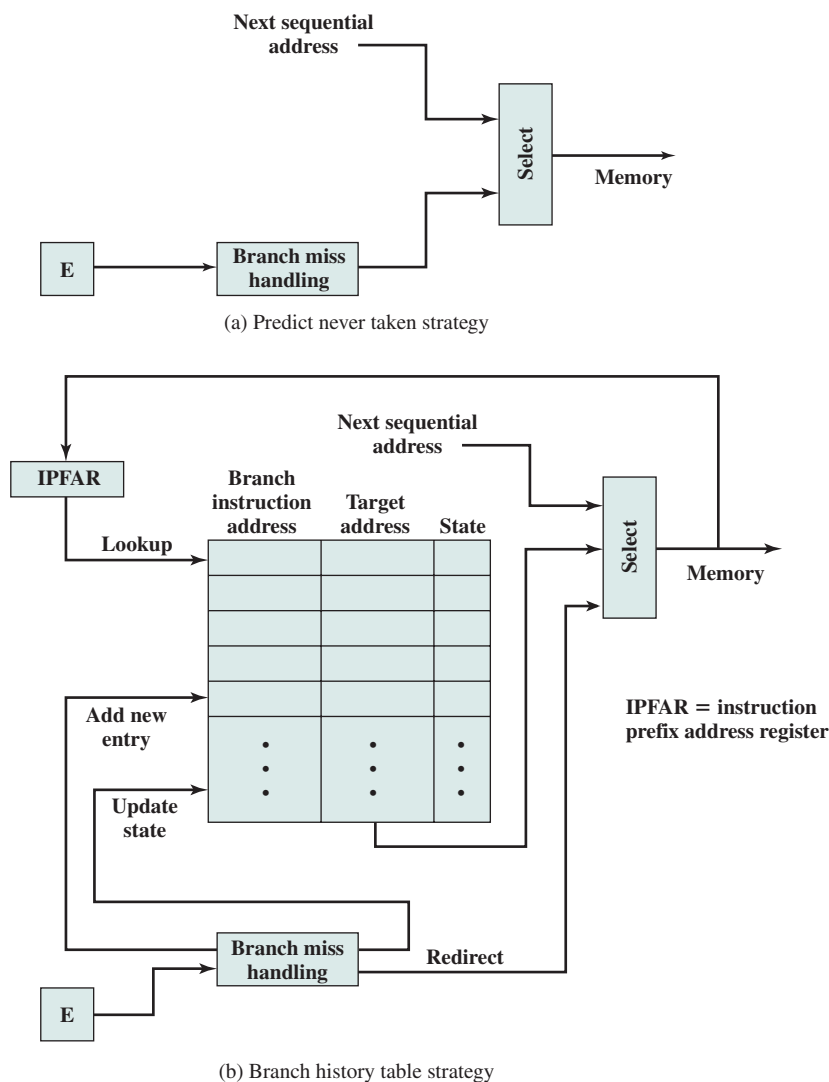
table but a greater instruction fetch time compared with storing the target instruction [RECH98].

Figure 14.20 contrasts this scheme with a predict-never-taken strategy. With the former strategy, the instruction fetch stage always fetches the next sequential address. If a branch is taken, some logic in the processor detects this and instructs that the next instruction be fetched from the target address (in addition to flushing the pipeline). The branch history table is treated as a cache. Each prefetch triggers a lookup in the branch history table. If no match is found, the next sequential address is used for the fetch. If a match is found, a prediction is made based on the state of the instruction: Either the next sequential address or the branch target address is fed to the select logic.

When the branch instruction is executed, the execute stage signals the branch history table logic with the result. The state of the instruction is updated to reflect a correct or incorrect prediction. If the prediction is incorrect, the select logic is redirected to the correct address for the next fetch. When a conditional branch instruction is encountered that is not in the table, it is added to the table and one of the existing entries is discarded, using one of the cache replacement algorithms discussed in Chapter 4.

A refinement of the branch history approach is referred to as two-level or correlation-based branch history [YEH91]. This approach is based on the assumption that whereas in loop-closing branches, the past history of a particular branch instruction is a good predictor of future behavior, with more complex control-flow structures, the direction of a branch is frequently correlated with the direction of related branches. An example is an if-then-else or case structure. There are a number of strategies possible. Typically, recent global branch history (i.e., the history of the most recent branches not just of this branch instruction) is used in addition to the history of the current branch instruction. The general structure is defined as an  $(m, n)$  correlator, which uses the behavior of the last  $m$  branches to choose from  $2^m$   $n$ -bit branch predictors for the current branch instruction. In other words, an





**Figure 14.20** Dealing with Branches

$n$ -bit history is kept for a give branch for each possible combination of branches taken by the most recent  $m$  branches.

**DELAYED BRANCH** It is possible to improve pipeline performance by automatically rearranging instructions within a program, so that branch instructions occur later than actually desired. This intriguing approach is examined in Chapter 15.

## Intel 80486 Pipelining

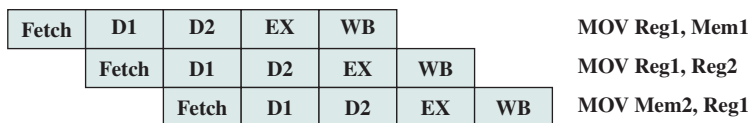
An instructive example of an instruction pipeline is that of the Intel 80486. The 80486 implements a five-stage pipeline:

- **Fetch:** Instructions are fetched from the cache or from external memory and placed into one of the two 16-byte prefetch buffers. The objective of the fetch stage is to fill the prefetch buffers with new data as soon as the old data have been consumed by the instruction decoder. Because instructions are of variable length (from 1 to 11 bytes not counting prefixes), the status of the prefetcher relative to the other pipeline stages varies from instruction to instruction. On average, about five instructions are fetched with each 16-byte load [CRAW90]. The fetch stage operates independently of the other stages to keep the prefetch buffers full.
- **Decode stage 1:** All opcode and addressing-mode information is decoded in the D1 stage. The required information, as well as instruction-length information, is included in at most the first 3 bytes of the instruction. Hence, 3 bytes are passed to the D1 stage from the prefetch buffers. The D1 decoder can then direct the D2 stage to capture the rest of the instruction (displacement and immediate data), which is not involved in the D1 decoding.
- **Decode stage 2:** The D2 stage expands each opcode into control signals for the ALU. It also controls the computation of the more complex addressing modes.
- **Execute:** This stage includes ALU operations, cache access, and register update.
- **Write back:** This stage, if needed, updates registers and status flags modified during the preceding execute stage. If the current instruction updates memory, the computed value is sent to the cache and to the bus-interface write buffers at the same time.

With the use of two decode stages, the pipeline can sustain a throughput of close to one instruction per clock cycle. Complex instructions and conditional branches can slow down this rate.

Figure 14.21 shows examples of the operation of the pipeline. Figure 14.21a shows that there is no delay introduced into the pipeline when a memory access is required. However, as Figure 14.21b shows, there can be a delay for values used to compute memory addresses. That is, if a value is loaded from memory into a register and that register is then used as a base register in the next instruction, the processor will stall for one cycle. In this example, the processor accesses the cache in the EX stage of the first instruction and stores the value retrieved in the register during the WB stage. However, the next instruction needs this register in its D2 stage. When the D2 stage lines up with the WB stage of the previous instruction, bypass signal paths allow the D2 stage to have access to the same data being used by the WB stage for writing, saving one pipeline stage.

Figure 14.21c illustrates the timing of a branch instruction, assuming that the branch is taken. The compare instruction updates condition codes in the WB stage, and bypass paths make this available to the EX stage of the jump instruction at the same time. In parallel, the processor runs a speculative fetch cycle to the target of the jump during the EX stage of the jump instruction. If the processor determines a false branch condition, it discards this prefetch and continues execution with the next sequential instruction (already fetched and decoded).



(a) No data load delay in the pipeline



(b) Pointer load delay



(c) Branch instruction timing

**Figure 14.21** 80486 Instruction Pipeline Examples

## 14.5 THE x86 PROCESSOR FAMILY

The x86 organization has evolved dramatically over the years. In this section we examine some of the details of the most recent processor organizations, concentrating on common elements in single processors. Chapter 16 looks at superscalar aspects of the x86, and Chapter 18 examines the multicore organization. An overview of the Pentium 4 processor organization is depicted in Figure 4.18.

### Register Organization

The register organization includes the following types of registers (Table 14.2):

- **General:** There are eight 32-bit general-purpose registers (see Figure 14.3c). These may be used for all types of x86 instructions; they can also hold operands for address calculations. In addition, some of these registers also serve special purposes. For example, string instructions use the contents of the ECX, ESI, and EDI registers as operands without having to reference these registers explicitly in the instruction. As a result, a number of instructions can be encoded more compactly. In 64-bit mode, there are 16 64-bit general-purpose registers.
- **Segment:** The six 16-bit segment registers contain segment selectors, which index into segment tables, as discussed in Chapter 8. The code segment (CS) register references the segment containing the instruction being executed. The stack segment (SS) register references the segment containing a user-visible stack. The remaining segment registers (DS, ES, FS, GS) enable the user to reference up to four separate data segments at a time.
- **Flags:** The 32-bit EFLAGS register contains condition codes and various mode bits. In 64-bit mode, this register is extended to 64 bits and referred

**Table 14.2** x86 Processor Registers

## (a) Integer Unit in 32-bit Mode

| Type                | Number | Length (bits) | Purpose                        |
|---------------------|--------|---------------|--------------------------------|
| General             | 8      | 32            | General-purpose user registers |
| Segment             | 6      | 16            | Contain segment selectors      |
| EFLAGS              | 1      | 32            | Status and control bits        |
| Instruction Pointer | 1      | 32            | Instruction pointer            |

## (b) Integer Unit in 64-bit Mode

| Type                | Number | Length (bits) | Purpose                        |
|---------------------|--------|---------------|--------------------------------|
| General             | 16     | 32            | General-purpose user registers |
| Segment             | 6      | 16            | Contain segment selectors      |
| RFLAGS              | 1      | 64            | Status and control bits        |
| Instruction Pointer | 1      | 64            | Instruction pointer            |

## (c) Floating-Point Unit

| Type                | Number | Length (bits) | Purpose                                        |
|---------------------|--------|---------------|------------------------------------------------|
| Numeric             | 8      | 80            | Hold floating-point numbers                    |
| Control             | 1      | 16            | Control bits                                   |
| Status              | 1      | 16            | Status bits                                    |
| Tag Word            | 1      | 16            | Specifies contents of numeric registers        |
| Instruction Pointer | 1      | 48            | Points to instruction interrupted by exception |
| Data Pointer        | 1      | 48            | Points to operand interrupted by exception     |

to as RFLAGS. In the current architecture definition, the upper 32 bits of RFLAGS are unused.

- **Instruction pointer:** Contains the address of the current instruction.

There are also registers specifically devoted to the floating-point unit:

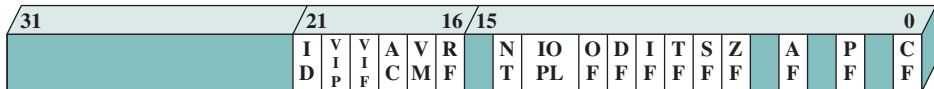
- **Numeric:** Each register holds an extended-precision 80-bit floating-point number. There are eight registers that function as a stack, with push and pop operations available in the instruction set.
- **Control:** The 16-bit control register contains bits that control the operation of the floating-point unit, including the type of rounding control; single, double, or extended precision; and bits to enable or disable various exception conditions.
- **Status:** The 16-bit status register contains bits that reflect the current state of the floating-point unit, including a 3-bit pointer to the top of the stack; condition codes reporting the outcome of the last operation; and exception flags.
- **Tag word:** This 16-bit register contains a 2-bit tag for each floating-point numeric register, which indicates the nature of the contents of the corresponding register.

The four possible values are valid, zero, special (NaN, infinity, denormalized), and empty. These tags enable programs to check the contents of a numeric register without performing complex decoding of the actual data in the register. For example, when a context switch is made, the processor need not save any floating-point registers that are empty.

The use of most of the aforementioned registers is easily understood. Let us elaborate briefly on several of the registers.

**EFLAGS REGISTER** The EFLAGS register (Figure 14.22) indicates the condition of the processor and helps to control its operation. It includes the six condition codes defined in Table 12.9 (carry, parity, auxiliary, zero, sign, overflow), which report the results of an integer operation. In addition, there are bits in the register that may be referred to as control bits:

- **Trap flag (TF):** When set, causes an interrupt after the execution of each instruction. This is used for debugging.
- **Interrupt enable flag (IF):** When set, the processor will recognize external interrupts.
- **Direction flag (DF):** Determines whether string processing instructions increment or decrement the 16-bit half-registers SI and DI (for 16-bit operations) or the 32-bit registers ESI and EDI (for 32-bit operations).
- **I/O privilege flag (IOPL):** When set, causes the processor to generate an exception on all accesses to I/O devices during protected-mode operation.
- **Resume flag (RF):** Allows the programmer to disable debug exceptions so that the instruction can be restarted after a debug exception without immediately causing another debug exception.
- **Alignment check (AC):** Activates if a word or doubleword is addressed on a nonword or nondoubleword boundary.
- **Identification flag (ID):** If this bit can be set and cleared, then this processor supports the processorID instruction. This instruction provides information about the vendor, family, and model.

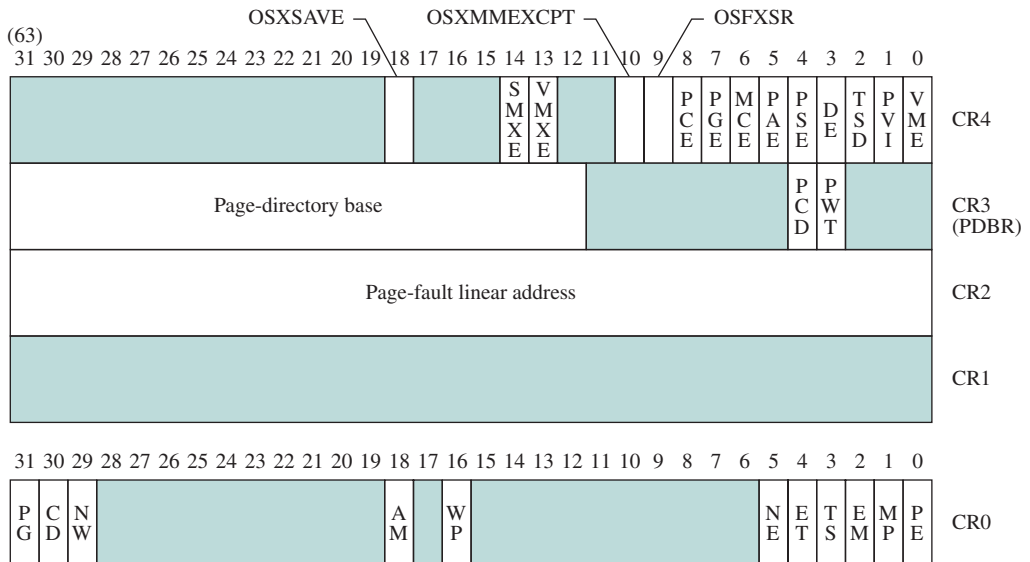


- |      |                             |    |                         |
|------|-----------------------------|----|-------------------------|
| ID   | = Identification flag       | DF | = Direction flag        |
| VIP  | = Virtual interrupt pending | IF | = Interrupt enable flag |
| VIF  | = Virtual interrupt flag    | TF | = Trap flag             |
| AC   | = Alignment check           | SF | = Sign flag             |
| VM   | = Virtual 8086 mode         | ZF | = Zero flag             |
| RF   | = Resume flag               | AF | = Auxiliary carry flag  |
| NT   | = Nested task flag          | PF | = Parity flag           |
| IOPL | = I/O privilege level       | CF | = Carry flag            |
| OF   | = Overflow flag             |    |                         |

**Figure 14.22** Pentium II EFLAGS Register

In addition, there are 4 bits that relate to operating mode. The Nested Task (NT) flag indicates that the current task is nested within another task in protected-mode operation. The Virtual Mode (VM) bit allows the programmer to enable or disable virtual 8086 mode, which determines whether the processor runs as an 8086 machine. The Virtual Interrupt Flag (VIF) and Virtual Interrupt Pending (VIP) flag are used in a multitasking environment.

**CONTROL REGISTERS** The x86 employs four control registers (register CR1 is unused) to control various aspects of processor operation (Figure 14.23). All of the registers except CR0 are either 32 bits or 64 bits long, depending on whether the implementation supports the x86 64-bit architecture. The CR0 register contains system control flags,



Shaded area indicates reserved bits.

|            |   |                                     |     |   |                               |
|------------|---|-------------------------------------|-----|---|-------------------------------|
| OSXSAVE    | = | XSAVE enable bit                    | PCD | = | Page-level cache disable      |
| SMXE       | = | Enable safer mode extensions        | PWT | = | Page-level writes transparent |
| VMXE       | = | Enable virtual machine extensions   | PG  | = | Paging                        |
| OSXMMEXCPT | = | Support unmasked SIMD FP exceptions | CD  | = | Cache disable                 |
| OSFXSR     | = | Support FXSAVE, FXSTOR              | NW  | = | Not write through             |
| PCE        | = | Performance counter enable          | AM  | = | Alignment mask                |
| PGE        | = | Page global enable                  | WP  | = | Write protect                 |
| MCE        | = | Machine check enable                | NE  | = | Numeric error                 |
| PAE        | = | Physical address extension          | ET  | = | Extension type                |
| PSE        | = | Page size extensions                | TS  | = | Task switched                 |
| DE         | = | Debug extensions                    | EM  | = | Emulation                     |
| TSD        | = | Time stamp disable                  | MP  | = | Monitor coprocessor           |
| PVI        | = | Protected mode virtual interrupt    | PE  | = | Protection enable             |
| VME        | = | Virtual 8086 mode extensions        |     |   |                               |

**Figure 14.23** x86 Control Registers

which control modes or indicate states that apply generally to the processor rather than to the execution of an individual task. The flags are as follows:

- **Protection Enable (PE):** Enable/disable protected mode of operation.
- **Monitor Coprocessor (MP):** Only of interest when running programs from earlier machines on the x86; it relates to the presence of an arithmetic coprocessor.
- **Emulation (EM):** Set when the processor does not have a floating-point unit, and causes an interrupt when an attempt is made to execute floating-point instructions.
- **Task Switched (TS):** Indicates that the processor has switched tasks.
- **Extension Type (ET):** Not used on the Pentium and later machines; used to indicate support of math coprocessor instructions on earlier machines.
- **Numeric Error (NE):** Enables the standard mechanism for reporting floating-point errors on external bus lines.
- **Write Protect (WP):** When this bit is clear, read-only user-level pages can be written by a supervisor process. This feature is useful for supporting process creation in some operating systems.
- **Alignment Mask (AM):** Enables/disables alignment checking.
- **Not Write Through (NW):** Selects mode of operation of the data cache. When this bit is set, the data cache is inhibited from cache write-through operations.
- **Cache Disable (CD):** Enables/disables the internal cache fill mechanism.
- **Paging (PG):** Enables/disables paging.

When paging is enabled, the CR2 and CR3 registers are valid. The CR2 register holds the 32-bit linear address of the last page accessed before a page fault interrupt. The leftmost 20 bits of CR3 hold the 20 most significant bits of the base address of the page directory; the remainder of the address contains zeros. Two bits of CR3 are used to drive pins that control the operation of an external cache. The page-level cache disable (PCD) enables or disables the external cache, and the page-level writes transparent (PWT) bit controls write through in the external cache.

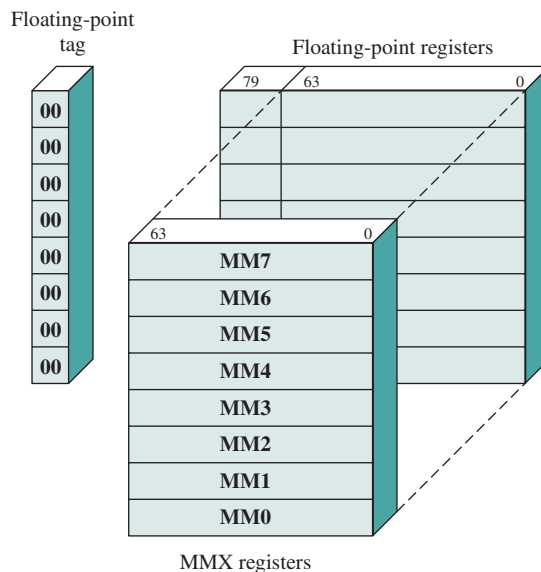
Nine additional control bits are defined in CR4:

- **Virtual-8086 Mode Extension (VME):** Enables support for the virtual interrupt flag in virtual-8086 mode.
- **Protected-mode Virtual Interrupts (PVI):** Enables support for the virtual interrupt flag in protected mode.
- **Time Stamp Disable (TSD):** Disables the read from time stamp counter (RDTSC) instruction, which is used for debugging purposes.
- **Debugging Extensions (DE):** Enables I/O breakpoints; this allows the processor to interrupt on I/O reads and writes.
- **Page Size Extensions (PSE):** Enables large page sizes (2 or 4-MByte pages) when set; restricts pages to 4 KBytes when clear.
- **Physical Address Extension (PAE):** Enables address lines A35 through A32 whenever a special new addressing mode, controlled by the PSE, is enabled.

- **Machine Check Enable (MCE):** Enables the machine check interrupt, which occurs when a data parity error occurs during a read bus cycle or when a bus cycle is not successfully completed.
- **Page Global Enable (PGE):** Enables the use of global pages. When  $PGE = 1$  and a task switch is performed, all of the TLB entries are flushed with the exception of those marked global.
- **Performance Counter Enable (PCE):** Enables the execution of the RDPMC (read performance counter) instruction at any privilege level. Two performance counters are used to measure the duration of a specific event type and the number of occurrences of a specific event type.

**MMX REGISTERS** Recall from Section 10.3 that the x86 MMX capability makes use of several 64-bit data types. The MMX instructions make use of 3-bit register address fields, so that eight MMX registers are supported. In fact, the processor does not include specific MMX registers. Rather, the processor uses an aliasing technique (Figure 14.24). The existing floating-point registers are used to store MMX values. Specifically, the low-order 64 bits (mantissa) of each floating-point register are used to form the eight MMX registers. Thus, the older 32-bit x86 architecture is easily extended to support the MMX capability. Some key characteristics of the MMX use of these registers are as follows:

- Recall that the floating-point registers are treated as a stack for floating-point operations. For MMX operations, these same registers are accessed directly.
- The first time that an MMX instruction is executed after any floating-point operations, the FP tag word is marked valid. This reflects the change from stack operation to direct register addressing.



**Figure 14.24** Mapping of MMX Registers to Floating-Point Registers



- The EMMS (Empty MMX State) instruction sets bits of the FP tag word to indicate that all registers are empty. It is important that the programmer insert this instruction at the end of an MMX code block so that subsequent floating-point operations function properly.
- When a value is written to an MMX register, bits [79:64] of the corresponding FP register (sign and exponent bits) are set to all ones. This sets the value in the FP register to NaN (not a number) or infinity when viewed as a floating-point value. This ensures that an MMX data value will not look like a valid floating-point value.

## Interrupt Processing

Interrupt processing within a processor is a facility provided to support the operating system. It allows an application program to be suspended, in order that a variety of interrupt conditions can be serviced and later resumed.

**INTERRUPTS AND EXCEPTIONS** Two classes of events cause the x86 to suspend execution of the current instruction stream and respond to the event: interrupts and exceptions. In both cases, the processor saves the context of the current process and transfers to a predefined routine to service the condition. An *interrupt* is generated by a signal from hardware, and it may occur at random times during the execution of a program. An *exception* is generated from software, and it is provoked by the execution of an instruction. There are two sources of interrupts and two sources of exceptions:

### 1. Interrupts

- **Maskable interrupts:** Received on the processor's INTR pin. The processor does not recognize a maskable interrupt unless the interrupt enable flag (IF) is set.
- **Nonmaskable interrupts:** Received on the processor's NMI pin. Recognition of such interrupts cannot be prevented.

### 2. Exceptions

- **Processor-detected exceptions:** Results when the processor encounters an error while attempting to execute an instruction.
- **Programmed exceptions:** These are instructions that generate an exception (e.g., INTO, INT3, INT, and BOUND).

**INTERRUPT VECTOR TABLE** Interrupt processing on the x86 uses the interrupt vector table. Every type of interrupt is assigned a number, and this number is used to index into the interrupt vector table. This table contains 256 32-bit interrupt vectors, which is the address (segment and offset) of the interrupt service routine for that interrupt number.

Table 14.3 shows the assignment of numbers in the interrupt vector table; shaded entries represent interrupts, while nonshaded entries are exceptions. The NMI hardware interrupt is type 2. INTR hardware interrupts are assigned numbers in the range of 32 to 255; when an INTR interrupt is generated, it must be accompanied on the bus with the interrupt vector number for this interrupt. The remaining vector numbers are used for exceptions.

**Table 14.3** x86 Exception and Interrupt Vector Table

| Vector Number | Description                                                                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0             | Divide error; division overflow or division by zero                                                                                                                           |
| 1             | Debug exception; includes various faults and traps related to debugging                                                                                                       |
| 2             | NMI pin interrupt; signal on NMI pin                                                                                                                                          |
| 3             | Breakpoint; caused by INT 3 instruction, which is a 1-byte instruction useful for debugging                                                                                   |
| 4             | INTO-detected overflow; occurs when the processor executes INTO with the OF flag set                                                                                          |
| 5             | BOUND range exceeded; the BOUND instruction compares a register with boundaries stored in memory and generates an interrupt if the contents of the register is out of bounds. |
| 6             | Undefined opcode                                                                                                                                                              |
| 7             | Device not available; attempt to use ESC or WAIT instruction fails due to lack of external device                                                                             |
| 8             | Double fault; two interrupts occur during the same instruction and cannot be handled serially                                                                                 |
| 9             | Reserved                                                                                                                                                                      |
| 10            | Invalid task state segment; segment describing a requested task is not initialized or not valid                                                                               |
| 11            | Segment not present; required segment not present                                                                                                                             |
| 12            | Stack fault; limit of stack segment exceeded or stack segment not present                                                                                                     |
| 13            | General protection; protection violation that does not cause another exception (e.g., writing to a read-only segment)                                                         |
| 14            | Page fault                                                                                                                                                                    |
| 15            | Reserved                                                                                                                                                                      |
| 16            | Floating-point error; generated by a floating-point arithmetic instruction                                                                                                    |
| 17            | Alignment check; access to a word stored at an odd byte address or a doubleword stored at an address not a multiple of 4                                                      |
| 18            | Machine check; model specific                                                                                                                                                 |
| 19–31         | Reserved                                                                                                                                                                      |
| 32–255        | User interrupt vectors; provided when INTR signal is activated                                                                                                                |

Unshaded: exceptions

Shaded: interrupts

If more than one exception or interrupt is pending, the processor services them in a predictable order. The location of vector numbers within the table does not reflect priority. Instead, priority among exceptions and interrupts is organized into five classes. In descending order of priority, these are

- **Class 1:** Traps on the previous instruction (vector number 1)
- **Class 2:** External interrupts (2, 32–255)
- **Class 3:** Faults from fetching next instruction (3, 14)
- **Class 4:** Faults from decoding the next instruction (6, 7)
- **Class 5:** Faults on executing an instruction (0, 4, 5, 8, 10–14, 16, 17)

**INTERRUPT HANDLING** Just as with a transfer of execution using a `CALL` instruction, a transfer to an interrupt-handling routine uses the system stack to store the processor state. When an interrupt occurs and is recognized by the processor, a sequence of events takes place:

1. If the transfer involves a change of privilege level, then the current stack segment register and the current extended stack pointer (ESP) register are pushed onto the stack.
2. The current value of the EFLAGS register is pushed onto the stack.
3. Both the interrupt (IF) and trap (TF) flags are cleared. This disables INTR interrupts and the trap or single-step feature.
4. The current code segment (CS) pointer and the current instruction pointer (IP or EIP) are pushed onto the stack.
5. If the interrupt is accompanied by an error code, then the error code is pushed onto the stack.
6. The interrupt vector contents are fetched and loaded into the CS and IP or EIP registers. Execution continues from the interrupt service routine.

To return from an interrupt, the interrupt service routine executes an `IRET` instruction. This causes all of the values saved on the stack to be restored; execution resumes from the point of the interrupt.

## 14.6 THE ARM PROCESSOR

In this section, we look at some of the key elements of the ARM architecture and organization. We defer a discussion of more complex aspects of organization and pipelining until Chapter 16. For the discussion in this section and in Chapter 16, it is useful to keep in mind key characteristics of the ARM architecture. ARM is primarily a RISC system with the following notable attributes:

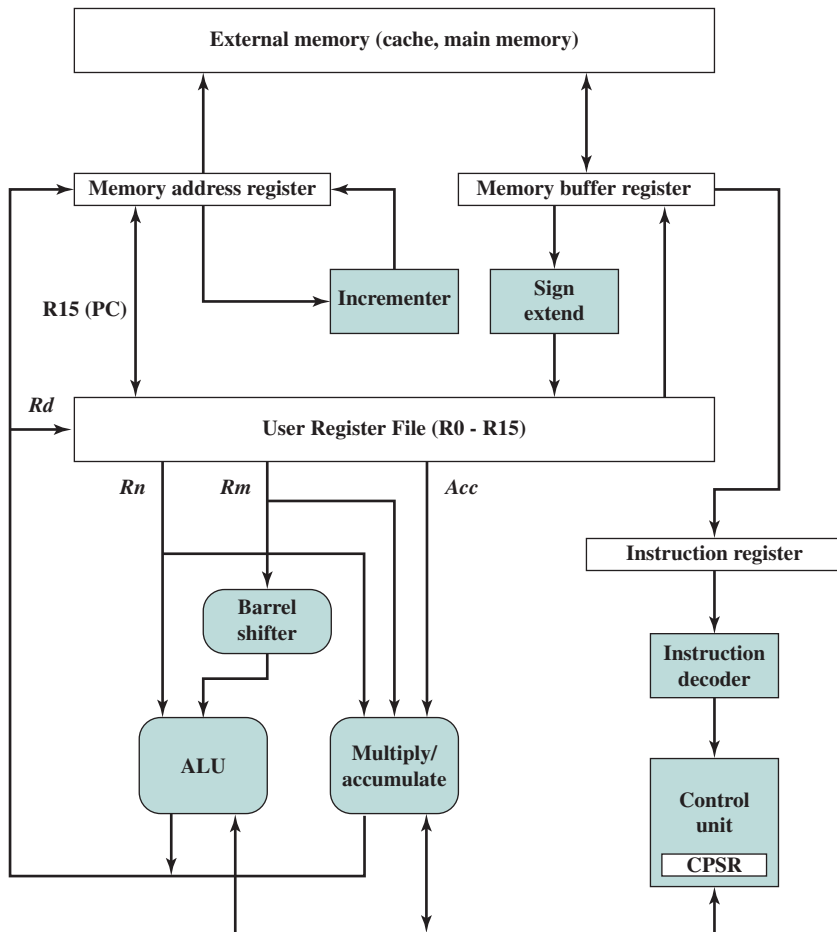
- A moderate array of uniform registers, more than are found on some CISC systems but fewer than are found on many RISC systems.
- A load/store model of data processing, in which operations only perform on operands in registers and not directly in memory. All data must be loaded into registers before an operation can be performed; the result can then be used for further processing or stored into memory.
- A uniform fixed-length instruction of 32 bits for the standard set and 16 bits for the Thumb instruction set.
- To make each data processing instruction more flexible, either a shift or rotation can preprocess one of the source registers. To efficiently support this feature, there are separate arithmetic logic unit (ALU) and shifter units.
- A small number of addressing modes with all load/store addressees determined from registers and instruction fields. Indirect or indexed addressing involving values in memory are not used.
- Auto-increment and auto-decrement addressing modes are used to improve the operation of program loops.

- Conditional execution of instructions minimizes the need for conditional branch instructions, thereby improving pipeline efficiency, because pipeline flushing is reduced.

## Processor Organization

The ARM processor organization varies substantially from one implementation to the next, particularly when based on different versions of the ARM architecture. However, it is useful for the discussion in this section to present a simplified, generic ARM organization, which is illustrated in Figure 14.25. In this figure, the arrows indicate the flow of data. Each box represents a functional hardware unit or a storage unit.

Data are exchanged with the processor from external memory through a data bus. The value transferred is either a data item, as a result of a load or store instruction, or an instruction fetch. Fetched instructions pass through an instruction decoder before execution, under control of a control unit. The latter includes



**Figure 14.25** Simplified ARM Organization

pipeline logic and provides control signals (not shown) to all the hardware elements of the processor. Data items are placed in the register file, consisting of a set of 32-bit registers. Byte or halfword items treated as twos-complement numbers are sign-extended to 32 bits.

ARM data processing instructions typically have two source registers, *Rn* and *Rm*, and a single result or destination register, *Rd*. The source register values feed into the ALU or a separate multiply unit that makes use of an additional register to accumulate partial results. The ARM processor also includes a hardware unit that can shift or rotate the *Rm* value before it enters the ALU. This shift or rotate occurs within the cycle time of the instruction and increases the power and flexibility of many data processing operations.

The results of an operation are fed back to the destination register. Load/store instructions may also use the output of the arithmetic units to generate the memory address for a load or store.

## Processor Modes

It is quite common for a processor to support only a small number of processor modes. For example, many operating systems make use of just two modes: a user mode and a kernel mode, with the latter mode used to execute privileged system software. In contrast, the ARM architecture provides a flexible foundation for operating systems to enforce a variety of protection policies.

The ARM architecture supports seven execution modes. Most application programs execute in **user mode**. While the processor is in user mode, the program being executed is unable to access protected system resources or to change mode, other than by causing an exception to occur.

The remaining six execution modes are referred to as privileged modes. These modes are used to run system software. There are two principal advantages to defining so many different privileged modes: (1) The OS can tailor the use of system software to a variety of circumstances, and (2) certain registers are dedicated for use for each of the privileged modes, allows swifter changes in context.

The exception modes have full access to system resources and can change modes freely. Five of these modes are known as exception modes. These are entered when specific exceptions occur. Each of these modes has some dedicated registers that substitute for some of the user mode registers, and which are used to avoid corrupting User mode state information when the exception occurs. The exception modes are as follows:

- **Supervisor mode:** Usually what the OS runs in. It is entered when the processor encounters a software interrupt instruction. Software interrupts are a standard way to invoke operating system services on ARM.
- **Abort mode:** Entered in response to memory faults.
- **Undefined mode:** Entered when the processor attempts to execute an instruction that is supported neither by the main integer core nor by one of the coprocessors.
- **Fast interrupt mode:** Entered whenever the processor receives an interrupt signal from the designated fast interrupt source. A fast interrupt cannot be interrupted, but a fast interrupt may interrupt a normal interrupt.

- **Interrupt mode:** Entered whenever the processor receives an interrupt signal from any other interrupt source (other than fast interrupt). An interrupt may only be interrupted by a fast interrupt.

The remaining privileged mode is the **System mode**. This mode is not entered by any exception and uses the same registers available in User mode. The System mode is used for running certain privileged operating system tasks. System mode tasks may be interrupted by any of the five exception categories.

## Register Organization

Figure 14.26 depicts the user-visible registers for the ARM. The ARM processor has a total of 37 32-bit registers, classified as follows:

- Thirty-one registers referred to in the ARM manual as general-purpose registers. In fact, some of these, such as the program counters, have special purposes.
- Six program status registers.

Registers are arranged in partially overlapping banks, with the current processor mode determining which bank is available. At any time, sixteen numbered registers and one or two program status registers are visible, for a total of 17 or 18 software-visible registers. Figure 14.26 is interpreted as follows:

- Registers R0 through R7, register R15 (the program counter) and the current program status register (CPSR) are visible in and shared by all modes.
- Registers R8 through R12 are shared by all modes except fast interrupt, which has its own dedicated registers R8\_fiq through R12\_fiq.
- All the exception modes have their own versions of registers R13 and R14.
- All the exception modes have a dedicated saved program status register (SPSR)

**GENERAL-PURPOSE REGISTERS** Register R13 is normally used as a stack pointer and is also known as the SP. Because each exception mode has a separate R13, each exception mode can have its own dedicated program stack. R14 is known as the link register (LR) and is used to hold subroutine return addresses and exception mode returns. Register R15 is the program counter (PC).

**PROGRAM STATUS REGISTERS** The CPSR is accessible in all processor modes. Each exception mode also has a dedicated SPSR that is used to preserve the value of the CPSR when the associated exception occurs.

The 16 most significant bits of the CPSR contain user flags visible in User mode, and which can be used to affect the operation of a program (Figure 14.27). These are as follows:

- **Condition code flags:** The N, Z, C, and V flags, which are discussed in Chapter 12.
- **Q flag:** used to indicate whether overflow and/or saturation has occurred in some SIMD-oriented instructions.
- **J bit:** indicates the use of special 8-bit instructions, known as Jazelle instructions, which are beyond the scope of our discussion.
- **GE[3:0] bits:** SIMD instructions use bits [19:16] as Greater than or Equal (GE) flags for individual bytes or halfwords of the result.

| Modes            |         |            |         |           |           |                |
|------------------|---------|------------|---------|-----------|-----------|----------------|
| Privileged modes |         |            |         |           |           |                |
| Exception modes  |         |            |         |           |           |                |
| User             | System  | Supervisor | Abort   | Undefined | Interrupt | Fast interrupt |
| R0               | R0      | R0         | R0      | R0        | R0        | R0             |
| R1               | R1      | R1         | R1      | R1        | R1        | R1             |
| R2               | R2      | R2         | R2      | R2        | R2        | R2             |
| R3               | R3      | R3         | R3      | R3        | R3        | R3             |
| R4               | R4      | R4         | R4      | R4        | R4        | R4             |
| R5               | R5      | R5         | R5      | R5        | R5        | R5             |
| R6               | R6      | R6         | R6      | R6        | R6        | R6             |
| R7               | R7      | R7         | R7      | R7        | R7        | R7             |
| R8               | R8      | R8         | R8      | R8        | R8        | R8_fiq         |
| R9               | R9      | R9         | R9      | R9        | R9        | R9_fiq         |
| R10              | R10     | R10        | R10     | R10       | R10       | R10_fiq        |
| R11              | R11     | R11        | R11     | R11       | R11       | R11_fiq        |
| R12              | R12     | R12        | R12     | R12       | R12       | R12_fiq        |
| R13(SP)          | R13(SP) | R13_svc    | R13_abt | R13_und   | R13_irq   | R13_fiq        |
| R14(LR)          | R14(LR) | R14_svc    | R14_abt | R14_und   | R14_irq   | R14_fiq        |
| R15(PC)          | R15(PC) | R15(PC)    | R15(PC) | R15(PC)   | R15(PC)   | R15(PC)        |

|      |      |          |          |          |          |          |
|------|------|----------|----------|----------|----------|----------|
| CPSR | CPSR | CPSR     | CPSR     | CPSR     | CPSR     | CPSR     |
|      |      | SPSR_svc | SPSR_abt | SPSR_und | SPSR_irq | SPSR_fiq |

Shading indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode.

SP = stack pointer      CPSR = current program status register

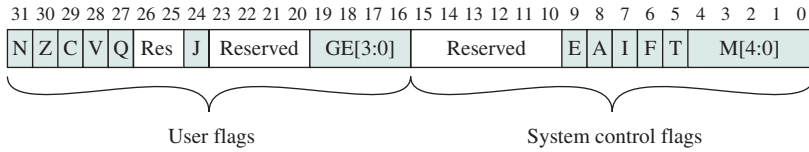
LR = link register      SPSR = saved program status register

PC = program counter

**Figure 14.26** ARM Register Organization

The 16 least significant bits of the CPSR contain system control flags that can only be altered when the processor is in a privileged mode. The fields are as follows:

- **E bit:** Controls load and store endianness for data; ignored for instruction fetches.
- **Interrupt disable bits:** The A bit disables imprecise data aborts when set; the I bit disables IRQ interrupts when set; and the F bit disables FIQ interrupts when set.
- **T bit:** Indicates whether instructions should be interpreted as normal ARM instructions or Thumb instructions.
- **Mode bits:** Indicates the processor mode.



**Figure 14.27** Format of ARM CPSR and SPSR

## Interrupt Processing

As with any processor, the ARM includes a facility that enables the processor to interrupt the currently executing program to deal with exception conditions. Exceptions are generated by internal and external sources to cause the processor to handle an event. The processor state just before handling the exception is normally preserved so that the original program can be resumed when the exception routine has completed. More than one exception can arise at the same time. The ARM architecture supports seven types of exception. Table 14.4 lists the types of exception and the processor mode that is used to process each type. When an exception occurs, execution is forced

**Table 14.4** ARM Interrupt Vector

| Exception type         | Mode       | Normal entry address | Description                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------|------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reset                  | Supervisor | 0x00000000           | Occurs when the system is initialized.                                                                                                                                                                                                                                                                                                                                                     |
| Data abort             | Abort      | 0x00000010           | Occurs when an invalid memory address has been accessed, such as if there is no physical memory for an address or the correct access permission is lacking.                                                                                                                                                                                                                                |
| FIQ (fast interrupt)   | FIQ        | 0x0000001C           | Occurs when an external device asserts the FIQ pin on the processor. An interrupt cannot be interrupted except by an FIQ. FIQ is designed to support a data transfer or channel process, and has sufficient private registers to remove the need for register saving in such applications, therefore minimizing the overhead of context switching. A fast interrupt cannot be interrupted. |
| IRQ (interrupt)        | IRQ        | 0x00000018           | Occurs when an external device asserts the IRQ pin on the processor. An interrupt cannot be interrupted except by an FIQ.                                                                                                                                                                                                                                                                  |
| Prefetch abort         | Abort      | 0x0000000C           | Occurs when an attempt to fetch an instruction results in a memory fault. The exception is raised when the instruction enters the execute stage of the pipeline.                                                                                                                                                                                                                           |
| Undefined instructions | Undefined  | 0x00000004           | Occurs when an instruction not in the instruction set reaches the execute stage of the pipeline.                                                                                                                                                                                                                                                                                           |
| Software interrupt     | Supervisor | 0x00000008           | Generally used to allow user mode programs to call the OS. The user program executes a SWI instruction with an argument that identifies the function the user wishes to perform.                                                                                                                                                                                                           |



from a fixed memory address corresponding to the type of exception. These fixed addresses are called the exception vectors.

If more than one interrupt is outstanding, they are handled in priority order. Table 14.4 lists the exceptions in priority order, highest to lowest.

When an exception occurs, the processor halts execution after the current instruction. The state of the processor is preserved in the SPSR that corresponds to the type of exception, so that the original program can be resumed when the exception routine has completed. The address of the instruction the processor was just about to execute is placed in the link register of the appropriate processor mode. To return after handling the exception, the SPSR is moved into the CPSR and R14 is moved into the PC.

## 14.7 RECOMMENDED READING

[PATT01] and [MOSH01] provide excellent coverage of the pipelining issues discussed in this chapter. [HENN91] contains a detailed discussions of pipelining. [SOHI90] provides an excellent, detailed discussion of the hardware design issues involved in an instruction pipeline. [RAMA77] is a classic paper on the subject still well worth reading.

[EVER01] examines the evolution of branch prediction strategies. [CRAG92] is a detailed study of branch prediction in instruction pipelines. [DUBE91] and [LILJ88] examine various branch prediction strategies that can be used to enhance the performance of instruction pipelining. [KAEL91] examines the difficulty introduced into branch prediction by instructions whose target address is variable.

[BREY09] provides good coverage of interrupt processing on the x86. [FOG08b] provides a detailed discussion of pipeline architecture for the x86 family.

**BREY09** Brey, B. *The Intel Microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and Core2 with 64-bit Extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.

**CRAG92** Cragon, H. *Branch Strategy Taxonomy and Performance Models*. Los Alamitos, CA: IEEE Computer Society Press, 1992.

**DUBE91** Dubey, P., and Flynn, M. "Branch Strategies: Modeling and Optimization." *IEEE Transactions on Computers*, October 1991.

**EVER01** Evers, M., and Yeh, T. "Understanding Branches and Designing Branch Predictors for High-Performance Microprocessors." *Proceedings of the IEEE*, November 2001.

**FOG08b** Fog, A. *The Microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. [www.agner.org/optimize/](http://www.agner.org/optimize/)

**HENN91** Hennessy, J., and Jouppi, N. "Computer Technology and Architecture: An Evolving Interaction." *Computer*, September 1991.

**KAEL91** Kaeli, D., and Emma, P. "Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns." *Proceedings, 18th Annual International Symposium on Computer Architecture*, May 1991.

**LILJ88** Lilja, D. "Reducing the Branch Penalty in Pipelined Processors." *Computer*, July 1988.

**MOSH01** Moshovos, A., and Sohi, G. "Microarchitectural Innovations: Boosting Microprocessor Performance Beyond Semiconductor Technology Scaling." *Proceedings of the IEEE*, November 2001.

- PATT01** Patt, Y. "Requirements, Bottlenecks, and Good Fortune: Agents for Micro-processor Evolution." *Proceedings of the IEEE*, November 2001.
- RAMA77** Ramamoorthy, C. "Pipeline Architecture." *Computing Surveys*, March 1977.
- SOHI90** Sohi, G. "Instruction Issue Logic for High-Performance Interruptable, Multiple Functional Unit, Pipelined Computers." *IEEE Transactions on Computers*, March 1990.

## 14.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

|                                                       |                                                   |                                                   |
|-------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| branch prediction<br>condition code<br>delayed branch | flag<br>instruction cycle<br>instruction pipeline | instruction prefetch<br>program status word (PSW) |
|-------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|

### Review Questions

- 14.1** What general roles are performed by processor registers?
- 14.2** What categories of data are commonly supported by user-visible registers?
- 14.3** What is the function of condition codes?
- 14.4** What is a program status word?
- 14.5** Why is a two-stage instruction pipeline unlikely to cut the instruction cycle time in half, compared with the use of no pipeline?
- 14.6** List and briefly explain various ways in which an instruction pipeline can deal with conditional branch instructions.
- 14.7** How are history bits used for branch prediction?

### Problems

- 14.1 a.** If the last operation performed on a computer with an 8-bit word was an addition in which the two operands were 00000010 and 00000011, what would be the value of the following flags?
  - Carry
  - Zero
  - Overflow
  - Sign
  - Even Parity
  - Half-Carry
- b.** Repeat for the addition of  $-1$  (twos complement) and  $+1$ .
- 14.2** Repeat Problem 14.1 for the operation  $A - B$ , where  $A$  contains 11110000 and  $B$  contains 0010100.
- 14.3** A microprocessor is clocked at a rate of 5 GHz.
  - a.** How long is a clock cycle?
  - b.** What is the duration of a particular type of machine instruction consisting of three clock cycles?

- 14.4** A microprocessor provides an instruction capable of moving a string of bytes from one area of memory to another. The fetching and initial decoding of the instruction takes 10 clock cycles. Thereafter, it takes 15 clock cycles to transfer each byte. The microprocessor is clocked at a rate of 10 GHz.
- Determine the length of the instruction cycle for the case of a string of 64 bytes.
  - What is the worst-case delay for acknowledging an interrupt if the instruction is noninterruptible?
  - Repeat part (b) assuming the instruction can be interrupted at the beginning of each byte transfer.
- 14.5** The Intel 8088 consists of a bus interface unit (BIU) and an execution unit (EU), which form a 2-stage pipeline. The BIU fetches instructions into a 4-byte instruction queue. The BIU also participates in address calculations, fetches operands, and writes results in memory as requested by the EU. If no such requests are outstanding and the bus is free, the BIU fills any vacancies in the instruction queue. When the EU completes execution of an instruction, it passes any results to the BIU (destined for memory or I/O) and proceeds to the next instruction.
- Suppose the tasks performed by the BIU and EU take about equal time. By what factor does pipelining improve the performance of the 8088? Ignore the effect of branch instructions.
  - Repeat the calculation assuming that the EU takes twice as long as the BIU.
- 14.6** Assume an 8088 is executing a program in which the probability of a program jump is 0.1. For simplicity, assume that all instructions are 2 bytes long.
- What fraction of instruction fetch bus cycles is wasted?
  - Repeat if the instruction queue is 8 bytes long.
- 14.7** Consider the timing diagram of Figures 14.10. Assume that there is only a two-stage pipeline (fetch, execute). Redraw the diagram to show how many time units are now needed for four instructions.
- 14.8** Assume a pipeline with four stages: fetch instruction (FI), decode instruction and calculate addresses (DA), fetch operand (FO), and execute (EX). Draw a diagram similar to Figure 14.10 for a sequence of 7 instructions, in which the third instruction is a branch that is taken and in which there are no data dependencies.
- 14.9** A pipelined processor has a clock rate of 2.5 GHz and executes a program with 1.5 million instructions. The pipeline has five stages, and instructions are issued at a rate of one per clock cycle. Ignore penalties due to branch instructions and out-of-sequence executions.
- What is the speedup of this processor for this program compared to a nonpipelined processor, making the same assumptions used in Section 14.4?
  - What is throughput (in MIPS) of the pipelined processor?
- 14.10** A nonpipelined processor has a clock rate of 2.5 GHz and an average CPI (cycles per instruction) of 4. An upgrade to the processor introduces a five-stage pipeline. However, due to internal pipeline delays, such as latch delay, the clock rate of the new processor has to be reduced to 2 GHz.
- What is the speedup achieved for a typical program?
  - What is the MIPS rate for each processor?
- 14.11** Consider an instruction sequence of length  $n$  that is streaming through the instruction pipeline. Let  $p$  be the probability of encountering a conditional or unconditional branch instruction, and let  $q$  be the probability that execution of a branch instruction  $I$  causes a jump to a nonconsecutive address. Assume that each such jump requires the pipeline to be cleared, destroying all ongoing instruction processing, when  $I$  emerges from the last stage. Revise Equations (14.1) and (14.2) to take these probabilities into account.
- 14.12** One limitation of the multiple-stream approach to dealing with branches in a pipeline is that additional branches will be encountered before the first branch is resolved. Suggest two additional limitations or drawbacks.
- 14.13** Consider the state diagrams of Figure 14.28.
- Describe the behavior of each.
  - Compare these with the branch prediction state diagram in Section 14.4. Discuss the relative merits of each of the three approaches to branch prediction.

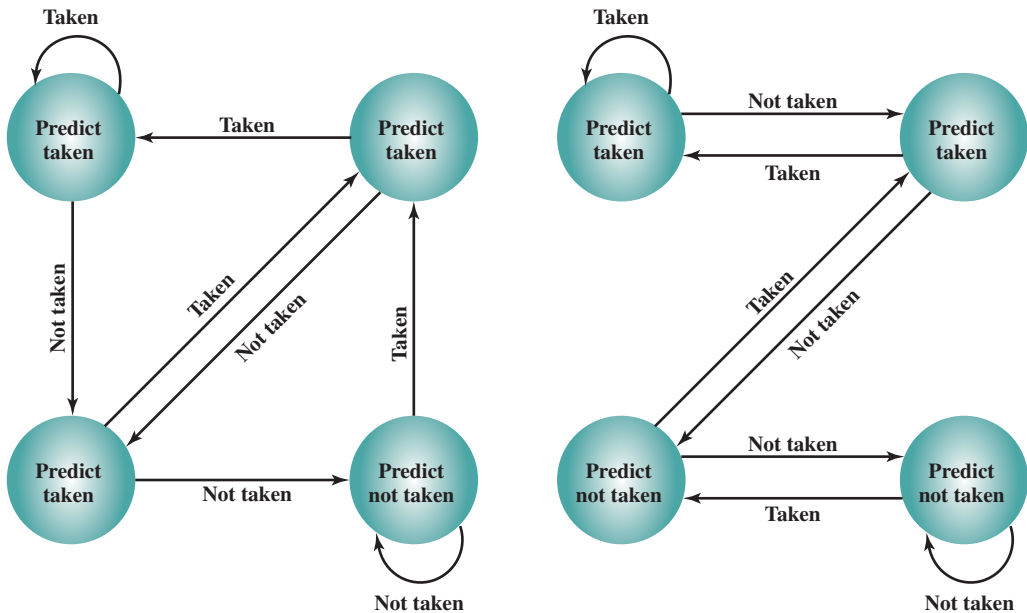


Figure 14.28 Two Branch Prediction State Diagrams

**14.14** The Motorola 680x0 machines include the instruction Decrement and Branch According to Condition, which has the following form:

DBcc Dn, displacement

where cc is one of the testable conditions, Dn is a general-purpose register, and displacement specifies the target address relative to the current address. The instruction can be defined as follows:

**if** (cc = False)

**then begin**

Dn := (Dn) - 1;

**if** Dn  $\neq$  -1 **then** PC := (PC) + displacement **end**

**else** PC := (PC) + 2;

When the instruction is executed, the condition is first tested to determine whether the termination condition for the loop is satisfied. If so, no operation is performed and execution continues at the next instruction in sequence. If the condition is false, the specified data register is decremented and checked to see if it is less than zero. If it is less than zero, the loop is terminated and execution continues at the next instruction in sequence. Otherwise, the program branches to the specified location. Now consider the following assembly-language program fragment:

```
AGAIN CMPM.L (A0)+, (A1)+
 DBNE D1, AGAIN
 NOP
```

Two strings addressed by A0 and A1 are compared for equality; the string pointers are incremented with each reference. D1 initially contains the number of longwords (4 bytes) to be compared.

- The initial contents of the registers are A0 = \$00004000, A1 = \$00005000 and D1 = \$000000FF (the \$ indicates hexadecimal notation). Memory between \$4000 and \$6000 is loaded with words \$AAAA. If the foregoing program is run, specify

**Table 14.5** Branch Behavior in Sample Applications

|                                           |                   |                   |                |
|-------------------------------------------|-------------------|-------------------|----------------|
| <b>Occurrence of branch classes:</b>      |                   |                   |                |
| Type 1: Branch                            | 72.5%             |                   |                |
| Type 2: Loop control                      | 9.8%              |                   |                |
| Type 3: Procedure call, return            | 17.7%             |                   |                |
| <b>Type 1 branch: where it goes</b>       | <b>Scientific</b> | <b>Commercial</b> | <b>Systems</b> |
| Unconditional—100% go to target           | 20%               | 40%               | 35%            |
| Conditional—went to target                | 43.2%             | 24.3%             | 32.5%          |
| Conditional—did not go to target (inline) | 36.8%             | 35.7%             | 32.5%          |
| <b>Type 2 branch (all environments)</b>   |                   |                   |                |
| That go to target                         | 91%               |                   |                |
| That go inline                            | 9%                |                   |                |
| <b>Type 3 branch</b>                      |                   |                   |                |
| 100% go to target                         |                   |                   |                |

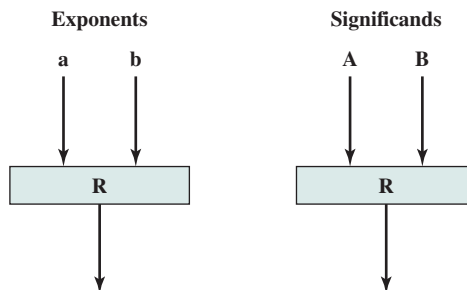
the number of times the DBNE loop is executed and the contents of the three registers when the NOP instruction is reached.

- b. Repeat (a), but now assume that memory between \$4000 and \$4FEE is loaded with \$0000 and between \$5000 and \$6000 is loaded with \$AAA.

**14.15** Redraw Figures 14.19c, assuming that the conditional branch is not taken.

**14.16** Table 14.5 summarizes statistics from [MACD84] concerning branch behavior for various classes of applications. With the exception of type 1 branch behavior, there is no noticeable difference among the application classes. Determine the fraction of all branches that go to the branch target address for the scientific environment. Repeat for commercial and systems environments.

**14.17** Pipelining can be applied within the ALU to speed up floating-point operations. Consider the case of floating-point addition and subtraction. In simplified terms, the pipeline could have four stages: (1) Compare the exponents; (2) Choose the exponent and align the significands; (3) Add or subtract significands; (4) Normalize the results. The pipeline can be considered to have two parallel threads, one handling exponents and one handling significands, and could start out like this:



In this figure, the boxes labeled R refer to a set of registers used to hold temporary results. Complete the block diagram that shows at a top level the structure of the pipeline.

# REDUCED INSTRUCTION SET COMPUTERS

## 15.1 Instruction Execution Characteristics

- Operations
- Operands
- Procedure Calls
- Implications

## 15.2 The Use of a Large Register File

- Register Windows
- Global Variables
- Large Register File versus Cache

## 15.3 Compiler-Based Register Optimization

## 15.4 Reduced Instruction Set Architecture

- Why CISC
- Characteristics of Reduced Instruction Set Architectures
- CISC versus RISC Characteristics

## 15.5 RISC Pipelining

- Pipelining with Regular Instructions
- Optimization of Pipelining

## 15.6 MIPS R4000

- Instruction Set
- Instruction Pipeline

## 15.7 SPARC

- SPARC Register Set
- Instruction Set
- Instruction Format

## 15.8 RISC versus CISC Controversy

## 15.9 Recommended Reading

## 15.10 Key Terms, Review Questions, and Problems

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- ◆ Provide an overview research results on instruction execution characteristics that motivated the development of the RISC approach.
- ◆ Summarize the key characteristics of RISC machines.
- ◆ Understand the design and performance implications of using a large register file.
- ◆ Understand the use of compiler-based register optimization to improve performance.
- ◆ Discuss the implication of a RISC architecture for pipeline design and performance.
- ◆ List and explain key approaches to pipeline optimization on a RISC machine.

Since the development of the stored-program computer around 1950, there have been remarkably few true innovations in the areas of computer organization and architecture. The following are some of the major advances since the birth of the computer:

- **The family concept:** Introduced by IBM with its System/360 in 1964, followed shortly thereafter by DEC, with its PDP-8. The family concept decouples the architecture of a machine from its implementation. A set of computers is offered, with different price/performance characteristics, that presents the same architecture to the user. The differences in price and performance are due to different implementations of the same architecture.
- **Microprogrammed control unit:** Suggested by Wilkes in 1951 and introduced by IBM on the S/360 line in 1964. Microprogramming eases the task of designing and implementing the control unit and provides support for the family concept.
- **Cache memory:** First introduced commercially on IBM S/360 Model 85 in 1968. The insertion of this element into the memory hierarchy dramatically improves performance.
- **Pipelining:** A means of introducing parallelism into the essentially sequential nature of a machine-instruction program. Examples are instruction pipelining and vector processing.
- **Multiple processors:** This category covers a number of different organizations and objectives.
- **Reduced instruction set computer (RISC) architecture:** This is the focus of this chapter.

When it appeared, RISC architecture was a dramatic departure from the historical trend in processor architecture. An analysis of the RISC architecture brings into focus many of the important issues in computer organization and architecture.

Although RISC architectures have been defined and designed in a variety of ways by different groups, the key elements shared by most designs are these:

- A large number of general-purpose registers, and/or the use of compiler technology to optimize register usage
- A limited and simple instruction set
- An emphasis on optimizing the instruction pipeline

Table 15.1 compares several RISC and non-RISC systems.

We begin this chapter with a brief survey of some results on instruction sets, and then examine each of the three topics just listed. This is followed by a description of two of the best-documented RISC designs.

## 15.1 INSTRUCTION EXECUTION CHARACTERISTICS

One of the most visible forms of evolution associated with computers is that of programming languages. As the cost of hardware has dropped, the relative cost of software has risen. Along with that, a chronic shortage of programmers has driven up software costs in absolute terms. Thus, the major cost in the life cycle of a system is software, not hardware. Adding to the cost, and to the inconvenience, is the element of unreliability: it is common for programs, both system and application, to continue to exhibit new bugs after years of operation.

The response from researchers and industry has been to develop ever more powerful and complex high-level programming languages. These **high-level languages (HLLs)**: (1) allow the programmer to express algorithms more concisely, (2) allow the compiler to take care of details that are not important in the programmer's expression of algorithms, and (3) often support naturally the use of structured programming and/or object-oriented design.

Alas, this solution gave rise to a perceived problem, known as the *semantic gap*, the difference between the operations provided in HLLs and those provided in computer architecture. Symptoms of this gap are alleged to include execution inefficiency, excessive machine program size, and compiler complexity. Designers responded with architectures intended to close this gap. Key features include large instruction sets, dozens of addressing modes, and various HLL statements implemented in hardware. An example of the latter is the CASE machine instruction on the VAX. Such complex instruction sets are intended to

- Ease the task of the compiler writer.
- Improve execution efficiency, because complex sequences of operations can be implemented in microcode.
- Provide support for even more complex and sophisticated HLLs.

Meanwhile, a number of studies have been done over the years to determine the characteristics and patterns of execution of machine instructions generated from HLL programs. The results of these studies inspired some researchers to look for a different approach: namely, to make the architecture that supports the HLL simpler, rather than more complex.



**Table 15.1** Characteristics of Some CISCs, RISCs, and Superscalar Processors

|                                                | <b>Complex Instruction Set<br/>(CISC) Computer</b> |               |                | <b>Reduced Instruction Set<br/>(RISC) Computer</b> |               | <b>Superscalar</b> |                |                |
|------------------------------------------------|----------------------------------------------------|---------------|----------------|----------------------------------------------------|---------------|--------------------|----------------|----------------|
| <b>Characteristic</b>                          | IBM<br>370/168                                     | VAX<br>11/780 | Intel<br>80486 | SPARC                                              | MIPS<br>R4000 | PowerPC            | Ultra<br>SPARC | MIPS<br>R10000 |
| <b>Year developed</b>                          | 1973                                               | 1978          | 1989           | 1987                                               | 1991          | 1993               | 1996           | 1996           |
| <b>Number of instructions</b>                  | 208                                                | 303           | 235            | 69                                                 | 94            | 225                | —              | —              |
| <b>Instruction size (bytes)</b>                | 2–6                                                | 2–57          | 1–11           | 4                                                  | 4             | 4                  | 4              | 4              |
| <b>Addressing modes</b>                        | 4                                                  | 22            | 11             | 1                                                  | 1             | 2                  | 1              | 1              |
| <b>Number of general-purpose<br/>registers</b> | 16                                                 | 16            | 8              | 40–520                                             | 32            | 32                 | 40–520         | 32             |
| <b>Control memory<br/>size (Kbits)</b>         | 420                                                | 480           | 246            | —                                                  | —             | —                  | —              | —              |
| <b>Cache size (Kbytes)</b>                     | 64                                                 | 64            | 8              | 32                                                 | 128           | 16–32              | 32             | 64             |

To understand the line of reasoning of the RISC advocates, we begin with a brief review of instruction execution characteristics. The aspects of computation of interest are as follows:

- **Operations performed:** These determine the functions to be performed by the processor and its interaction with memory.
- **Operands used:** The types of operands and the frequency of their use determine the memory organization for storing them and the addressing modes for accessing them.
- **Execution sequencing:** This determines the control and pipeline organization.

In the remainder of this section, we summarize the results of a number of studies of high-level-language programs. All of the results are based on dynamic measurements. That is, measurements are collected by executing the program and counting the number of times some feature has appeared or a particular property has held true. In contrast, static measurements merely perform these counts on the source text of a program. They give no useful information on performance, because they are not weighted relative to the number of times each statement is executed.

## Operations

A variety of studies have been made to analyze the behavior of HLL programs. Table 4.8, discussed in Chapter 4, includes key results from a number of studies. There is quite good agreement in the results of this mixture of languages and applications. Assignment statements predominate, suggesting that the simple movement of data is of high importance. There is also a preponderance of conditional statements (IF, LOOP). These statements are implemented in machine language with some sort of compare and branch instruction. This suggests that the sequence control mechanism of the instruction set is important.

These results are instructive to the machine instruction set designer, indicating which types of statements occur most often and therefore should be supported in an “optimal” fashion. However, these results do not reveal which statements use the most time in the execution of a typical program. That is, we want to answer the question: Given a compiled machine-language program, which statements in the source language cause the execution of the most machine-language instructions and what is the execution time of these instructions?

To get at this underlying phenomenon, the Patterson programs [PATT82a], described in Appendix 4A, were compiled on the VAX, PDP-11, and Motorola 68000 to determine the average number of machine instructions and memory references per statement type. The second and third columns in Table 15.2 show the relative frequency of occurrence of various HLL statements in a variety of programs; the data were obtained by observing the occurrences in running programs rather than just the number of times that statements occur in the source code. Hence these metrics capture dynamic behavior. To obtain the data in columns four and five (machine-instruction weighted), each value in the second and third columns is multiplied by the number of machine instructions produced by the compiler. These results are then normalized so that columns four and five show the relative frequency of occurrence, weighted by the number of machine instructions per HLL

**Table 15.2** Weighted Relative Dynamic Frequency of HLL Operations [PATT82a]

|        | Dynamic Occurrence |     | Machine-Instruction Weighted |     | Memory-Reference Weighted |     |
|--------|--------------------|-----|------------------------------|-----|---------------------------|-----|
|        | Pascal             | C   | Pascal                       | C   | Pascal                    | C   |
| ASSIGN | 45%                | 38% | 13%                          | 13% | 14%                       | 15% |
| LOOP   | 5%                 | 3%  | 42%                          | 32% | 33%                       | 26% |
| CALL   | 15%                | 12% | 31%                          | 33% | 44%                       | 45% |
| IF     | 29%                | 43% | 11%                          | 21% | 7%                        | 13% |
| GOTO   | —                  | 3%  | —                            | —   | —                         | —   |
| OTHER  | 6%                 | 1%  | 3%                           | 1%  | 2%                        | 1%  |

statement. Similarly, the sixth and seventh columns are obtained by multiplying the frequency of occurrence of each statement type by the relative number of memory references caused by each statement. The data in columns four through seven provide surrogate measures of the actual time spent executing the various statement types. The results suggest that the procedure call/return is the most time-consuming operation in typical HLL programs.

The reader should be clear on the significance of Table 15.2. This table indicates the relative performance impact of various statement types in an HLL, when that HLL is compiled for a typical contemporary instruction set architecture. Some other architecture could conceivably produce different results. However, this study produces results that are representative for contemporary **complex instruction set computer (CISC)** architectures. Thus, they can provide guidance to those looking for more efficient ways to support HLLs.

## Operands

Much less work has been done on the occurrence of types of operands, despite the importance of this topic. There are several aspects that are significant.

The Patterson study already referenced [PATT82a] also looked at the dynamic frequency of occurrence of classes of variables (Table 15.3). The results, consistent between Pascal and C programs, show that most references are to simple scalar variables. Further, more than 80% of the scalars were local (to the procedure) variables. In addition, each reference to an array or a structure requires a reference to an index or pointer, which again is usually a local scalar. Thus, there is a preponderance of references to scalars, and these are highly localized.

The Patterson study examined the dynamic behavior of HLL programs, independent of the underlying architecture. As discussed before, it is necessary

**Table 15.3** Dynamic Percentage of Operands

|                  | Pascal | C   | Average |
|------------------|--------|-----|---------|
| Integer Constant | 16%    | 23% | 20%     |
| Scalar Variable  | 58%    | 53% | 55%     |
| Array/Structure  | 26%    | 24% | 25%     |

to deal with actual architectures to examine program behavior more deeply. One study, [LUND77], examined DEC-10 instructions dynamically and found that each instruction on the average references 0.5 operand in memory and 1.4 registers. Similar results are reported in [HUCK83] for C, Pascal, and FORTRAN programs on S/370, PDP-11, and VAX. Of course, these figures depend highly on both the architecture and the compiler, but they do illustrate the frequency of operand accessing.

These latter studies suggest the importance of an architecture that lends itself to fast operand accessing, because this operation is performed so frequently. The Patterson study suggests that a prime candidate for optimization is the mechanism for storing and accessing local scalar variables.

### Procedure Calls

We have seen that procedure calls and returns are an important aspect of HLL programs. The evidence (Table 15.2) suggests that these are the most time-consuming operations in compiled HLL programs. Thus, it will be profitable to consider ways of implementing these operations efficiently. Two aspects are significant: the number of parameters and variables that a procedure deals with, and the depth of nesting.

Tanenbaum's study [TANE78] found that 98% of dynamically called procedures were passed fewer than six arguments and that 92% of them used fewer than six local scalar variables. Similar results were reported by the Berkeley RISC team [KATE83], as shown in Table 15.4. These results show that the number of words required per procedure activation is not large. The studies reported earlier indicated that a high proportion of operand references is to local scalar variables. These studies show that those references are in fact confined to relatively few variables.

The same Berkeley group also looked at the pattern of procedure calls and returns in HLL programs. They found that it is rare to have a long uninterrupted sequence of procedure calls followed by the corresponding sequence of returns. Rather, they found that a program remains confined to a rather narrow window of procedure-invocation depth. This is illustrated in Figure 4.21, which was discussed in Chapter 4. These results reinforce the conclusion that operand references are highly localized.

### Implications

A number of groups have looked at results such as those just reported and have concluded that the attempt to make the instruction set architecture close to HLLs

**Table 15.4** Procedure Arguments and Local Scalar Variables

| Percentage of Executed Procedure Calls With | Compiler, Interpreter, and Typesetter | Small Nonnumeric Programs |
|---------------------------------------------|---------------------------------------|---------------------------|
| >3 arguments                                | 0–7%                                  | 0–5%                      |
| >5 arguments                                | 0–3%                                  | 0%                        |
| >8 words of arguments and local scalars     | 1–20%                                 | 0–6%                      |
| >12 words of arguments and local scalars    | 1–6%                                  | 0–3%                      |

is not the most effective design strategy. Rather, the HLLs can best be supported by optimizing performance of the most time-consuming features of typical HLL programs.

Generalizing from the work of a number of researchers, three elements emerge that, by and large, characterize RISC architectures. First, use a large number of registers or use a compiler to optimize register usage. This is intended to optimize operand referencing. The studies just discussed show that there are several references per HLL statement and that there is a high proportion of move (assignment) statements. This, coupled with the locality and predominance of scalar references, suggests that performance can be improved by reducing memory references at the expense of more register references. Because of the locality of these references, an expanded register set seems practical.

Second, careful attention needs to be paid to the design of instruction pipelines. Because of the high proportion of conditional branch and procedure call instructions, a straightforward instruction pipeline will be inefficient. This manifests itself as a high proportion of instructions that are prefetched but never executed.

Finally, an instruction set consisting of high-performance primitives is indicated. Instructions should have predictable costs (measured in execution time and code size, and increasingly, in energy dissipation) and be consistent with a high-performance implementation (which harmonizes with predictable execution-time cost).

## 15.2 THE USE OF A LARGE REGISTER FILE

The results summarized in Section 15.1 point out the desirability of quick access to operands. We have seen that there is a large proportion of assignment statements in HLL programs, and many of these are of the simple form  $A \leftarrow B$ . Also, there is a significant number of operand accesses per HLL statement. If we couple these results with the fact that most accesses are to local scalars, heavy reliance on register storage is suggested.

The reason that register storage is indicated is that it is the fastest available storage device, faster than both main memory and cache. The register file is physically small, on the same chip as the ALU and control unit, and employs much shorter addresses than addresses for cache and memory. Thus, a strategy is needed that will allow the most frequently accessed operands to be kept in registers and to minimize register-memory operations.

Two basic approaches are possible, one based on software and the other on hardware. The software approach is to rely on the compiler to maximize register usage. The compiler will attempt to assign registers to those variables that will be used the most in a given time period. This approach requires the use of sophisticated program-analysis algorithms. The hardware approach is simply to use more registers so that more variables can be held in registers for longer periods of time.

In this section, we will discuss the hardware approach. This approach has been pioneered by the Berkeley RISC group [PATT82a]; was used in the first commercial RISC product, the Pyramid [RAGA83]; and is currently used in the popular **SPARC** architecture.

## Register Windows

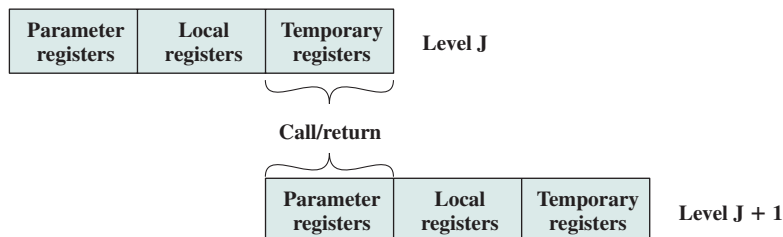
On the face of it, the use of a large set of registers should decrease the need to access memory. The design task is to organize the registers in such a fashion that this goal is realized.

Because most operand references are to local scalars, the obvious approach is to store these in registers, with perhaps a few registers reserved for global variables. The problem is that the definition of *local* changes with each procedure call and return, operations that occur frequently. On every call, local variables must be saved from the registers into memory, so that the registers can be reused by the called procedure. Furthermore, parameters must be passed. On return, the variables of the calling procedure must be restored (loaded back into registers) and results must be passed back to the calling procedure.

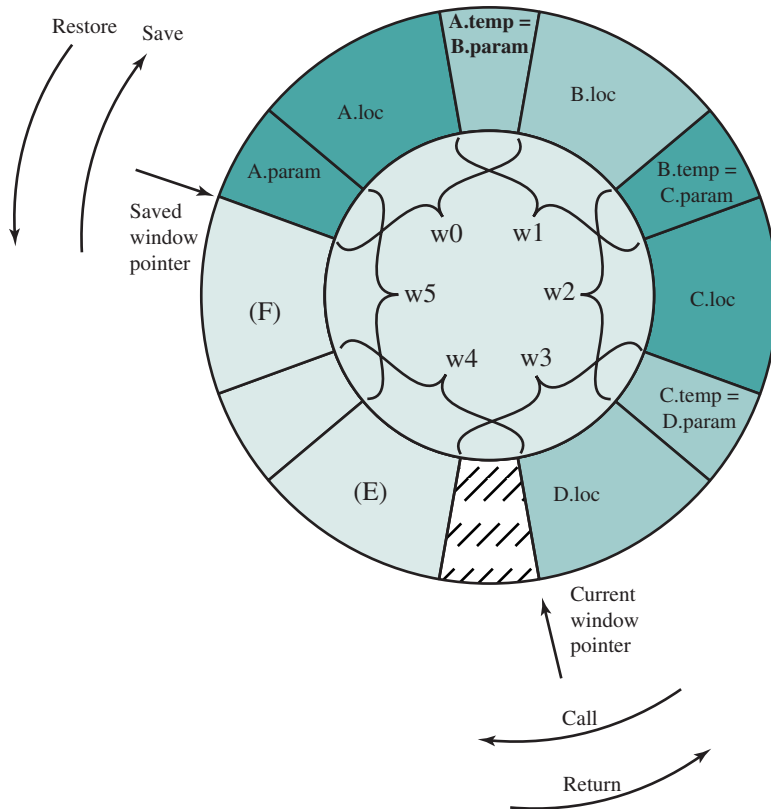
The solution is based on two other results reported in Section 15.1. First, a typical procedure employs only a few passed parameters and local variables (Table 15.4). Second, the depth of procedure activation fluctuates within a relatively narrow range (Figure 4.21). To exploit these properties, multiple small sets of registers are used, each assigned to a different procedure. A procedure call automatically switches the processor to use a different fixed-size window of registers, rather than saving registers in memory. Windows for adjacent procedures are overlapped to allow parameter passing.

The concept is illustrated in Figure 15.1. At any time, only one window of registers is visible and is addressable as if it were the only set of registers (e.g., addresses 0 through  $N - 1$ ). The window is divided into three fixed-size areas. Parameter registers hold parameters passed down from the procedure that called the current procedure and hold results to be passed back up. Local registers are used for local variables, as assigned by the compiler. Temporary registers are used to exchange parameters and results with the next lower level (procedure called by current procedure). The temporary registers at one level are physically the same as the parameter registers at the next lower level. This overlap permits parameters to be passed without the actual movement of data. Keep in mind that, except for the overlap, the registers at two different levels are physically distinct. That is, the parameter and local registers at level  $J$  are disjoint from the local and temporary registers at level  $J + 1$ .

To handle any possible pattern of calls and returns, the number of **register windows** would have to be unbounded. Instead, the register windows can be used to hold the few most recent procedure activations. Older activations must be saved



**Figure 15.1** Overlapping Register Windows



**Figure 15.2** Circular-Buffer Organization of Overlapped Windows

in memory and later restored when the nesting depth decreases. Thus, the actual organization of the register file is as a circular buffer of overlapping windows. Two notable examples of this approach are Sun's SPARC architecture, described in Section 15.7, and the IA-64 architecture used in Intel's Itanium processor.

The circular organization is shown in Figure 15.2, which depicts a circular buffer of six windows. The buffer is filled to a depth of 4 (A called B; B called C; C called D) with procedure D active. The current-window pointer (CWP) points to the window of the currently active procedure. Register references by a machine instruction are offset by this pointer to determine the actual physical register. The saved-window pointer (SWP) identifies the window most recently saved in memory. If procedure D now calls procedure E, arguments for E are placed in D's temporary registers (the overlap between w3 and w4) and the CWP is advanced by one window.

If procedure E then makes a call to procedure F, the call cannot be made with the current status of the buffer. This is because F's window overlaps A's window. If F begins to load its temporary registers, preparatory to a call, it will overwrite the parameter registers of A (A.in). Thus, when CWP is incremented (modulo 6) so that it becomes equal to SWP, an interrupt occurs, and A's window is saved. Only the first two portions (A.in and A.loc) need be saved. Then, the SWP is incremented

and the call to F proceeds. A similar interrupt can occur on returns. For example, subsequent to the activation of F, when B returns to A, CWP is decremented and becomes equal to SWP. This causes an interrupt that results in the restoration of A's window.

From the preceding, it can be seen that an  $N$ -window register file can hold only  $N - 1$  procedure activations. The value of  $N$  need not be large. As was mentioned in Appendix 4A, one study [TAMI83] found that, with 8 windows, a save or restore is needed on only 1% of the calls or returns. The Berkeley RISC computers use 8 windows of 16 registers each. The Pyramid computer employs 16 windows of 32 registers each.

## Global Variables

The window scheme just described provides an efficient organization for storing local scalar variables in registers. However, this scheme does not address the need to store global variables, those accessed by more than one procedure. Two options suggest themselves. First, variables declared as global in an HLL can be assigned memory locations by the compiler, and all machine instructions that reference these variables will use memory-reference operands. This is straightforward, from both the hardware and software (compiler) points of view. However, for frequently accessed global variables, this scheme is inefficient.

An alternative is to incorporate a set of global registers in the processor. These registers would be fixed in number and available to all procedures. A unified numbering scheme can be used to simplify the instruction format. For example, references to registers 0 through 7 could refer to unique global registers, and references to registers 8 through 31 could be offset to refer to physical registers in the current window. There is an increased hardware burden to accommodate the split in register addressing. In addition, the linker must decide which global variables should be assigned to registers.

## Large Register File versus Cache

The register file, organized into windows, acts as a small, fast buffer for holding a subset of all variables that are likely to be used the most heavily. From this point of view, the register file acts much like a cache memory, although a much faster memory. The question therefore arises as to whether it would be simpler and better to use a cache and a small traditional register file.

Table 15.5 compares characteristics of the two approaches. The window-based register file holds all the local scalar variables (except in the rare case of window overflow) of the most recent  $N - 1$  procedure activations. The cache holds a selection of recently used scalar variables. The register file should save time, because all local scalar variables are retained. On the other hand, the cache may make more efficient use of space, because it is reacting to the situation dynamically. Furthermore, caches generally treat all memory references alike, including instructions and other types of data. Thus, savings in these other areas are possible with a cache and not a register file.



**Table 15.5** Characteristics of Large-Register-File and Cache Organizations

| Large Register File                                   | Cache                                             |
|-------------------------------------------------------|---------------------------------------------------|
| All local scalars                                     | Recently-used local scalars                       |
| Individual variables                                  | Blocks of memory                                  |
| Compiler-assigned global variables                    | Recently-used global variables                    |
| Save/Restore based on procedure nesting depth         | Save/Restore based on cache replacement algorithm |
| Register addressing                                   | Memory addressing                                 |
| Multiple operands addressed and accessed in one cycle | One operand addressed and accessed per cycle      |

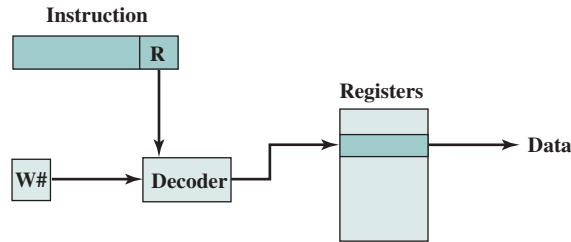
A register file may make inefficient use of space, because not all procedures will need the full window space allotted to them. On the other hand, the cache suffers from another sort of inefficiency: Data are read into the cache in blocks. Whereas the register file contains only those variables in use, the cache reads in a block of data, some or much of which will not be used.

The cache is capable of handling global as well as local variables. There are usually many global scalars, but only a few of them are heavily used [KATE83]. A cache will dynamically discover these variables and hold them. If the window-based register file is supplemented with global registers, it too can hold some global scalars. However, when program modules are separately compiled, it is impossible for the compiler to assign global values to registers; the linker must perform this task.

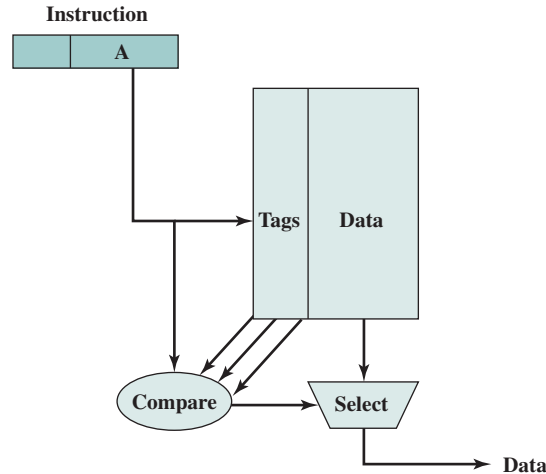
With the register file, the movement of data between registers and memory is determined by the procedure nesting depth. Because this depth usually fluctuates within a narrow range, the use of memory is relatively infrequent. Most cache memories are set associative with a small set size. Thus, there is the danger that other data or instructions will compete for cache residency.

Based on the discussion so far, the choice between a large window-based register file and a cache is not clear-cut. There is one characteristic, however, in which the register approach is clearly superior and which suggests that a cache-based system will be noticeably slower. This distinction shows up in the amount of addressing overhead experienced by the two approaches.

Figure 15.3 illustrates the difference. To reference a local scalar in a window-based register file, a “virtual” register number and a window number are used. These can pass through a relatively simple decoder to select one of the physical registers. To reference a memory location in cache, a full-width memory address must be generated. The complexity of this operation depends on the addressing mode. In a set associative cache, a portion of the address is used to read a number of words and tags equal to the set size. Another portion of the address is compared with the tags, and one of the words that were read is selected. It should be clear that even if the cache is as fast as the register file, the access time will be considerably longer. Thus, from the point of view of performance, the window-based register file is superior for local scalars. Further performance improvement could be achieved by the addition of a cache for instructions only.



(a) Windows-based register file



(b) Cache

**Figure 15.3** Referencing a Scalar

## 15.3 COMPILER-BASED REGISTER OPTIMIZATION

Let us assume now that only a small number (e.g., 16–32) of registers is available on the target RISC machine. In this case, optimized register usage is the responsibility of the compiler. A program written in a high-level language has, of course, no explicit references to registers (the C-language keyword `register` notwithstanding). Rather, program quantities are referred to symbolically. The objective of the compiler is to keep the operands for as many computations as possible in registers rather than main memory, and to minimize load-and-store operations.

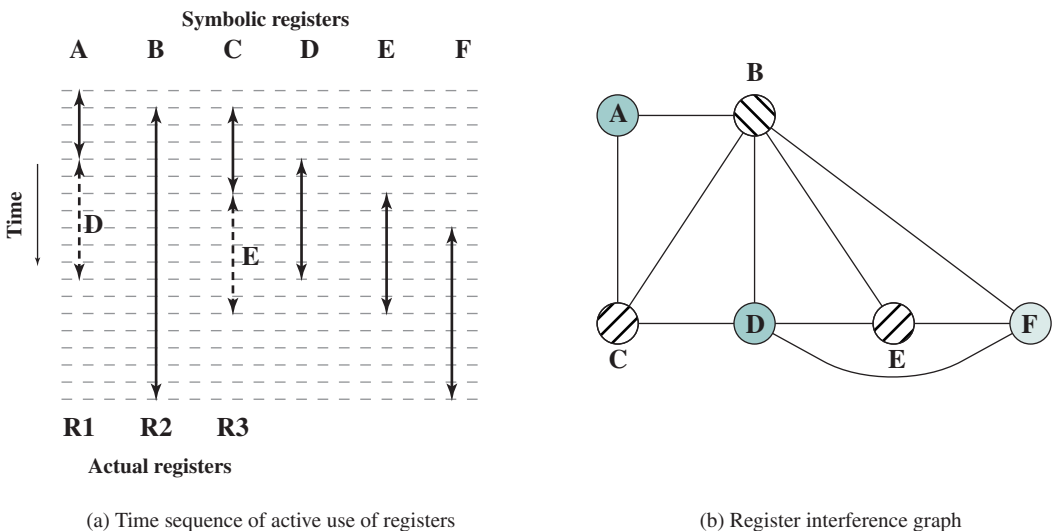
In general, the approach taken is as follows. Each program quantity that is a candidate for residing in a register is assigned to a symbolic or virtual register. The compiler then maps the unlimited number of symbolic registers into a fixed number of real registers. Symbolic registers whose usage does not overlap can share the same real register. If, in a particular portion of the program, there are more quantities to deal with than real registers, then some of the quantities are assigned to memory locations. Load-and-store instructions are used to position quantities in registers temporarily for computational operations.

The essence of the optimization task is to decide which quantities are to be assigned to registers at any given point in the program. The technique most commonly used in RISC compilers is known as graph coloring, which is a technique borrowed from the discipline of topology [CHAI82, CHOW86, COUT86, CHOW90].

The graph coloring problem is this. Given a graph consisting of nodes and edges, assign colors to nodes such that adjacent nodes have different colors, and do this in such a way as to minimize the number of different colors. This problem is adapted to the compiler problem in the following way. First, the program is analyzed to build a register interference graph. The nodes of the graph are the symbolic registers. If two symbolic registers are “live” during the same program fragment, then they are joined by an edge to depict interference. An attempt is then made to color the graph with  $n$  colors, where  $n$  is the number of registers. Nodes that share the same color can be assigned to the same register. If this process does not fully succeed, then those nodes that cannot be colored must be placed in memory, and loads and stores must be used to make space for the affected quantities when they are needed.

Figure 15.4 is a simple example of the process. Assume a program with six symbolic registers to be compiled into three actual registers. Figure 15.4a shows the time sequence of active use of each symbolic register. The dashed horizontal lines indicate successive instruction executions. Figure 15.4b shows the register interference graph (shading and cross-hatching are used instead of colors). A possible coloring with three colors is indicated. Because symbolic registers A and D do not interfere, the compile can assign both of these to physical register R1. Similarly, symbolic registers C and E can be assigned to register R3. One symbolic register, F, is left uncolored and must be dealt with using loads and stores.

In general, there is a trade-off between the use of a large set of registers and compiler-based register optimization. For example, [BRAD91a] reports on a study



**Figure 15.4** Graph Coloring Approach

that modeled a RISC architecture with features similar to the Motorola 88000 and the MIPS R2000. The researchers varied the number of registers from 16 to 128, and they considered both the use of all general-purpose registers and registers split between integer and floating-point use. Their study showed that with even simple register optimization, there is little benefit to the use of more than 64 registers. With reasonably sophisticated register optimization techniques, there is only marginal performance improvement with more than 32 registers. Finally, they noted that with a small number of registers (e.g., 16), a machine with a shared register organization executes faster than one with a split organization. Similar conclusions can be drawn from [HUGU91], which reports on a study that is primarily concerned with optimizing the use of a small number of registers rather than comparing the use of large register sets with optimization efforts.

## 15.4 REDUCED INSTRUCTION SET ARCHITECTURE

In this section, we look at some of the general characteristics of and the motivation for a reduced instruction set architecture. Specific examples will be seen later in this chapter. We begin with a discussion of motivations for contemporary complex instruction set architectures.

### Why CISC

We have noted the trend to richer instruction sets, which include a larger number of instructions and more complex instructions. Two principal reasons have motivated this trend: a desire to simplify compilers and a desire to improve performance. Underlying both of these reasons was the shift to HLLs on the part of programmers; architects attempted to design machines that provided better support for HLLs.

It is not the intent of this chapter to say that the CISC designers took the wrong direction. Indeed, because technology continues to evolve and because architectures exist along a spectrum rather than in two neat categories, a black-and-white assessment is unlikely ever to emerge. Thus, the comments that follow are simply meant to point out some of the potential pitfalls in the CISC approach and to provide some understanding of the motivation of the RISC adherents.

The first of the reasons cited, compiler simplification, seems obvious, but it is not. The task of the compiler writer is to build a compiler that generates good (fast, small, fast and small) sequences of machine instructions for HLL programs (i.e., the compiler views individual HLL statements in the context of surrounding HLL statements). If there are machine instructions that resemble HLL statements, this task is simplified. This reasoning has been disputed by the RISC researchers ([HENN82], [RADI83], [PATT82b]). They have found that complex machine instructions are often hard to exploit because the compiler must find those cases that exactly fit the construct. The task of optimizing the generated code to minimize code size, reduce instruction execution count, and enhance pipelining is much more difficult with a complex instruction set. As evidence of this, studies cited earlier in this chapter indicate that most of the instructions in a compiled program are the relatively simple ones.

The other major reason cited is the expectation that a CISC will yield smaller, faster programs. Let us examine both aspects of this assertion: that programs will be smaller and that they will execute faster.

There are two advantages to smaller programs. First, because the program takes up less memory, there is a savings in that resource. With memory today being so inexpensive, this potential advantage is no longer compelling. More important, smaller programs should improve performance, and this will happen in three ways. First, fewer instructions means fewer instruction bytes to be fetched. Second, in a paging environment, smaller programs occupy fewer pages, reducing page faults. Third, more instructions fit in cache(s).

The problem with this line of reasoning is that it is far from certain that a CISC program will be smaller than a corresponding RISC program. In many cases, the CISC program, expressed in symbolic machine language, may be *shorter* (i.e., fewer instructions), but the number of bits of memory occupied may not be noticeably *smaller*. Table 15.6 shows results from three studies that compared the size of compiled C programs on a variety of machines, including RISC I, which has a reduced instruction set architecture. Note that there is little or no savings using a CISC over a RISC. It is also interesting to note that the VAX, which has a much more complex instruction set than the PDP-11, achieves very little savings over the latter. These results were confirmed by IBM researchers [RADI83], who found that the IBM 801 (a RISC) produced code that was 0.9 times the size of code on an IBM S/370. The study used a set of PL/I programs.

There are several reasons for these rather surprising results. We have already noted that compilers on CISCs tend to favor simpler instructions, so that the conciseness of the complex instructions seldom comes into play. Also, because there are more instructions on a CISC, longer opcodes are required, producing longer instructions. Finally, RISCs tend to emphasize register rather than memory references, and the former require fewer bits. An example of this last effect is discussed presently.

So the expectation that a CISC will produce smaller programs, with the attendant advantages, may not be realized. The second motivating factor for increasingly complex instruction sets was that instruction execution would be faster. It seems to make sense that a complex HLL operation will execute more quickly as a single machine instruction rather than as a series of more primitive instructions. However, because of the bias toward the use of those simpler instructions, this may not be so.

**Table 15.6** Code Size Relative to RISC I

|            | [PATT82a]<br>11 C Programs | [KATE83]<br>12 C Programs | [HEAT84]<br>5 C Programs |
|------------|----------------------------|---------------------------|--------------------------|
| RISC I     | 1.0                        | 1.0                       | 1.0                      |
| VAX-11/780 | 0.8                        | 0.67                      |                          |
| M68000     | 0.9                        |                           | 0.9                      |
| Z8002      | 1.2                        |                           | 1.12                     |
| PDP-11/70  | 0.9                        | 0.71                      |                          |

The entire control unit must be made more complex, and/or the microprogram control store must be made larger, to accommodate a richer instruction set. Either factor increases the execution time of the simple instructions.

In fact, some researchers have found that the speedup in the execution of complex functions is due not so much to the power of the complex machine instructions as to their residence in high-speed control store [RADI83]. In effect, the control store acts as an instruction cache. Thus, the hardware architect is in the position of trying to determine which subroutines or functions will be used most frequently and assigning those to the control store by implementing them in microcode. The results have been less than encouraging. On S/390 systems, instructions such as Translate and Extended-Precision-Floating-Point-Divide reside in high-speed storage, while the sequence involved in setting up procedure calls or initiating an interrupt handler are in slower main memory.

Thus, it is far from clear that a trend to increasingly complex instruction sets is appropriate. This has led a number of groups to pursue the opposite path.

### Characteristics of Reduced Instruction Set Architectures

Although a variety of different approaches to reduced instruction set architecture have been taken, certain characteristics are common to all of them:

- One instruction per cycle
- Register-to-register operations
- Simple addressing modes
- Simple instruction formats

Here, we provide a brief discussion of these characteristics. Specific examples are explored later in this chapter.

The first characteristic listed is that there is **one machine instruction per machine cycle**. A *machine cycle* is defined to be the time it takes to fetch two operands from registers, perform an ALU operation, and store the result in a register. Thus, RISC machine instructions should be no more complicated than, and execute about as fast as, microinstructions on CISC machines (discussed in Part Four). With simple, one-cycle instructions, there is little or no need for microcode; the machine instructions can be hardwired. Such instructions should execute faster than comparable machine instructions on other machines, because it is not necessary to access a microprogram control store during instruction execution.

A second characteristic is that most operations should be **register to register**, with only simple LOAD and STORE operations accessing memory. This design feature simplifies the instruction set and therefore the control unit. For example, a RISC instruction set may include only one or two ADD instructions (e.g., integer add, add with carry); the VAX has 25 different ADD instructions. Another benefit is that such an architecture encourages the optimization of register use, so that frequently accessed operands remain in high-speed storage.

This emphasis on register-to-register operations is notable for RISC designs. Contemporary CISC machines provide such instructions but also include memory-to-memory and mixed register/memory operations. Attempts to compare these

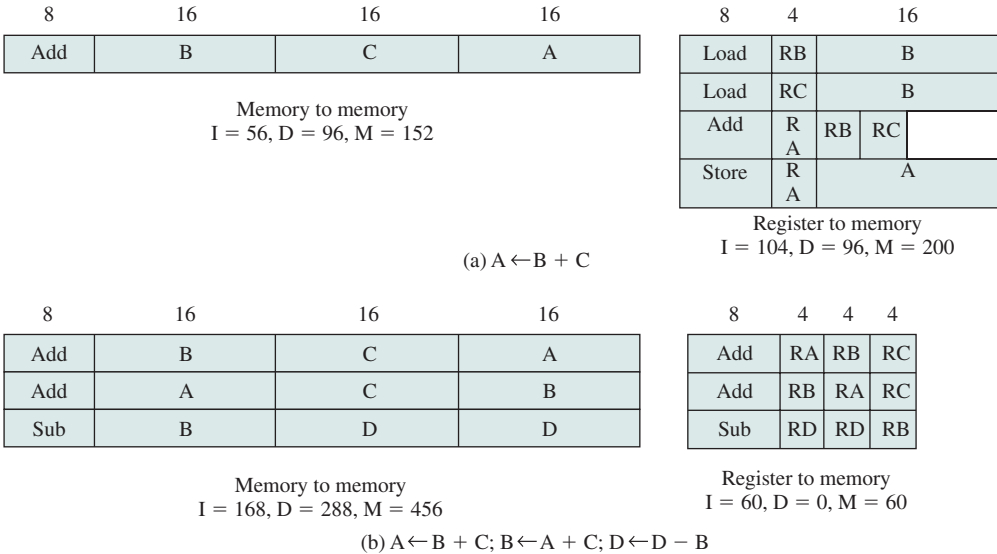
approaches were made in the 1970s, before the appearance of RISCs. Figure 15.5a illustrates the approach taken. Hypothetical architectures were evaluated on program size and the number of bits of memory traffic. Results such as this one led one researcher to suggest that future architectures should contain no registers at all [MYER78]. One wonders what he would have thought, at the time, of the RISC machine once produced by Pyramid, which contained no less than 528 registers!

What was missing from those studies was a recognition of the frequent access to a small number of local scalars and that, with a large bank of registers or an optimizing compiler, most operands could be kept in registers for long periods of time. Thus, Figure 15.5b may be a fairer comparison.

A third characteristic is the use of **simple addressing modes**. Almost all RISC instructions use simple register addressing. Several additional modes, such as displacement and PC-relative, may be included. Other, more complex modes can be synthesized in software from the simple ones. Again, this design feature simplifies the instruction set and the control unit.

A final common characteristic is the use of **simple instruction formats**. Generally, only one or a few formats are used. Instruction length is fixed and aligned on word boundaries. Field locations, especially the opcode, are fixed. This design feature has a number of benefits. With fixed fields, opcode decoding and register operand accessing can occur simultaneously. Simplified formats simplify the control unit. Instruction fetching is optimized because word-length units are fetched. Alignment on a word boundary also means that a single instruction does not cross page boundaries.

Taken together, these characteristics can be assessed to determine the potential performance benefits of the RISC approach. A certain amount of “circumstantial



$I$  = number of bytes occupied by executed instructions  
 $D$  = number of bytes occupied by data  
 $M$  = total memory traffic =  $I + D$

**Figure 15.5** Two Comparisons of Register-to-Register and Memory-to-Memory Approaches

evidence” can be presented. First, more effective optimizing compilers can be developed. With more-primitive instructions, there are more opportunities for moving functions out of loops, reorganizing code for efficiency, maximizing register utilization, and so forth. It is even possible to compute parts of complex instructions at compile time. For example, the S/390 Move Characters (MVC) instruction moves a string of characters from one location to another. Each time it is executed, the move will depend on the length of the string, whether and in which direction the locations overlap, and what the alignment characteristics are. In most cases, these will all be known at compile time. Thus, the compiler could produce an optimized sequence of primitive instructions for this function.

A second point, already noted, is that most instructions generated by a compiler are relatively simple anyway. It would seem reasonable that a control unit built specifically for those instructions and using little or no microcode could execute them faster than a comparable CISC.

A third point relates to the use of instruction pipelining. RISC researchers feel that the instruction pipelining technique can be applied much more effectively with a reduced instruction set. We examine this point in some detail presently.

A final, and somewhat less significant, point is that RISC processors are more responsive to interrupts because interrupts are checked between rather elementary operations. Architectures with complex instructions either restrict interrupts to instruction boundaries or must define specific interruptible points and implement mechanisms for restarting an instruction.

The case for improved performance for a reduced instruction set architecture is strong, but one could perhaps still make an argument for CISC. A number of studies have been done but not on machines of comparable technology and power. Further, most studies have not attempted to separate the effects of a reduced instruction set and the effects of a large register file. The “circumstantial evidence,” however, is suggestive.

### CISC versus RISC Characteristics

After the initial enthusiasm for RISC machines, there has been a growing realization that (1) RISC designs may benefit from the inclusion of some CISC features and that (2) CISC designs may benefit from the inclusion of some RISC features. The result is that the more recent RISC designs, notably the PowerPC, are no longer “pure” RISC and the more recent CISC designs, notably the Pentium II and later Pentium models, do incorporate some RISC characteristics.

An interesting comparison in [MASH95] provides some insight into this issue. Table 15.7 lists a number of processors and compares them across a number of characteristics. For purposes of this comparison, the following are considered typical of a classic RISC:

1. A single instruction size.
2. That size is typically 4 bytes.
3. A small number of data addressing modes, typically less than five. This parameter is difficult to pin down. In the table, register and literal modes are not counted and different formats with different offset sizes are counted separately.



**Table 15.7** Characteristics of Some Processors

| Processor   | Number of instruction sizes | Max instruction size in bytes | Number of addressing modes | Indirect addressing | Load/store combined with arithmetic | Max number of memory operands | Unaligned addressing allowed | Max number of MMU uses | Number of bits for integer register specifier | Number of bits for FP register specifier |
|-------------|-----------------------------|-------------------------------|----------------------------|---------------------|-------------------------------------|-------------------------------|------------------------------|------------------------|-----------------------------------------------|------------------------------------------|
| AMD29000    | 1                           | 4                             | 1                          | no                  | no                                  | 1                             | no                           | 1                      | 8                                             | 3 <sup>a</sup>                           |
| MIPS R2000  | 1                           | 4                             | 1                          | no                  | no                                  | 1                             | no                           | 1                      | 5                                             | 4                                        |
| SPARC       | 1                           | 4                             | 2                          | no                  | no                                  | 1                             | no                           | 1                      | 5                                             | 4                                        |
| MC88000     | 1                           | 4                             | 3                          | no                  | no                                  | 1                             | no                           | 1                      | 5                                             | 4                                        |
| HP PA       | 1                           | 4                             | 10 <sup>a</sup>            | no                  | no                                  | 1                             | no                           | 1                      | 5                                             | 4                                        |
| IBM RT/PC   | 2 <sup>a</sup>              | 4                             | 1                          | no                  | no                                  | 1                             | no                           | 1                      | 4 <sup>a</sup>                                | 3 <sup>a</sup>                           |
| IBM RS/6000 | 1                           | 4                             | 4                          | no                  | no                                  | 1                             | yes                          | 1                      | 5                                             | 5                                        |
| Intel i860  | 1                           | 4                             | 4                          | no                  | no                                  | 1                             | no                           | 1                      | 5                                             | 4                                        |
| IBM 3090    | 4                           | 8                             | 2 <sup>b</sup>             | no <sup>b</sup>     | yes                                 | 2                             | yes                          | 4                      | 4                                             | 2                                        |
| Intel 80486 | 12                          | 12                            | 15                         | no <sup>b</sup>     | yes                                 | 2                             | yes                          | 4                      | 3                                             | 3                                        |
| NSC 32016   | 21                          | 21                            | 23                         | yes                 | yes                                 | 2                             | yes                          | 4                      | 3                                             | 3                                        |
| MC68040     | 11                          | 22                            | 44                         | yes                 | yes                                 | 2                             | yes                          | 8                      | 4                                             | 3                                        |
| VAX         | 56                          | 56                            | 22                         | yes                 | yes                                 | 6                             | yes                          | 24                     | 4                                             | 0                                        |
| Clipper     | 4 <sup>a</sup>              | 8 <sup>a</sup>                | 9 <sup>a</sup>             | no                  | no                                  | 1                             | 0                            | 2                      | 4 <sup>a</sup>                                | 3 <sup>a</sup>                           |
| Intel 80960 | 2 <sup>a</sup>              | 8 <sup>a</sup>                | 9 <sup>a</sup>             | no                  | no                                  | 1                             | yes <sup>a</sup>             | —                      | 5                                             | 3 <sup>a</sup>                           |

Notes: <sup>a</sup>RISC that does not conform to this characteristic.

<sup>b</sup>CISC that does not conform to this characteristic.

4. No indirect addressing that requires you to make one memory access to get the address of another operand in memory.
5. No operations that combine load/store with arithmetic (e.g., add from memory, add to memory).
6. No more than one memory-addressed operand per instruction.
7. Does not support arbitrary alignment of data for load/store operations.
8. Maximum number of uses of the memory management unit (MMU) for a data address in an instruction.
9. Number of bits for integer register specifier equal to five or more. This means that at least 32 integer registers can be explicitly referenced at a time.
10. Number of bits for floating-point register specifier equal to four or more. This means that at least 16 floating-point registers can be explicitly referenced at a time.

Items 1 through 3 are an indication of instruction decode complexity. Items 4 through 8 suggest the ease or difficulty of pipelining, especially in the presence of virtual memory requirements. Items 9 and 10 are related to the ability to take good advantage of compilers.

In the table, the first eight processors are clearly RISC architectures, the next five are clearly CISC, and the last two are processors often thought of as RISC that in fact have many CISC characteristics.

## 15.5 RISC PIPELINING

### Pipelining with Regular Instructions

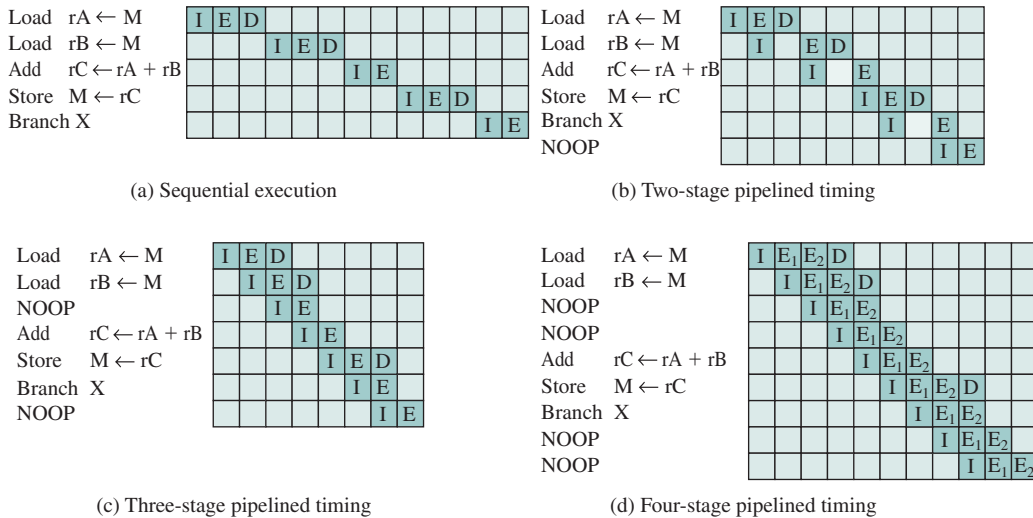
As we discussed in Section 12.4, instruction pipelining is often used to enhance performance. Let us reconsider this in the context of a RISC architecture. Most instructions are register to register, and an instruction cycle has the following two stages:

- I: Instruction fetch.
- E: Execute. Performs an ALU operation with register input and output.

For load and store operations, three stages are required:

- I: Instruction fetch.
- E: Execute. Calculates memory address.
- D: Memory. Register-to-memory or memory-to-register operation.

Figure 15.6a depicts the timing of a sequence of instructions using no pipelining. Clearly, this is a wasteful process. Even very simple pipelining can substantially improve performance. Figure 15.6b shows a two-stage pipelining scheme, in which the I and E stages of two different instructions are performed simultaneously. The two stages of the pipeline are an instruction fetch stage, and an execute/memory stage that executes the instruction, including register-to-memory and memory-to-register operations. Thus we see that the instruction fetch stage of the

**Figure 15.6** The Effects of Pipelining

second instruction can be performed in parallel with the first part of the execute/memory stage. However, the execute/memory stage of the second instruction must be delayed until the first instruction clears the second stage of the pipeline. This scheme can yield up to twice the execution rate of a serial scheme. Two problems prevent the maximum speedup from being achieved. First, we assume that a single-port memory is used and that only one memory access is possible per stage. This requires the insertion of a wait state in some instructions. Second, a branch instruction interrupts the sequential flow of execution. To accommodate this with minimum circuitry, a NOOP instruction can be inserted into the instruction stream by the compiler or assembler.

Pipelining can be improved further by permitting two memory accesses per stage. This yields the sequence shown in Figure 15.6c. Now, up to three instructions can be overlapped, and the improvement is as much as a factor of 3. Again, branch instructions cause the speedup to fall short of the maximum possible. Also, note that data dependencies have an effect. If an instruction needs an operand that is altered by the preceding instruction, a delay is required. Again, this can be accomplished by a NOOP.

The pipelining discussed so far works best if the three stages are of approximately equal duration. Because the E stage usually involves an ALU operation, it may be longer. In this case, we can divide into two substages:

- E<sub>1</sub>: Register file read
- E<sub>2</sub>: ALU operation and register write

Because of the simplicity and regularity of a RISC instruction set, the design of the phasing into three or four stages is easily accomplished. Figure 15.6d shows the result with a four-stage pipeline. Up to four instructions at a time can be under way, and the maximum potential speedup is a factor of 4. Note again the use of NOOPs to account for data and branch delays.

## Optimization of Pipelining

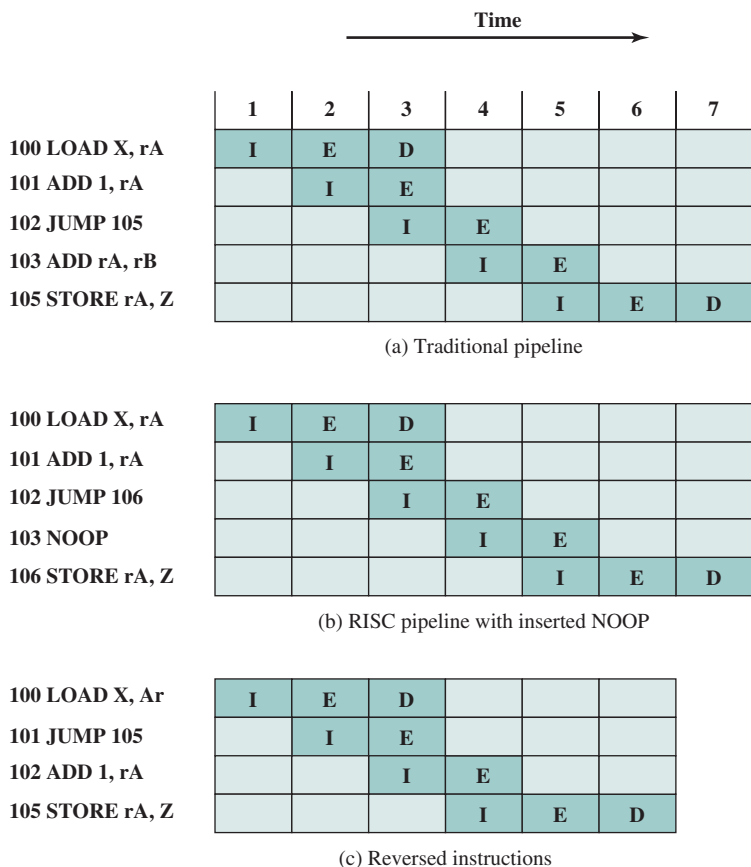
Because of the simple and regular nature of RISC instructions, it is easier for a hardware designer to implement a simple, fast pipeline. There are few variations in instruction execution duration, and the pipeline can be tailored to reflect this. However, we have seen that data and branch dependencies reduce the overall execution rate.

**DELAYED BRANCH** To compensate for these dependencies, code reorganization techniques have been developed. First, let us consider branching instructions. **Delayed branch**, a way of increasing the efficiency of the pipeline, makes use of a branch that does not take effect until after execution of the following instruction (hence the term *delayed*). The instruction location immediately following the branch is referred to as the *delay slot*. This strange procedure is illustrated in Table 15.8. In the column labeled “normal branch,” we see a normal symbolic instruction machine-language program. After 102 is executed, the next instruction to be executed is 105. To regularize the pipeline, a NOOP is inserted after this branch. However, increased performance is achieved if the instructions at 101 and 102 are interchanged.

Figure 15.7 shows the result. Figure 15.7a shows the traditional approach to pipelining, of the type discussed in Chapter 14 (e.g., see Figures 14.11 and 14.12). The JUMP instruction is fetched at time 3. At time 4, the JUMP instruction is executed at the same time that instruction 103 (ADD instruction) is fetched. Because a JUMP occurs, which updates the program counter, the pipeline must be cleared of instruction 103; at time 5, instruction 105, which is the target of the JUMP, is loaded. Figure 15.7b shows the same pipeline handled by a typical RISC organization. The timing is the same. However, because of the insertion of the NOOP instruction, we do not need special circuitry to clear the pipeline; the NOOP simply executes with no effect. Figure 15.7c shows the use of the delayed branch. The JUMP instruction is fetched at time 2, before the ADD instruction, which is fetched at time 3. Note, however, that the ADD instruction is fetched before the execution of the JUMP instruction has a chance to alter the program counter. Therefore, during time 4, the ADD instruction is executed at the same time that instruction 105 is fetched.

**Table 15.8** Normal and Delayed Branch

| Address | Normal Branch |        | Delayed Branch |        | Optimized Delayed Branch |        |
|---------|---------------|--------|----------------|--------|--------------------------|--------|
| 100     | LOAD          | X, rA  | LOAD           | X, rA  | LOAD                     | X, rA  |
| 101     | ADD           | 1, rA  | ADD            | 1, rA  | JUMP                     | 105    |
| 102     | JUMP          | 105    | JUMP           | 106    | ADD                      | 1, rA  |
| 103     | ADD           | rA, rB | NOOP           |        | ADD                      | rA, rB |
| 104     | SUB           | rC, rB | ADD            | rA, rB | SUB                      | rC, rB |
| 105     | STORE         | rA, Z  | SUB            | rC, rB | STORE                    | rA, Z  |
| 106     |               |        | STORE          | rA, Z  |                          |        |



**Figure 15.7** Use of the Delayed Branch

Thus, the original semantics of the program are retained but one less clock cycle is required for execution.

This interchange of instructions will work successfully for unconditional branches, calls, and returns. For conditional branches, this procedure cannot be blindly applied. If the condition that is tested for the branch can be altered by the immediately preceding instruction, then the compiler must refrain from doing the interchange and instead insert a NOOP. Otherwise, the compiler can seek to insert a useful instruction after the branch. The experience with both the Berkeley RISC and IBM 801 systems is that the majority of conditional branch instructions can be optimized in this fashion ([PATT82a], [RADI83]).

**DELAYED LOAD** A similar sort of tactic, called the **delayed load**, can be used on LOAD instructions. On LOAD instructions, the register that is to be the target of the load is locked by the processor. The processor then continues execution of the instruction stream until it reaches an instruction requiring that register, at which point it idles until the load is complete. If the compiler can rearrange instructions

so that useful work can be done while the load is in the pipeline, efficiency is increased.



### Loop Unrolling Simulator

**LOOP UNROLLING** Another compiler technique to improve instruction parallelism is loop unrolling [BACO94]. Unrolling replicates the body of a loop some number of times called the unrolling factor ( $u$ ) and iterates by step  $u$  instead of step 1.

Unrolling can improve the performance by

- reducing loop overhead
- increasing instruction parallelism by improving pipeline performance
- improving register, data cache, or TLB locality

Figure 15.8 illustrates all three of these improvements in an example. Loop overhead is cut in half because two iterations are performed before the test and branch at the end of the loop. Instruction parallelism is increased because the second assignment can be performed while the results of the first are being stored and the loop variables are being updated. If array elements are assigned to registers, register locality will improve because  $a[i]$  and  $a[i + 1]$  are used twice in the loop body, reducing the number of loads per iteration from three to two.

```
do i=2, n-1
 a[i] = a[i] + a[i-1] * a[i+1]
end do
```

(a) Original loop

```
do i=2, n-2, 2
 a[i] = a[i] + a[i-1] * a[i+1]
 a[i+1] = a[i+1] + a[i] * a[i+2]
end do

if (mod(n-2, 2) = 1) then
 a[n-1] = a[n-1] + a[n-2] * a[n]
end if
```

(b) Loop unrolled twice

**Figure 15.8** Loop Unrolling

As a final note, we should point out that the design of the instruction pipeline should not be carried out in isolation from other optimization techniques applied to the system. For example, [BRAD91b] shows that the scheduling of instructions for the pipeline and the dynamic allocation of registers should be considered together to achieve the greatest efficiency.

## 15.6 MIPS R4000

One of the first commercially available RISC chip sets was developed by MIPS Technology Inc. The system was inspired by an experimental system, also using the name MIPS, developed at Stanford [HENN84]. In this section we look at the MIPS R4000. It has substantially the same architecture and instruction set of the earlier MIPS designs: the R2000 and R3000. The most significant difference is that the R4000 uses 64 rather than 32 bits for all internal and external data paths and for addresses, registers, and the ALU.

The use of 64 bits has a number of advantages over a 32-bit architecture. It allows a bigger address space—large enough for an operating system to map more than a terabyte of files directly into virtual memory for easy access. With 1-terabyte and larger disk drives now common, the 4-gigabyte address space of a 32-bit machine becomes limiting. Also, the 64-bit capacity allows the R4000 to process data such as IEEE double-precision floating-point numbers and character strings, up to eight characters in a single action.

The R4000 processor chip is partitioned into two sections, one containing the CPU and the other containing a coprocessor for memory management. The processor has a very simple architecture. The intent was to design a system in which the instruction execution logic was as simple as possible, leaving space available for logic to enhance performance (e.g., the entire memory-management unit).

The processor supports thirty-two 64-bit registers. It also provides for up to 128 Kbytes of high-speed cache, half each for instructions and data. The relatively large cache (the IBM 3090 provides 128 to 256 Kbytes of cache) enables the system to keep large sets of program code and data local to the processor, off-loading the main memory bus and avoiding the need for a large register file with the accompanying windowing logic.

### Instruction Set

Table 15.9 lists the basic instruction set for all MIPS R series processors. All processor instructions are encoded in a single 32-bit word format. All data operations are register to register; the only memory references are pure load/store operations.

The R4000 makes no use of condition codes. If an instruction generates a condition, the corresponding flags are stored in a general-purpose register. This avoids the need for special logic to deal with condition codes as they affect the pipelining mechanism and the reordering of instructions by the compiler. Instead, the mechanisms already implemented to deal with register-value dependencies are employed. Further, conditions mapped onto the register files are subject

**Table 15.9** MIPS R-Series Instruction Set

| OP    | Description                                            | OP      | Description                                         |
|-------|--------------------------------------------------------|---------|-----------------------------------------------------|
|       | <b>Load/Store Instructions</b>                         | SRLV    | Shift Right Logical Variable                        |
| LB    | Load Byte                                              | SRAV    | Shift Right Arithmetic Variable                     |
| LBU   | Load Byte Unsigned                                     |         | <b>Multiply/Divide Instructions</b>                 |
| LH    | Load Halfword                                          | MULT    | Multiply                                            |
| LHU   | Load Halfword Unsigned                                 | MULTU   | Multiply Unsigned                                   |
| LW    | Load Word                                              | DIV     | Divide                                              |
| LWL   | Load Word Left                                         | DIVU    | Divide Unsigned                                     |
| LWR   | Load Word Right                                        | MFHI    | Move From HI                                        |
| SB    | Store Byte                                             | MTHI    | Move To HI                                          |
| SH    | Store Halfword                                         | MFLO    | Move From LO                                        |
| SW    | Store Word                                             | MTLO    | Move To LO                                          |
| SWL   | Store Word Left                                        |         | <b>Jump and Branch Instructions</b>                 |
| SWR   | Store Word Right                                       | J       | Jump                                                |
|       | <b>Arithmetic Instructions<br/>(ALU Immediate)</b>     | JAL     | Jump and Link                                       |
| ADDI  | Add Immediate                                          | JR      | Jump to Register                                    |
| ADDIU | Add Immediate Unsigned                                 | JALR    | Jump and Link Register                              |
| SLTI  | Set on Less Than Immediate                             | BEQ     | Branch on Equal                                     |
| SLTIU | Set on Less Than Immediate<br>Unsigned                 | BNE     | Branch on Not Equal                                 |
| ANDI  | AND Immediate                                          | BLEZ    | Branch on Less Than or Equal to<br>Zero             |
| ORI   | OR Immediate                                           | BGTZ    | Branch on Greater Than Zero                         |
| XORI  | Exclusive-OR Immediate                                 | BLTZ    | Branch on Less Than Zero                            |
| LUI   | Load Upper Immediate                                   | BGEZ    | Branch on Greater Than or Equal<br>to Zero          |
|       | <b>Arithmetic Instructions<br/>(3-operand, R-type)</b> | BLTZAL  | Branch on Less Than Zero<br>And Link                |
| ADD   | Add                                                    | BGEZAL  | Branch on Greater Than or Equal to<br>Zero And Link |
| ADDU  | Add Unsigned                                           |         | <b>Coprocessor Instructions</b>                     |
| SUB   | Subtract                                               | LWCz    | Load Word to Coprocessor                            |
| SUBU  | Subtract Unsigned                                      | SWCz    | Store Word to Coprocessor                           |
| SLT   | Set on Less Than                                       | MTCz    | Move To Coprocessor                                 |
| SLTU  | Set on Less Than Unsigned                              | MFCz    | Move From Coprocessor                               |
| AND   | AND                                                    | CTCz    | Move Control To Coprocessor                         |
| OR    | OR                                                     | CFCz    | Move Control From Coprocessor                       |
| XOR   | Exclusive-OR                                           | COPz    | Coprocessor Operation                               |
| NOR   | NOR                                                    | BCzT    | Branch on Coprocessor z True                        |
|       | <b>Shift Instructions</b>                              | BCzF    | Branch on Coprocessor z False                       |
| SLL   | Shift Left Logical                                     |         | <b>Special Instructions</b>                         |
| SRL   | Shift Right Logical                                    | SYSCALL | System Call                                         |
| SRA   | Shift Right Arithmetic                                 | BREAK   | Break                                               |
| SLLV  | Shift Left Logical Variable                            |         |                                                     |



to the same compile-time optimizations in allocation and reuse as other values stored in registers.

As with most RISC-based machines, the MIPS uses a single 32-bit instruction length. This single instruction length simplifies instruction fetch and decode, and it also simplifies the interaction of instruction fetch with the virtual memory management unit (i.e., instructions do not cross word or page boundaries). The three instruction formats (Figure 15.9) share common formatting of opcodes and register references, simplifying instruction decode. The effect of more complex instructions can be synthesized at compile time.

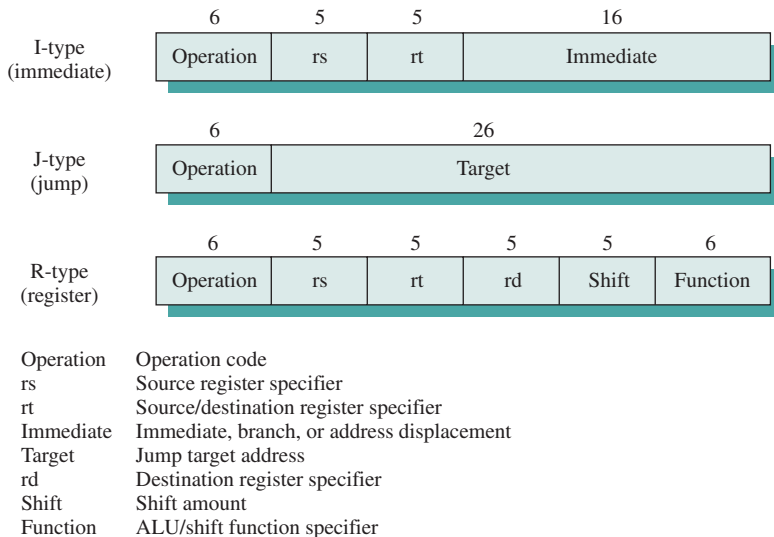
Only the simplest and most frequently used memory-addressing mode is implemented in hardware. All memory references consist of a 16-bit offset from a 32-bit register. For example, the “load word” instruction is of the form

```
lw r2, 128(r3) /* load word at address 128 offset from
 register 3 into register 2
```

Each of the 32 general-purpose registers can be used as the base register. One register, r0, always contains 0.

The compiler makes use of multiple machine instructions to synthesize typical addressing modes in conventional machines. Here is an example from [CHOW87], which uses the instruction lui (load upper immediate). This instruction loads the upper half of a register with a 16-bit immediate value, setting the lower half to zero. Consider an assembly-language instruction that uses a 32-bit immediate argument

```
lw r2, #imm(r4) /* load word at address using a 32-bit
 immediate offset #imm
 /* offset from register 4 into register 2
```



**Figure 15.9** MIPS Instruction Formats

This instruction can be compiled into the following MIPS instructions

```
lui r1, #imm-hi /* where #imm-hi is the high-order
 16 bits of #imm
addu r1, r1, r4 /* add unsigned #imm-hi to r4 and
 put in r1
lw r2, #imm-lo(r1) /* where #imm-lo is the low-order
 16 bits of #imm
```

## Instruction Pipeline

With its simplified instruction architecture, the MIPS can achieve very efficient pipelining. It is instructive to look at the evolution of the MIPS pipeline, as it illustrates the evolution of RISC pipelining in general.

The initial experimental RISC systems and the first generation of commercial RISC processors achieve execution speeds that approach one instruction per system clock cycle. To improve on this performance, two classes of processors have evolved to offer execution of multiple instructions per clock cycle: superscalar and superpipelined architectures. In essence, a superscalar architecture replicates each of the pipeline stages so that two or more instructions at the same stage of the pipeline can be processed simultaneously. A superpipelined architecture is one that makes use of more, and more fine-grained, pipeline stages. With more stages, more instructions can be in the pipeline at the same time, increasing parallelism.

Both approaches have limitations. With superscalar pipelining, dependencies between instructions in different pipelines can slow down the system. Also, overhead logic is required to coordinate these dependencies. With superpipelining, there is overhead associated with transferring instructions from one stage to the next.

Chapter 16 is devoted to a study of superscalar architecture. The MIPS R4000 is a good example of a RISC-based superpipeline architecture.

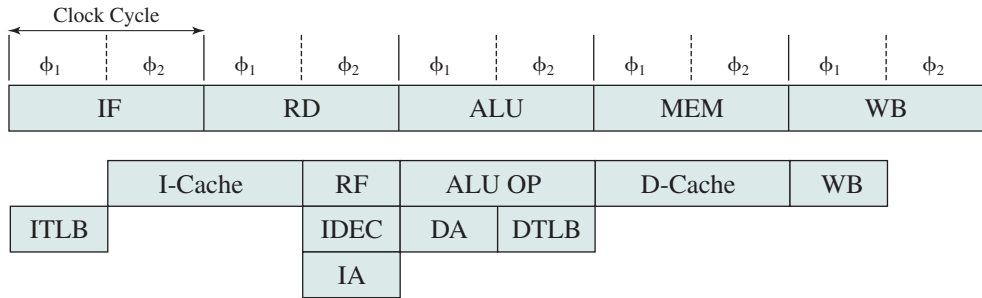


### MIPS R3000 Five-Stage Pipeline Simulator

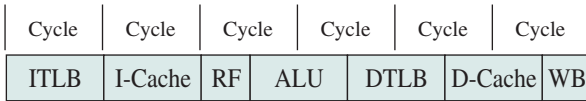
Figure 15.10a shows the instruction pipeline of the R3000. In the R3000, the pipeline advances once per clock cycle. The MIPS compiler is able to reorder instructions to fill delay slots with code 70 to 90% of the time. All instructions follow the same sequence of five pipeline stages:

- Instruction fetch
- Source operand fetch from register file
- ALU operation or data operand address generation
- Data memory reference
- Write back into register file

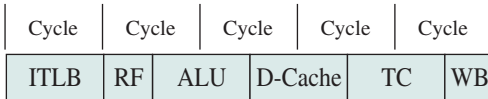
As illustrated in Figure 15.10a, there is not only parallelism due to pipelining but also parallelism within the execution of a single instruction. The 60-ns clock cycle



(a) Detailed R3000 pipeline



(b) Modified R3000 pipeline with reduced latencies



(c) Optimized R3000 pipeline with parallel TLB and cache accesses

- IF = Instruction fetch
- RD = Read
- MEM = Memory access
- WB = Write back to register file
- I-Cache = Instruction cache access
- RF = Fetch operand from register
- D-Cache = Data cache access
- ITLB = Instruction address translation
- IDEC = Instruction decode
- IA = Compute instruction address
- DA = Calculate data virtual address
- DTLB = Data address translation
- TC = Data cache tag check

**Figure 15.10** Enhancing the R3000 Pipeline

is divided into two 30-ns stages. The external instruction and data access operations to the cache each require 60 ns, as do the major internal operations (OP, DA, IA). Instruction decode is a simpler operation, requiring only a single 30-ns stage, overlapped with register fetch in the same instruction. Calculation of an address for a branch instruction also overlaps instruction decode and register fetch, so that a branch at instruction  $i$  can address the ICACHE access of instruction  $i + 2$ . Similarly, a load at instruction  $i$  fetches data that are immediately used by the OP of instruction  $i + 1$ , while an ALU/shift result gets passed directly into instruction  $i + 1$  with no delay. This tight coupling between instructions makes for a highly efficient pipeline.

In detail, then, each clock cycle is divided into separate stages, denoted as  $\phi_1$  and  $\phi_2$ . The functions performed in each stage are summarized in Table 15.10.

The R4000 incorporates a number of technical advances over the R3000. The use of more advanced technology allows the clock cycle time to be cut in half, to 30 ns, and for the access time to the register file to be cut in half. In addition, there is greater density on the chip, which enables the instruction and data caches to be incorporated on the chip. Before looking at the final R4000 pipeline, let us consider how the R3000 pipeline can be modified to improve performance using R4000 technology.

Figure 15.10b shows a first step. Remember that the cycles in this figure are half as long as those in Figure 15.10a. Because they are on the same chip, the instruction and data cache stages take only half as long; so they still occupy only one clock cycle. Again, because of the speedup of the register file access, register read and write still occupy only half of a clock cycle.

**Table 15.10** R3000 Pipeline Stages

| Pipeline Stage | Phase             | Function                                                                                                                                |
|----------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| IF             | $\phi 1$          | Using the TLB, translate an instruction virtual address to a physical address (after a branching decision).                             |
| IF             | $\phi 2$          | Send the physical address to the instruction address.                                                                                   |
| RD             | $\phi 1$          | Return instruction from instruction cache.<br>Compare tags and validity of fetched instruction.                                         |
| RD             | $\phi 2$          | Decode instruction.<br>Read register file.<br>If branch, calculate branch target address.                                               |
| ALU            | $\phi 1 + \phi 2$ | If register-to-register operation, the arithmetic or logical operation is performed.                                                    |
| ALU            | $\phi 1$          | If a branch, decide whether the branch is to be taken or not.<br>If a memory reference (load or store), calculate data virtual address. |
| ALU            | $\phi 2$          | If a memory reference, translate data virtual address to physical using TLB.                                                            |
| MEM            | $\phi 1$          | If a memory reference, send physical address to data cache.                                                                             |
| MEM            | $\phi 2$          | If a memory reference, return data from data cache, and check tags.                                                                     |
| WB             | $\phi 1$          | Write to register file.                                                                                                                 |

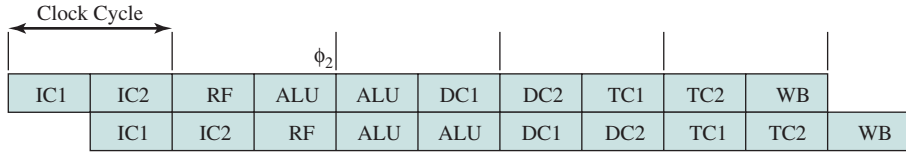
Because the R4000 caches are on-chip, the virtual-to-physical address translation can delay the cache access. This delay is reduced by implementing virtually indexed caches and going to a parallel cache access and address translation. Figure 15.10c shows the optimized R3000 pipeline with this improvement. Because of the compression of events, the data cache tag check is performed separately on the next cycle after cache access. This check determines whether the data item is in the cache.

In a superpipelined system, existing hardware is used several times per cycle by inserting pipeline registers to split up each pipe stage. Essentially, each superpipeline stage operates at a multiple of the base clock frequency, the multiple depending on the degree of superpipelining. The R4000 technology has the speed and density to permit superpipelining of degree 2. Figure 15.11a shows the optimized R3000 pipeline using this superpipelining. Note that this is essentially the same dynamic structure as Figure 15.10c.

Further improvements can be made. For the R4000, a much larger and specialized adder was designed. This makes it possible to execute ALU operations at twice the rate. Other improvements allow the execution of loads and stores at twice the rate. The resulting pipeline is shown in Figure 15.11b.

The R4000 has eight pipeline stages, meaning that as many as eight instructions can be in the pipeline at the same time. The pipeline advances at the rate of two stages per clock cycle. The eight pipeline stages are as follows:

- **Instruction fetch first half:** Virtual address is presented to the instruction cache and the translation lookaside buffer.
- **Instruction fetch second half:** Instruction cache outputs the instruction and the TLB generates the physical address.



(a) Superpipelined implementation of the optimized R3000 pipeline



(b) R4000 pipeline

IF = Instruction fetch first half      DC = Data cache  
 IS = Instruction fetch second half    DF = Data cache first half  
 RF = Fetch operands from register    DS = Data cache second half  
 EX = Instruction execute              TC = Tag check  
 IC = Instruction cache                WB = Write back to register file

**Figure 15.11** Theoretical R3000 and Actual R4000 Superpipelines

- **Register file:** Three activities occur in parallel:
  - Instruction is decoded and check made for interlock conditions (i.e., this instruction depends on the result of a preceding instruction).
  - Instruction cache tag check is made.
  - Operands are fetched from the register file.
- **Instruction execute:** One of three activities can occur:
  - If the instruction is a register-to-register operation, the ALU performs the arithmetic or logical operation.
  - If the instruction is a load or store, the data virtual address is calculated.
  - If the instruction is a branch, the branch target virtual address is calculated and branch conditions are checked.
- **Data cache first:** Virtual address is presented to the data cache and TLB.
- **Data cache second:** The TLB generates the physical address, and the data cache outputs the data.
- **Tag check:** Cache tag checks are performed for loads and stores.
- **Write back:** Instruction result is written back to register file.

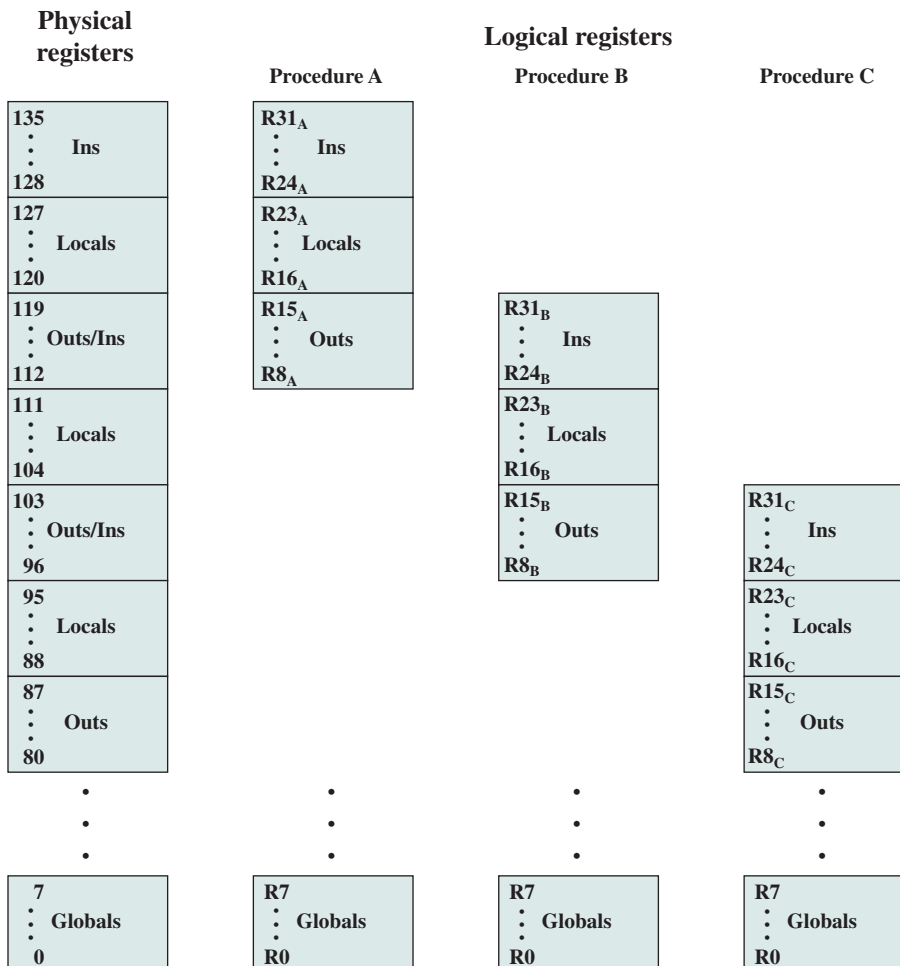
## 15.7 SPARC

SPARC (Scalable Processor Architecture) refers to an architecture defined by Sun Microsystems. Sun developed its own SPARC implementation but also licenses the architecture to other vendors to produce SPARC-compatible machines. The SPARC

architecture is inspired by the Berkeley RISC I machine, and its instruction set and register organization is based closely on the Berkeley RISC model.

### SPARC Register Set

As with the Berkeley RISC, the SPARC makes use of register windows. Each window gives addressability to 24 registers, and the total number of windows is implementation dependent and ranges from 2 to 32 windows. Figure 15.12 illustrates an implementation that supports 8 windows, using a total of 136 physical registers; as the discussion in Section 15.2 indicates, this seems a reasonable number of windows. Physical registers 0 through 7 are global registers shared by all procedures. Each procedure sees logical registers 0 through 31. Logical registers 24 through 31, referred to as *ins*, are shared with the calling (parent) procedure; and logical registers 8 through 15, referred to as *outs*, are shared with any called (child) procedure. These two portions overlap with other windows. Logical registers 16 through 23,



**Figure 15.12** SPARC Register Window Layout with Three Procedures

referred to as *locals*, are not shared and do not overlap with other windows. Again, as the discussion of Section 12.1 indicates, the availability of 8 registers for parameter passing should be adequate in most cases (e.g., see Table 15.4).

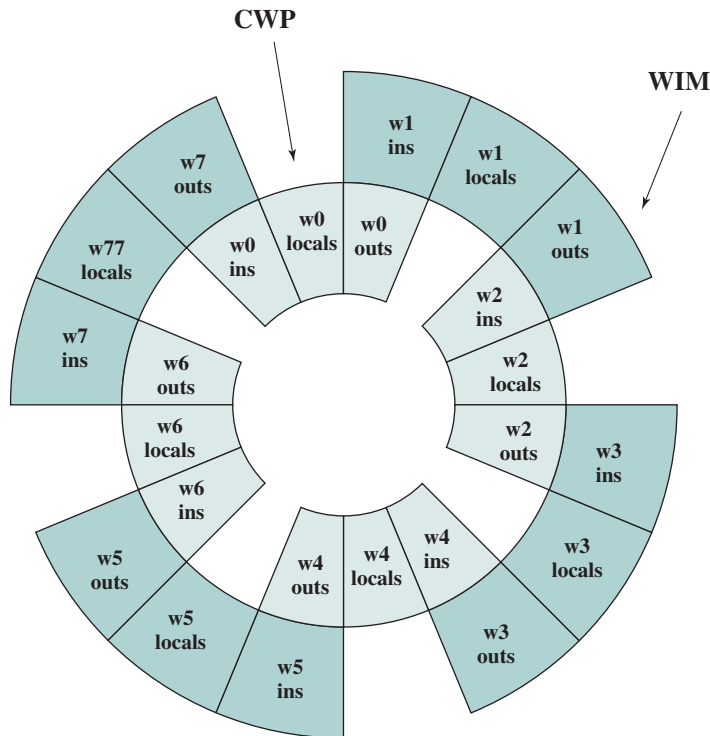
Figure 15.13 is another view of the register overlap. The calling procedure places any parameters to be passed in its *outs* registers; the called procedure treats these same physical registers as its *ins* registers. The processor maintains a current window pointer (CWP), located in the processor status register (PSR), that points to the window of the currently executing procedure. The window invalid mask (WIM), also in the PSR, indicates which windows are invalid.

With the SPARC register architecture, it is usually not necessary to save and restore registers for a procedure call. The compiler is simplified because the compiler need be concerned only with allocating the local registers for a procedure in an efficient manner and need not be concerned with register allocation between procedures.

### Instruction Set

Table 15.11 lists the instructions for the SPARC architecture. Most of the instructions reference only register operands. Register-to-register instructions have three operands and can be expressed in the form

$$R_d \rightarrow R_{S1} \text{ op } S2$$



**Figure 15.13** Eight Register Windows Forming a Circular Stack in SPARC

**Table 15.11** SPARC Instruction Set

| OP                             | Description            | OP                                | Description                        |
|--------------------------------|------------------------|-----------------------------------|------------------------------------|
| <b>Load/Store Instructions</b> |                        | <b>Arithmetic Instructions</b>    |                                    |
| LDSB                           | Load signed byte       | ADD                               | Add                                |
| LDSH                           | Load signed halfword   | ADDCC                             | Add, set icc                       |
| LDUB                           | Load unsigned byte     | ADDX                              | Add with carry                     |
| LDUH                           | Load unsigned halfword | ADDXCC                            | Add with carry, set icc            |
| LD                             | Load word              | SUB                               | Subtract                           |
| LDD                            | Load doubleword        | SUBCC                             | Subtract, set icc                  |
| STB                            | Store byte             | SUBX                              | Subtract with carry                |
| STH                            | Store halfword         | SUBXCC                            | Subtract with carry, set icc       |
| STD                            | Store word             | MULSCC                            | Multiply step, set icc             |
| STDD                           | Store doubleword       | <b>Jump/Branch Instructions</b>   |                                    |
| <b>Shift Instructions</b>      |                        | BCC                               | Branch on condition                |
| SLL                            | Shift left logical     | FBCC                              | Branch on floating-point condition |
| SRL                            | Shift right logical    | CBCC                              | Branch on coprocessor condition    |
| SRA                            | Shift right arithmetic | CALL                              | Call procedure                     |
| <b>Boolean Instructions</b>    |                        | JMPL                              | Jump and link                      |
| AND                            | AND                    | TCC                               | Trap on condition                  |
| ANDCC                          | AND, set icc           | SAVE                              | Advance register window            |
| ANDN                           | NAND                   | RESTORE                           | Move windows backward              |
| ANDNCC                         | NAND, set icc          | RETT                              | Return from trap                   |
| OR                             | OR                     | <b>Miscellaneous Instructions</b> |                                    |
| ORCC                           | OR, set icc            | SETHI                             | Set high 22 bits                   |
| ORN                            | NOR                    | UNIMP                             | Unimplemented instruction (trap)   |
| ORNCC                          | NOR, set icc           | RD                                | Read a special register            |
| XOR                            | XOR                    | WR                                | Write a special register           |
| XORCC                          | XOR, set icc           | IFLUSH                            | Instruction cache flush            |
| XNOR                           | Exclusive NOR          |                                   |                                    |
| XNORCC                         | Exclusive NOR, set icc |                                   |                                    |

where  $R_d$  and  $R_{S1}$  are register references;  $S2$  can refer either to a register or to a 13-bit immediate operand. Register zero ( $R_0$ ) is hardwired with the value 0. This form is well suited to typical programs, which have a high proportion of local scalars and constants.

The available ALU operations can be grouped as follows:

- Integer addition (with or without carry)
- Integer subtraction (with or without carry)
- Bitwise Boolean AND, OR, XOR and their negations
- Shift left logical, right logical, or right arithmetic



All of these instructions, except the shifts, can optionally set the four condition codes (ZERO, NEGATIVE, OVERFLOW, CARRY). Signed integers are represented in 32-bit two's complement form.

Only simple load and store instructions reference memory. There are separate load and store instructions for word (32 bits), doubleword, halfword, and byte. For the latter two cases, there are instructions for loading these quantities as signed or unsigned numbers. Signed numbers are sign extended to fill out the 32-bit destination register. Unsigned numbers are padded with zeros.

The only available addressing mode, other than register, is a displacement mode. That is, the effective address (EA) of an operand consists of a displacement from an address contained in a register:

$$\begin{aligned} \text{EA} &= (\text{R}_{\text{S1}}) + \text{S2} \\ \text{or EA} &= (\text{R}_{\text{S1}}) + (\text{R}_{\text{S2}}) \end{aligned}$$

depending on whether the second operand is immediate or a register reference. To perform a load or store, an extra stage is added to the instruction cycle. During the second stage, the memory address is calculated using the ALU; the load or store occurs in a third stage. This single addressing mode is quite versatile and can be used to synthesize other addressing modes, as indicated in Table 15.12.

It is instructive to compare the SPARC addressing capability with that of the MIPS. The MIPS makes use of a 16-bit offset, compared with a 13-bit offset on the SPARC. On the other hand, the MIPS does not permit an address to be constructed from the contents of two registers.

### Instruction Format

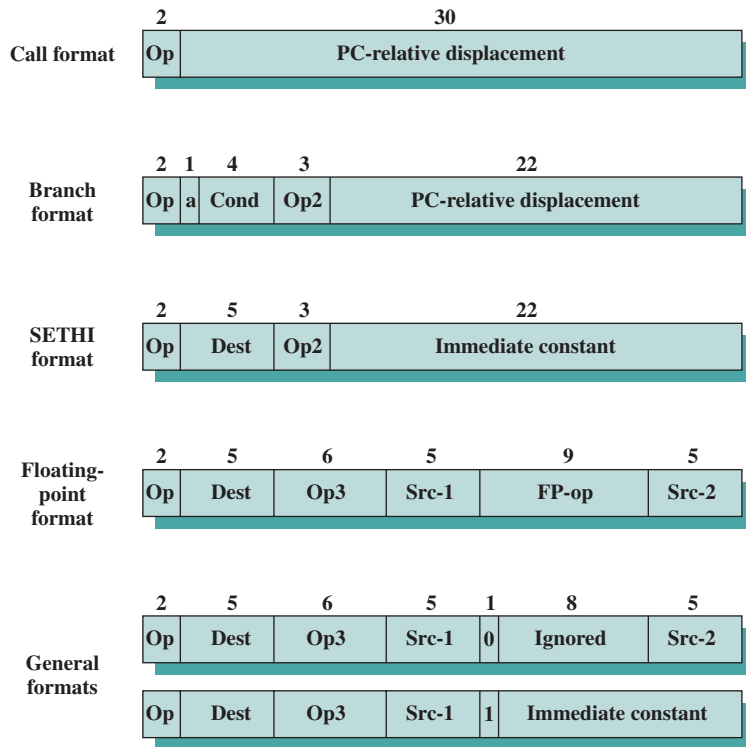
As with the MIPS R4000, SPARC uses a simple set of 32-bit instruction formats (Figure 15.14). All instructions begin with a 2-bit opcode. For most instructions, this is extended with additional opcode bits elsewhere in the format. For the Call instruction, a 30-bit immediate operand is extended with two zero bits to the right to form a 32-bit PC-relative address in two's complement form. Instructions are aligned on a 32-bit boundary so that this form of addressing suffices.

The Branch instruction includes a 4-bit condition field that corresponds to the four standard condition code bits, so that any combination of conditions can be tested. The 22-bit PC-relative address is extended with two zero bits on the right to form a 24-bit two's complement relative address. An unusual feature of the Branch instruction is the annul bit. When the annul bit is not set, the instruction after the

**Table 15.12** Synthesizing Other Addressing Modes with SPARC Addressing Modes

| Instruction Type     | Addressing Mode   | Algorithm    | SPARC Equivalent                  |
|----------------------|-------------------|--------------|-----------------------------------|
| Register-to-register | Immediate         | operand = A  | S2                                |
| Load, store          | Direct            | EA = A       | R <sub>0</sub> + S <sub>2</sub>   |
| Register-to-register | Register          | EA = R       | R <sub>S1</sub> , R <sub>S2</sub> |
| Load, store          | Register Indirect | EA = (R)     | R <sub>S1</sub> + 0               |
| Load, store          | Displacement      | EA = (R) + A | R <sub>S1</sub> + S2              |

*Note:* S2 = either a register operand or a 13-bit immediate operand.



**Figure 15.14** SPARC Instruction Formats

branch is always executed, regardless of whether the branch is taken. This is the typical delayed branch operation found on many RISC machines and described in Section 15.5 (see Figure 15.7). However, when the annul bit is set, the instruction following the branch is executed only if the branch is taken. The processor suppresses the effect of that instruction even though it is already in the pipeline. This annul bit is useful because it makes it easier for the compiler to fill the delay slot following a conditional branch. The instruction that is the target of the branch can always be put in the delay slot, because if the branch is not taken, the instruction can be annulled. The reason this technique is desirable is that conditional branches are generally taken more than half the time.

The SETHI instruction is a special instruction used to form a 32-bit constant. This feature is needed to form large data constants; for example, it can be used to form a large offset for a load or store instruction. The SETHI instruction sets the 22 high-order bits of a register with its 22-bit immediate operand, and zeros out the low-order 10 bits. An immediate constant of up to 13 bits can be specified in one of the general formats, and such an instruction could be used to fill in the remaining 10 bits of the register. A load or store instruction can also be used to achieve a direct addressing mode. To load a value from location K in memory, we could use the following SPARC instructions:

```
sethi %hi(K), %r8 ;load high-order 22 bits of address of location
 ;K into register r8
ld [%r8 + %lo(K)], %r8 ;load contents of location K into r8
```

The macros %hi and %lo are used to define immediate operands consisting of the appropriate address bits of a location. This use of SETHI is similar to the use of the lui instruction on the MIPS.

The floating-point format is used for floating-point operations. Two source and one destination registers are designated.

Finally, all other operations, including loads, stores, arithmetic, and logical operations use one of the last two formats shown in Figure 15.14. One of the formats makes use of two source registers and a destination register, while the other uses one source register, one 13-bit immediate operand, and one destination register.

## 15.8 RISC VERSUS CISC CONTROVERSY

For many years, the general trend in computer architecture and organization has been toward increasing processor complexity: more instructions, more addressing modes, more specialized registers, and so on. The RISC movement represents a fundamental break with the philosophy behind that trend. Naturally, the appearance of RISC systems, and the publication of papers by its proponents extolling RISC virtues, led to a reaction from those involved in the design of CISC architectures.

The work that has been done on assessing merits of the RISC approach can be grouped into two categories:

- **Quantitative:** Attempts to compare program size and execution speed of programs on RISC and CISC machines that use comparable technology
- **Qualitative:** Examines issues such as high-level language support and optimum use of VLSI real estate

Most of the work on quantitative assessment has been done by those working on RISC systems [PATT82b, HEAT84, PATT84], and it has been, by and large, favorable to the RISC approach. Others have examined the issue and come away unconvinced [COLW85a, FLYN87, DAVI87]. There are several problems with attempting such comparisons [SERL86]:

- There is no pair of RISC and CISC machines that are comparable in life-cycle cost, level of technology, gate complexity, sophistication of compiler, operating system support, and so on.
- No definitive test set of programs exists. Performance varies with the program.
- It is difficult to sort out hardware effects from effects due to skill in compiler writing.
- Most of the comparative analysis on RISC has been done on “toy” machines rather than commercial products. Furthermore, most commercially available machines advertised as RISC possess a mixture of RISC and CISC characteristics. Thus, a fair comparison with a commercial, “pure-play” CISC machine (e.g., VAX, Pentium) is difficult.

The qualitative assessment is, almost by definition, subjective. Several researchers have turned their attention to such an assessment [COLW85a, WALL85], but the results are, at best, ambiguous, and certainly subject to rebuttal [PATT85b] and, of course, counterrebuttal [COLW85b].

In more recent years, the RISC versus CISC controversy has died down to a great extent. This is because there has been a gradual convergence of the technologies. As chip densities and raw hardware speeds increase, RISC systems have become more complex. At the same time, in an effort to squeeze out maximum performance, CISC designs have focused on issues traditionally associated with RISC, such as an increased number of general-purpose registers and increased emphasis on instruction pipeline design.

## 15.9 RECOMMENDED READING

Two classic overview papers on RISC are [PATT85a] and [HENN84]. Another survey article is [STAL88]. Accounts of two pioneering RISC efforts are provided by [RADI83] and [PATT82a].

[KANE92] covers the commercial MIPS machine in detail. [MIRA92] provides a good overview of the MIPS R4000. [BASH91] discusses the evolution from the R3000 pipeline to the R4000 superpipeline. The SPARC is covered in some detail in [DEWA90].

**BASH91** Bashteen, A.; Lui, I.; and Mullan, J. "A Superpipeline Approach to the MIPS Architecture." *Proceedings, COMPCON Spring '91*, February 1991.

**DEWA90** Dewar, R., and Smosna, M. *Microprocessors: A Programmer's View*. New York: McGraw-Hill, 1990.

**HENN84** Hennessy, J. "VLSI Processor Architecture." *IEEE Transactions on Computers*, December 1984.

**KANE92** Kane, G., and Heinrich, J. *MIPS RISC Architecture*. Englewood Cliffs, NJ: Prentice Hall, 1992.

**MIRA92** Mirapuri, S.; Woodacre, M.; and Vasseghi, N. "The MIPS R4000 Processor." *IEEE Micro*, April 1992.

**PATT82a** Patterson, D., and Sequin, C. "A VLSI RISC." *Computer*, September 1982.

**PATT85a** Patterson, D. "Reduced Instruction Set Computers." *Communications of the ACM*, January 1985.

**RADI83** Radin, G. "The 801 Minicomputer." *IBM Journal of Research and Development*, May 1983.

**STAL88** Stallings, W. "Reduced Instruction Set Computer Architecture." *Proceedings of the IEEE*, January 1988.

## 15.10 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

|                                                                           |                                                                                       |                          |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------|
| complex instruction set computer (CISC)<br>delayed branch<br>delayed load | high-level language (HLL)<br>reduced instruction set computer (RISC)<br>register file | register window<br>SPARC |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------|

## Review Questions

- 15.1 What are some typical distinguishing characteristics of RISC organization?
- 15.2 Briefly explain the two basic approaches used to minimize register-memory operations on RISC machines.
- 15.3 If a circular register buffer is used to handle local variables for nested procedures, describe two approaches for handling global variables.
- 15.4 What are some typical characteristics of a RISC instruction set architecture?
- 15.5 What is a delayed branch?

## Problems

- 15.1 Considering the call-return pattern in Figure 4.21, how many overflows and underflows (each of which causes a register save/restore) will occur with a window size of
  - a. 5?
  - b. 8?
  - c. 16?
- 15.2 In the discussion of Figure 15.2, it was stated that only the first two portions of a window are saved or restored. Why is it not necessary to save the temporary registers?
- 15.3 We wish to determine the execution time for a given program using the various pipelining schemes discussed in Section 15.5. Let

$N$  = number of executed instructions

$D$  = number of memory accesses

$J$  = number of jump instructions

For the simple sequential scheme (Figure 15.6a), the execution time is  $2N + D$  stages. Derive formulas for two-stage, three-stage, and four-stage pipelining.

- 15.4 Reorganize the code sequence in Figure 15.6d to reduce the number of NOOPs.
- 15.5 Consider the following code fragment in a high-level language:

```

for I in 1...100 loop
 S ← S + Q(I).VAL
end loop;
```

Assume that Q is an array of 32-byte records and the VAL field is in the first 4 bytes of each record. Using x86 code, we can compile this program fragment as follows:

|     |      |            |                             |
|-----|------|------------|-----------------------------|
|     | MOV  | ECX,1      | ;use register ECX to hold I |
| LP: | IMUL | EAX,ECX,32 | ;get offset in EAX          |
|     | MOV  | EBX,Q[EAX] | ;load VAL field             |
|     | ADD  | S,EBX      | ;add to S                   |
|     | INC  | ECX        | ;increment I                |
|     | CMP  | ECX,101    | ;compare to 101             |
|     | JNE  | LP         | ;loop until I = 100         |

This program makes use of the IMUL instruction, which multiplies the second operand by the immediate value in the third operand and places the result in the first operand (see Problem 10.13). A RISC advocate would like to demonstrate that a clever compiler can eliminate unnecessarily complex instructions such as IMUL. Provide the demonstration by rewriting the above x86 program without using the IMUL instruction.

- 15.6 Consider the following loop:

```

S := 0;
for K := 1 to 100 do
 S := S - K;
```

A straightforward translation of this into a generic assembly language would look something like this:

```

 LD R1,0 ;keep value of S in R1
 LD R2,1 ;keep value of K in R2
LP SUB R1,R1,R2 ;S := S - K
 BEQ R2,100,EXIT ;done if K = 100
 ADD R2,R2,1 ;else increment K
 JMP LP ;back to start of loop

```

A compiler for a RISC machine will introduce delay slots into this code so that the processor can employ the delayed branch mechanism. The JMP instruction is easy to deal with, because this instruction is always followed by the SUB instruction; therefore, we can simply place a copy of the SUB instruction in the delay slot after the JMP. The BEQ presents a difficulty. We can't leave the code as is, because the ADD instruction would then be executed one too many times. Therefore, a NOP instruction is needed. Show the resulting code.

- 15.7** A RISC machine's compiler may do both a mapping of symbolic registers to actual registers and a rearrangement of instructions for pipeline efficiency. An interesting question arises as to the order in which these two operations should be done. Consider the following program fragment:

```

 LD SR1,A ;load A into symbolic register 1
 LD SR2,B ;load B into symbolic register 2
 ADD SR3,SR1,SR2 ;add contents of SR1 and SR2 and store in SR3
 LD SR4,C
 LD SR5,D
 ADD SR6,SR4,SR5

```

- First do the register mapping and then any possible instruction reordering. How many machine registers are used? Has there been any pipeline improvement?
- Starting with the original program, now do instruction reordering and then any possible mapping. How many machine registers are used? Has there been any pipeline improvement?

- 15.8** Add entries for the following processors to Table 15.7:

- Pentium II
- ARM

- 15.9** In many cases, common machine instructions that are not listed as part of the MIPS instruction set can be synthesized with a single MIPS instruction. Show this for the following:

- Register-to-register move
- Increment, decrement
- Complement
- Negate
- Clear

- 15.10** A SPARC implementation has  $K$  register windows. What is the number  $N$  of physical registers?

- 15.11** SPARC is lacking a number of instructions commonly found on CISC machines. Some of these are easily simulated using either register R0, which is always set to 0, or a constant operand. These simulated instructions are called pseudoinstructions and are recognized by the SPARC assembler. Show how to simulate the following pseudoinstructions, each with a single SPARC instruction. In all of these, src and dst refer to registers. (*Hint: A store to R0 has no effect.*)

- |                       |            |            |
|-----------------------|------------|------------|
| a. MOV src, dst       | d. NOT dst | g. DEC dst |
| b. COMPARE src1, src2 | e. NEG dst | h. CLR dst |
| c. TEST src1          | f. INC dst | i. NOP     |

**15.12** Consider the following code fragment:

```

if K > 10
 L := K + 1
else
 L := K - 1;

```

A straightforward translation of this statement into SPARC assembler could take the following form:

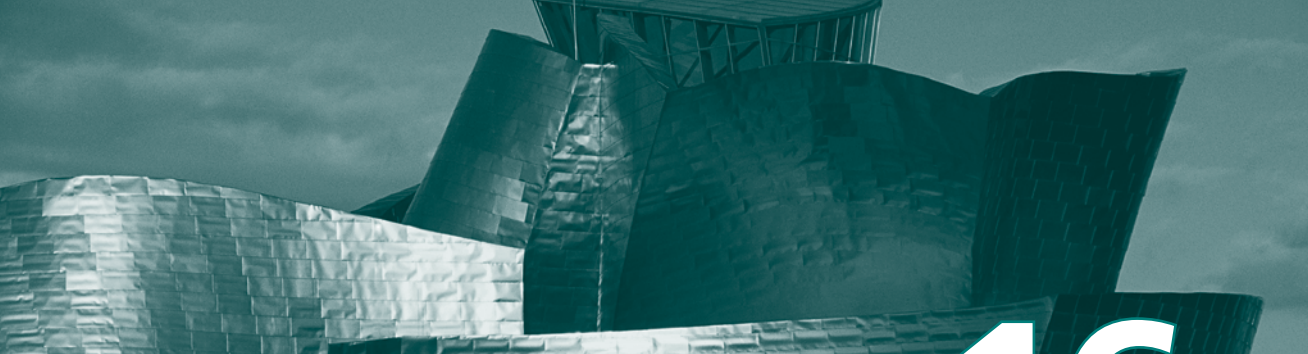
```

 sethi %hi(K), %r8 ;load high-order 22 bits of address of location
 ;K into register r8
ld [%r8 + %lo(K)], %r8 ;load contents of location K into r8
cmp %r8, 10 ;compare contents of r8 with 10
ble L1 ;branch if (r8) ≤ 10
nop
sethi %hi(K), %r9
ld [%r9 + %lo(K)], %r9 ;load contents of location K into r9
inc %r9 ;add 1 to (r9)
sethi %hi(L), %r10
st %r9, [%r10 + %lo(L)] ;store (r9) into location L
b L2
nop
L1: sethi %hi(K), %r11
ld [%r11 + %lo(K)], %r12 ;load contents of location K into r12
dec %r12 ;subtract 1 from (r12)
sethi %hi(L), %r13
st %r12, [%r13 + %lo(L)] ;store (r12) into location L
L2:

```

The code contains a nop after each branch instruction to permit delayed branch operation.

- a.** Standard compiler optimizations that have nothing to do with RISC machines are generally effective in being able to perform two transformations on the foregoing code. Notice that two of the loads are unnecessary and that the two stores can be merged if the store is moved to a different place in the code. Show the program after making these two changes.
- b.** It is now possible to perform some optimizations peculiar to SPARC. The nop after the ble can be replaced by moving another instruction into that delay slot and setting the annul bit on the ble instruction (expressed as ble,a L1). Show the program after this change.
- c.** There are now two unnecessary instructions. Remove these and show the resulting program.



# CHAPTER 16

## INSTRUCTION-LEVEL PARALLELISM AND SUPERSCALAR PROCESSORS

### 16.1 Overview

- Superscalar versus Superpipelined Constraints

### 16.2 Design Issues

- Instruction-Level Parallelism and Machine Parallelism
- Instruction Issue Policy
- Register Renaming
- Machine Parallelism
- Branch Prediction
- Superscalar Execution
- Superscalar Implementation

### 16.3 Pentium 4

- Front End
- Out-of-Order Execution Logic
- Integer and Floating-Point Execution Units

### 16.4 ARM Cortex-A8

- Instruction Fetch Unit
- Instruction Decode Unit
- Integer Execute Unit
- SIMD and Floating-Point Pipeline

### 16.5 Recommended Reading

### 16.6 Key Terms, Review Questions, and Problems



**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- ◆ Explain the difference between superscalar and superpipelined approaches.
- ◆ Define instruction-level parallelism.
- ◆ Discuss dependencies and resource conflicts as limitations to instruction-level parallelism
- ◆ Present an overview of the design issues involved in instruction-level parallelism.
- ◆ Compare and contrast techniques of improving pipeline performance in RISC machines and superscalar machines.

A superscalar implementation of a processor architecture is one in which common instructions—integer and floating-point arithmetic, loads, stores, and conditional branches—can be initiated simultaneously and executed independently. Such implementations raise a number of complex design issues related to the instruction pipeline.

Superscalar design arrived on the scene hard on the heels of RISC architecture. Although the simplified instruction set architecture of a RISC machine lends itself readily to superscalar techniques, the superscalar approach can be used on either a RISC or CISC architecture.

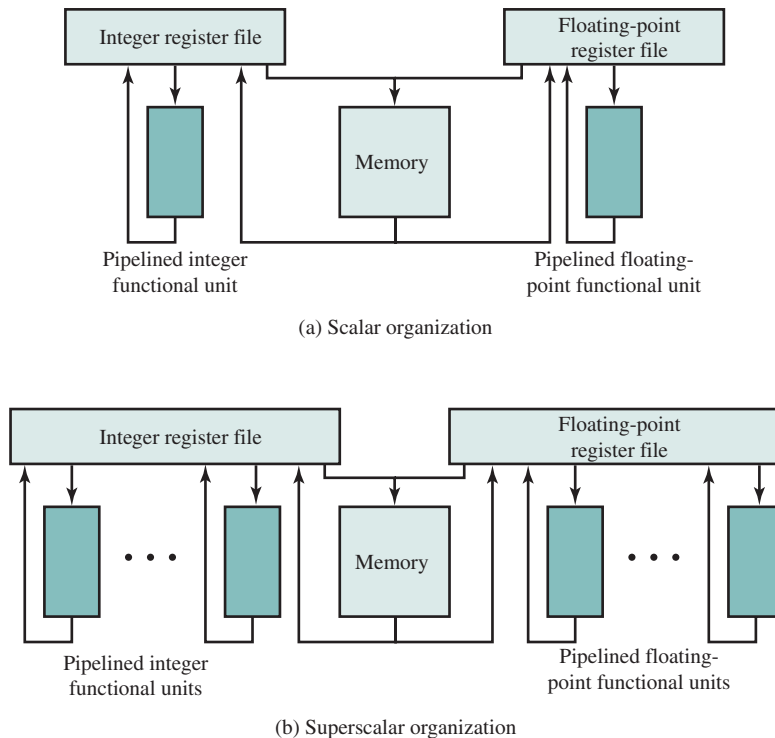
Whereas the gestation period for the arrival of commercial RISC machines from the beginning of true RISC research with the IBM 801 and the Berkeley RISC I was seven or eight years, the first superscalar machines became commercially available within just a year or two of the coining of the term **superscalar**. The superscalar approach has now become the standard method for implementing high-performance microprocessors.

In this chapter, we begin with an overview of the superscalar approach, contrasting it with superpipelining. Next, we present the key design issues associated with superscalar implementation. Then we look at several important examples of superscalar architecture.

## 16.1 OVERVIEW

The term *superscalar*, first coined in 1987 [AGER87], refers to a machine that is designed to improve the performance of the execution of scalar instructions. In most applications, the bulk of the operations are on scalar quantities. Accordingly, the superscalar approach represents the next step in the evolution of high-performance general-purpose processors.

The essence of the superscalar approach is the ability to execute instructions independently and concurrently in different pipelines. The concept can be further exploited by allowing instructions to be executed in an order different from the program order. Figure 16.1 compares, in general terms, the scalar and superscalar approaches. In a traditional scalar organization, there is a single pipelined functional unit for integer operations and one for floating-point operations. Parallelism is achieved by enabling multiple instructions to be at different stages of the pipeline at



**Figure 16.1** Superscalar Organization Compared to Ordinary Scalar Organization

one time. In the superscalar organization, there are multiple functional units, each of which is implemented as a pipeline. Each individual functional unit provides a degree of parallelism by virtue of its pipelined structure. The use of multiple functional units enables the processor to execute streams of instructions in parallel, one stream for each pipeline. It is the responsibility of the hardware, in conjunction with the compiler, to assure that the parallel execution does not violate the intent of the program.

Many researchers have investigated superscalar-like processors, and their research indicates that some degree of performance improvement is possible. Table 16.1 presents the reported performance advantages. The differences in the

**Table 16.1** Reported Speedups of Superscalar-Like Machines

| Reference | Speedup |
|-----------|---------|
| [TJAD70]  | 1.8     |
| [KUCK77]  | 8       |
| [WEIS84]  | 1.58    |
| [ACOS86]  | 2.7     |
| [SOHI90]  | 1.8     |
| [SMIT89]  | 2.3     |
| [JOUP89b] | 2.2     |
| [LEE91]   | 7       |

results arise from differences both in the hardware of the simulated machine and in the applications being simulated.

### Superscalar versus Superpipelined

An alternative approach to achieving greater performance is referred to as superpipelining, a term first coined in 1988 [JOUN88]. Superpipelining exploits the fact that many pipeline stages perform tasks that require less than half a clock cycle. Thus, a doubled internal clock speed allows the performance of two tasks in one external clock cycle. We have seen one example of this approach with the MIPS R4000.

Figure 16.2 compares the two approaches. The upper part of the diagram illustrates an ordinary pipeline, used as a base for comparison. The base pipeline issues

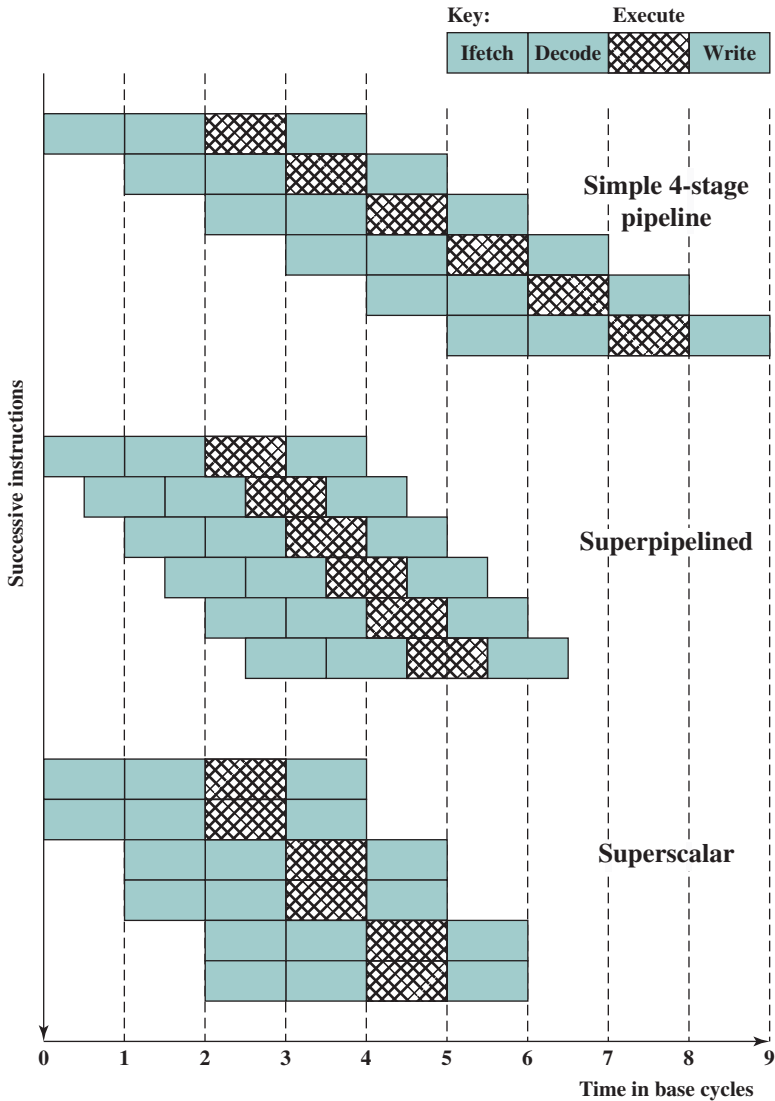


Figure 16.2 Comparison of Superscalar and Superpipeline Approaches

one instruction per clock cycle and can perform one pipeline stage per clock cycle. The pipeline has four stages: instruction fetch, operation decode, operation execution, and result write back. The execution stage is crosshatched for clarity. Note that although several instructions are executing concurrently, only one instruction is in its execution stage at any one time.

The next part of the diagram shows a **superpipelined** implementation that is capable of performing two pipeline stages per clock cycle. An alternative way of looking at this is that the functions performed in each stage can be split into two nonoverlapping parts and each can execute in half a clock cycle. A superpipeline implementation that behaves in this fashion is said to be of degree 2. Finally, the lowest part of the diagram shows a superscalar implementation capable of executing two instances of each stage in parallel. Higher-degree superpipeline and superscalar implementations are of course possible.

Both the superpipeline and the superscalar implementations depicted in Figure 16.2 have the same number of instructions executing at the same time in the steady state. The superpipelined processor falls behind the superscalar processor at the start of the program and at each branch target.

## Constraints

The superscalar approach depends on the ability to execute multiple instructions in parallel. The term **instruction-level parallelism** refers to the degree to which, on average, the instructions of a program can be executed in parallel. A combination of compiler-based optimization and hardware techniques can be used to maximize instruction-level parallelism. Before examining the design techniques used in superscalar machines to increase instruction-level parallelism, we need to look at the fundamental limitations to parallelism with which the system must cope. [JOHN91] lists five limitations:

- True data dependency
- Procedural dependency
- Resource conflicts
- Output dependency
- Antidependency

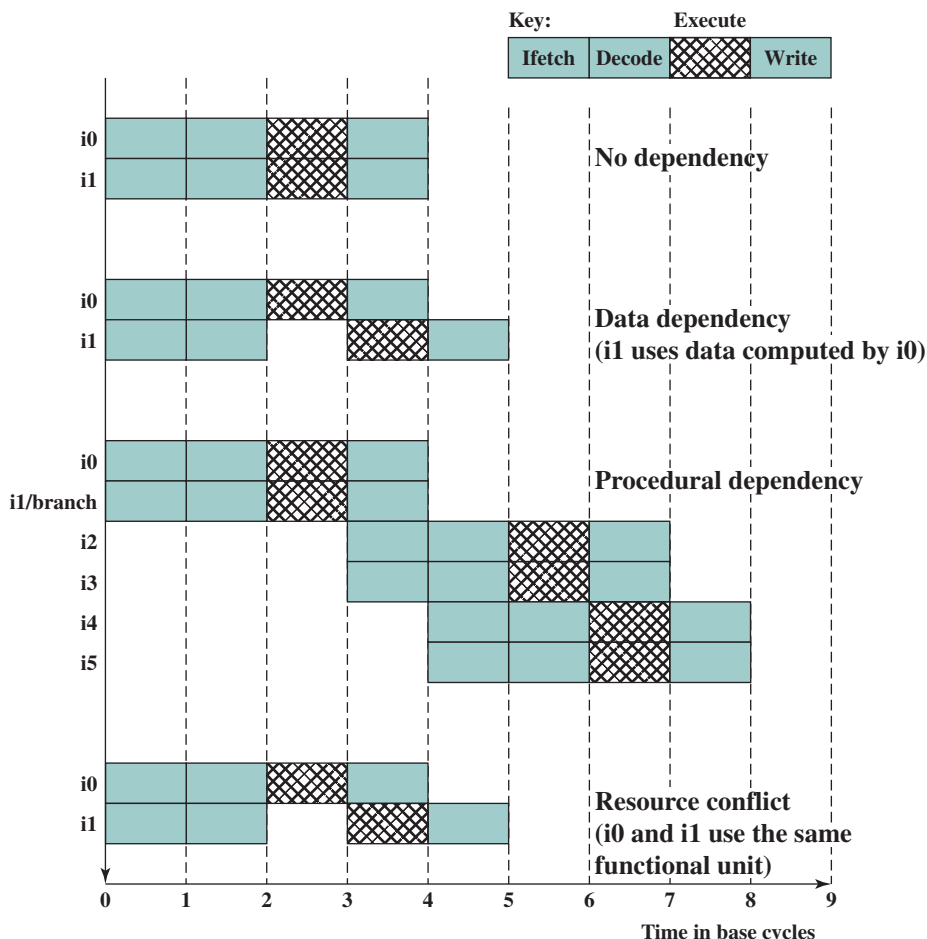
We examine the first three of these limitations in the remainder of this section. A discussion of the last two must await some of the developments in the next section.

**TRUE DATA DEPENDENCY** Consider the following sequence:<sup>1</sup>

```
ADD EAX, ECX ;load register EAX with the con-
 ;tents of ECX plus the contents
 ;of EAX
MOV EBX, EAX ;load EBX with the contents of EAX
```

The second instruction can be fetched and decoded but cannot execute until the first instruction executes. The reason is that the second instruction needs data

<sup>1</sup>For the Intel x86 assembly language, a semicolon starts a comment field.



**Figure 16.3** Effect of Dependencies

produced by the first instruction. This situation is referred to as a **true data dependency** (also called **flow dependency** or **read after write [RAW] dependency**).

Figure 16.3 illustrates this **dependency** in a superscalar machine of degree 2. With no dependency, two instructions can be fetched and executed in parallel. If there is a data dependency between the first and second instructions, then the second instruction is delayed as many clock cycles as required to remove the dependency. In general, any instruction must be delayed until all of its input values have been produced.

In a simple pipeline, such as illustrated in the upper part of Figure 16.2, the aforementioned sequence of instructions would cause no delay. However, consider the following, in which one of the loads is from memory rather than from a register:

```
MOV EAX, eff ;load register EAX with the con-
 tents of effective memory add-
 ress eff
MOV EBX, EAX ;load EBX with the contents of EAX
```

A typical RISC processor takes two or more cycles to perform a load from memory when the load is a cache hit. It can take tens or even hundreds of cycles for a cache miss on all cache levels, because of the delay of an off-chip memory access. One way to compensate for this delay is for the compiler to reorder instructions so that one or more subsequent instructions that do not depend on the memory load can begin flowing through the pipeline. This scheme is less effective in the case of a superscalar pipeline: The independent instructions executed during the load are likely to be executed on the first cycle of the load, leaving the processor with nothing to do until the load completes.

**PROCEDURAL DEPENDENCIES** As was discussed in Chapter 14, the presence of branches in an instruction sequence complicates the pipeline operation. The instructions following a branch (taken or not taken) have a **procedural dependency** on the branch and cannot be executed until the branch is executed. Figure 16.3 illustrates the effect of a branch on a superscalar pipeline of degree 2.

As we have seen, this type of procedural dependency also affects a scalar pipeline. The consequence for a superscalar pipeline is more severe, because a greater magnitude of opportunity is lost with each delay.

If variable-length instructions are used, then another sort of procedural dependency arises. Because the length of any particular instruction is not known, it must be at least partially decoded before the following instruction can be fetched. This prevents the simultaneous fetching required in a superscalar pipeline. This is one of the reasons that superscalar techniques are more readily applicable to a RISC or RISC-like architecture, with its fixed instruction length.

**RESOURCE CONFLICT** A **resource conflict** is a competition of two or more instructions for the same resource at the same time. Examples of resources include memories, caches, buses, register-file ports, and functional units (e.g., ALU adder).

In terms of the pipeline, a resource conflict exhibits similar behavior to a data dependency (Figure 16.3). There are some differences, however. For one thing, resource conflicts can be overcome by duplication of resources, whereas a true data dependency cannot be eliminated. Also, when an operation takes a long time to complete, resource conflicts can be minimized by pipelining the appropriate functional unit.

## 16.2 DESIGN ISSUES

### Instruction-Level Parallelism and Machine Parallelism

[JOUP89a] makes an important distinction between the two related concepts of instruction-level parallelism and machine parallelism. **Instruction-level parallelism** exists when instructions in a sequence are independent and thus can be executed in parallel by overlapping.

As an example of the concept of instruction-level parallelism, consider the following two code fragments [JOUP89b]:

|                  |                  |
|------------------|------------------|
| Load R1 ← R2     | Add R3 ← R3, "1" |
| Add R3 ← R3, "1" | Add R4 ← R3, R2  |
| Add R4 ← R4, R2  | Store [R4] ← R0  |

The three instructions on the left are independent, and in theory all three could be executed in parallel. In contrast, the three instructions on the right cannot be executed in parallel because the second instruction uses the result of the first, and the third instruction uses the result of the second.

The degree of instruction-level parallelism is determined by the frequency of true data dependencies and procedural dependencies in the code. These factors, in turn, are dependent on the instruction set architecture and on the application. Instruction-level parallelism is also determined by what [JOUP89a] refers to as operation latency: the time until the result of an instruction is available for use as an operand in a subsequent instruction. The latency determines how much of a delay a data or procedural dependency will cause.

**Machine parallelism** is a measure of the ability of the processor to take advantage of instruction-level parallelism. Machine parallelism is determined by the number of instructions that can be fetched and executed at the same time (the number of parallel pipelines) and by the speed and sophistication of the mechanisms that the processor uses to find independent instructions.

Both instruction-level and machine parallelism are important factors in enhancing performance. A program may not have enough instruction-level parallelism to take full advantage of machine parallelism. The use of a fixed-length instruction set architecture, as in a RISC, enhances instruction-level parallelism. On the other hand, limited machine parallelism will limit performance no matter what the nature of the program.

### Instruction Issue Policy

As was mentioned, machine parallelism is not simply a matter of having multiple instances of each pipeline stage. The processor must also be able to identify instruction-level parallelism and orchestrate the fetching, decoding, and execution of instructions in parallel. [JOHN91] uses the term **instruction issue** to refer to the process of initiating instruction execution in the processor's functional units and the term **instruction issue policy** to refer to the protocol used to issue instructions. In general, we can say that instruction issue occurs when instruction moves from the decode stage of the pipeline to the first execute stage of the pipeline.

In essence, the processor is trying to look ahead of the current point of execution to locate instructions that can be brought into the pipeline and executed. Three types of orderings are important in this regard:

- The order in which instructions are fetched
- The order in which instructions are executed
- The order in which instructions update the contents of register and memory locations

The more sophisticated the processor, the less it is bound by a strict relationship between these orderings. To optimize utilization of the various pipeline elements, the processor will need to alter one or more of these orderings with respect to the ordering to be found in a strict sequential execution. The one constraint on the processor is that the result must be correct. Thus, the processor must accommodate the various dependencies and conflicts discussed earlier.

In general terms, we can group superscalar instruction issue policies into the following categories:

- In-order issue with in-order completion
- In-order issue with **out-of-order** completion
- Out-of-order issue with out-of-order completion

**IN-ORDER ISSUE WITH IN-ORDER COMPLETION** The simplest instruction issue policy is to issue instructions in the exact order that would be achieved by sequential execution (**in-order issue**) and to write results in that same order (**in-order completion**). Not even scalar pipelines follow such a simple-minded policy. However, it is useful to consider this policy as a baseline for comparing more sophisticated approaches.

Figure 16.4a gives an example of this policy. We assume a superscalar pipeline capable of fetching and decoding two instructions at a time, having three separate functional units (e.g., two integer arithmetic and one floating-point arithmetic), and having two instances of the write-back pipeline stage. The example assumes the following constraints on a six-instruction code fragment:

- I1 requires two cycles to execute.
- I3 and I4 conflict for the same functional unit.
- I5 depends on the value produced by I4.
- I5 and I6 conflict for a functional unit.

Instructions are fetched two at a time and passed to the decode unit. Because instructions are fetched in pairs, the next two instructions must wait until the pair of decode pipeline stages has cleared. To guarantee **in-order completion**, when there is a conflict for a functional unit or when a functional unit requires more than one cycle to generate a result, the issuing of instructions temporarily stalls.

In this example, the elapsed time from decoding the first instruction to writing the last results is eight cycles.

**IN-ORDER ISSUE WITH OUT-OF-ORDER COMPLETION** Out-of-order completion is used in scalar RISC processors to improve the performance of instructions that require multiple cycles. Figure 16.4b illustrates its use on a superscalar processor. Instruction I2 is allowed to run to completion prior to I1. This allows I3 to be completed earlier, with the net result of a savings of one cycle.

With out-of-order completion, any number of instructions may be in the execution stage at any one time, up to the maximum degree of machine parallelism across all functional units. Instruction issuing is stalled by a resource conflict, a data dependency, or a procedural dependency.

In addition to the aforementioned limitations, a new dependency, which we referred to earlier as an **output dependency** (also called **write after write [WAW] dependency**), arises. The following code fragment illustrates this dependency (*op* represents any operation):

```

I1: R3 ← R3 op R5
I2: R4 ← R3 + 1
I3: R3 ← R5 + 1
I4: R7 ← R3 op R4

```



| Decode |    | Execute |    |    | Write |    | Cycle |
|--------|----|---------|----|----|-------|----|-------|
| I1     | I2 |         |    |    |       |    | 1     |
| I3     | I4 | I1      | I2 |    |       |    | 2     |
| I3     | I4 | I1      |    |    |       |    | 3     |
|        | I4 |         |    | I3 | I1    | I2 | 4     |
| I5     | I6 |         |    | I4 |       |    | 5     |
|        | I6 |         | I5 |    | I3    | I4 | 6     |
|        |    |         | I6 |    |       |    | 7     |
|        |    |         |    |    | I5    | I6 | 8     |

(a) In-order issue and in-order completion

| Decode |    | Execute |    |    | Write |    | Cycle |
|--------|----|---------|----|----|-------|----|-------|
| I1     | I2 |         |    |    |       |    | 1     |
| I3     | I4 | I1      | I2 |    |       |    | 2     |
|        | I4 | I1      |    | I3 | I2    |    | 3     |
| I5     | I6 |         |    | I4 | I1    | I3 | 4     |
|        | I6 |         | I5 |    | I4    |    | 5     |
|        |    |         | I6 |    | I5    |    | 6     |
|        |    |         |    |    | I6    |    | 7     |

(b) In-order issue and out-of-order completion

| Decode |    | Window     | Execute |    |    | Write |    | Cycle |
|--------|----|------------|---------|----|----|-------|----|-------|
| I1     | I2 |            |         |    |    |       |    | 1     |
| I3     | I4 | I1, I2     | I1      | I2 |    |       |    | 2     |
| I5     | I6 | I3, I4     | I1      |    | I3 | I2    |    | 3     |
|        |    | I4, I5, I6 |         | I6 | I4 | I1    | I3 | 4     |
|        |    | I5         |         | I5 |    | I4    | I6 | 5     |
|        |    |            |         |    |    | I5    |    | 6     |

(c) Out-of-order issue and out-of-order completion

**Figure 16.4** Superscalar Instruction Issue and Completion Policies

Instruction I2 cannot execute before instruction I1, because it needs the result in register R3 produced in I1; this is an example of a true data dependency, as described in Section 16.1. Similarly, I4 must wait for I3, because it uses a result produced by I3. What about the relationship between I1 and I3? There is no data dependency here, as we have defined it. However, if I3 executes to completion prior to I1, then the wrong value of the contents of R3 will be fetched for the execution of I4. Consequently, I3 must complete after I1 to produce the correct output values. To ensure this, the issuing of the third instruction must be stalled if its result might later be overwritten by an older instruction that takes longer to complete.

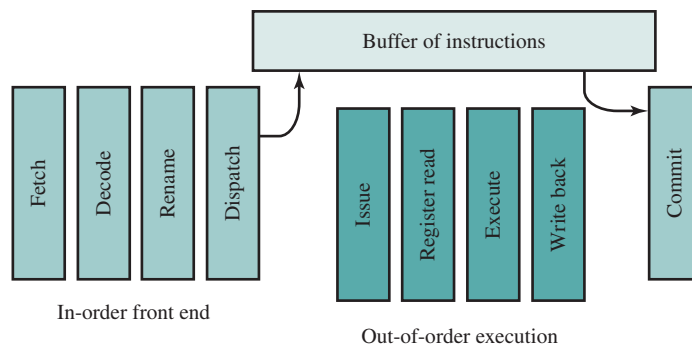
Out-of-order completion requires more complex instruction issue logic than in-order completion. In addition, it is more difficult to deal with instruction interrupts and exceptions. When an interrupt occurs, instruction execution at the current point is suspended, to be resumed later. The processor must assure that the resumption takes into account that, at the time of interruption, instructions ahead of the instruction that caused the interrupt may already have completed.

**OUT-OF-ORDER ISSUE WITH OUT-OF-ORDER COMPLETION** With in-order issue, the processor will only decode instructions up to the point of a dependency or conflict. No additional instructions are decoded until the conflict is resolved. As a result, the processor cannot look ahead of the point of conflict to subsequent instructions that may be independent of those already in the pipeline and that may be usefully introduced into the pipeline.

To allow **out-of-order issue**, it is necessary to decouple the decode and execute stages of the pipeline. This is done with a buffer referred to as an **instruction window**. With this organization, after a processor has finished decoding an instruction, it is placed in the instruction window. As long as this buffer is not full, the processor can continue to fetch and decode new instructions. When a functional unit becomes available in the execute stage, an instruction from the instruction window may be issued to the execute stage. Any instruction may be issued, provided that (1) it needs the particular functional unit that is available, and (2) no conflicts or dependencies block this instruction. Figure 16.5 suggests this organization.

The result of this organization is that the processor has a lookahead capability, allowing it to identify independent instructions that can be brought into the execute stage. Instructions are issued from the instruction window with little regard for their original program order. As before, the only constraint is that the program execution behaves correctly.

Figures 16.4c illustrates this policy. During each of the first three cycles, two instructions are fetched into the decode stage. During each cycle, subject to the constraint of the buffer size, two instructions move from the decode stage to the instruction window. In this example, it is possible to issue instruction I6 ahead of I5 (recall that I5 depends on I4, but I6 does not). Thus, one cycle is saved in both the execute and write-back stages, and the end-to-end savings, compared with Figure 16.4b, is one cycle.



**Figure 16.5** Organization for Out-of-Order Issue with Out-of-Order Completion

The instruction window is depicted in Figure 16.4c to illustrate its role. However, this window is not an additional pipeline stage. An instruction being in the window simply implies that the processor has sufficient information about that instruction to decide when it can be issued.

The out-of-order issue, out-of-order completion policy is subject to the same constraints described earlier. An instruction cannot be issued if it violates a dependency or conflict. The difference is that more instructions are available for issuing, reducing the probability that a pipeline stage will have to stall. In addition, a new dependency, which we referred to earlier as an **antidependency** (also called **write after read [WAR] dependency**), arises. The code fragment considered earlier illustrates this dependency:

```
I1: R3 ← R3 op R5
I2: R4 ← R3 + 1
I3: R3 ← R5 + 1
I4: R7 ← R3 op R4
```

Instruction I3 cannot complete execution before instruction I2 begins execution and has fetched its operands. This is so because I3 updates register R3, which is a source operand for I2. The term *antidependency* is used because the constraint is similar to that of a true data dependency, but reversed: Instead of the first instruction producing a value that the second instruction uses, the second instruction destroys a value that the first instruction uses.



**Reorder Buffer Simulator**  
**Tomasulo's Algorithm Simulator**  
**Alternative Simulation of Tomasulo's Algorithm**

One common technique that is used to support out-of-order completion is the reorder buffer. The reorder buffer is temporary storage for results completed out of order that are then committed to the register file in program order. A related concept is Tomasulo's algorithm. Appendix I examines these concepts.

## Register Renaming

When out-of-order instruction issuing and/or out-of-order instruction completion are allowed, we have seen that this gives rise to the possibility of WAW dependencies and WAR dependencies. These dependencies differ from RAW data dependencies and resource conflicts, which reflect the flow of data through a program and the sequence of execution. WAW dependencies and WAR dependencies, on the other hand, arise because the values in registers may no longer reflect the sequence of values dictated by the program flow.

When instructions are issued in sequence and complete in sequence, it is possible to specify the contents of each register at each point in the execution. When out-of-order techniques are used, the values in registers cannot be fully known at each point in time just from a consideration of the sequence of instructions dictated

by the program. In effect, values are in conflict for the use of registers, and the processor must resolve those conflicts by occasionally stalling a pipeline stage.

Antidependencies and output dependencies are both examples of storage conflicts. Multiple instructions are competing for the use of the same register locations, generating pipeline constraints that retard performance. The problem is made more acute when register optimization techniques are used (as discussed in Chapter 15), because these compiler techniques attempt to maximize the use of registers, hence maximizing the number of storage conflicts.

One method for coping with these types of storage conflicts is based on a traditional resource-conflict solution: duplication of resources. In this context, the technique is referred to as **register renaming**. In essence, registers are allocated dynamically by the processor hardware, and they are associated with the values needed by instructions at various points in time. When a new register value is created (i.e., when an instruction executes that has a register as a destination operand), a new register is allocated for that value. Subsequent instructions that access that value as a source operand in that register must go through a renaming process: the register references in those instructions must be revised to refer to the register containing the needed value. Thus, the same original register reference in several different instructions may refer to different actual registers, if different values are intended.

Let us consider how register renaming could be used on the code fragment we have been examining:

```

I1: R3b ← R3a op R5a
I2: R4b ← R3b + 1
I3: R3c ← R5a + 1
I4: R7b ← R3c op R4b

```

The register reference without the subscript refers to the logical register reference found in the instruction. The register reference with the subscript refers to a hardware register allocated to hold a new value. When a new allocation is made for a particular logical register, subsequent instruction references to that logical register as a source operand are made to refer to the most recently allocated hardware register (recent in terms of the program sequence of instructions).

In this example, the creation of register R3<sub>c</sub> in instruction I3 avoids the WAR dependency on the second instruction and the WAW on the first instruction, and it does not interfere with the correct value being accessed by I4. The result is that I3 can be issued immediately; without renaming, I3 cannot be issued until the first instruction is complete and the second instruction is issued.



### Scoreboarding Simulator

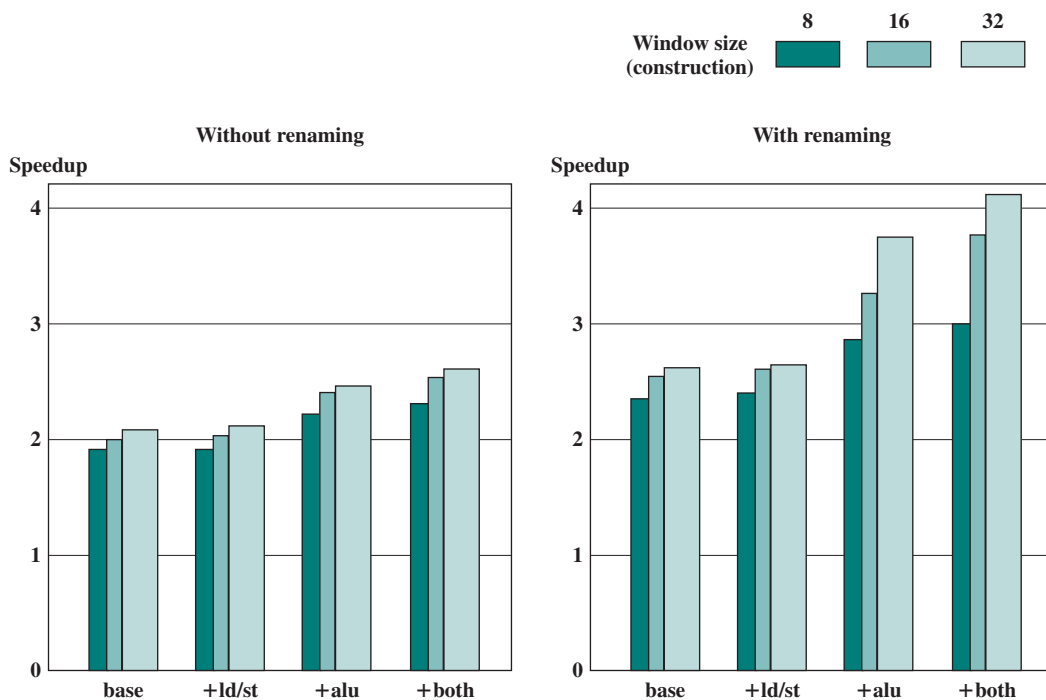
An alternative to register renaming is a scoreboarding. In essence, scoreboarding is a bookkeeping technique that allows instructions to execute whenever they are not dependent on previous instructions and no structural hazards are present. See Appendix I for a discussion.

## Machine Parallelism

In the preceding discussion, we have looked at three hardware techniques that can be used in a superscalar processor to enhance performance: duplication of resources, out-of-order issue, and renaming. One study that illuminates the relationship among these techniques was reported in [SMIT89]. The study made use of a simulation that modeled a machine with the characteristics of the MIPS R2000, augmented with various superscalar features. A number of different program sequences were simulated.

Figure 16.6 shows the results. In each of the graphs, the vertical axis corresponds to the mean speedup of the superscalar machine over the scalar machine. The horizontal axis shows the results for four alternative processor organizations. The base machine does not duplicate any of the functional units, but it can issue instructions out of order. The second configuration duplicates the load/store functional unit that accesses a data cache. The third configuration duplicates the ALU, and the fourth configuration duplicates both load/store and ALU. In each graph, results are shown for instruction window sizes of 8, 16, and 32 instructions, which dictates the amount of lookahead the processor can do. The difference between the two graphs is that, in the second, register renaming is allowed. This is equivalent to saying that the first graph reflects a machine that is limited by all dependencies, whereas the second graph corresponds to a machine that is limited only by true dependencies.

The two graphs, combined, yield some important conclusions. The first is that it is probably not worthwhile to add functional units without register renaming.



**Figure 16.6** Speedups of Various Machine Organizations without Procedural Dependencies

There is some slight improvement in performance, but at the cost of increased hardware complexity. With register renaming, which eliminates antidependencies and output dependencies, noticeable gains are achieved by adding more functional units. Note, however, that there is a significant difference in the amount of gain achievable between using an instruction window of 8 versus a larger instruction window. This indicates that if the instruction window is too small, data dependencies will prevent effective utilization of the extra functional units; the processor must be able to look quite far ahead to find independent instructions to utilize the hardware more fully.



### Pipeline with Static vs. Dynamic Scheduling—Simulator

#### Branch Prediction

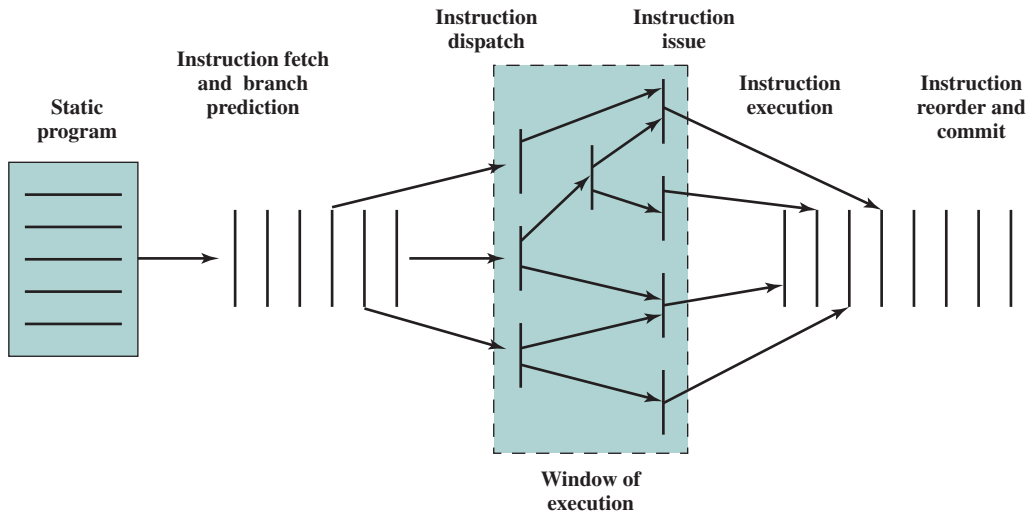
Any high-performance pipelined machine must address the issue of dealing with branches. For example, the Intel 80486 addressed the problem by fetching both the next sequential instruction after a branch and speculatively fetching the branch target instruction. However, because there are two pipeline stages between prefetch and execution, this strategy incurs a two-cycle delay when the branch gets taken.

With the advent of RISC machines, the delayed branch strategy was explored. This allows the processor to calculate the result of conditional branch instructions before any unusable instructions have been prefetched. With this method, the processor always executes the single instruction that immediately follows the branch. This keeps the pipeline full while the processor fetches a new instruction stream.

With the development of superscalar machines, the delayed branch strategy has less appeal. The reason is that multiple instructions need to execute in the delay slot, raising several problems relating to instruction dependencies. Thus, superscalar machines have returned to pre-RISC techniques of **branch prediction**. Some, like the PowerPC 601, use a simple static branch prediction technique. More sophisticated processors, such as the PowerPC 620 and the Pentium 4, use dynamic branch prediction based on branch history analysis.

#### Superscalar Execution

We are now in a position to provide an overview of superscalar execution of programs; this is illustrated in Figure 16.7. The program to be executed consists of a linear sequence of instructions. This is the static program as written by the programmer or generated by the compiler. The instruction fetch stage, which includes branch prediction, is used to form a dynamic stream of instructions. This stream is examined for dependencies, and the processor may remove artificial dependencies. The processor then dispatches the instructions into a window of execution. In this window, instructions no longer form a sequential stream but are structured according to their true data dependencies. The processor executes each instruction in an order determined by the true data dependencies and hardware resource availability. Finally, instructions are conceptually put back into sequential order and their results are recorded.



**Figure 16.7** Conceptual Depiction of Superscalar Processing

The final step mentioned in the preceding paragraph is referred to as **committing**, or **retiring**, the instruction. This step is needed for the following reason. Because of the use of parallel, multiple pipelines, instructions may complete in an order different from that shown in the static program. Further, the use of branch prediction and speculative execution means that some instructions may complete execution and then must be abandoned because the branch they represent is not taken. Therefore, permanent storage and program-visible registers cannot be updated immediately when instructions complete execution. Results must be held in some sort of temporary storage that is usable by dependent instructions and then made permanent when it is determined that the sequential model would have executed the instruction.

### Superscalar Implementation

Based on our discussion so far, we can make some general comments about the processor hardware required for the superscalar approach. [SMIT95] lists the following key elements:

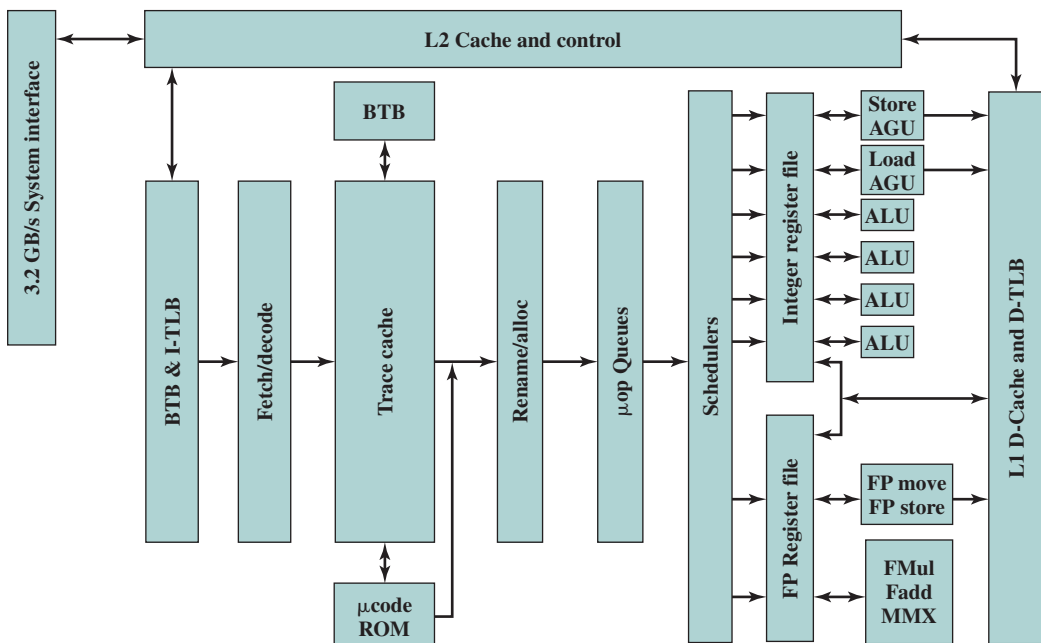
- Instruction fetch strategies that simultaneously fetch multiple instructions, often by predicting the outcomes of, and fetching beyond, conditional branch instructions. These functions require the use of multiple pipeline fetch and decode stages, and branch prediction logic.
- Logic for determining true dependencies involving register values, and mechanisms for communicating these values to where they are needed during execution.
- Mechanisms for initiating, or issuing, multiple instructions in parallel.
- Resources for parallel execution of multiple instructions, including multiple pipelined functional units and memory hierarchies capable of simultaneously servicing multiple memory references.
- Mechanisms for committing the process state in correct order.

## 16.3 PENTIUM 4

Although the concept of superscalar design is generally associated with the RISC architecture, the same superscalar principles can be applied to a CISC machine. Perhaps the most notable example of this is the Pentium. The evolution of superscalar concepts in the Intel line is interesting to note. The 386 is a traditional CISC non-pipelined machine. The 486 introduced the first pipelined x86 processor, reducing the average latency of integer operations from between two and four cycles to one cycle, but still limited to executing a single instruction each cycle, with no superscalar elements. The original Pentium had a modest superscalar component, consisting of the use of two separate integer execution units. The Pentium Pro introduced a full-blown superscalar design with out-of-order execution. Subsequent x86 models have refined and enhanced the superscalar design.

A general block diagram of the Pentium 4 was shown in Figure 4.18. Figure 16.8 depicts the same structure in a way more suitable for the pipeline discussion in this section. The operation of the Pentium 4 can be summarized as follows:

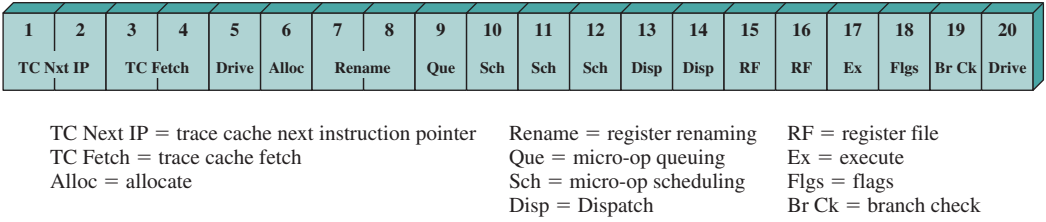
1. The processor fetches instructions from memory in the order of the static program.
2. Each instruction is translated into one or more fixed-length RISC instructions, known as **micro-operations**, or **micro-ops**.



AGU = address generation unit  
 BTB = branch target buffer  
 D-TLB = data translation lookaside buffer  
 I-TLB = instruction translation lookaside buffer

**Figure 16.8** Pentium 4 Block Diagram





**Figure 16.9** Pentium 4 Pipeline

3. The processor executes the micro-ops on a superscalar pipeline organization, so that the micro-ops may be executed out of order.
4. The processor commits the results of each micro-op execution to the processor's register set in the order of the original program flow.

In effect, the Pentium 4 architecture implements a CISC instruction set architecture on a RISC microarchitecture. The inner RISC micro-ops pass through a pipeline with at least 20 stages (Figure 16.9); in some cases, the micro-op requires multiple execution stages, resulting in an even longer pipeline. This contrasts with the five-stage pipeline (Figure 14.21) used on the earlier Intel x86 processors and on the Pentium.

We now trace the operation of the Pentium 4 pipeline, using Figure 16.10 to illustrate its operation.

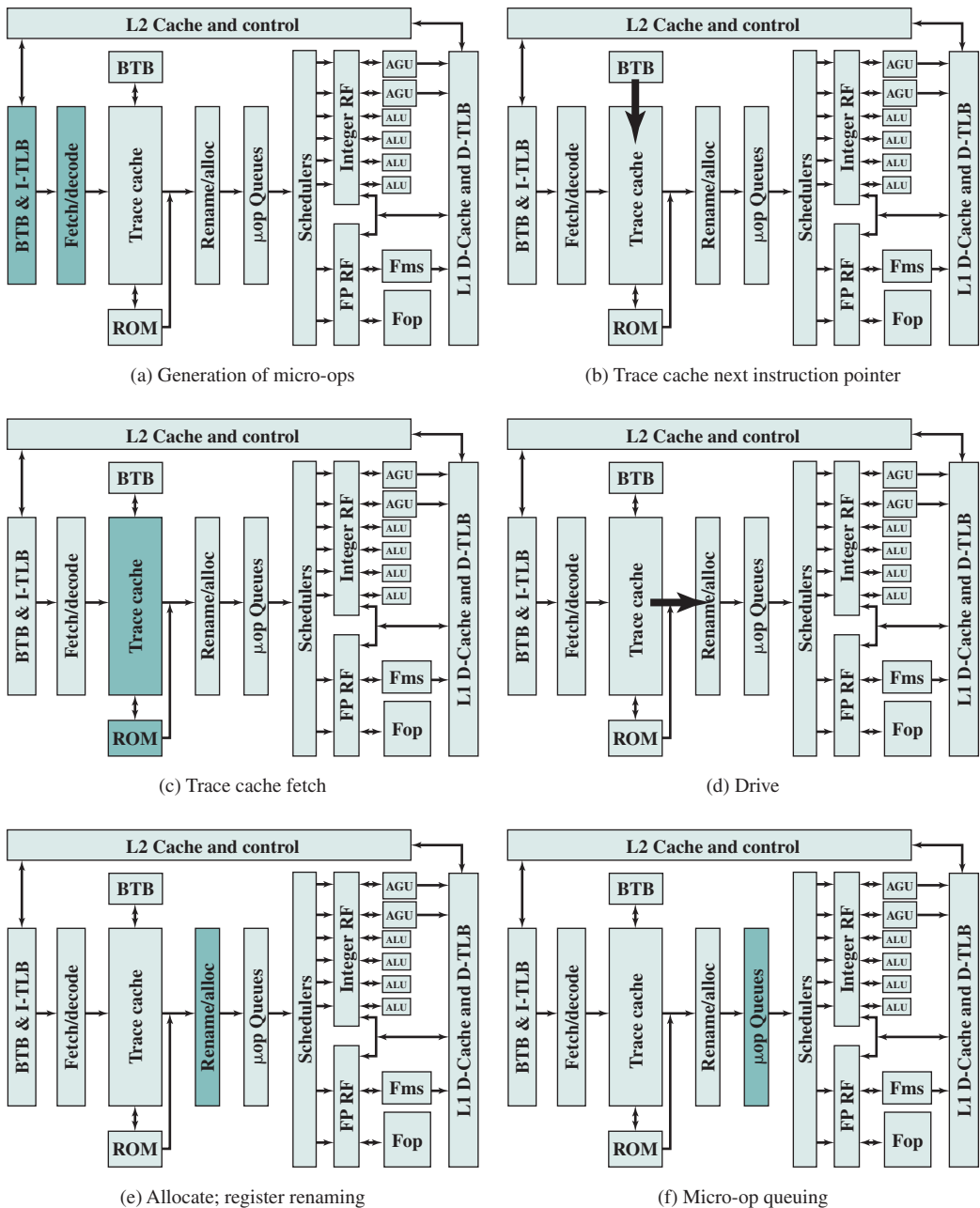
### Front End

**GENERATION OF MICRO-OPS** The Pentium 4 organization includes an in-order front end (Figure 16.10a) that can be considered outside the scope of the pipeline depicted in Figure 16.9. This front end feeds into an L1 instruction cache, called the trace cache, which is where the pipeline proper begins. Usually, the processor operates from the trace cache; when a trace cache miss occurs, the in-order front end feeds new instructions into the trace cache.

With the aid of the branch target buffer and the instruction lookaside buffer (BTB & I-TLB), the fetch/decode unit fetches x86 machine instructions from the L2 cache 64 bytes at a time. As a default, instructions are fetched sequentially, so that each L2 cache line fetch includes the next instruction to be fetched. Branch prediction via the BTB & I-TLB unit may alter this sequential fetch operation. The ITLB translates the linear instruction pointer address given it into physical addresses needed to access the L2 cache. Static branch prediction in the front-end BTB is used to determine which instructions to fetch next.

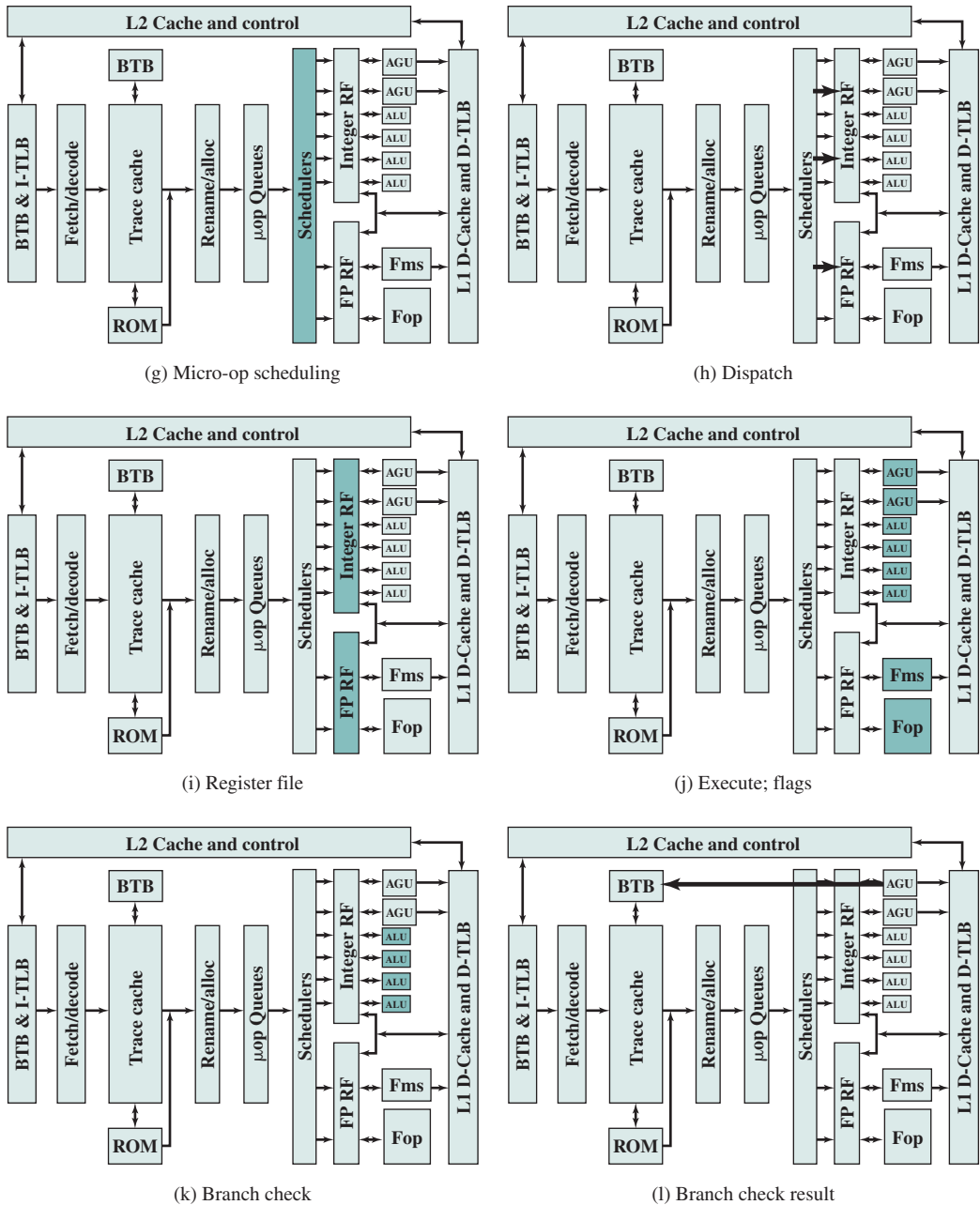
Once instructions are fetched, the fetch/decode unit scans the bytes to determine instruction boundaries; this is a necessary operation because of the variable length of x86 instructions. The decoder translates each machine instruction into from one to four micro-ops, each of which is a 118-bit RISC instruction. Note for comparison that most pure RISC machines have an instruction length of just 32 bits. The longer micro-op length is required to accommodate the more complex x86 instructions. Nevertheless, the micro-ops are easier to manage than the original instructions from which they derive.

The generated micro-ops are stored in the trace cache.



**Figure 16.10** Pentium Pipeline Operation

**TRACE CACHE NEXT INSTRUCTION POINTER** The first two pipeline stages (Figure 16.10b) deal with the selection of instructions in the trace cache and involve a separate branch prediction mechanism from that described in the previous section. The Pentium 4 uses a dynamic branch prediction strategy based on the



**Figure 16.10** Pentium Pipeline Operation (*continued*)

history of recent executions of branch instructions. A branch target buffer (BTB) is maintained that caches information about recently encountered branch instructions. Whenever a branch instruction is encountered in the instruction stream, the BTB is checked. If an entry already exists in the BTB, then the instruction unit is guided

by the history information for that entry in determining whether to predict that the branch is taken. If a branch is predicted, then the branch destination address associated with this entry is used for prefetching the branch target instruction.

Once the instruction is executed, the history portion of the appropriate entry is updated to reflect the result of the branch instruction. If this instruction is not represented in the BTB, then the address of this instruction is loaded into an entry in the BTB; if necessary, an older entry is deleted.

The description of the preceding two paragraphs fits, in general terms, the branch prediction strategy used on the original Pentium model, as well as the later Pentium models, including Pentium 4. However, in the case of the Pentium, a relatively simple 2-bit history scheme is used. The later Pentium models have much longer pipelines (20 stages for the Pentium 4 compared with 5 stages for the Pentium) and therefore the penalty for misprediction is greater. Accordingly, the later Pentium models use a more elaborate branch prediction scheme with more history bits to reduce the misprediction rate.

The Pentium 4 BTB is organized as a four-way set-associative cache with 512 lines. Each entry uses the address of the branch as a tag. The entry also includes the branch destination address for the last time this branch was taken and a 4-bit history field. Thus use of four history bits contrasts with the 2 bits used in the original Pentium and used in most superscalar processors. With 4 bits, the Pentium 4 mechanism can take into account a longer history in predicting branches. The algorithm that is used is referred to as Yeh's algorithm [YEH91]. The developers of this algorithm have demonstrated that it provides a significant reduction in misprediction compared to algorithms that use only 2 bits of history [EVER98].

Conditional branches that do not have a history in the BTB are predicted using a static prediction algorithm, according to the following rules:

- For branch addresses that are not IP relative, predict taken if the branch is a return and not taken otherwise.
- For IP-relative backward conditional branches, predict taken. This rule reflects the typical behavior of loops.
- For IP-relative forward conditional branches, predict not taken.

**TRACE CACHE FETCH** The trace cache (Figure 16.10c) takes the already-decoded micro-ops from the instruction decoder and assembles them in to program-ordered sequences of micro-ops called traces. Micro-ops are fetched sequentially from the trace cache, subject to the branch prediction logic.

A few instructions require more than four micro-ops. These instructions are transferred to microcode ROM, which contains the series of micro-ops (five or more) associated with a complex machine instruction. For example, a string instruction may translate into a very large (even hundreds), repetitive sequence of micro-ops. Thus, the microcode ROM is a microprogrammed control unit in the sense discussed in Part Four. After the microcode ROM finishes sequencing micro-ops for the current Pentium instruction, fetching resumes from the trace cache.

**DRIVE** The fifth stage (Figure 16.10d) of the Pentium 4 pipeline delivers decoded instructions from the trace cache to the rename/allocator module.

## Out-of-Order Execution Logic

This part of the processor reorders micro-ops to allow them to execute as quickly as their input operands are ready.

**ALLOCATE** The allocate stage (Figure 16.10e) allocates resources required for execution. It performs the following functions:

- If a needed resource, such as a register, is unavailable for one of the three micro-ops arriving at the allocator during a clock cycle, the allocator stalls the pipeline.
- The allocator allocates a reorder buffer (ROB) entry, which tracks the completion status of one of the 126 micro-ops that could be in process at any time.<sup>2</sup>
- The allocator allocates one of the 128 integer or floating-point register entries for the result data value of the micro-op, and possibly a load or store buffer used to track one of the 48 loads or 24 stores in the machine pipeline.
- The allocator allocates an entry in one of the two micro-op queues in front of the instruction schedulers.

The ROB is a circular buffer that can hold up to 126 micro-ops and also contains the 128 hardware registers. Each buffer entry consists of the following fields:

- **State:** Indicates whether this micro-op is scheduled for execution, has been dispatched for execution, or has completed execution and is ready for retirement.
- **Memory Address:** The address of the Pentium instruction that generated the micro-op.
- **Micro-op:** The actual operation.
- **Alias Register:** If the micro-op references one of the 16 architectural registers, this entry redirects that reference to one of the 128 hardware registers.

Micro-ops enter the ROB in order. Micro-ops are then dispatched from the ROB to the Dispatch/Execute unit out of order. The criterion for dispatch is that the appropriate execution unit and all necessary data items required for this micro-op are available. Finally, micro-ops are retired from the ROB in order. To accomplish in-order retirement, micro-ops are retired oldest first after each micro-op has been designated as ready for retirement.

**REGISTER RENAMING** The rename stage (Figure 16.10e) remaps references to the 16 architectural registers (8 floating-point registers, plus EAX, EBX, ECX, EDX, ESI, EDI, EBP, and ESP) into a set of 128 physical registers. The stage removes false dependencies caused by a limited number of architectural registers while preserving the true data dependencies (reads after writes).

**MICRO-OP QUEUING** After resource allocation and register renaming, micro-ops are placed in one of two micro-op queues (Figure 16.10f), where they are held until there is room in the schedulers. One of the two queues is for memory operations

<sup>2</sup>See Appendix I for a discussion of reorder buffers.

(loads and stores) and the other for micro-ops that do not involve memory references. Each queue obeys a FIFO (first-in-first-out) discipline, but no order is maintained between queues. That is, a micro-op may be read out of one queue out of order with respect to micro-ops in the other queue. This provides greater flexibility to the schedulers.

**MICRO-OP SCHEDULING AND DISPATCHING** The schedulers (Figure 16.10g) are responsible for retrieving micro-ops from the micro-op queues and dispatching these for execution. Each scheduler looks for micro-ops in whose status indicates that the micro-op has all of its operands. If the execution unit needed by that micro-op is available, then the scheduler fetches the micro-op and dispatches it to the appropriate execution unit (Figure 16.10h). Up to six micro-ops can be dispatched in one cycle. If more than one micro-op is available for a given execution unit, then the scheduler dispatches them in sequence from the queue. This is a sort of FIFO discipline that favors in-order execution, but by this time the instruction stream has been so rearranged by dependencies and branches that it is substantially out of order.

Four ports attach the schedulers to the execution units. Port 0 is used for both integer and floating-point instructions, with the exception of simple integer operations and the handling of branch mispredictions, which are allocated to Port 1. In addition, MMX execution units are allocated between these two ports. The remaining ports are for memory loads and stores.

### Integer and Floating-Point Execution Units

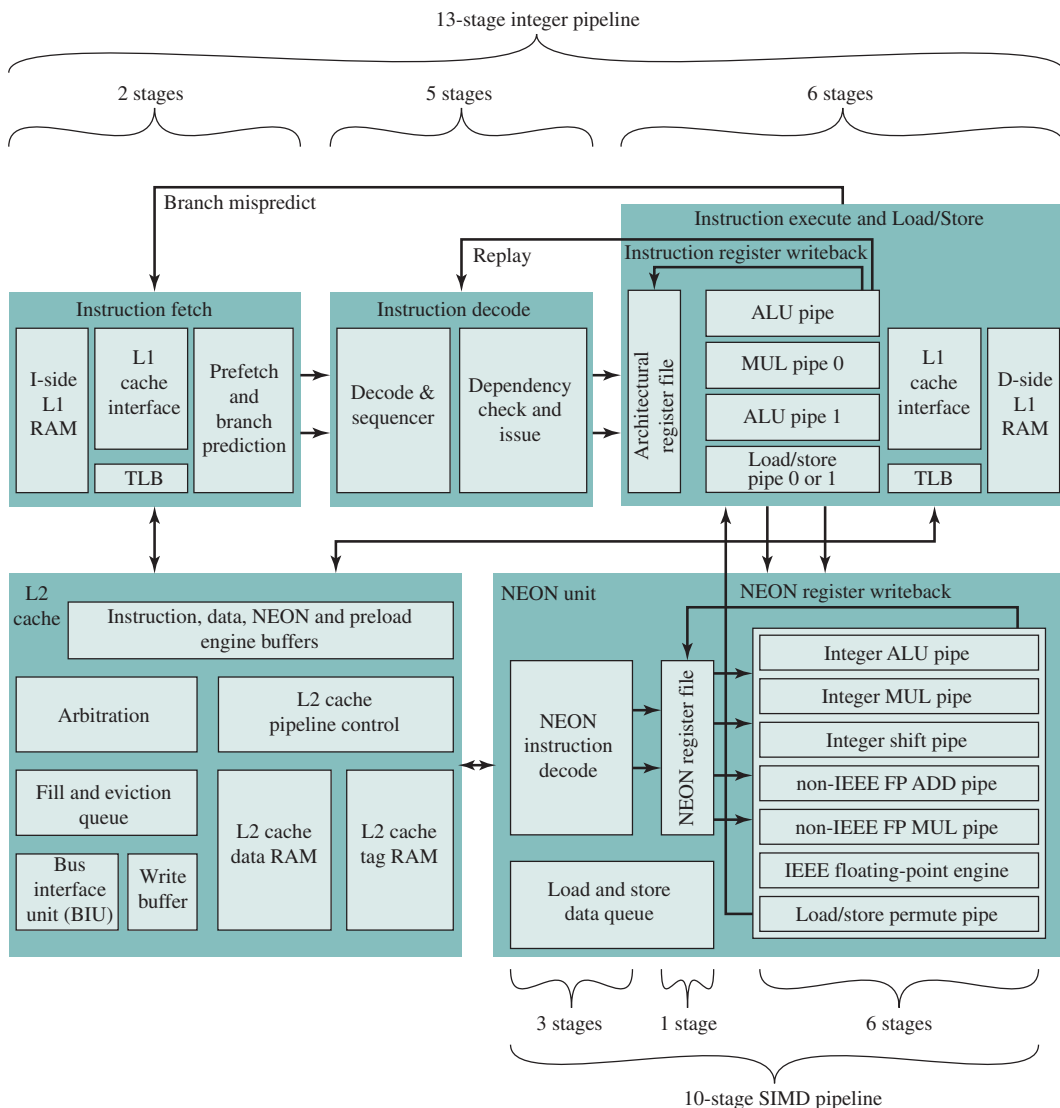
The integer and floating-point register files are the source for pending operations by the execution units (Figure 16.10i). The execution units retrieve values from the register files as well as from the L1 data cache (Figure 16.10j). A separate pipeline stage is used to compute flags (e.g., zero, negative); these are typically the input to a branch instruction.

A subsequent pipeline stage performs branch checking (Figure 16.10k). This function compares the actual branch result with the prediction. If a branch prediction turns out to have been wrong, then there are micro-operations in various stages of processing that must be removed from the pipeline. The proper branch destination is then provided to the Branch Predictor during a drive stage (Figure 16.10l), which restarts the whole pipeline from the new target address.

## 16.4 ARM CORTEX-A8

Recent implementations of the ARM architecture have seen the introduction of superscalar techniques in the instruction pipeline. In this section, we focus on the ARM Cortex-A8, which provides a good example of a RISC-based superscalar design.

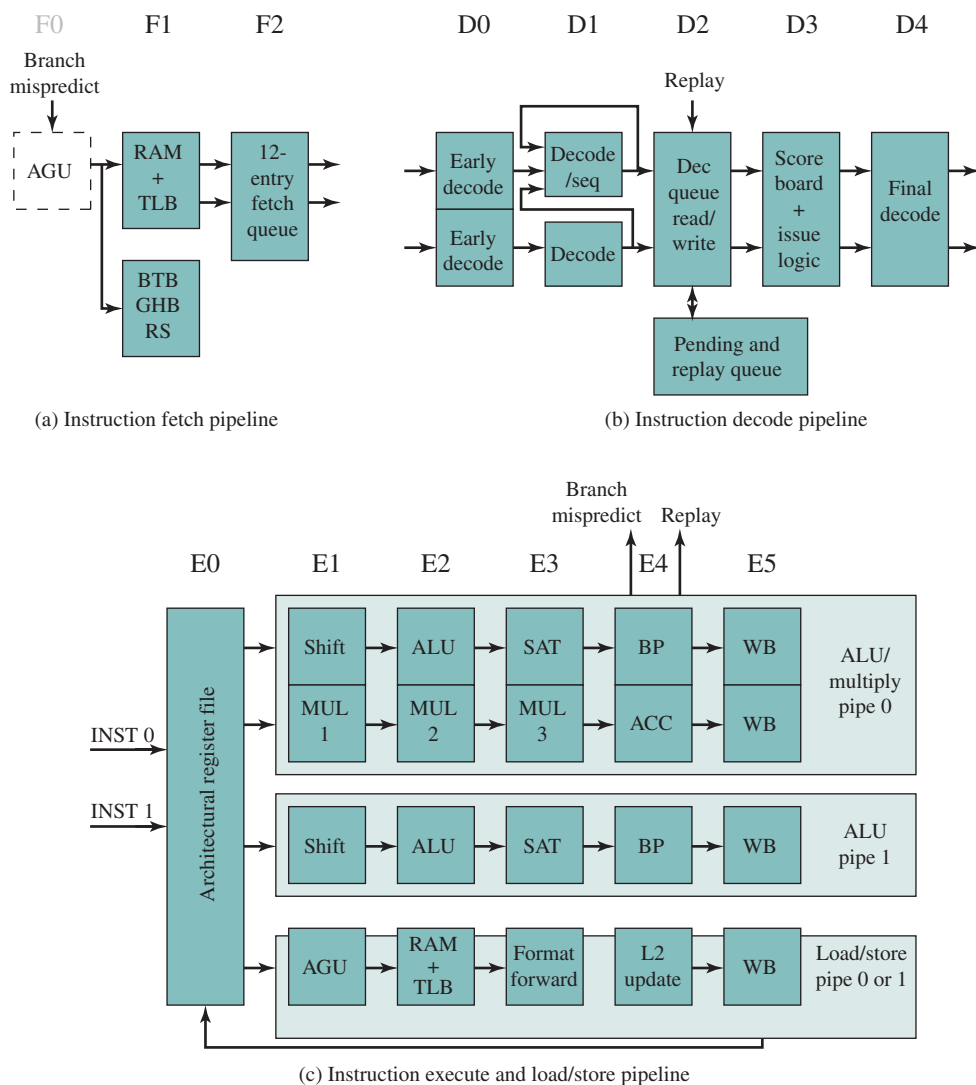
The Cortex-A8 is in the ARM family of processors that ARM refers to as application processors. An ARM application processor is an embedded processor running complex operating systems for wireless, consumer and imaging applications.



**Figure 16.11** Architectural Block Diagram of ARM Cortex-A8

The Cortex-A8 targets a wide variety of mobile and consumer applications including mobile phones, set-top boxes, gaming consoles and automotive navigation/entertainment systems.

Figure 16.11 shows a logical view of the Cortex-A8 architecture, emphasizing the flow of instructions among functional units. The main instruction flow is through three functional units that implement a dual, in-order-issue, 13-stage pipeline. The Cortex designers decided to stay with in-order issue to keep additional power required to a minimum. Out-of-order issue and **retire** can require extensive amounts of logic consuming extra power.



**Figure 16.12** ARM Cortex-A8 Integer Pipeline

Figure 16.12 shows the details of the main Cortex-A8 pipeline. There is a separate unit for SIMD (single-instruction-multiple-data) unit that implements a 10-stage pipeline.

### Instruction Fetch Unit

The instruction fetch unit predicts the instruction stream, fetches instructions from the L1 instruction cache, and places the fetched instructions into a buffer for consumption by the decode pipeline. The instruction fetch unit also includes the L1 instruction cache. Because there can be several unresolved branches in the pipeline, instruction fetches are speculative, meaning there is no guarantee that they are



executed. A branch or exceptional instruction in the code stream can cause a pipeline flush, discarding the currently fetched instructions. The instruction fetch unit can fetch up to four instructions per cycle, and goes through the following stages:

**F0** The address generation unit (AGU) generates a new virtual address. Normally, this address is the next address sequentially from the preceding fetch address. The address can also be a branch target address provided by a branch prediction for a previous instruction. F0 is not counted as part of the 13-stage pipeline, because ARM processors have traditionally defined instruction cache access as the first stage.

**F1** The calculated address is used to fetch instructions from the L1 instruction cache. In parallel, the fetch address is used to access the branch prediction arrays to determine if the next fetch address should be based on a branch prediction.

**F3** Instruction data are placed into the instruction queue. If an instruction results in branch prediction, the new target address is sent to the address generation unit.

To minimize the branch penalties typically associated with a deeper pipeline, the Cortex-A8 processor implements a two-level global history branch predictor, consisting of the branch target buffer (BTB) and the global history buffer (GHB). These data structures are accessed in parallel with instruction fetches. The BTB indicates whether or not the current fetch address will return a branch instruction and its branch target address. It contains 512 entries. On a hit in the BTB a branch is predicted and the GHB is accessed. The GHB consists of 4096 2-bit counters that encode the strength and direction information of branches. The GHB is indexed by 10-bit history of the direction of the last ten branches encountered and 4 bits of the PC. In addition to the dynamic branch predictor, a return stack is used to predict subroutine return addresses. The return stack has eight 32-bit entries that store the link register value in r14 and the ARM or Thumb state of the calling function. When a return-type instruction is predicted taken, the return stack provides the last pushed address and state.

The instruction fetch unit can fetch and queue up to 12 instructions. It issues instructions to the decode unit two at a time. The queue enables the instruction fetch unit to prefetch ahead of the rest of the integer pipeline and build up a backlog of instructions ready for decoding.

## Instruction Decode Unit

The instruction decode unit decodes and sequences all ARM and Thumb instructions. It has a dual pipeline structure, called *pipe0* and *pipe1*, so that two instructions can progress through the unit at a time. When two instructions are issued from the instruction decode pipeline, *pipe0* will always contain the older instruction in program order. This means that if the instruction in *pipe0* cannot issue, then the instruction in *pipe1* will not issue. All issued instructions progress in order down the execution pipeline with results written back into the register file at the end of the execution pipeline. This in-order instruction issue and retire prevents WAR hazards and keeps tracking of WAW hazards and recovery from flush conditions

straightforward. Thus, the main concern of the instruction decode pipeline is the prevention of RAW hazards.

Each instruction goes through five stages of processing.

**D0** Thumb instructions are decompressed into 32-bit ARM instructions. A preliminary decode function is performed.

**D1** The instruction decode function is completed.

**D2** This stage writes instructions into and read instructions from the pending/replay queue structure.

**D3** This stage contains the instruction scheduling logic. A scoreboard predicts register availability using static scheduling techniques.<sup>3</sup> Hazard checking is also done at this stage.

**D4** Performs the final decode for all the control signals required by the integer execute and load/store units.

In the first two stages, the instruction type, the source and destination operands, and resource requirements for the instruction are determined. A few less commonly used instructions are referred to as multicycle instructions. The D1 stage breaks these instructions down into multiple instruction opcodes that are sequenced individually through the execution pipeline.

The pending queue serves two purposes. First, it prevents a stall signal from D3 from rippling any further up the pipeline. Second, by buffering instructions, there should always be two instructions available for the dual pipeline. In the case where only one instruction is issued, the pending queue enables two instructions to proceed down the pipeline together, even if they were originally sent from the fetch unit in different cycles.

The replay operation is designed to deal with the effects of the memory system on instruction timing. Instructions are statically scheduled in the D3 stage based on a prediction of when the source operand will be available. Any stall from the memory system can result in the minimum of an 8-cycle delay. This 8-cycle delay minimum is balanced with the minimum number of possible cycles to receive data from the L2 cache in the case of an L1 load miss. Table 16.2 gives the most common cases that can result in an instruction replay because of a memory system stall.

To deal with these stalls, a recovery mechanism is used to flush all subsequent instructions in the execution pipeline and reissue (replay) them. To support replay, instructions are copied into the replay queue before they are issued and removed as they write back their results and retire. If a replay signal is issued instructions are retrieved from the replay queue and reenter the pipeline.

The decode unit issues two instructions in parallel to the execution unit, unless it encounters an issue restriction. Table 16.3 shows the most common restriction cases.

## Integer Execute Unit

The instruction execute unit consists of two symmetric arithmetic logic unit (ALU) pipelines, an address generator for load and store instructions, and the multiply

<sup>3</sup>See Appendix I for a discussion of scoreboarding.

**Table 16.2** Cortex-A8 Memory System Effects on Instruction Timings

| Replay Event                    | Delay                                      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Load data miss                  | 8 cycles                                   | <ol style="list-style-type: none"> <li>1. A load instruction misses in the L1 data cache.</li> <li>2. A request is then made to the L2 data cache.</li> <li>3. If a miss also occurs in the L2 data cache, then a second replay occurs. The number of stall cycles depends on the external system memory timing. The minimum time required to receive the critical word for an L2 cache miss is approximately 25 cycles, but can be much longer because of L3 memory latencies.</li> </ol> |
| Data TLB miss                   | 24 cycles                                  | <ol style="list-style-type: none"> <li>1. A table walk because of a miss in the L1 TLB causes a 24-cycle delay, assuming the translation table entries are found in the L2 cache.</li> <li>2. If the translation table entries are not present in the L2 cache, the number of stall cycles depends on the external system memory timing.</li> </ol>                                                                                                                                        |
| Store buffer full               | 8 cycles plus latency to drain fill buffer | <ol style="list-style-type: none"> <li>1. A store instruction miss does not result in any stalls unless the store buffer is full.</li> <li>2. In the case of a full store buffer, the delay is at least eight cycles. The delay can be more if it takes longer to drain some entries from the store buffer.</li> </ol>                                                                                                                                                                     |
| Unaligned load or store request | 8 cycles                                   | <ol style="list-style-type: none"> <li>1. If a load instruction address is unaligned and the full access is not contained within a 128-bit boundary, there is a 8-cycle penalty.</li> <li>2. If a store instruction address is unaligned and the full access is not contained within a 64-bit boundary, there is a 8-cycle penalty.</li> </ol>                                                                                                                                             |

pipeline. The execute pipelines also perform register write back. The instruction execute unit:

- Executes all integer ALU and multiply operations, including flag generation
- Generates the virtual addresses for loads and stores and the base write-back value, when required
- Supplies formatted data for stores and forwards data and flags
- Processes branches and other changes of instruction stream and evaluates instruction condition codes

For ALU instructions, either pipeline can be used, consisting of the following stages:

**E0** Access register file. Up to six registers can be read from the register file for two instructions.

**E1** The barrel shifter (see Figure 14.25) performs its function, if needed.

**E2** The ALU unit (see Figure 14.25) performs its function.

**E3** If needed, this stage completes saturation arithmetic used by some ARM data processing instructions.

**Table 16.3** Cortex-A8 Dual-Issue Restrictions

| Restriction Type           | Description                                                                                                                         | Example                                                                                | Cycle                     | Restriction                                                                                                |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------|
| Load/store resource hazard | There is only one LS pipeline. Only one LS instruction can be issued per cycle. It can be in pipeline 0 or pipeline 1               | LDR r5, [r6]<br>STR r7, [r8]<br>MOV r9, r10                                            | 1<br>2<br>2               | Wait for LS unit<br>Dual issue possible                                                                    |
| Multiply resource hazard   | There is only one multiply pipeline, and it is only available in pipeline 0.                                                        | ADD r1, r2, r3<br>MUL r4, r5, r6<br>MUL r7, r8, r9                                     | 1<br>2<br>3               | Wait for pipeline 0<br>Wait for multiply unit                                                              |
| Branch resource hazard     | There can be only one branch per cycle. It can be in pipeline 0 or pipeline 1. A branch is any instruction that changes the PC.     | BX r1<br>BEQ 0x1000<br>ADD r1, r2, r3                                                  | 1<br>2<br>2               | Wait for branch<br>Dual issue possible                                                                     |
| Data output hazard         | Instructions with the same destination cannot be issued in the same cycle. This can happen with conditional code.                   | MOVEQ r1, r2<br>MOVNE r1, r3<br>LDR r5, [r6]                                           | 1<br>2<br>2               | Wait because of output dependency<br>Dual issue possible                                                   |
| Data source hazard         | Instructions cannot be issued if their data is not available. See the scheduling tables for source requirements and stages results. | ADD r1, r2, r3<br>ADD r4, r1, r6<br>LDR r7, [r4]                                       | 1<br>2<br>4               | Wait for r1<br>Wait two cycles for r4                                                                      |
| Multi-cycle instructions   | Multi-cycle instructions must issue in pipeline 0 and can only dual issue in their last iteration.                                  | MOV r1, r2<br>LDM r3, {r4-r7}<br>LDM (cycle 2)<br>LDM (cycle 3)<br><br>ADD r8, r9, r10 | 1<br>2<br>3<br>4<br><br>4 | Wait for pipeline 0, transfer r4<br>Transfer r5, r6<br>Transfer r7<br>Dual issue possible on last transfer |

**E4** Any change in control flow, including branch misprediction, exceptions, and memory system replays are prioritized and processed.

**E5** Results of ARM instructions are written back into the register file.

Instructions that invoke the multiply unit (see Figure 14.25) are routed to pipe0; the multiply operation is performed in stages E1 through E3, and the multiply accumulate operation in stage E4.

The load/store pipeline runs parallel to the integer pipeline. The stages are as follows:

**E1** The memory address is generated from the base and index register.

**E2** The address is applied to the cache arrays.

**E3** In the case of a load, data are returned and formatted for forwarding to the ALU or MUL unit. In the case of a store, the data are formatted and ready to be written into the cache.

**E4** Performs updates to the L2 cache, if required.

**E5** Results of ARM instructions are written back into the register file.

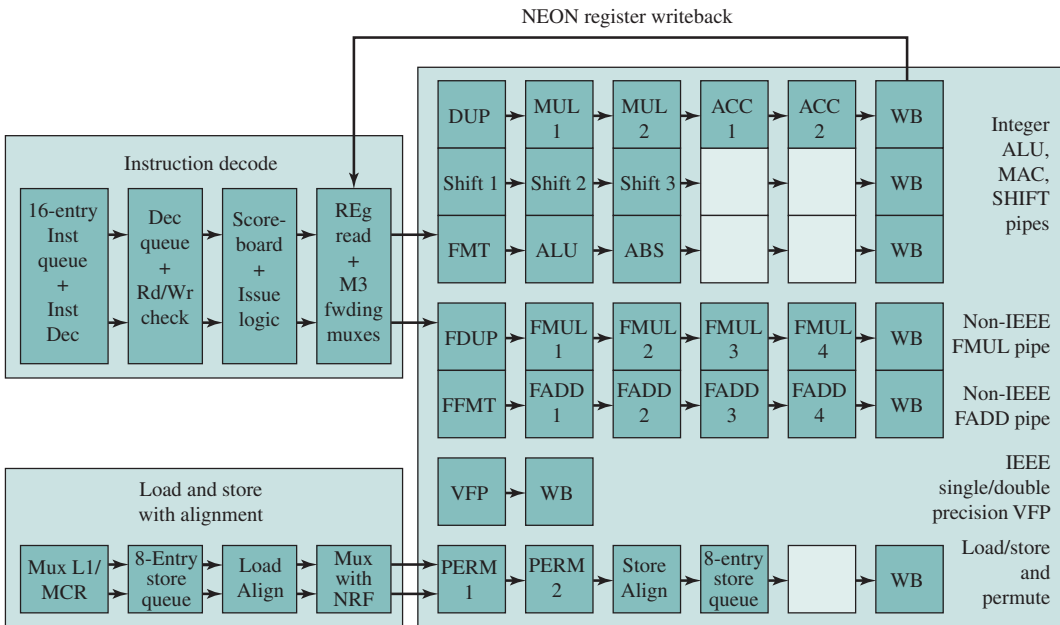
**Table 16.4** Cortex-A8 Example Dual Issue Instruction Sequence for Integer Pipeline

| Cycle | Program Counter | Instruction             | Timing Description                                                                                               |
|-------|-----------------|-------------------------|------------------------------------------------------------------------------------------------------------------|
| 1     | 0x00000ed0      | BX r14                  | Dual issue pipeline 0                                                                                            |
| 1     | 0x00000ee4      | CMP r0,#0               | Dual issue in pipeline 1                                                                                         |
| 2     | 0x00000ee8      | MOV r3,#3               | Dual issue pipeline 0                                                                                            |
| 2     | 0x00000eec      | MOV r0,#0               | Dual issue in pipeline 1                                                                                         |
| 3     | 0x00000ef0      | STREQ r3,[r1,#0]        | Dual issue in pipeline 0, r3 not needed until E3                                                                 |
| 3     | 0x00000ef4      | CMP r2,#4               | Dual issue in pipeline 1                                                                                         |
| 4     | 0x00000ef8      | LDRLS pc,[pc,r2,LSL #2] | Single issue pipeline 0, +1 cycle for load to pc, no extra cycle for shift since LSL #2                          |
| 5     | 0x00000f2c      | MOV r0,#1               | Dual issue with 2nd iteration of load in pipeline 1                                                              |
| 6     | 0x00000f30      | B {pc} + 8              | #0xf38 dual issue pipeline 0                                                                                     |
| 6     | 0x00000f38      | STR r0,[r1,#0]          | Dual issue pipeline 1                                                                                            |
| 7     | 0x00000f3c:     | LDR pc,[r13],#4         | Single issue pipeline 0, +1 cycle for load to pc                                                                 |
| 8     | 0x0000017c      | ADD r2,r4,#0xc          | Dual issue with 2nd iteration of load in pipeline 1                                                              |
| 9     | 0x00000180      | LDR r0,[r6,#4]          | Dual issue pipeline 0                                                                                            |
| 9     | 0x00000184      | MOV r1,#0xa             | Dual issue pipeline 1                                                                                            |
| 12    | 0x00000188      | LDR r0,[r0,#0]          | Single issue pipeline 0: r0 produced in E3, required in E1, so +2 cycle stall                                    |
| 13    | 0x0000018c      | STR r0,[r4,#0]          | Single issue pipeline 0 due to LS resource hazard, no extra delay for r0 since produced in E3 and consumed in E3 |
| 14    | 0x00000190      | LDR r0,[r4,#0xc]        | Single issue pipeline 0 due to LS resource hazard                                                                |
| 15    | 0x00000194      | LDMFD r13!,{r4-r6,r14}  | Load multiple: loads r4 in 1st cycle, r5 and r6 in 2nd cycle, r14 in 3rd cycle, 3 cycles total                   |
| 17    | 0x00000198      | B {pc}+0xda8            | #0xf40 dual issue in pipeline 1 with 3rd cycle of LDM                                                            |
| 18    | 0x00000f40      | ADD r0,r0,#2 ARM        | Single issue in pipeline 0                                                                                       |
| 19    | 0x00000f44      | ADD r0,r1,r0 ARM        | Single issue in pipeline 0, no dual issue due to hazard on r0 produced in E2 and required in E2                  |

Table 16.4 shows a sample code segment and indicates how the processor might schedule it.

### SIMD and Floating-Point Pipeline

All SIMD and floating-point instructions pass through the integer pipeline and are processed in a separate 10-stage pipeline (Figure 16.13). This unit, referred to as the



**Figure 16.13** ARM Cortex-A8 NEON and Floating-Point Pipeline

NEON unit, handles packed SIMD instructions, and provides two types of floating-point support. If implemented, a vector floating-point (VFP) coprocessor performs floating-point operations in compliance with IEEE 754. If the coprocessor is not present, then separate multiply and add pipelines implement the floating-point operations

## 16.5 RECOMMENDED READING

Two good book-length treatments of superscalar design are [SHEN05] and [OMON99]. Worthwhile survey articles on the subject are [SMIT95] and [SIMA97]. [JOUP89a] examines instruction-level parallelism, looks at various techniques for maximizing parallelism, and compares superscalar and superpipelined approaches using simulation. Recent papers that provide good coverage of superscalar design issues include [SIMA04], [PATT01], and [MOSH01].

[POPE91] provides a detailed look at a proposed superscalar machine. It also provides an excellent tutorial on the design issues related to out-of-order instruction policies. Another look at a proposed system is found in [KUGA91]; this article raises and considers most of the important design issues for superscalar implementation. [LEE91] examines software techniques that can be used to enhance superscalar performance. [WALL91] is an interesting study of the extent to which instruction-level parallelism can be exploited in a superscalar processor.

Volume I of [INTE04a] provides general description of the Pentium 4 pipeline; more detail is provided in [INTE01a] and [INTE01b]. Another detailed treatment is [FOG08b].

[JOHN08] and [ARM08a] provide thorough coverage of the ARM Cortex-A8 pipeline. [RICH07] is a good overview.

- ARM08a** ARM Limited. *Cortex-A8 Technical Reference Manual*. ARM DDI 0344E, 2008. [www.arm.com](http://www.arm.com)
- FOG08b** Fog, A. *The Microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. <http://www.agner.org/optimize/>
- HINT01** Hinton, G., et al. "The Microarchitecture of the Pentium 4 Processor." *Intel Technology Journal*, Q1 2001. <http://developer.intel.com/technology/itj/>
- INTE01a** Intel Corp. *Intel Pentium 4 Processor Optimization Reference Manual*. Document 248966-04 2001. <http://developer.intel.com/design/Pentium4/documentation.htm>
- INTE01b** Intel Corp. *Desktop Performance and Optimization for Intel Pentium 4 Processor*. Document 248966-04 2001. <http://developer.intel.com/design/Pentium4/documentation.htm>
- INTE04a** Intel Corp. *IA-32 Intel Architecture Software Developer's Manual (4 volumes)*. Document 253665 through 253668. 2004. <http://developer.intel.com/design/Pentium4/documentation.htm>
- JOHN08** John, E., and Rubio, J. *Unique Chips and Systems*. Boca Raton, FL: CRC Press, 2008.
- JOUP89a** Jouppi, N., and Wall, D. "Available Instruction-Level Parallelism for Superscalar and Superpipelined Machines." *Proceedings, Third International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1989.
- KUGA91** Kuga, M.; Murakami, K.; and Tomita, S. "DSNS (Dynamically-hazard resolved, Statically-code-scheduled, Nonuniform Superscalar): Yet Another Superscalar Processor Architecture." *Computer Architecture News*, June 1991.
- LEE91** Lee, R.; Kwok, A.; and Briggs, F. "The Floating Point Performance of a Superscalar SPARC Processor." *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1991.
- MOSH01** Moshovos, A., and Sohi, G. "Microarchitectural Innovations: Boosting Microprocessor Performance Beyond Semiconductor Technology Scaling." *Proceedings of the IEEE*, November 2001.
- OMON99** Omondi, A. *The Microarchitecture of Pipelined and Superscalar Computers*. Boston: Kluwer, 1999.
- PATT01** Patt, Y. "Requirements, Bottlenecks, and Good Fortune: Agents for Microprocessor Evolution." *Proceedings of the IEEE*, November 2001.
- POPE91** Popescu, V., et al. "The Metaflow Architecture." *IEEE Micro*, June 1991.
- RICH07** Riches, S., et al. "A Fully Automated High Performance Implementation of ARM Cortex-A8." *IQ Online*, Vol. 6, No. 3, 2007. [www.arm.com/iqonline](http://www.arm.com/iqonline)
- SHEN05** Shen, J., and Lipasti, M. *Modern Processor Design: Fundamentals of Superscalar Processors*. New York: McGraw-Hill, 2005.
- SIMA97** Sima, D. "Superscalar Instruction Issue." *IEEE Micro*, September/October 1997.
- SIMA04** Sima, D. "Decisive Aspects in the Evolution of Microprocessors." *Proceedings of the IEEE*, December 2004.

**SMIT95** Smith, J., and Sohi, G. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE*, December 1995.

**WALL91** Wall, D. “Limits of Instruction-Level Parallelism.” *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1991.

## 16.6 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

|                               |                       |                       |
|-------------------------------|-----------------------|-----------------------|
| antidependency                | machine parallelism   | register renaming     |
| branch prediction             | micro-operations      | resource conflict     |
| commit                        | micro-ops             | retire                |
| flow dependency               | out-of-order          | superpipelined        |
| in-order completion           | completion            | superscalar           |
| in-order issue                | out-of-order issue    | true data dependency  |
| instruction issue             | output dependency     | write-read dependency |
| instruction-level parallelism | procedural dependency | write-write           |
| instruction window            | read-write dependency | dependency            |

### Review Questions

- 16.1** What is the essential characteristic of the superscalar approach to processor design?
- 16.2** What is the difference between the superscalar and superpipelined approaches?
- 16.3** What is instruction-level parallelism?
- 16.4** Briefly define the following terms:
  - True data dependency
  - Procedural dependency
  - Resource conflicts
  - Output dependency
  - Antidependency
- 16.5** What is the distinction between instruction-level parallelism and machine parallelism?
- 16.6** List and briefly define three types of superscalar instruction issue policies.
- 16.7** What is the purpose of an instruction window?
- 16.8** What is register renaming and what is its purpose?
- 16.9** What are the key elements of a superscalar processor organization?

### Problems

- 16.1** When out-of-order completion is used in a superscalar processor, resumption of execution after interrupt processing is complicated, because the exceptional condition may have been detected as an instruction that produced its result out of order. The program cannot be restarted at the instruction following the exceptional instruction, because subsequent instructions have already completed, and doing so would cause these instructions to be executed twice. Suggest a mechanism or mechanisms for dealing with this situation.



- 16.2** Consider the following sequence of instructions, where the syntax consists of an opcode followed by the destination register followed by one or two source registers:

```

0 ADD R3, R1, R2
1 LOAD R6, [R3]
2 AND R7, R5, 3
3 ADD R1, R6, R7
4 SRL R7, R0, 8
5 OR R2, R4, R7
6 SUB R5, R3, R4
7 ADD R0, R1, 10
8 LOAD R6, [R5]
9 SUB R2, R1, R6
10 AND R3, R7, 15

```

Assume the use of a four-stage pipeline: fetch, decode/issue, execute, write back. Assume that all pipeline stages take one clock cycle except for the execute stage. For simple integer arithmetic and logical instructions, the execute stage takes one cycle, but for a LOAD from memory, five cycles are consumed in the execute stage.

If we have a simple scalar pipeline but allow out-of-order execution, we can construct the following table for the execution of the first seven instructions:

| Instruction | Fetch | Decode | Execute | Write Back |
|-------------|-------|--------|---------|------------|
| 0           | 0     | 1      | 2       | 3          |
| 1           | 1     | 2      | 4       | 9          |
| 2           | 2     | 3      | 5       | 6          |
| 3           | 3     | 4      | 10      | 11         |
| 4           | 4     | 5      | 6       | 7          |
| 5           | 5     | 6      | 8       | 10         |
| 6           | 6     | 7      | 9       | 12         |

The entries under the four pipeline stages indicate the clock cycle at which each instruction begins each phase. In this program, the second ADD instruction (instruction 3) depends on the LOAD instruction (instruction 1) for one of its operands, r6. Because the LOAD instruction takes five clock cycles, and the issue logic encounters the dependent ADD instruction after two clocks, the issue logic must delay the ADD instruction for three clock cycles. With an out-of-order capability, the processor can stall instruction 3 at clock cycle 4, and then move on to issue the following three independent instructions, which enter execution at clocks 6, 8, and 9. The LOAD finishes execution at clock 9, and so the dependent ADD can be launched into execution on clock 10.

- Complete the preceding table.
- Redo the table assuming no out-of-order capability. What is the savings using the capability?
- Redo the table assuming a superscalar implementation that can handle two instructions at a time at each stage.

- 16.3** Consider the following assembly language program:

```

I1: Move R3, R7 /R3 ← (R7)/
I2: Load R8, (R3) /R8 ← Memory (R3)/
I3: Add R3, R3, 4 /R3 ← (R3) + 4/
I4: Load R9, (R3) /R9 ← Memory (R3)/
I5: BLE R8, R9, L3 /Branch if (R9) > (R8)/

```

This program includes WAW, RAW, and WAR dependencies. Show these.

| Decode |    | Execute |    |    | Write |    | Cycle |
|--------|----|---------|----|----|-------|----|-------|
| I1     | I2 |         |    |    |       |    | 1     |
|        | I2 |         |    | I1 |       |    | 2     |
|        | I2 |         |    | I1 |       |    | 3     |
| I3     | I4 |         | I2 |    |       |    | 4     |
| I5     | I6 |         | I4 | I3 | I1    | I2 | 5     |
| I5     | I6 |         |    | I3 |       |    | 6     |
|        |    | I5      | I6 |    | I3    | I4 | 7     |
|        |    | I5      |    |    |       |    | 8     |
|        |    |         |    |    | I5    | I6 | 9     |

**Figure 16.14** An In-Order Issue, In-Order-Completion Execution Sequence

- 16.4 a.** Identify the RAW, WAR, and WAW dependencies in the following instruction sequence:

```

I1: R1 = 100
I2: R1 = R2 + R4
I3: R2 = R4 - 25
I4: R4 = R1 + R3
I5: R1 = R1 + 30

```

- b.** Rename the registers from part (a) to prevent dependency problems. Identify references to initial register values using the subscript “a” to the register reference.
- 16.5** Consider the “in-order-issue/in-order-completion” execution sequence shown in Figure 16.14.
- a.** Identify the most likely reason why I2 could not enter the execute stage until the fourth cycle. Will “in-order issue/out-of-order completion” or “out-of-order issue/out-of-order completion” fix this? If so, which?
- b.** Identify the reason why I6 could not enter the write stage until the ninth cycle. Will “in-order issue/out-of-order completion” or “out-of-order issue/out-of-order completion” fix this? If so, which?
- 16.6** Figure 16.15 shows an example of a superscalar processor organization. The processor can issue two instructions per cycle if there is no resource conflict and no data dependence problem. There are essentially two pipelines, with four processing stages (fetch, decode, execute, and store). Each pipeline has its own fetch decode and store unit. Four functional units (multiplier, adder, logic unit, and load unit) are available for use in the execute stage and are shared by the two pipelines on a dynamic basis. The two store units can be dynamically used by the two pipelines, depending on availability at a particular cycle. There is a lookahead window with its own fetch and decoding logic. This window is used for instruction lookahead for out-of-order instruction issue.

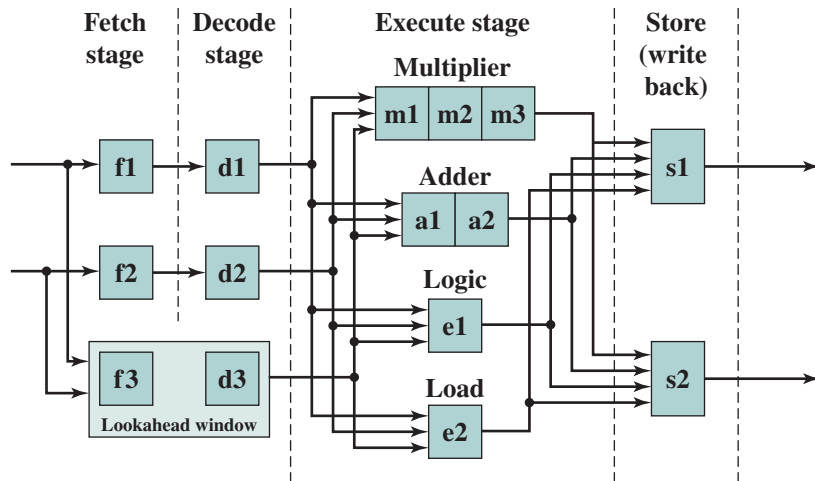
Consider the following program to be executed on this processor:

```

I1: Load R1, A /R1 ← Memory (A) /
I2: Add R2, R1 /R2 ← (R2) + R(1) /
I3: Add R3, R4 /R3 ← (R3) + R(4) /
I4: Mul R4, R5 /R4 ← (R4) + R(5) /
I5: Comp R6 /R6 ← (R6) /
I6: Mul R6, R7 /R6 ← (R6) × R(7) /

```

- a.** What dependencies exist in the program?
- b.** Show the pipeline activity for this program on the processor of Figure 16.15 using in-order issue with in-order completion policies and using a presentation similar to Figure 16.2.

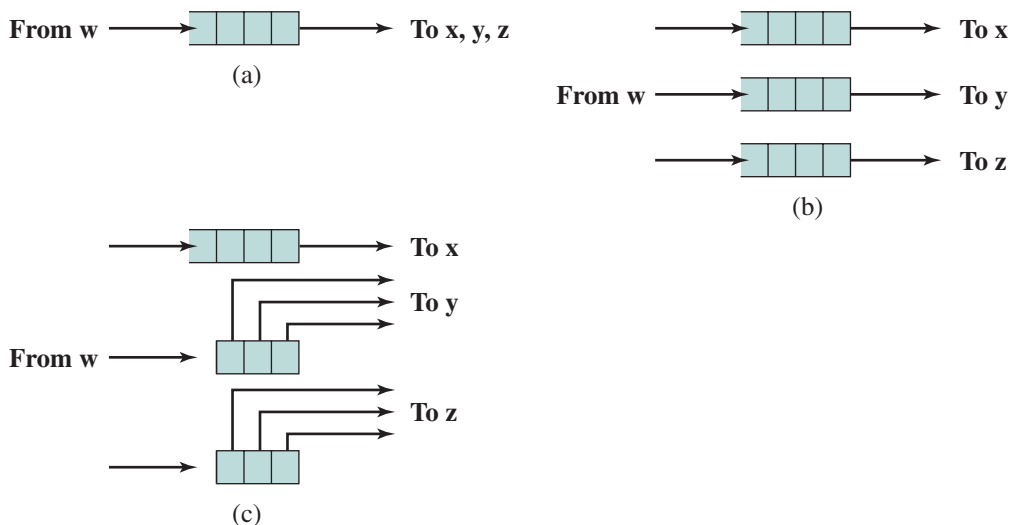


**Figure 16.15** A Dual-Pipeline Superscalar Processor

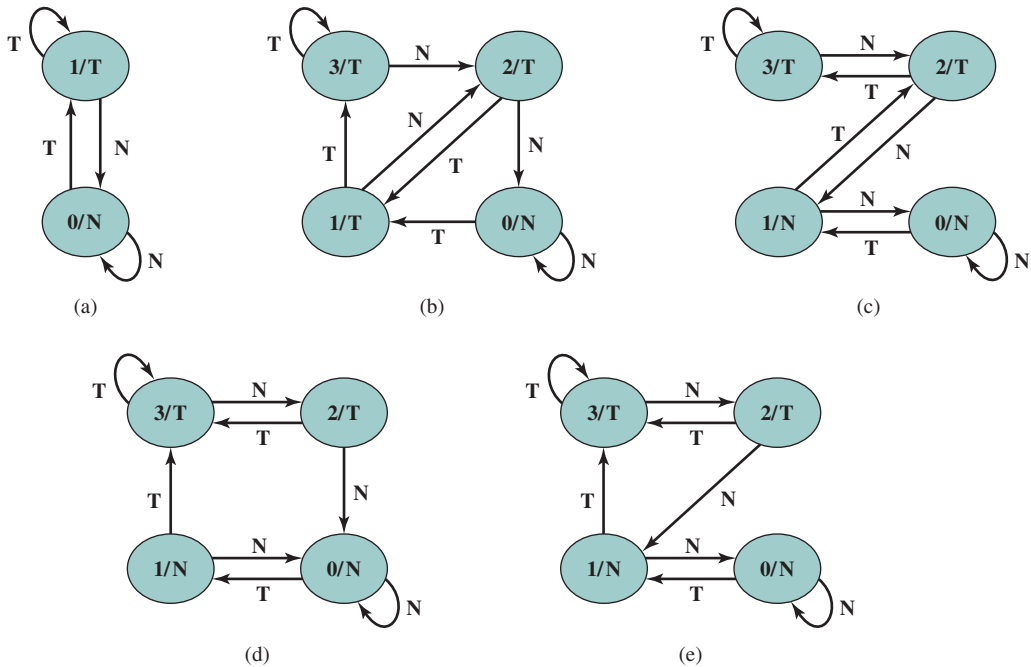
- c. Repeat for in-order issue with out-of-order completion.
- d. Repeat for out-of-order issue with out-of-order completion.

**16.7** Figure 16.16 is from a paper on superscalar design. Explain the three parts of the figure, and define  $w$ ,  $x$ ,  $y$ , and  $z$ .

**16.8** Yeh's dynamic branch prediction algorithm, used on the Pentium 4, is a two-level branch prediction algorithm. The first level is the history of the last  $n$  branches. The second level is the branch behavior of the last  $s$  occurrences of that unique pattern of the last  $n$  branches. For each conditional branch instruction in a program, there is an entry in a Branch History Table (BHT). Each entry consists of  $n$  bits corresponding to the last  $n$  executions of the branch instruction, with a 1 if the branch was taken



**Figure 16.16** Figure for Problem 16.7



**Figure 16.17** Figure for Problem 16.8

and a 0 if the branch was not. Each BHT entry indexes into a Pattern Table (PT) that has  $2n$  entries, one for each possible pattern of  $n$  bits. Each PT entry consists of  $s$  bits that are used in branch prediction, as was described in Chapter 14 (e.g., Figure 14.19). When a conditional branch is encountered during instruction fetch and decode, the address of the instruction is used to retrieve the appropriate BHT entry, which shows the recent history of the instruction. Then, the BHT entry is used to retrieve the appropriate PT entry for branch prediction. After the branch is executed, the BHT entry is updated, and then the appropriate PT entry is updated.

- In testing the performance of this scheme, Yeh tried five different prediction schemes, illustrated in Figure 16.17. Identify which three of these schemes correspond to those shown in Figures 14.19 and 14.28. Describe the remaining two schemes.
- With this algorithm, the prediction is not based on just the recent history of this particular branch instruction. Rather, it is based on the recent history of all patterns of branches that match the  $n$ -bit pattern in the BHT entry for this instruction. Suggest a rationale for such a strategy.

*This page intentionally left blank*

# PARALLEL PROCESSING

## 17.1 Multiple Processor Organizations

- Types of Parallel Processor Systems
- Parallel Organizations

## 17.2 Symmetric Multiprocessors

- Organization
- Multiprocessor Operating System Design Considerations

## 17.3 Cache Coherence and the MESI Protocol

- Software Solutions
- Hardware Solutions
- The MESI Protocol

## 17.4 Multithreading and Chip Multiprocessors

- Implicit and Explicit Multithreading
- Approaches to Explicit Multithreading
- Example Systems

## 17.5 Clusters

- Cluster Configurations
- Operating System Design Issues
- Cluster Computer Architecture
- Blade Servers
- Clusters Compared to SMP

## 17.6 Nonuniform Memory Access

- Motivation
- Organization
- NUMA Pros and Cons

## 17.7 Vector Computation

- Approaches to Vector Computation
- IBM 3090 Vector Facility

## 17.8 Recommended Reading

## 17.9 Key Terms, Review Questions, and Problems

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- ◆ Summarize the types of parallel processor organizations.
- ◆ Present an overview of design features of symmetric multiprocessors.
- ◆ Understand the issue of cache coherence in a multiple processor system.
- ◆ Explain the key features of the MESI protocol.
- ◆ Explain the difference between implicit and explicit multithreading.
- ◆ Summarize key design issues for clusters.
- ◆ Explain the concept of nonuniform memory access.
- ◆ Present an overview of vector computation.

Traditionally, the computer has been viewed as a sequential machine. Most computer programming languages require the programmer to specify algorithms as sequences of instructions. Processors execute programs by executing machine instructions in a sequence and one at a time. Each instruction is executed in a sequence of operations (fetch instruction, fetch operands, perform operation, store results).

This view of the computer has never been entirely true. At the micro-operation level, multiple control signals are generated at the same time. Instruction pipelining, at least to the extent of overlapping fetch and execute operations, has been around for a long time. Both of these are examples of performing functions in parallel. This approach is taken further with superscalar organization, which exploits instruction-level parallelism. With a superscalar machine, there are multiple execution units within a single processor, and these may execute multiple instructions from the same program in parallel.

As computer technology has evolved, and as the cost of computer hardware has dropped, computer designers have sought more and more opportunities for parallelism, usually to enhance performance and, in some cases, to increase availability. After an overview, this chapter looks at some of the most prominent approaches to parallel organization. First, we examine symmetric multiprocessors (SMPs), one of the earliest and still the most common example of parallel organization. In an SMP organization, multiple processors share a common memory. This organization raises the issue of cache coherence, to which a separate section is devoted. Next, the chapter examines multithreaded processors and chip multiprocessors. Then we describe clusters, which consist of multiple independent computers organized in a cooperative fashion. Clusters have become increasingly common to support workloads that are beyond the capacity of a single SMP. Another approach to the use of multiple processors that we examine is that of nonuniform memory access (NUMA) machines. The NUMA approach is relatively new and not yet proven in the marketplace, but is often considered as an alternative to the SMP or cluster approach. Finally, this chapter looks at hardware organizational approaches to vector computation. These approaches optimize the ALU for processing vectors or arrays of floating-point numbers. They are common on the class of systems known as *supercomputers*.

## 17.1 MULTIPLE PROCESSOR ORGANIZATIONS

### Types of Parallel Processor Systems

A taxonomy first introduced by Flynn [FLYN72] is still the most common way of categorizing systems with parallel processing capability. Flynn proposed the following categories of computer systems:

- **Single instruction, single data (SISD) stream:** A single processor executes a single instruction stream to operate on data stored in a single memory. Uniprocessors fall into this category.
- **Single instruction, multiple data (SIMD) stream:** A single machine instruction controls the simultaneous execution of a number of processing elements on a lockstep basis. Each processing element has an associated data memory, so that instructions are executed on different sets of data by different processors. Vector and array processors fall into this category, and are discussed in Section 18.7.
- **Multiple instruction, single data (MISD) stream:** A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence. This structure is not commercially implemented.
- **Multiple instruction, multiple data (MIMD) stream:** A set of processors simultaneously execute different instruction sequences on different data sets. SMPs, clusters, and NUMA systems fit into this category.

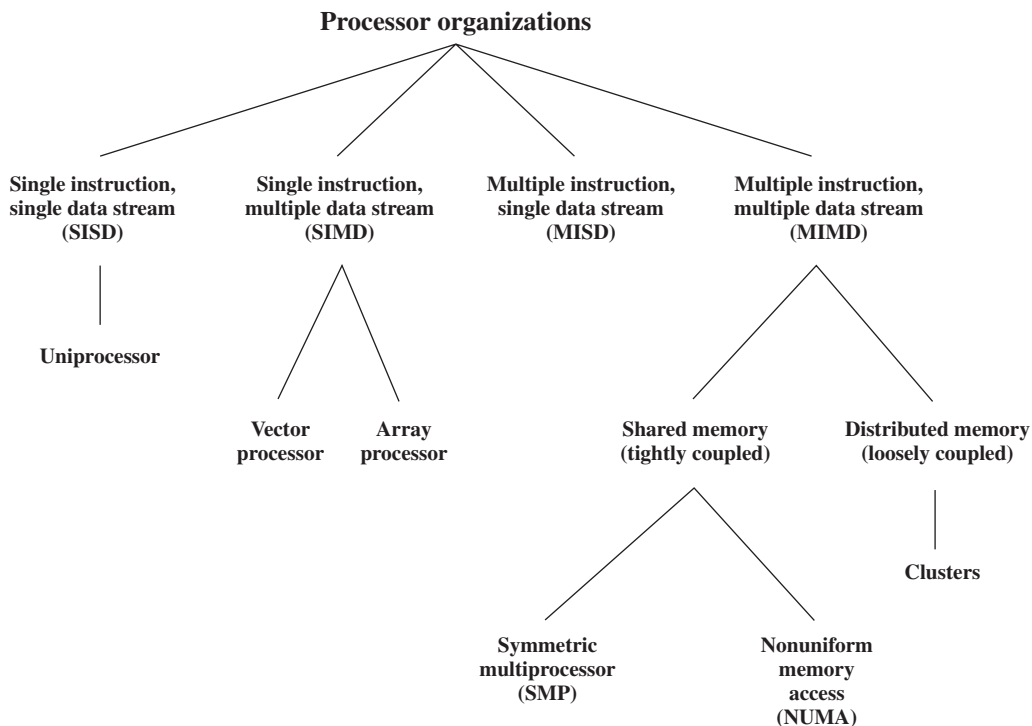
With the MIMD organization, the processors are general purpose; each is able to process all of the instructions necessary to perform the appropriate data transformation. MIMDs can be further subdivided by the means in which the processors communicate (Figure 17.1). If the processors share a common memory, then each processor accesses programs and data stored in the shared memory, and processors communicate with each other via that memory. The most common form of such system is known as a **symmetric multiprocessor (SMP)**, which we examine in Section 17.2. In an SMP, multiple processors share a single memory or pool of memory by means of a shared bus or other interconnection mechanism; a distinguishing feature is that the memory access time to any region of memory is approximately the same for each processor. A more recent development is the **nonuniform memory access (NUMA)** organization, which is described in Section 17.5. As the name suggests, the memory access time to different regions of memory may differ for a NUMA processor.

A collection of independent uniprocessors or SMPs may be interconnected to form a **cluster**. Communication among the computers is either via fixed paths or via some network facility.

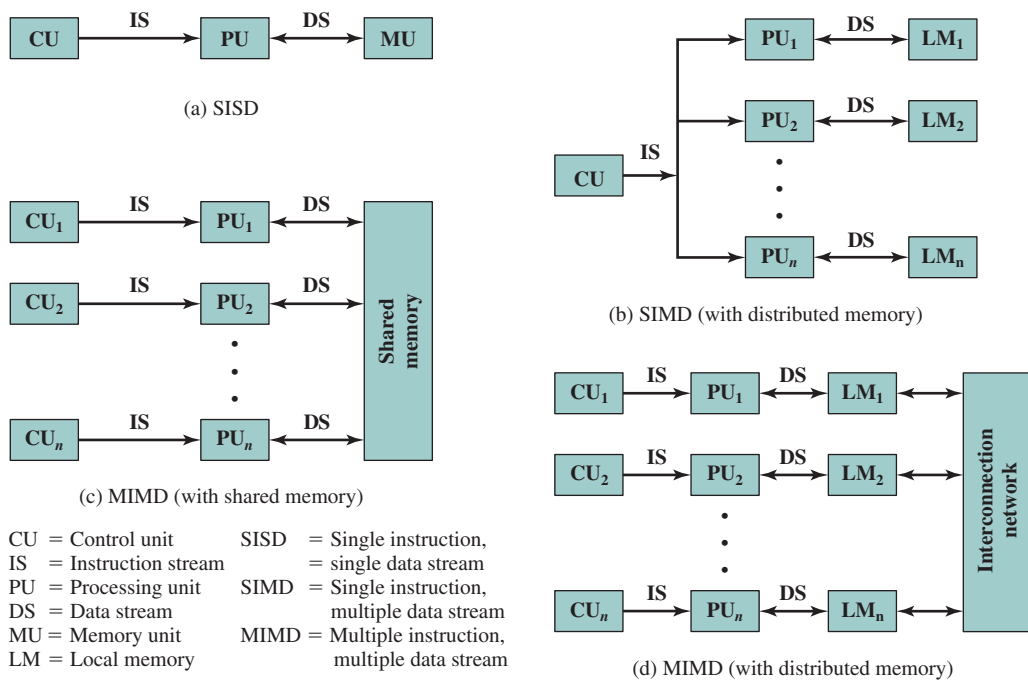
### Parallel Organizations

Figure 17.2 illustrates the general organization of the taxonomy of Figure 17.1. Figure 17.2a shows the structure of an SISD. There is some sort of control unit (CU) that provides an instruction stream (IS) to a processing unit (PU). The processing unit operates on a single data stream (DS) from a memory unit (MU). With an





**Figure 17.1** A Taxonomy of Parallel Processor Architectures



**Figure 17.2** Alternative Computer Organizations

SIMD, there is still a single control unit, now feeding a single instruction stream to multiple PUs. Each PU may have its own dedicated memory (illustrated in Figure 17.2b), or there may be a shared memory. Finally, with the MIMD, there are multiple control units, each feeding a separate instruction stream to its own PU. The MIMD may be a shared-memory multiprocessor (Figure 17.2c) or a distributed-memory multicomputer (Figure 17.2d).

The design issues relating to SMPs, clusters, and NUMAs are complex, involving issues relating to physical organization, interconnection structures, interprocessor communication, operating system design, and application software techniques. Our concern here is primarily with organization, although we touch briefly on operating system design issues.

## 17.2 SYMMETRIC MULTIPROCESSORS

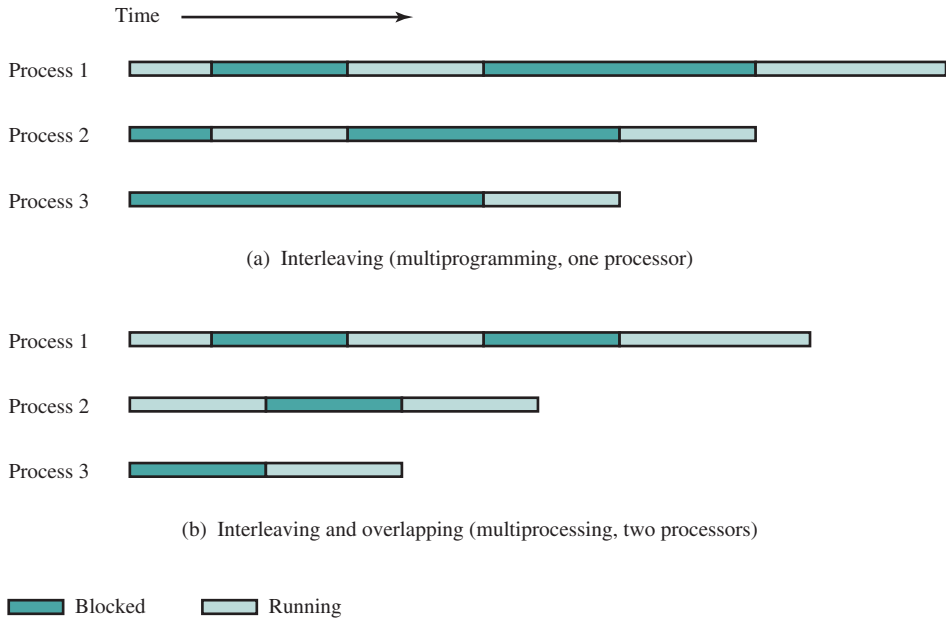
Until fairly recently, virtually all single-user personal computers and most workstations contained a single general-purpose microprocessor. As demands for performance increase and as the cost of microprocessors continues to drop, vendors have introduced systems with an SMP organization. The term *SMP* refers to a computer hardware architecture and also to the operating system behavior that reflects that architecture. An SMP can be defined as a standalone computer system with the following characteristics:

1. There are two or more similar processors of comparable capability.
2. These processors share the same main memory and I/O facilities and are interconnected by a bus or other internal connection scheme, such that memory access time is approximately the same for each processor.
3. All processors share access to I/O devices, either through the same channels or through different channels that provide paths to the same device.
4. All processors can perform the same functions (hence the term *symmetric*).
5. The system is controlled by an integrated operating system that provides interaction between processors and their programs at the job, task, file, and data element levels.

Points 1 to 4 should be self-explanatory. Point 5 illustrates one of the contrasts with a loosely coupled multiprocessing system, such as a cluster. In the latter, the physical unit of interaction is usually a message or complete file. In an SMP, individual data elements can constitute the level of interaction, and there can be a high degree of cooperation between processes.

The operating system of an SMP schedules processes or threads across all of the processors. An SMP organization has a number of potential advantages over a uniprocessor organization, including the following:

- **Performance:** If the work to be done by a computer can be organized so that some portions of the work can be done in parallel, then a system with multiple processors will yield greater performance than one with a single processor of the same type (Figure 17.3).



**Figure 17.3** Multiprogramming and Multiprocessing

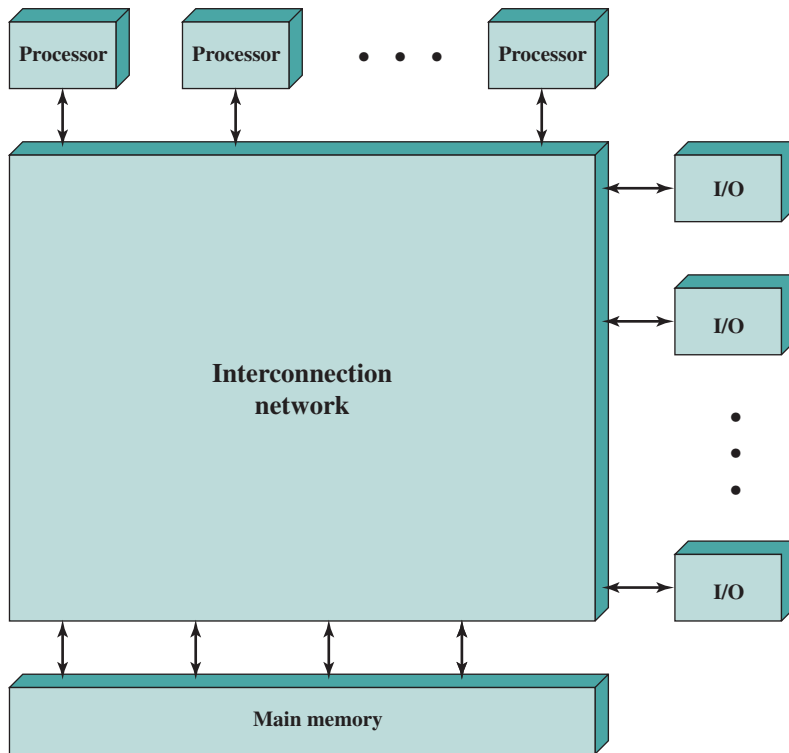
- **Availability:** In a symmetric multiprocessor, because all processors can perform the same functions, the failure of a single processor does not halt the machine. Instead, the system can continue to function at reduced performance.
- **Incremental growth:** A user can enhance the performance of a system by adding an additional processor.
- **Scaling:** Vendors can offer a range of products with different price and performance characteristics based on the number of processors configured in the system.

It is important to note that these are potential, rather than guaranteed, benefits. The operating system must provide tools and functions to exploit the parallelism in an SMP system.

An attractive feature of an SMP is that the existence of multiple processors is transparent to the user. The operating system takes care of scheduling of threads or processes on individual processors and of synchronization among processors.

## Organization

Figure 17.4 depicts in general terms the organization of a multiprocessor system. There are two or more processors. Each processor is self-contained, including a control unit, ALU, registers, and, typically, one or more levels of cache. Each processor has access to a shared main memory and the I/O devices through some form of interconnection mechanism. The processors can communicate with each other through memory (messages and status information left in common data areas). It may also be possible for processors to exchange signals directly. The memory is



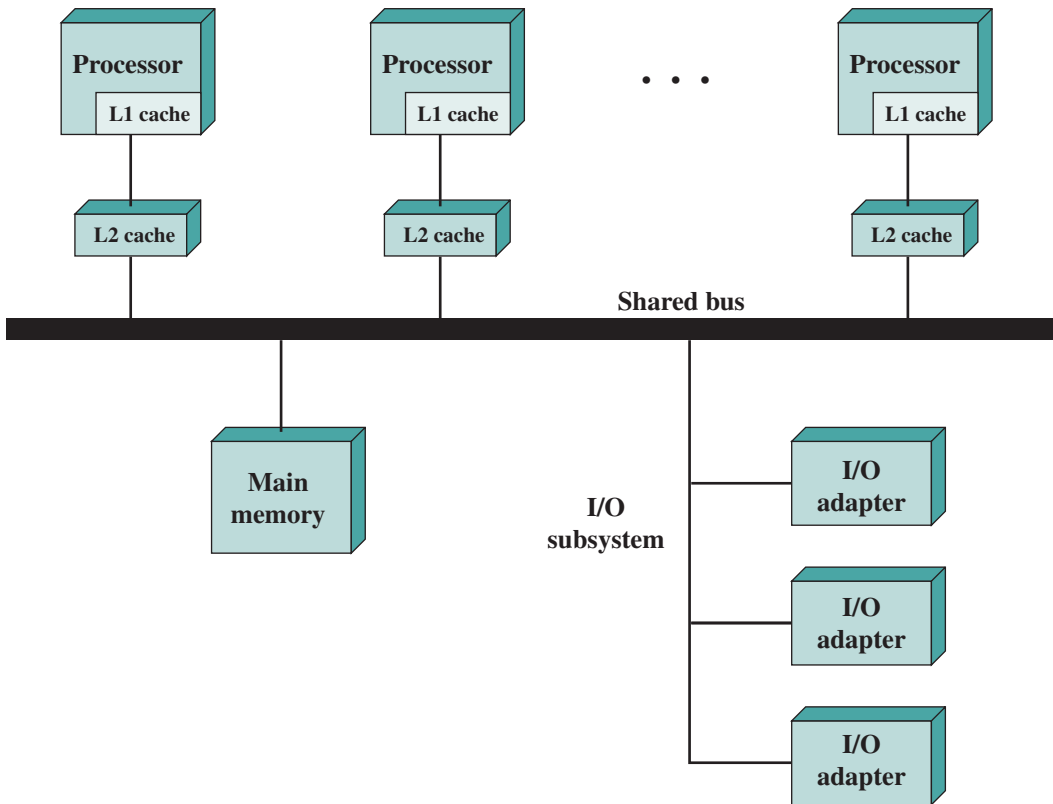
**Figure 17.4** Generic Block Diagram of a Tightly Coupled Multiprocessor

often organized so that multiple simultaneous accesses to separate blocks of memory are possible. In some configurations, each processor may also have its own private main memory and I/O channels in addition to the shared resources.

The most common organization for personal computers, workstations, and servers is the time-shared bus. The time-shared bus is the simplest mechanism for constructing a multiprocessor system (Figure 17.5). The structure and interfaces are basically the same as for a single-processor system that uses a bus interconnection. The bus consists of control, address, and data lines. To facilitate DMA transfers from I/O subsystems to processors, the following features are provided:

- **Addressing:** It must be possible to distinguish modules on the bus to determine the source and destination of data.
- **Arbitration:** Any I/O module can temporarily function as “master.” A mechanism is provided to arbitrate competing requests for bus control, using some sort of priority scheme.
- **Time-sharing:** When one module is controlling the bus, other modules are locked out and must, if necessary, suspend operation until bus access is achieved.

These uniprocessor features are directly usable in an SMP organization. In this latter case, there are now multiple processors as well as multiple I/O processors all attempting to gain access to one or more memory modules via the bus.



**Figure 17.5** Symmetric Multiprocessor Organization

The bus organization has several attractive features:

- **Simplicity:** This is the simplest approach to multiprocessor organization. The physical interface and the addressing, arbitration, and time-sharing logic of each processor remain the same as in a single-processor system.
- **Flexibility:** It is generally easy to expand the system by attaching more processors to the bus.
- **Reliability:** The bus is essentially a passive medium, and the failure of any attached device should not cause failure of the whole system.

The main drawback to the bus organization is performance. All memory references pass through the common bus. Thus, the bus cycle time limits the speed of the system. To improve performance, it is desirable to equip each processor with a cache memory. This should reduce the number of bus accesses dramatically. Typically, workstation and PC SMPs have two levels of cache, with the L1 cache internal (same chip as the processor) and the L2 cache either internal or external. Some processors now employ a L3 cache as well.

The use of caches introduces some new design considerations. Because each local cache contains an image of a portion of memory, if a word is altered in one

cache, it could conceivably invalidate a word in another cache. To prevent this, the other processors must be alerted that an update has taken place. This problem is known as the *cache coherence* problem and is typically addressed in hardware rather than by the operating system. We address this issue in Section 17.4.

### Multiprocessor Operating System Design Considerations

An SMP operating system manages processor and other computer resources so that the user perceives a single operating system controlling system resources. In fact, such a configuration should appear as a single-processor multiprogramming system. In both the SMP and uniprocessor cases, multiple jobs or processes may be active at one time, and it is the responsibility of the operating system to schedule their execution and to allocate resources. A user may construct applications that use multiple processes or multiple threads within processes without regard to whether a single processor or multiple processors will be available. Thus, a multiprocessor operating system must provide all the functionality of a multiprogramming system plus additional features to accommodate multiple processors. Among the key design issues:

- **Simultaneous concurrent processes:** OS routines need to be reentrant to allow several processors to execute the same IS code simultaneously. With multiple processors executing the same or different parts of the OS, OS tables and management structures must be managed properly to avoid deadlock or invalid operations.
- **Scheduling:** Any processor may perform scheduling, so conflicts must be avoided. The scheduler must assign ready processes to available processors.
- **Synchronization:** With multiple active processes having potential access to shared address spaces or shared I/O resources, care must be taken to provide effective synchronization. Synchronization is a facility that enforces mutual exclusion and event ordering.
- **Memory management:** Memory management on a multiprocessor must deal with all of the issues found on uniprocessor machines, as is discussed in Chapter 8. In addition, the operating system needs to exploit the available hardware parallelism, such as multiported memories, to achieve the best performance. The paging mechanisms on different processors must be coordinated to enforce consistency when several processors share a page or segment and to decide on page replacement.
- **Reliability and fault tolerance:** The operating system should provide graceful degradation in the face of processor failure. The scheduler and other portions of the operating system must recognize the loss of a processor and restructure management tables accordingly.

## 17.3 CACHE COHERENCE AND THE MESI PROTOCOL

In contemporary multiprocessor systems, it is customary to have one or two levels of cache associated with each processor. This organization is essential to achieve reasonable performance. It does, however, create a problem known as the *cache*

*coherence* problem. The essence of the problem is this: Multiple copies of the same data can exist in different caches simultaneously, and if processors are allowed to update their own copies freely, an inconsistent view of memory can result. In Chapter 4 we defined two common write policies:

- **Write back:** Write operations are usually made only to the cache. Main memory is only updated when the corresponding cache line is flushed from the cache.
- **Write through:** All write operations are made to main memory as well as to the cache, ensuring that main memory is always valid.

It is clear that a write-back policy can result in inconsistency. If two caches contain the same line, and the line is updated in one cache, the other cache will unknowingly have an invalid value. Subsequent reads to that invalid line produce invalid results. Even with the write-through policy, inconsistency can occur unless other caches monitor the memory traffic or receive some direct notification of the update.

In this section, we will briefly survey various approaches to the cache coherence problem and then focus on the approach that is most widely used: the MESI (modified/exclusive/shared/invalid) protocol. A version of this protocol is used on both the Pentium 4 and PowerPC implementations.

For any cache coherence protocol, the objective is to let recently used local variables get into the appropriate cache and stay there through numerous reads and write, while using the protocol to maintain consistency of shared variables that might be in multiple caches at the same time. Cache coherence approaches have generally been divided into software and hardware approaches. Some implementations adopt a strategy that involves both software and hardware elements. Nevertheless, the classification into software and hardware approaches is still instructive and is commonly used in surveying cache coherence strategies.

## Software Solutions

Software cache coherence schemes attempt to avoid the need for additional hardware circuitry and logic by relying on the compiler and operating system to deal with the problem. Software approaches are attractive because the overhead of detecting potential problems is transferred from run time to compile time, and the design complexity is transferred from hardware to software. On the other hand, compile-time software approaches generally must make conservative decisions, leading to inefficient cache utilization.

Compiler-based coherence mechanisms perform an analysis on the code to determine which data items may become unsafe for caching, and they mark those items accordingly. The operating system or hardware then prevents noncacheable items from being cached.

The simplest approach is to prevent any shared data variables from being cached. This is too conservative, because a shared data structure may be exclusively used during some periods and may be effectively read-only during other periods. It is only during periods when at least one process may update the variable and at least one other process may access the variable that cache coherence is an issue.

More efficient approaches analyze the code to determine safe periods for shared variables. The compiler then inserts instructions into the generated code to enforce cache coherence during the critical periods. A number of techniques have been developed for performing the analysis and for enforcing the results; see [LILJ93] and [STEN90] for surveys.

## Hardware Solutions

Hardware-based solutions are generally referred to as cache coherence protocols. These solutions provide dynamic recognition at run time of potential inconsistency conditions. Because the problem is only dealt with when it actually arises, there is more effective use of caches, leading to improved performance over a software approach. In addition, these approaches are transparent to the programmer and the compiler, reducing the software development burden.

Hardware schemes differ in a number of particulars, including where the state information about data lines is held, how that information is organized, where coherence is enforced, and the enforcement mechanisms. In general, hardware schemes can be divided into two categories: directory protocols and snoopy protocols.

**DIRECTORY PROTOCOLS** Directory protocols collect and maintain information about where copies of lines reside. Typically, there is a centralized controller that is part of the main memory controller, and a directory that is stored in main memory. The directory contains global state information about the contents of the various local caches. When an individual cache controller makes a request, the centralized controller checks and issues necessary commands for data transfer between memory and caches or between caches. It is also responsible for keeping the state information up to date; therefore, every local action that can affect the global state of a line must be reported to the central controller.

Typically, the controller maintains information about which processors have a copy of which lines. Before a processor can write to a local copy of a line, it must request exclusive access to the line from the controller. Before granting this exclusive access, the controller sends a message to all processors with a cached copy of this line, forcing each processor to invalidate its copy. After receiving acknowledgments back from each such processor, the controller grants exclusive access to the requesting processor. When another processor tries to read a line that is exclusively granted to another processor, it will send a miss notification to the controller. The controller then issues a command to the processor holding that line that requires the processor to do a write back to main memory. The line may now be shared for reading by the original processor and the requesting processor.

Directory schemes suffer from the drawbacks of a central bottleneck and the overhead of communication between the various cache controllers and the central controller. However, they are effective in large-scale systems that involve multiple buses or some other complex interconnection scheme.

**SNOOPY PROTOCOLS** Snoopy protocols distribute the responsibility for maintaining cache coherence among all of the cache controllers in a multiprocessor. A cache must recognize when a line that it holds is shared with other caches.



When an update action is performed on a shared cache line, it must be announced to all other caches by a broadcast mechanism. Each cache controller is able to “snoop” on the network to observe these broadcasted notifications, and react accordingly.

Snoopy protocols are ideally suited to a bus-based multiprocessor, because the shared bus provides a simple means for broadcasting and snooping. However, because one of the objectives of the use of local caches is to avoid bus accesses, care must be taken that the increased bus traffic required for broadcasting and snooping does not cancel out the gains from the use of local caches.

Two basic approaches to the snoopy protocol have been explored: write invalidate and write update (or write broadcast). With a write-invalidate protocol, there can be multiple readers but only one writer at a time. Initially, a line may be shared among several caches for reading purposes. When one of the caches wants to perform a write to the line, it first issues a notice that invalidates that line in the other caches, making the line exclusive to the writing cache. Once the line is exclusive, the owning processor can make cheap local writes until some other processor requires the same line.

With a write-update protocol, there can be multiple writers as well as multiple readers. When a processor wishes to update a shared line, the word to be updated is distributed to all others, and caches containing that line can update it.

Neither of these two approaches is superior to the other under all circumstances. Performance depends on the number of local caches and the pattern of memory reads and writes. Some systems implement adaptive protocols that employ both write-invalidate and write-update mechanisms.

The write-invalidate approach is the most widely used in commercial multiprocessor systems, such as the Pentium 4 and PowerPC. It marks the state of every cache line (using two extra bits in the cache tag) as modified, exclusive, shared, or invalid. For this reason, the write-invalidate protocol is called MESI. In the remainder of this section, we will look at its use among local caches across a multiprocessor. For simplicity in the presentation, we do not examine the mechanisms involved in coordinating among both level 1 and level 2 locally as well as at the same time coordinating across the distributed multiprocessor. This would not add any new principles but would greatly complicate the discussion.

## The MESI Protocol

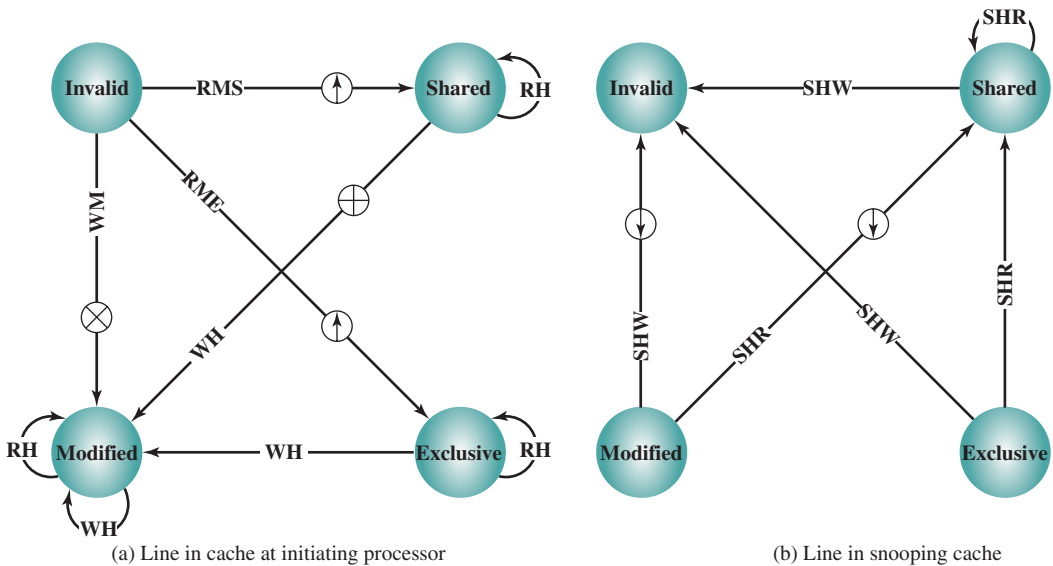
To provide cache consistency on an SMP, the data cache often supports a protocol known as MESI. For MESI, the data cache includes two status bits per tag, so that each line can be in one of four states:

- **Modified:** The line in the cache has been modified (different from main memory) and is available only in this cache.
- **Exclusive:** The line in the cache is the same as that in main memory and is not present in any other cache.
- **Shared:** The line in the cache is the same as that in main memory and may be present in another cache.
- **Invalid:** The line in the cache does not contain valid data.

**Table 17.1** MESI Cache Line States

|                               | <b>M</b><br><b>Modified</b> | <b>E</b><br><b>Exclusive</b> | <b>S</b><br><b>Shared</b>     | <b>I</b><br><b>Invalid</b> |
|-------------------------------|-----------------------------|------------------------------|-------------------------------|----------------------------|
| This cache line valid?        | Yes                         | Yes                          | Yes                           | No                         |
| The memory copy is ...        | out of date                 | valid                        | valid                         | —                          |
| Copies exist in other caches? | No                          | No                           | Maybe                         | Maybe                      |
| A write to this line ...      | does not go to bus          | does not go to bus           | goes to bus and updates cache | goes directly to bus       |

Table 17.1 summarizes the meaning of the four states. Figure 17.6 displays a state diagram for the MESI protocol. Keep in mind that each line of the cache has its own state bits and therefore its own realization of the state diagram. Figure 17.6a shows the transitions that occur due to actions initiated by the processor attached to this cache. Figure 17.6b shows the transitions that occur due to events that are snooped on the common bus. This presentation of separate state diagrams for processor-initiated and bus-initiated actions helps to clarify the logic of the MESI protocol.



|     |                                                  |   |                            |
|-----|--------------------------------------------------|---|----------------------------|
| RH  | Read hit                                         | ⬇ | Dirty line copyback        |
| RMS | Read miss, shared                                | ⊕ | Invalidate transaction     |
| RME | Read miss, exclusive                             | ⊗ | Read-with-intent-to-modify |
| WH  | Write hit                                        | ⬆ | Cache line fill            |
| WM  | Write miss                                       |   |                            |
| SHR | Snoop hit on read                                |   |                            |
| SHW | Snoop hit on write or read-with-intent-to-modify |   |                            |

**Figure 17.6** MESI State Transition Diagram

At any time a cache line is in a single state. If the next event is from the attached processor, then the transition is dictated by Figure 17.6a and if the next event is from the bus, the transition is dictated by Figure 17.6b. Let us look at these transitions in more detail.

**READ MISS** When a read miss occurs in the local cache, the processor initiates a memory read to read the line of main memory containing the missing address. The processor inserts a signal on the bus that alerts all other processor/cache units to snoop the transaction. There are a number of possible outcomes:

- If one other cache has a clean (unmodified since read from memory) copy of the line in the exclusive state, it returns a signal indicating that it shares this line. The responding processor then transitions the state of its copy from exclusive to shared, and the initiating processor reads the line from main memory and transitions the line in its cache from invalid to shared.
- If one or more caches have a clean copy of the line in the shared state, each of them signals that it shares the line. The initiating processor reads the line and transitions the line in its cache from invalid to shared.
- If one other cache has a modified copy of the line, then that cache blocks the memory read and provides the line to the requesting cache over the shared bus. The responding cache then changes its line from modified to shared.<sup>1</sup> The line sent to the requesting cache is also received and processed by the memory controller, which stores the block in memory.
- If no other cache has a copy of the line (clean or modified), then no signals are returned. The initiating processor reads the line and transitions the line in its cache from invalid to exclusive.

**READ HIT** When a read hit occurs on a line currently in the local cache, the processor simply reads the required item. There is no state change: The state remains modified, shared, or exclusive.

**WRITE MISS** When a write miss occurs in the local cache, the processor initiates a memory read to read the line of main memory containing the missing address. For this purpose, the processor issues a signal on the bus that means *read-with-intent-to-modify* (RWITM). When the line is loaded, it is immediately marked modified. With respect to other caches, two possible scenarios precede the loading of the line of data.

First, some other cache may have a modified copy of this line (state = modify). In this case, the alerted processor signals the initiating processor that another processor has a modified copy of the line. The initiating processor surrenders the bus and waits. The other processor gains access to the bus, writes the modified cache

---

<sup>1</sup>In some implementations, the cache with the modified line signals the initiating processor to retry. Meanwhile, the processor with the modified copy seizes the bus, writes the modified line back to main memory, and transitions the line in its cache from modified to shared. Subsequently, the requesting processor tries again and finds that one or more processors have a clean copy of the line in the shared state, as described in the preceding point.

line back to main memory, and transitions the state of the cache line to invalid (because the initiating processor is going to modify this line). Subsequently, the initiating processor will again issue a signal to the bus of RWITM and then read the line from main memory, modify the line in the cache, and mark the line in the modified state.

The second scenario is that no other cache has a modified copy of the requested line. In this case, no signal is returned, and the initiating processor proceeds to read in the line and modify it. Meanwhile, if one or more caches have a clean copy of the line in the shared state, each cache invalidates its copy of the line, and if one cache has a clean copy of the line in the exclusive state, it invalidates its copy of the line.

**WRITE HIT** When a write hit occurs on a line currently in the local cache, the effect depends on the current state of that line in the local cache:

- **Shared:** Before performing the update, the processor must gain exclusive ownership of the line. The processor signals its intent on the bus. Each processor that has a shared copy of the line in its cache transitions the sector from shared to invalid. The initiating processor then performs the update and transitions its copy of the line from shared to modified.
- **Exclusive:** The processor already has exclusive control of this line, and so it simply performs the update and transitions its copy of the line from exclusive to modified.
- **Modified:** The processor already has exclusive control of this line and has the line marked as modified, and so it simply performs the update.

**L1-L2 CACHE CONSISTENCY** We have so far described cache coherency protocols in terms of the cooperate activity among caches connected to the same bus or other SMP interconnection facility. Typically, these caches are L2 caches, and each processor also has an L1 cache that does not connect directly to the bus and that therefore cannot engage in a snoopy protocol. Thus, some scheme is needed to maintain data integrity across both levels of cache and across all caches in the SMP configuration.

The strategy is to extend the MESI protocol (or any cache coherency protocol) to the L1 caches. Thus, each line in the L1 cache includes bits to indicate the state. In essence, the objective is the following: for any line that is present in both an L2 cache and its corresponding L1 cache, the L1 line state should track the state of the L2 line. A simple means of doing this is to adopt the write-through policy in the L1 cache; in this case the write through is to the L2 cache and not to the memory. The L1 write-through policy forces any modification to an L1 line out to the L2 cache and therefore makes it visible to other L2 caches. The use of the L1 write-through policy requires that the L1 content must be a subset of the L2 content. This in turn suggests that the associativity of the L2 cache should be equal to or greater than that of the L1 associativity. The L1 write-through policy is used in the IBM S/390 SMP.

If the L1 cache has a write-back policy, the relationship between the two caches is more complex. There are several approaches to maintaining coherence. For example, the approach used on the Pentium II is described in detail in [SHAN05].

## 17.4 MULTITHREADING AND CHIP MULTIPROCESSORS

The most important measure of performance for a processor is the rate at which it executes instructions. This can be expressed as

$$\text{MIPS rate} = f \times \text{IPC}$$

where  $f$  is the processor clock frequency, in MHz, and  $\text{IPC}$  (instructions per cycle) is the average number of instructions executed per cycle. Accordingly, designers have pursued the goal of increased performance on two fronts: increasing clock frequency and increasing the number of instructions executed or, more properly, the number of instructions that complete during a processor cycle. As we have seen in earlier chapters, designers have increased IPC by using an instruction pipeline and then by using multiple parallel instruction pipelines in a superscalar architecture. With pipelined and multiple-pipeline designs, the principal problem is to maximize the utilization of each pipeline stage. To improve throughput, designers have created ever more complex mechanisms, such as executing some instructions in a different order from the way they occur in the instruction stream and beginning execution of instructions that may never be needed. But as was discussed in Section 2.2, this approach may be reaching a limit due to complexity and power consumption concerns.

An alternative approach, which allows for a high degree of instruction-level parallelism without increasing circuit complexity or power consumption, is called multithreading. In essence, the instruction stream is divided into several smaller streams, known as threads, such that the threads can be executed in parallel.

The variety of specific multithreading designs, realized in both commercial systems and experimental systems, is vast. In this section, we give a brief survey of the major concepts.

### Implicit and Explicit Multithreading

The concept of thread used in discussing multithreaded processors may or may not be the same as the concept of software threads in a multiprogrammed operating system. It will be useful to define terms briefly:

- **Process:** An instance of a program running on a computer. A process embodies two key characteristics:
  - **Resource ownership:** A process includes a virtual address space to hold the process image; the process image is the collection of program, data, stack, and attributes that define the process. From time to time, a process may be allocated control or ownership of resources, such as main memory, I/O channels, I/O devices, and files.
  - **Scheduling/execution:** The execution of a process follows an execution path (trace) through one or more programs. This execution may be interleaved with that of other processes. Thus, a process has an execution state (Running, Ready, etc.) and a dispatching priority and is the entity that is scheduled and dispatched by the operating system.

- **Process switch:** An operation that switches the processor from one process to another, by saving all the process control data, registers, and other information for the first and replacing them with the process information for the second.<sup>2</sup>
- **Thread:** A dispatchable unit of work within a process. It includes a processor context (which includes the program counter and stack pointer) and its own data area for a stack (to enable subroutine branching). A thread executes sequentially and is interruptible so that the processor can turn to another thread.
- **Thread switch:** The act of switching processor control from one thread to another within the same process. Typically, this type of switch is much less costly than a process switch.

Thus, a thread is concerned with scheduling and execution, whereas a process is concerned with both scheduling/execution and resource ownership. The multiple threads within a process share the same resources. This is why a thread switch is much less time consuming than a process switch. Traditional operating systems, such as earlier versions of UNIX, did not support threads. Most modern operating systems, such as Linux, other versions of UNIX, and Windows, do support thread. A distinction is made between user-level threads, which are visible to the application program, and kernel-level threads, which are visible only to the operating system. Both of these may be referred to as explicit threads, defined in software.

All of the commercial processors and most of the experimental processors so far have used explicit multithreading. These systems concurrently execute instructions from different explicit threads, either by interleaving instructions from different threads on shared pipelines or by parallel execution on parallel pipelines. Implicit multithreading refers to the concurrent execution of multiple threads extracted from a single sequential program. These implicit threads may be defined either statically by the compiler or dynamically by the hardware. In the remainder of this section we consider explicit multithreading.

### Approaches to Explicit Multithreading

At minimum, a multithreaded processor must provide a separate program counter for each thread of execution to be executed concurrently. The designs differ in the amount and type of additional hardware used to support concurrent thread execution. In general, instruction fetching takes place on a thread basis. The processor treats each thread separately and may use a number of techniques for optimizing single-thread execution, including branch prediction, register renaming, and superscalar techniques. What is achieved is thread-level parallelism, which may provide for greatly improved performance when married to instruction-level parallelism.

Broadly speaking, there are four principal approaches to multithreading:

- **Interleaved multithreading:** This is also known as **fine-grained multithreading**. The processor deals with two or more thread contexts at a time, switching from one thread to another at each clock cycle. If a thread is blocked because

---

<sup>2</sup>The term *context switch* is often found in OS literature and textbooks. Unfortunately, although most of the literature uses this term to mean what is here called a process switch, other sources use it to mean a thread switch. To avoid ambiguity, the term is not used in this book.

of data dependencies or memory latencies, that thread is skipped and a ready thread is executed.

- **Blocked multithreading:** This is also known as **coarse-grained multithreading**. The instructions of a thread are executed successively until an event occurs that may cause delay, such as a cache miss. This event induces a switch to another thread. This approach is effective on an in-order processor that would stall the pipeline for a delay event such as a cache miss.
- **Simultaneous multithreading (SMT):** Instructions are simultaneously issued from multiple threads to the execution units of a superscalar processor. This combines the wide superscalar instruction issue capability with the use of multiple thread contexts.
- **Chip multiprocessing:** In this case, the entire processor is replicated on a single chip and each processor handles separate threads. The advantage of this approach is that the available logic area on a chip is used effectively without depending on ever-increasing complexity in pipeline design. This is referred to as multicore; we examine this topic separately in Chapter 18.

For the first two approaches, instructions from different threads are not executed simultaneously. Instead, the processor is able to rapidly switch from one thread to another, using a different set of registers and other context information. This results in a better utilization of the processor's execution resources and avoids a large penalty due to cache misses and other latency events. The SMT approach involves true simultaneous execution of instructions from different threads, using replicated execution resources. Chip multiprocessing also enables simultaneous execution of instructions from different threads.

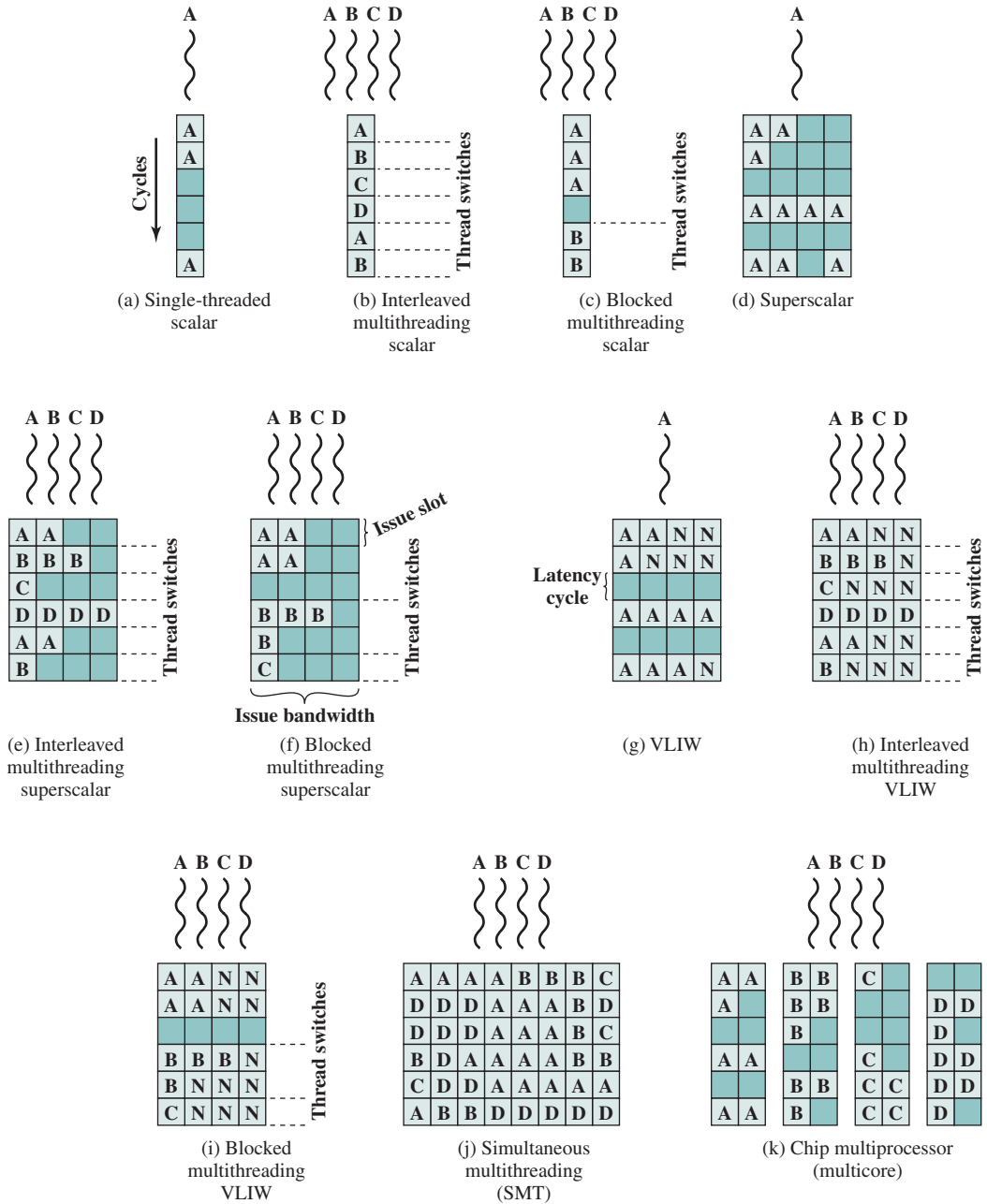
Figure 17.7, based on one in [UNGE02], illustrates some of the possible pipeline architectures that involve multithreading and contrasts these with approaches that do not use multithreading. Each horizontal row represents the potential issue slot or slots for a single execution cycle; that is, the width of each row corresponds to the maximum number of instructions that can be issued in a single clock cycle.<sup>3</sup> The vertical dimension represents the time sequence of clock cycles. An empty (shaded) slot represents an unused execution slot in one pipeline. A no-op is indicated by N.

The first three illustrations in Figure 17.7 show different approaches with a scalar (i.e., single-issue) processor:

- **Single-threaded scalar:** This is the simple pipeline found in traditional RISC and CISC machines, with no multithreading.
- **Interleaved multithreaded scalar:** This is the easiest multithreading approach to implement. By switching from one thread to another at each clock cycle, the pipeline stages can be kept fully occupied, or close to fully occupied. The hardware must be capable of switching from one thread context to another between cycles.

---

<sup>3</sup>Issue slots are the position from which instructions can be issued in a given clock cycle. Recall from Chapter 16 that instruction issue is the process of initiating instruction execution in the processor's functional units. This occurs when an instruction moves from the decode stage of the pipeline to the first execute stage of the pipeline.



**Figure 17.7** Approaches to Executing Multiple Threads

- **Blocked multithreaded scalar:** In this case, a single thread is executed until a latency event occurs that would stop the pipeline, at which time the processor switches to another thread.

Figure 17.7c shows a situation in which the time to perform a thread switch is one cycle, whereas Figure 17.7b shows that thread switching occurs in zero cycles.



In the case of interleaved multithreading, it is assumed that there are no control or data dependencies between threads, which simplifies the pipeline design and therefore should allow a thread switch with no delay. However, depending on the specific design and implementation, block multithreading may require a clock cycle to perform a thread switch, as illustrated in Figure 17.7. This is true if a fetched instruction triggers the thread switch and must be discarded from the pipeline [UNGE03].

Although interleaved multithreading appears to offer better processor utilization than blocked multithreading, it does so at the sacrifice of single-thread performance. The multiple threads compete for cache resources, which raises the probability of a cache miss for a given thread.

More opportunities for parallel execution are available if the processor can issue multiple instructions per cycle. Figures 17.7d through 17.7i illustrate a number of variations among processors that have hardware for issuing four instructions per cycle. In all these cases, only instructions from a single thread are issued in a single cycle. The following alternatives are illustrated:

- **Superscalar:** This is the basic superscalar approach with no multithreading. Until relatively recently, this was the most powerful approach to providing parallelism within a processor. Note that during some cycles, not all of the available issue slots are used. During these cycles, less than the maximum number of instructions is issued; this is referred to as *horizontal loss*. During other instruction cycles, no issue slots are used; these are cycles when no instructions can be issued; this is referred to as *vertical loss*.
- **Interleaved multithreading superscalar:** During each cycle, as many instructions as possible are issued from a single thread. With this technique, potential delays due to thread switches are eliminated, as previously discussed. However, the number of instructions issued in any given cycle is still limited by dependencies that exist within any given thread.
- **Blocked multithreaded superscalar:** Again, instructions from only one thread may be issued during any cycle, and blocked multithreading is used.
- **Very long instruction word (VLIW):** A VLIW architecture, such as IA-64, places multiple instructions in a single word. Typically, a VLIW is constructed by the compiler, which places operations that may be executed in parallel in the same word. In a simple VLIW machine (Figure 17.7g), if it is not possible to completely fill the word with instructions to be issued in parallel, no-ops are used.
- **Interleaved multithreading VLIW:** This approach should provide similar efficiencies to those provided by interleaved multithreading on a superscalar architecture.
- **Blocked multithreaded VLIW:** This approach should provide similar efficiencies to those provided by blocked multithreading on a superscalar architecture.

The final two approaches illustrated in Figure 17.7 enable the parallel, simultaneous execution of multiple threads:

- **Simultaneous multithreading:** Figure 17.7j shows a system capable of issuing 8 instructions at a time. If one thread has a high degree of instruction-level parallelism, it may on some cycles be able fill all of the horizontal slots. On

other cycles, instructions from two or more threads may be issued. If sufficient threads are active, it should usually be possible to issue the maximum number of instructions on each cycle, providing a high level of efficiency.

- **Chip multiprocessor (multicore):** Figure 17.7k shows a chip containing four processors, each of which has a two-issue superscalar processor. Each processor is assigned a thread, from which it can issue up to two instructions per cycle. We discuss multicore computers in Chapter 18.

Comparing Figures 17.7j and 17.7k, we see that a chip multiprocessor with the same instruction issue capability as an SMT cannot achieve the same degree of instruction-level parallelism. This is because the chip multiprocessor is not able to hide latencies by issuing instructions from other threads. On the other hand, the chip multiprocessor should outperform a superscalar processor with the same instruction issue capability, because the horizontal losses will be greater for the superscalar processor. In addition, it is possible to use multithreading within each of the processors on a chip multiprocessor, and this is done on some contemporary machines.

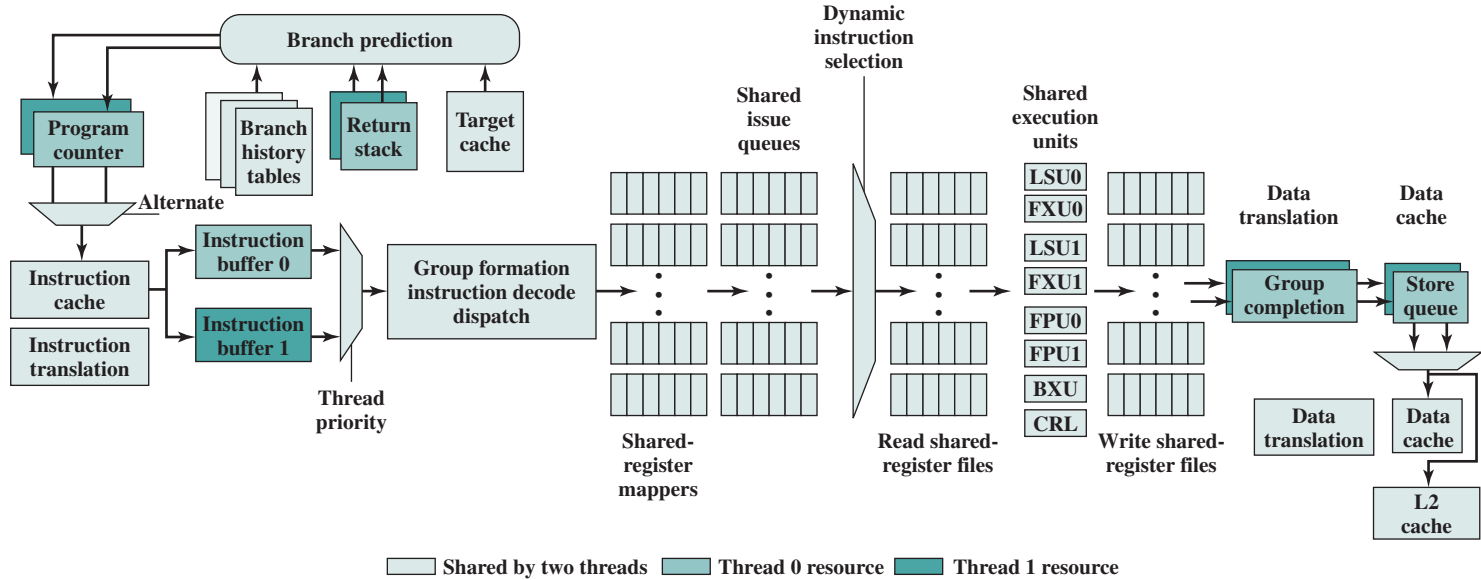
### Example Systems

**PENTIUM 4** More recent models of the Pentium 4 use a multithreading technique that the Intel literature refers to as *hyperthreading* [MARR02]. In essence, the Pentium 4 approach is to use SMT with support for two threads. Thus, the single multithreaded processor is logically two processors.

**IBM POWER5** The IBM Power5 chip, which is used in high-end PowerPC products, combines chip multiprocessing with SMT [KALL04]. The chip has two separate processors, each of which is a multithreaded processor capable of supporting two threads concurrently using SMT. Interestingly, the designers simulated various alternatives and found that having two two-way SMT processors on a single chip provided superior performance to a single four-way SMT processor. The simulations showed that additional multithreading beyond the support for two threads might decrease performance because of cache thrashing, as data from one thread displaces data needed by another thread.

Figure 17.8 shows the IBM Power5's instruction flow diagram. Only a few of the elements in the processor need to be replicated, with separate elements dedicated to separate threads. Two program counters are used. The processor alternates fetching instructions, up to eight at a time, between the two threads. All the instructions are stored in a common instruction cache and share an instruction translation facility, which does a partial instruction decode. When a conditional branch is encountered, the branch prediction facility predicts the direction of the branch and, if possible, calculates the target address. For predicting the target of a subroutine return, the processor uses a return stack, one for each thread.

Instructions then move into two separate instruction buffers. Then, on the basis of thread priority, a group of instructions is selected and decoded in parallel. Next, instructions flow through a register-renaming facility in program order. Logical registers are mapped to physical registers. The Power5 has 120 physical general-purpose registers and 120 physical floating-point registers. The instructions are then moved into issue queues. From the issue queues, instructions are issued



BXU = Branch execution unit and  
 CRL = Condition register logical execution unit  
 FPU = Floating-point execution unit  
 FXU = Fixed-point execution unit  
 LSU = Load/store unit

**Figure 17.8** Power5 Instruction Data Flow

using symmetric multithreading. That is, the processor has a superscalar architecture and can issue instructions from one or both threads in parallel. At the end of the pipeline, separate thread resources are needed to commit the instructions.

## 17.5 CLUSTERS

An important and relatively recent development computer system design is clustering. Clustering is an alternative to symmetric multiprocessing as an approach to providing high performance and high availability and is particularly attractive for server applications. We can define a cluster as a group of interconnected, whole computers working together as a unified computing resource that can create the illusion of being one machine. The term *whole computer* means a system that can run on its own, apart from the cluster; in the literature, each computer in a cluster is typically referred to as a *node*.

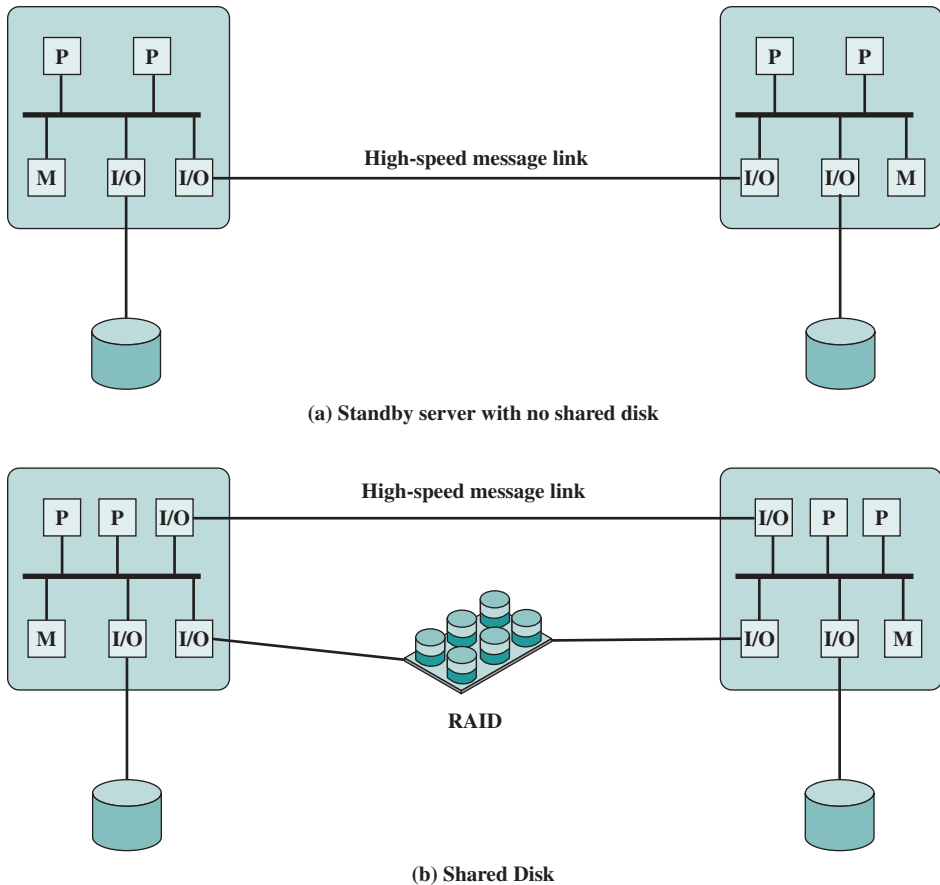
[BREW97] lists four benefits that can be achieved with clustering. These can also be thought of as objectives or design requirements:

- **Absolute scalability:** It is possible to create large clusters that far surpass the power of even the largest standalone machines. A cluster can have tens, hundreds, or even thousands of machines, each of which is a multiprocessor.
- **Incremental scalability:** A cluster is configured in such a way that it is possible to add new systems to the cluster in small increments. Thus, a user can start out with a modest system and expand it as needs grow, without having to go through a major upgrade in which an existing small system is replaced with a larger system.
- **High availability:** Because each node in a cluster is a standalone computer, the failure of one node does not mean loss of service. In many products, fault tolerance is handled automatically in software.
- **Superior price/performance:** By using commodity building blocks, it is possible to put together a cluster with equal or greater computing power than a single large machine, at much lower cost.

### Cluster Configurations

In the literature, clusters are classified in a number of different ways. Perhaps the simplest classification is based on whether the computers in a cluster share access to the same disks. Figure 17.9a shows a two-node cluster in which the only interconnection is by means of a high-speed link that can be used for message exchange to coordinate cluster activity. The link can be a LAN that is shared with other computers that are not part of the cluster or the link can be a dedicated interconnection facility. In the latter case, one or more of the computers in the cluster will have a link to a LAN or WAN so that there is a connection between the server cluster and remote client systems. Note that in the figure, each computer is depicted as being a multiprocessor. This is not necessary but does enhance both performance and availability.

In the simple classification depicted in Figure 17.9, the other alternative is a shared-disk cluster. In this case, there generally is still a message link between



**Figure 17.9** Cluster Configurations

nodes. In addition, there is a disk subsystem that is directly linked to multiple computers within the cluster. In this figure, the common disk subsystem is a RAID system. The use of RAID or some similar redundant disk technology is common in clusters so that the high availability achieved by the presence of multiple computers is not compromised by a shared disk that is a single point of failure.

A clearer picture of the range of cluster options can be gained by looking at functional alternatives. Table 17.2 provides a useful classification along functional lines, which we now discuss.

A common, older method, known as **passive standby**, is simply to have one computer handle all of the processing load while the other computer remains inactive, standing by to take over in the event of a failure of the primary. To coordinate the machines, the active, or primary, system periodically sends a “heartbeat” message to the standby machine. Should these messages stop arriving, the standby assumes that the primary server has failed and puts itself into operation. This approach increases availability but does not improve performance. Further, if the only information that is exchanged between the two systems is a heartbeat message,

**Table 17.2** Clustering Methods: Benefits and Limitations

| Clustering Method          | Description                                                                                                                              | Benefits                                                                          | Limitations                                                                                |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Passive Standby</b>     | A secondary server takes over in case of primary server failure.                                                                         | Easy to implement.                                                                | High cost because the secondary server is unavailable for other processing tasks.          |
| <b>Active Secondary:</b>   | The secondary server is also used for processing tasks.                                                                                  | Reduced cost because secondary servers can be used for processing.                | Increased complexity.                                                                      |
| Separate Servers           | Separate servers have their own disks. Data is continuously copied from primary to secondary server.                                     | High availability.                                                                | High network and server overhead due to copying operations.                                |
| Servers Connected to Disks | Servers are cabled to the same disks, but each server owns its disks. If one server fails, its disks are taken over by the other server. | Reduced network and server overhead due to elimination of copying operations.     | Usually requires disk mirroring or RAID technology to compensate for risk of disk failure. |
| Servers Share Disks        | Multiple servers simultaneously share access to disks.                                                                                   | Low network and server overhead. Reduced risk of downtime caused by disk failure. | Requires lock manager software. Usually used with disk mirroring or RAID technology.       |

and if the two systems do not share common disks, then the standby provides a functional backup but has no access to the databases managed by the primary.

The passive standby is generally not referred to as a cluster. The term *cluster* is reserved for multiple interconnected computers that are all actively doing processing while maintaining the image of a single system to the outside world. The term **active secondary** is often used in referring to this configuration. Three classifications of clustering can be identified: separate servers, shared nothing, and shared memory.

In one approach to clustering, each computer is a **separate server** with its own disks and there are no disks shared between systems (Figure 17.9a). This arrangement provides high performance as well as high availability. In this case, some type of management or scheduling software is needed to assign incoming client requests to servers so that the load is balanced and high utilization is achieved. It is desirable to have a failover capability, which means that if a computer fails while executing an application, another computer in the cluster can pick up and complete the application. For this to happen, data must constantly be copied among systems so that each system has access to the current data of the other systems. The overhead of this data exchange ensures high availability at the cost of a performance penalty.

To reduce the communications overhead, most clusters now consist of servers connected to common disks (Figure 17.9b). In one variation on this approach, called **shared nothing**, the common disks are partitioned into volumes, and each volume is owned by a single computer. If that computer fails, the cluster must be reconfigured so that some other computer has ownership of the volumes of the failed computer.

It is also possible to have multiple computers share the same disks at the same time (called the **shared disk** approach), so that each computer has access to all of the volumes on all of the disks. This approach requires the use of some type of locking facility to ensure that data can only be accessed by one computer at a time.

## Operating System Design Issues

Full exploitation of a cluster hardware configuration requires some enhancements to a single-system operating system.

**FAILURE MANAGEMENT** How failures are managed by a cluster depends on the clustering method used (Table 17.2). In general, two approaches can be taken to dealing with failures: highly available clusters and fault-tolerant clusters. A highly available cluster offers a high probability that all resources will be in service. If a failure occurs, such as a system goes down or a disk volume is lost, then the queries in progress are lost. Any lost query, if retried, will be serviced by a different computer in the cluster. However, the cluster operating system makes no guarantee about the state of partially executed transactions. This would need to be handled at the application level.

A fault-tolerant cluster ensures that all resources are always available. This is achieved by the use of redundant shared disks and mechanisms for backing out uncommitted transactions and committing completed transactions.

The function of switching applications and data resources over from a failed system to an alternative system in the cluster is referred to as **failover**. A related function is the restoration of applications and data resources to the original system once it has been fixed; this is referred to as **failback**. Failback can be automated, but this is desirable only if the problem is truly fixed and unlikely to recur. If not, automatic failback can cause subsequently failed resources to bounce back and forth between computers, resulting in performance and recovery problems.

**LOAD BALANCING** A cluster requires an effective capability for balancing the load among available computers. This includes the requirement that the cluster be incrementally scalable. When a new computer is added to the cluster, the load-balancing facility should automatically include this computer in scheduling applications. Middleware mechanisms need to recognize that services can appear on different members of the cluster and may migrate from one member to another.

**PARALLELIZING COMPUTATION** In some cases, effective use of a cluster requires executing software from a single application in parallel. [KAPP00] lists three general approaches to the problem:

- **Parallelizing compiler:** A parallelizing compiler determines, at compile time, which parts of an application can be executed in parallel. These are then split off to be assigned to different computers in the cluster. Performance depends on the nature of the problem and how well the compiler is designed. In general, such compilers are difficult to develop.
- **Parallelized application:** In this approach, the programmer writes the application from the outset to run on a cluster, and uses message passing to move data, as required, between cluster nodes. This places a high burden on the programmer but may be the best approach for exploiting clusters for some applications.

- **Parametric computing:** This approach can be used if the essence of the application is an algorithm or program that must be executed a large number of times, each time with a different set of starting conditions or parameters. A good example is a simulation model, which will run a large number of different scenarios and then develop statistical summaries of the results. For this approach to be effective, parametric processing tools are needed to organize, run, and manage the jobs in an effective manner.

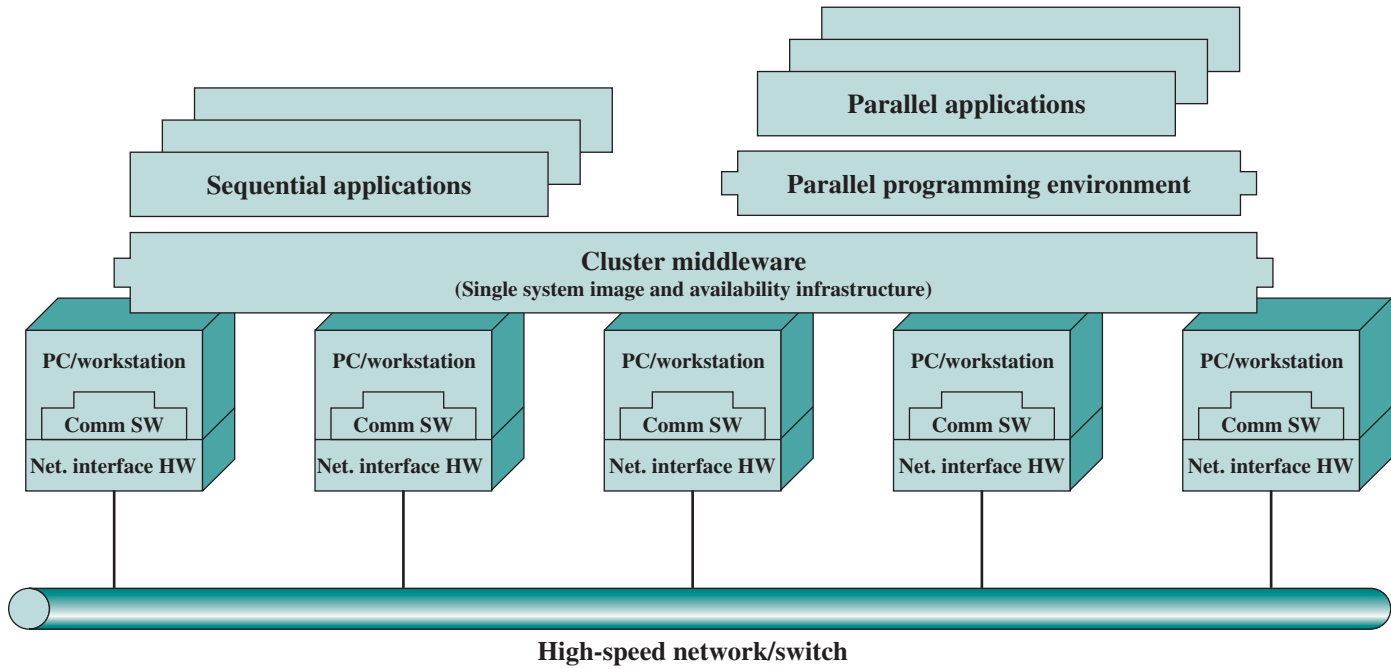
## Cluster Computer Architecture

Figure 17.10 shows a typical cluster architecture. The individual computers are connected by some high-speed LAN or switch hardware. Each computer is capable of operating independently. In addition, a middleware layer of software is installed in each computer to enable cluster operation. The cluster middleware provides a unified system image to the user, known as a **single-system image**. The middleware is also responsible for providing high availability, by means of load balancing and responding to failures in individual components. [HWAN99] lists the following as desirable cluster middleware services and functions:

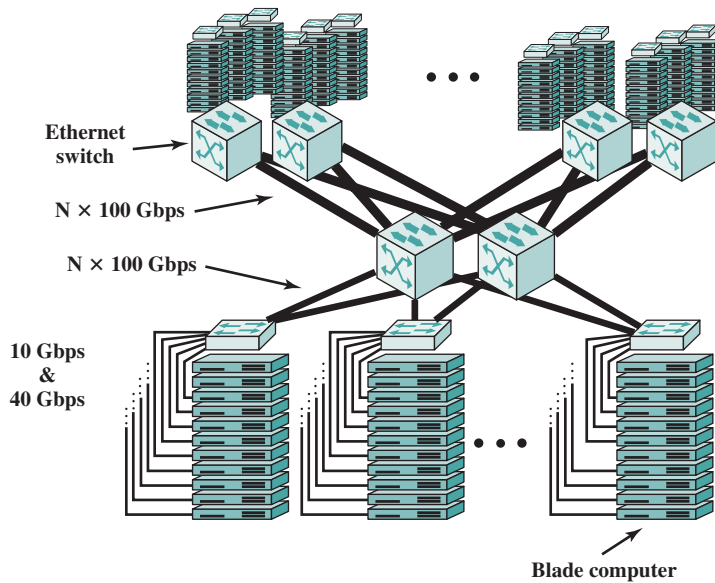
- **Single entry point:** A user logs onto the cluster rather than to an individual computer.
- **Single file hierarchy:** The user sees a single hierarchy of file directories under the same root directory.
- **Single control point:** There is a default workstation used for cluster management and control.
- **Single virtual networking:** Any node can access any other point in the cluster, even though the actual cluster configuration may consist of multiple interconnected networks. There is a single virtual network operation.
- **Single memory space:** Distributed shared memory enables programs to share variables.
- **Single job-management system:** Under a cluster job scheduler, a user can submit a job without specifying the host computer to execute the job.
- **Single user interface:** A common graphic interface supports all users, regardless of the workstation from which they enter the cluster.
- **Single I/O space:** Any node can remotely access any I/O peripheral or disk device without knowledge of its physical location.
- **Single process space:** A uniform process-identification scheme is used. A process on any node can create or communicate with any other process on a remote node.
- **Checkpointing:** This function periodically saves the process state and intermediate computing results, to allow rollback recovery after a failure.
- **Process migration:** This function enables load balancing.

The last four items on the preceding list enhance the availability of the cluster. The remaining items are concerned with providing a single system image.





**Figure 17.10** Cluster Computer Architecture [BUY99a]



**Figure 17.11** Example 100-Gbps Ethernet Configuration for Massive Blade Server Site

Returning to Figure 17.10, a cluster will also include software tools for enabling the efficient execution of programs that are capable of parallel execution.

### Blade Servers

A common implementation of the cluster approach is the blade server. A blade server is a server architecture that houses multiple server modules (“blades”) in a single chassis. It is widely used in data centers to save space and improve system management. Either self-standing or rack mounted, the chassis provides the power supply, and each blade has its own processor, memory, and hard disk.

An example of the application is shown in Figure 17.11, taken from [NOWE07]. The trend at large data centers, with substantial banks of blade servers, is the deployment of 10-Gbps ports on individual servers to handle the massive multimedia traffic provided by these servers. Such arrangements are stressing the on-site Ethernet switches needed to interconnect large numbers of servers. A 100-Gbps rate provides the bandwidth required to handle the increased traffic load. The 100-Gbps Ethernet switches are deployed in switch uplinks inside the data center as well as providing interbuilding, intercampus, wide area connections for enterprise networks.

### Clusters Compared to SMP

Both clusters and symmetric multiprocessors provide a configuration with multiple processors to support high-demand applications. Both solutions are commercially available, although SMP schemes have been around far longer.

The main strength of the SMP approach is that an SMP is easier to manage and configure than a cluster. The SMP is much closer to the original single-processor

model for which nearly all applications are written. The principal change required in going from a uniprocessor to an SMP is to the scheduler function. Another benefit of the SMP is that it usually takes up less physical space and draws less power than a comparable cluster. A final important benefit is that the SMP products are well established and stable.

Over the long run, however, the advantages of the cluster approach are likely to result in clusters dominating the high-performance server market. Clusters are far superior to SMPs in terms of incremental and absolute scalability. Clusters are also superior in terms of availability, because all components of the system can readily be made highly redundant.

## 17.6 NONUNIFORM MEMORY ACCESS

In terms of commercial products, the two common approaches to providing a multiple-processor system to support applications are SMPs and clusters. For some years, another approach, known as nonuniform memory access (NUMA), has been the subject of research and commercial NUMA products are now available.

Before proceeding, we should define some terms often found in the NUMA literature.

- **Uniform memory access (UMA):** All processors have access to all parts of main memory using loads and stores. The memory access time of a processor to all regions of memory is the same. The access times experienced by different processors are the same. The SMP organization discussed in Sections 17.2 and 17.3 is UMA.
- **Nonuniform memory access (NUMA):** All processors have access to all parts of main memory using loads and stores. The memory access time of a processor differs depending on which region of main memory is accessed. The last statement is true for all processors; however, for different processors, which memory regions are slower and which are faster differ.
- **Cache-coherent NUMA (CC-NUMA):** A NUMA system in which cache coherence is maintained among the caches of the various processors.

A NUMA system without cache coherence is more or less equivalent to a cluster. The commercial products that have received much attention recently are CC-NUMA systems, which are quite distinct from both SMPs and clusters. Usually, but unfortunately not always, such systems are in fact referred to in the commercial literature as CC-NUMA systems. This section is concerned only with CC-NUMA systems.

### Motivation

With an SMP system, there is a practical limit to the number of processors that can be used. An effective cache scheme reduces the bus traffic between any one processor and main memory. As the number of processors increases, this bus traffic also increases. Also, the bus is used to exchange cache-coherence signals, further adding to the burden. At some point, the bus becomes a performance bottleneck. Performance degradation seems to limit the number of processors in an SMP

configuration to somewhere between 16 and 64 processors. For example, Silicon Graphics' Power Challenge SMP is limited to 64 R10000 processors in a single system; beyond this number performance degrades substantially.

The processor limit in an SMP is one of the driving motivations behind the development of cluster systems. However, with a cluster, each node has its own private main memory; applications do not see a large global memory. In effect, coherency is maintained in software rather than hardware. This memory granularity affects performance and, to achieve maximum performance, software must be tailored to this environment. One approach to achieving large-scale multiprocessing while retaining the flavor of SMP is NUMA. For example, the Silicon Graphics Origin NUMA system is designed to support up to 1024 MIPS R10000 processors [WHIT97] and the Sequent NUMA-Q system is designed to support up to 252 Pentium II processors [LOVE96].

The objective with NUMA is to maintain a transparent system wide memory while permitting multiple multiprocessor nodes, each with its own bus or other internal interconnect system.

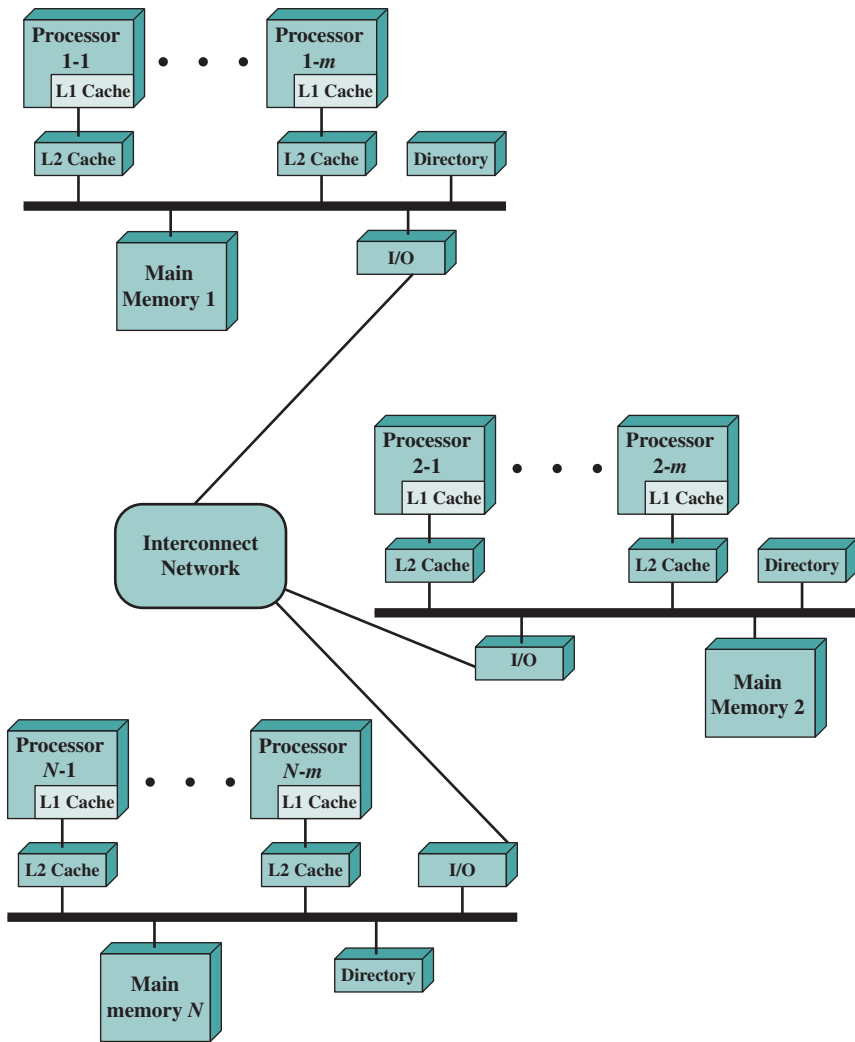
## Organization

Figure 17.12 depicts a typical CC-NUMA organization. There are multiple independent nodes, each of which is, in effect, an SMP organization. Thus, each node contains multiple processors, each with its own L1 and L2 caches, plus main memory. The node is the basic building block of the overall CC-NUMA organization. For example, each Silicon Graphics Origin node includes two MIPS R10000 processors; each Sequent NUMA-Q node includes four Pentium II processors. The nodes are interconnected by means of some communications facility, which could be a switching mechanism, a ring, or some other networking facility.

Each node in the CC-NUMA system includes some main memory. From the point of view of the processors, however, there is only a single addressable memory, with each location having a unique system wide address. When a processor initiates a memory access, if the requested memory location is not in that processor's cache, then the L2 cache initiates a fetch operation. If the desired line is in the local portion of the main memory, the line is fetched across the local bus. If the desired line is in a remote portion of the main memory, then an automatic request is sent out to fetch that line across the interconnection network, deliver it to the local bus, and then deliver it to the requesting cache on that bus. All of this activity is automatic and transparent to the processor and its cache.

In this configuration, cache coherence is a central concern. Although implementations differ as to details, in general terms we can say that each node must maintain some sort of directory that gives it an indication of the location of various portions of memory and also cache status information. To see how this scheme works, we give an example taken from [PFIS98]. Suppose that processor 3 on node 2 (P2-3) requests a memory location 798, which is in the memory of node 1. The following sequence occurs:

1. P2-3 issues a read request on the snoopy bus of node 2 for location 798.
2. The directory on node 2 sees the request and recognizes that the location is in node 1.



**Figure 17.12** CC-NUMA Organization

3. Node 2's directory sends a request to node 1, which is picked up by node 1's directory.
4. Node 1's directory, acting as a surrogate of P2-3, requests the contents of 798, as if it were a processor.
5. Node 1's main memory responds by putting the requested data on the bus.
6. Node 1's directory picks up the data from the bus.
7. The value is transferred back to node 2's directory.
8. Node 2's directory places the data back on node 2's bus, acting as a surrogate for the memory that originally held it.
9. The value is picked up and placed in P2-3's cache and delivered to P2-3.

The preceding sequence explains how data are read from a remote memory using hardware mechanisms that make the transaction transparent to the processor. On top of these mechanisms, some form of cache coherence protocol is needed. Various systems differ on exactly how this is done. We make only a few general remarks here. First, as part of the preceding sequence, node 1's directory keeps a record that some remote cache has a copy of the line containing location 798. Then, there needs to be a cooperative protocol to take care of modifications. For example, if a modification is done in a cache, this fact can be broadcast to other nodes. Each node's directory that receives such a broadcast can then determine if any local cache has that line and, if so, cause it to be purged. If the actual memory location is at the node receiving the broadcast notification, then that node's directory needs to maintain an entry indicating that that line of memory is invalid and remains so until a write back occurs. If another processor (local or remote) requests the invalid line, then the local directory must force a write back to update memory before providing the data.

### NUMA Pros and Cons

The main advantage of a CC-NUMA system is that it can deliver effective performance at higher levels of parallelism than SMP, without requiring major software changes. With multiple NUMA nodes, the bus traffic on any individual node is limited to a demand that the bus can handle. However, if many of the memory accesses are to remote nodes, performance begins to break down. There is reason to believe that this performance breakdown can be avoided. First, the use of L1 and L2 caches is designed to minimize all memory accesses, including remote ones. If much of the software has good temporal locality, then remote memory accesses should not be excessive. Second, if the software has good spatial locality, and if virtual memory is in use, then the data needed for an application will reside on a limited number of frequently used pages that can be initially loaded into the memory local to the running application. The Sequent designers report that such spatial locality does appear in representative applications [LOVE96]. Finally, the virtual memory scheme can be enhanced by including in the operating system a page migration mechanism that will move a virtual memory page to a node that is frequently using it; the Silicon Graphics designers report success with this approach [WHIT97].

Even if the performance breakdown due to remote access is addressed, there are two other disadvantages for the CC-NUMA approach [PFIS98]. First, a CC-NUMA does not transparently look like an SMP; software changes will be required to move an operating system and applications from an SMP to a CC-NUMA system. These include page allocation, already mentioned, process allocation, and load balancing by the operating system. A second concern is that of availability. This is a rather complex issue and depends on the exact implementation of the CC-NUMA system; the interested reader is referred to [PFIS98].



## 17.7 VECTOR COMPUTATION

Although the performance of mainframe general-purpose computers continues to improve relentlessly, there continue to be applications that are beyond the reach of the contemporary mainframe. There is a need for computers to solve mathematical problems of physical processes, such as occur in disciplines including aerodynamics, seismology, meteorology, and atomic, nuclear, and plasma physics.

Typically, these problems are characterized by the need for high precision and a program that repetitively performs floating-point arithmetic operations on large arrays of numbers. Most of these problems fall into the category known as *continuous-field simulation*. In essence, a physical situation can be described by a surface or region in three dimensions (e.g., the flow of air adjacent to the surface of a rocket). This surface is approximated by a grid of points. A set of differential equations defines the physical behavior of the surface at each point. The equations are represented as an array of values and coefficients, and the solution involves repeated arithmetic operations on the arrays of data.

Supercomputers were developed to handle these types of problems. These machines are typically capable of billions of floating-point operations per second. In contrast to mainframes, which are designed for multiprogramming and intensive I/O, the supercomputer is optimized for the type of numerical calculation just described.

The supercomputer has limited use and, because of its price tag, a limited market. Comparatively few of these machines are operational, mostly at research centers and some government agencies with scientific or engineering functions. As with other areas of computer technology, there is a constant demand to increase the performance of the supercomputer. Thus, the technology and performance of the supercomputer continues to evolve.

There is another type of system that has been designed to address the need for vector computation, referred to as the *array processor*. Although a supercomputer is optimized for vector computation, it is a general-purpose computer, capable of handling scalar processing and general data processing tasks. Array processors do not include scalar processing; they are configured as peripheral devices by both mainframe and minicomputer users to run the vectorized portions of programs.

### Approaches to Vector Computation

The key to the design of a supercomputer or array processor is to recognize that the main task is to perform arithmetic operations on arrays or vectors of floating-point numbers. In a general-purpose computer, this will require iteration through each element of the array. For example, consider two vectors (one-dimensional arrays) of numbers,  $A$  and  $B$ . We would like to add these and place the result in  $C$ . In the example of Figure 17.13, this requires six separate additions. How could we speed up this computation? The answer is to introduce some form of parallelism.

Several approaches have been taken to achieving parallelism in vector computation. We illustrate this with an example. Consider the vector multiplication  $C = A \times B$ , where  $A$ ,  $B$ , and  $C$  are  $N \times N$  matrices. The formula for each element of  $C$  is

$$c_{i,j} = \sum_{k=1}^N a_{i,k} \times b_{k,j}$$

$$\begin{array}{rcccl}
 \begin{bmatrix} 1.5 \\ 7.1 \\ 6.9 \\ 100.5 \\ 0 \\ 59.7 \end{bmatrix} & + & \begin{bmatrix} 2.0 \\ 39.7 \\ 1000.003 \\ 11 \\ 21.1 \\ 19.7 \end{bmatrix} & = & \begin{bmatrix} 3.5 \\ 46.8 \\ 1006.093 \\ 111.5 \\ 21.1 \\ 79.4 \end{bmatrix} \\
 A & + & B & = & C
 \end{array}$$

**Figure 17.13** Example of Vector Addition

where  $A$ ,  $B$ , and  $C$  have elements  $a_{i,j}$ ,  $b_{i,j}$ , and  $c_{i,j}$  respectively. Figure 17.14a shows a FORTRAN program for this computation that can be run on an ordinary scalar processor.

One approach to improving performance can be referred to as *vector processing*. This assumes that it is possible to operate on a one-dimensional vector of data. Figure 17.14b is a FORTRAN program with a new form of instruction that allows

```

DO 100 I = 1, N
DO 100 J = 1, N
C(I, J) = 0.0
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
100 CONTINUE

```

(a) Scalar processing

```

DO 100 I = 1, N
C(I, J) = 0.0 (J = 1, N)
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J) (J = 1, N)
100 CONTINUE

```

(b) Vector processing

```

DO 50 J = 1, N - 1
FORK 100
50 CONTINUE
J = N
100 DO 200 I = 1, N
C(I, J) = 0.0
DO 200 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
200 CONTINUE
JOIN N

```

(c) Parallel processing

**Figure 17.14** Matrix Multiplication ( $C = A \times B$ )



vector computation to be specified. The notation ( $J = 1, N$ ) indicates that operations on all indices  $J$  in the given interval are to be carried out as a single operation. How this can be achieved is addressed shortly.

The program in Figure 17.14b indicates that all the elements of the  $i$ th row are to be computed in parallel. Each element in the row is a summation, and the summations (across  $K$ ) are done serially rather than in parallel. Even so, only  $N^2$  vector multiplications are required for this algorithm as compared with  $N^3$  scalar multiplications for the scalar algorithm.

Another approach, *parallel processing*, is illustrated in Figure 17.14c. This approach assumes that we have  $N$  independent processors that can function in parallel. To utilize processors effectively, we must somehow parcel out the computation to the various processors. Two primitives are used. The primitive `FORK  $n$`  causes an independent process to be started at location  $n$ . In the meantime, the original process continues execution at the instruction immediately following the `FORK`. Every execution of a `FORK` spawns a new process. The `JOIN` instruction is essentially the inverse of the `FORK`. The statement `JOIN  $N$`  causes  $N$  independent processes to be merged into one that continues execution at the instruction following the `JOIN`. The operating system must coordinate this merger, and so the execution does not continue until all  $N$  processes have reached the `JOIN` instruction.

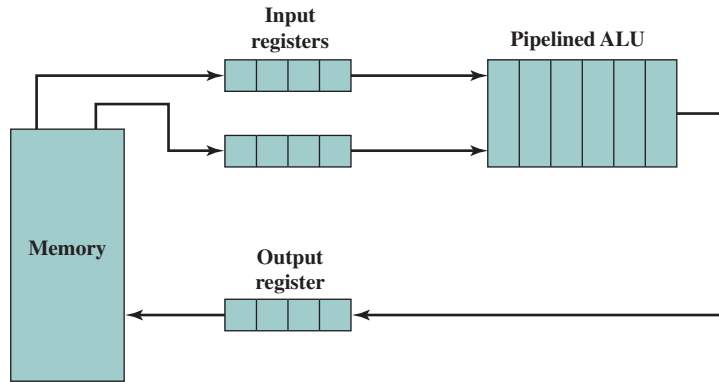
The program in Figure 17.15c is written to mimic the behavior of the vector-processing program. In the parallel processing program, each column of  $C$  is computed by a separate process. Thus, the elements in a given row of  $C$  are computed in parallel.

The preceding discussion describes approaches to vector computation in logical or architectural terms. Let us turn now to a consideration of types of processor organization that can be used to implement these approaches. A wide variety of organizations have been and are being pursued. Three main categories stand out:

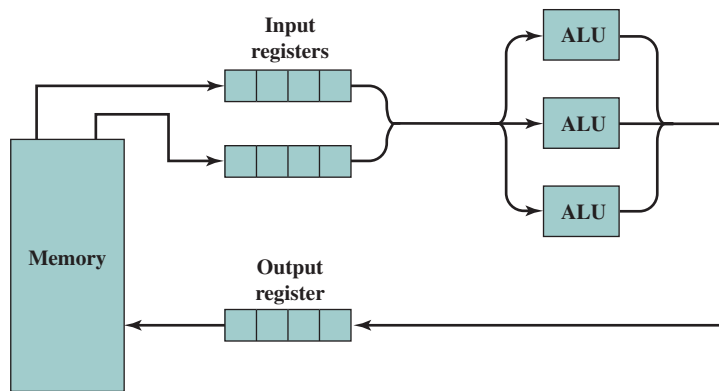
- Pipelined ALU
- Parallel ALUs
- Parallel processors

Figure 17.15 illustrates the first two of these approaches. We have already discussed pipelining in Chapter 14. Here the concept is extended to the operation of the ALU. Because floating-point operations are rather complex, there is opportunity for decomposing a floating-point operation into stages, so that different stages can operate on different sets of data concurrently. This is illustrated in Figure 17.16a. Floating-point addition is broken up into four stages (see Figure 10.22): compare, shift, add, and normalize. A vector of numbers is presented sequentially to the first stage. As the processing proceeds, four different sets of numbers will be operated on concurrently in the pipeline.

It should be clear that this organization is suitable for vector processing. To see this, consider the instruction pipelining described in Chapter 14. The processor goes through a repetitive cycle of fetching and processing instructions. In the absence of branches, the processor is continuously fetching instructions from sequential locations. Consequently, the pipeline is kept full and a savings in time is achieved. Similarly, a pipelined ALU will save time only if it is fed a stream of data from



(a) Pipelined ALU



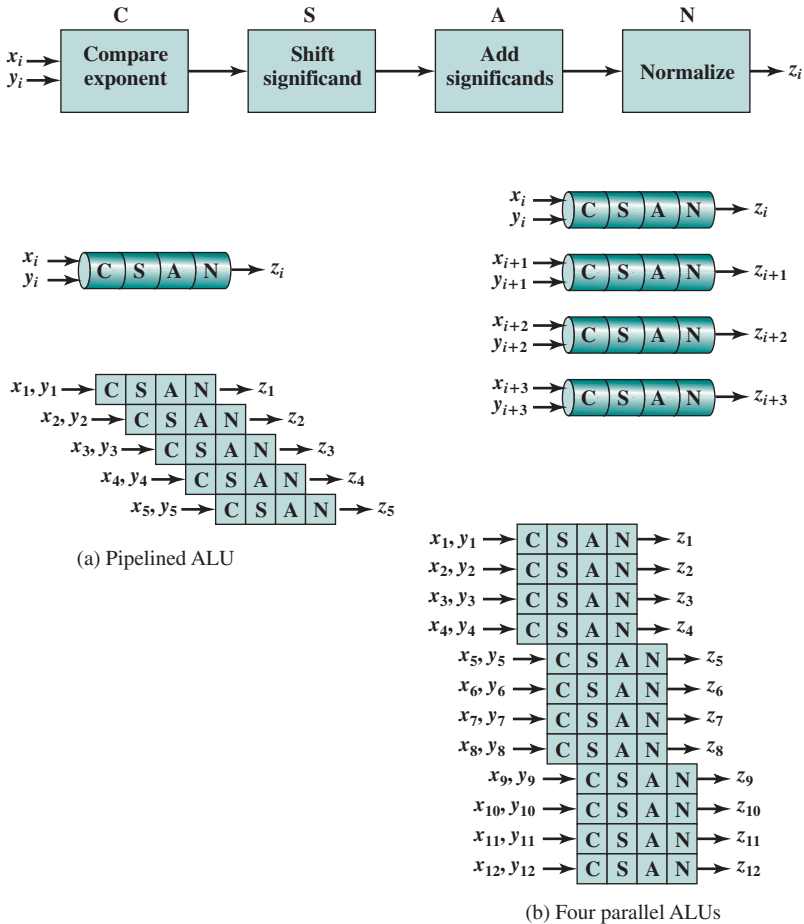
(b) Parallel ALUs

**Figure 17.15** Approaches to Vector Computation

sequential locations. A single, isolated floating-point operation is not speeded up by a pipeline. The speedup is achieved when a vector of operands is presented to the ALU. The control unit cycles the data through the ALU until the entire vector is processed.

The pipeline operation can be further enhanced if the vector elements are available in registers rather than from main memory. This is in fact suggested by Figure 17.15a. The elements of each vector operand are loaded as a block into a vector register, which is simply a large bank of identical registers. The result is also placed in a vector register. Thus, most operations involve only the use of registers, and only load and store operations and the beginning and end of a vector operation require access to memory.

The mechanism illustrated in Figure 17.16 could be referred to as *pipelining within an operation*. That is, we have a single arithmetic operation (e.g.,  $C = A + B$ ) that is to be applied to vector operands, and pipelining allows multiple vector elements to be processed in parallel. This mechanism can be augmented with *pipelining across operations*. In this latter case, there is a sequence of arithmetic vector operations, and instruction pipelining is used to speed up processing. One approach



**Figure 17.16** Pipelined Processing of Floating-Point Operations

to this, referred to as **chaining**, is found on the Cray supercomputers. The basic rule for chaining is this: A vector operation may start as soon as the first element of the operand vector(s) is available and the functional unit (e.g., add, subtract, multiply, divide) is free. Essentially, chaining causes results issuing from one functional unit to be fed immediately into another functional unit and so on. If vector registers are used, intermediate results do not have to be stored into memory and can be used even before the vector operation that created them runs to completion.

For example, when computing  $C = (s \times A) + B$ , where  $A$ ,  $B$ , and  $C$  are vectors and  $s$  is a scalar, the Cray may execute three instructions at once. Elements fetched for a load immediately enter a pipelined multiplier, the products are sent to a pipelined adder, and the sums are placed in a vector register as soon as the adder completes them:

1. Vector load  $A \rightarrow \text{Vector Register (VR1)}$
2. Vector load  $B \rightarrow \text{VR2}$

3. Vector multiply  $s \times \text{VR1} \rightarrow \text{VR3}$
4. Vector add  $\text{VR3} + \text{VR2} \rightarrow \text{VR4}$
5. Vector store  $\text{VR4} \rightarrow \text{C}$

Instructions 2 and 3 can be chained (pipelined) because they involve different memory locations and registers. Instruction 4 needs the results of instructions 2 and 3, but it can be chained with them as well. As soon as the first elements of vector registers 2 and 3 are available, the operation in instruction 4 can begin.

Another way to achieve vector processing is by the use of multiple ALUs in a single processor, under the control of a single control unit. In this case, the control unit routes data to ALUs so that they can function in parallel. It is also possible to use pipelining on each of the parallel ALUs. This is illustrated in Figure 17.16b. The example shows a case in which four ALUs operate in parallel.

As with pipelined organization, a parallel ALU organization is suitable for vector processing. The control unit routes vector elements to ALUs in a round-robin fashion until all elements are processed. This type of organization is more complex than a single-ALU CPI.

Finally, vector processing can be achieved by using multiple parallel processors. In this case, it is necessary to break the task up into multiple processes to be executed in parallel. This organization is effective only if the software and hardware for effective coordination of parallel processors is available.

We can expand our taxonomy of Section 17.1 to reflect these new structures, as shown in Figure 17.17. Computer organizations can be distinguished by the presence of one or more control units. Multiple control units imply multiple processors. Following our previous discussion, if the multiple processors can function cooperatively on a given task, they are termed *parallel processors*.

The reader should be aware of some unfortunate terminology likely to be encountered in the literature. The term *vector processor* is often equated with a pipelined ALU organization, although a parallel ALU organization is also designed for vector processing, and, as we have discussed, a parallel processor organization may also be designed for vector processing. *Array processing* is sometimes used to refer to a parallel ALU, although, again, any of the three organizations is optimized for the processing of arrays. To make matters worse, *array processor* usually refers to an auxiliary processor attached to a general-purpose processor and used to perform vector computation. An array processor may use either the pipelined or parallel ALU approach.

At present, the pipelined ALU organization dominates the marketplace. Pipelined systems are less complex than the other two approaches. Their control



**Figure 17.17** A Taxonomy of Computer Organizations

unit and operating system design are well developed to achieve efficient resource allocation and high performance. The remainder of this section is devoted to a more detailed examination of this approach, using a specific example.

### IBM 3090 Vector Facility

A good example of a pipelined ALU organization for vector processing is the vector facility developed for the IBM 370 architecture and implemented on the high-end 3090 series [PADE88, TUCK87]. This facility is an optional add-on to the basic system but is highly integrated with it. It resembles vector facilities found on supercomputers, such as the Cray family.

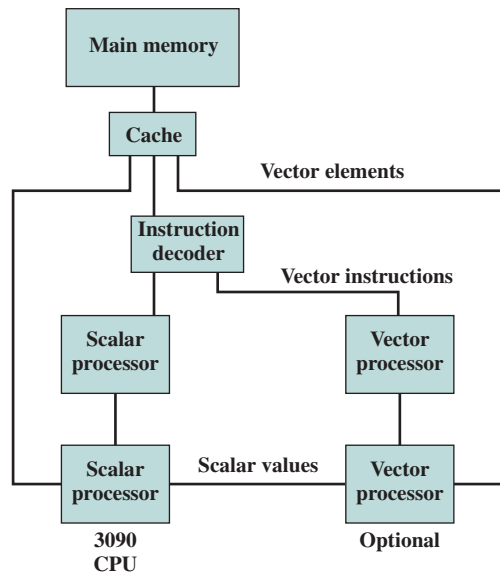
The IBM facility makes use of a number of vector registers. Each register is actually a bank of scalar registers. To compute the vector sum  $C = A + B$ , the vectors  $A$  and  $B$  are loaded into two vector registers. The data from these registers are passed through the ALU as fast as possible, and the results are stored in a third vector register. The computation overlap, and the loading of the input data into the registers in a block, results in a significant speeding up over an ordinary ALU operation.

**ORGANIZATION** The IBM vector architecture, and similar pipelined vector ALUs, provides increased performance over loops of scalar arithmetic instructions in three ways:

- The fixed and predetermined structure of vector data permits housekeeping instructions inside the loop to be replaced by faster internal (hardware or microcoded) machine operations.
- Data-access and arithmetic operations on several successive vector elements can proceed concurrently by overlapping such operations in a pipelined design or by performing multiple-element operations in parallel.
- The use of vector registers for intermediate results avoids additional storage reference.

Figure 17.18 shows the general organization of the vector facility. Although the vector facility is seen to be a physically separate add-on to the processor, its architecture is an extension of the System/370 architecture and is compatible with it. The vector facility is integrated into the System/370 architecture in the following ways:

- Existing System/370 instructions are used for all scalar operations.
- Arithmetic operations on individual vector elements produce exactly the same result as do corresponding System/370 scalar instructions. For example, one design decision concerned the definition of the result in a floating-point DIVIDE operation. Should the result be exact, as it is for scalar floating-point division, or should an approximation be allowed that would permit higher-speed implementation but could sometimes introduce an error in one or more low-order bit positions? The decision was made to uphold complete compatibility with the System/370 architecture at the expense of a minor performance degradation.
- Vector instructions are interruptible, and their execution can be resumed from the point of interruption after appropriate action has been taken, in a manner compatible with the System/370 program-interruption scheme.



**Figure 17.18** IBM 3090 with Vector Facility

- Arithmetic exceptions are the same as, or extensions of, exceptions for the scalar arithmetic instructions of the System/370, and similar fix-up routines can be used. To accommodate this, a vector interruption index is employed that indicates the location in a vector register that is affected by an exception (e.g., overflow). Thus, when execution of the vector instruction resumes, the proper place in a vector register is accessed.
- Vector data reside in virtual storage, with page faults being handled in a standard manner.

This level of integration provides a number of benefits. Existing operating systems can support the vector facility with minor extensions. Existing application programs, language compilers, and other software can be run unchanged. Software that could take advantage of the vector facility can be modified as desired.

**REGISTERS** A key issue in the design of a vector facility is whether operands are located in registers or memory. The IBM organization is referred to as *register to register*, because the vector operands, both input and output, can be staged in vector registers. This approach is also used on the Cray supercomputer. An alternative approach, used on Control Data machines, is to obtain operands directly from memory. The main disadvantage of the use of vector registers is that the programmer or compiler must take them into account for good performance. For example, suppose that the length of the vector registers is  $K$  and the length of the vectors to be processed is  $N > K$ . In this case, a vector loop must be performed, in which the operation is performed on  $K$  elements at a time and the loop is repeated  $N/K$  times. The main advantage of the vector register approach is that the operation is decoupled from slower main memory and instead takes place primarily with registers.

FORTTRAN ROUTINE:

```

DO 100 J = 1, 50
 CR(J) = AR(J) * BR(J) - AI(J) * BI(J)
100 CI(J) = AR(J) * BI(J) + AI(J) * BR(J)

```

| Operation             | Cycles    |
|-----------------------|-----------|
| AR(J) * BR(J) → T1(J) | 3         |
| AI(J) * BI(J) → T2(J) | 3         |
| T1(J) - T2(J) → CR(J) | 3         |
| AR(J) * BI(J) → T3(J) | 3         |
| AI(J) * BR(J) → T4(J) | 3         |
| T3(J) + T4(J) → CI(J) | 3         |
| <b>TOTAL</b>          | <b>18</b> |

(a) Storage to storage

| Operation             | Cycles    |
|-----------------------|-----------|
| AR(J) → V1(J)         | 1         |
| V1(J) * BR(J) → V2(J) | 1         |
| AI(J) → V3(J)         | 1         |
| V3(J) * BI(J) → V4(J) | 1         |
| V2(J) - V4(J) → V5(J) | 1         |
| V5(J) → CR(J)         | 1         |
| V1(J) * BI(J) → V6(J) | 1         |
| V4(J) * BR(J) → V7(J) | 1         |
| V6(J) + V7(J) → V8(J) | 1         |
| V8(J) → CI(J)         | 1         |
| <b>TOTAL</b>          | <b>10</b> |

(c) Storage to register

$V_i$  = Vector registers  
 AR, BR, AI, BI = Operands in memory  
 Ti = Temporary locations in memory

| Operation             | Cycles    |
|-----------------------|-----------|
| AR(J) → V1(J)         | 1         |
| BR(J) → V2(J)         | 1         |
| V1(J) * V2(J) → V3(J) | 1         |
| AI(J) → V4(J)         | 1         |
| BI(J) → V5(J)         | 1         |
| V4(J) * V5(J) → V6(J) | 1         |
| V3(J) - V6(J) → V7(J) | 1         |
| V7(J) → CR(J)         | 1         |
| V1(J) * V5(J) → V8(J) | 1         |
| V4(J) * V2(J) → V9(J) | 1         |
| V8(J) + V9(J) → V0(J) | 1         |
| V0(J) → CI(J)         | 1         |
| <b>TOTAL</b>          | <b>12</b> |

(b) Register to register

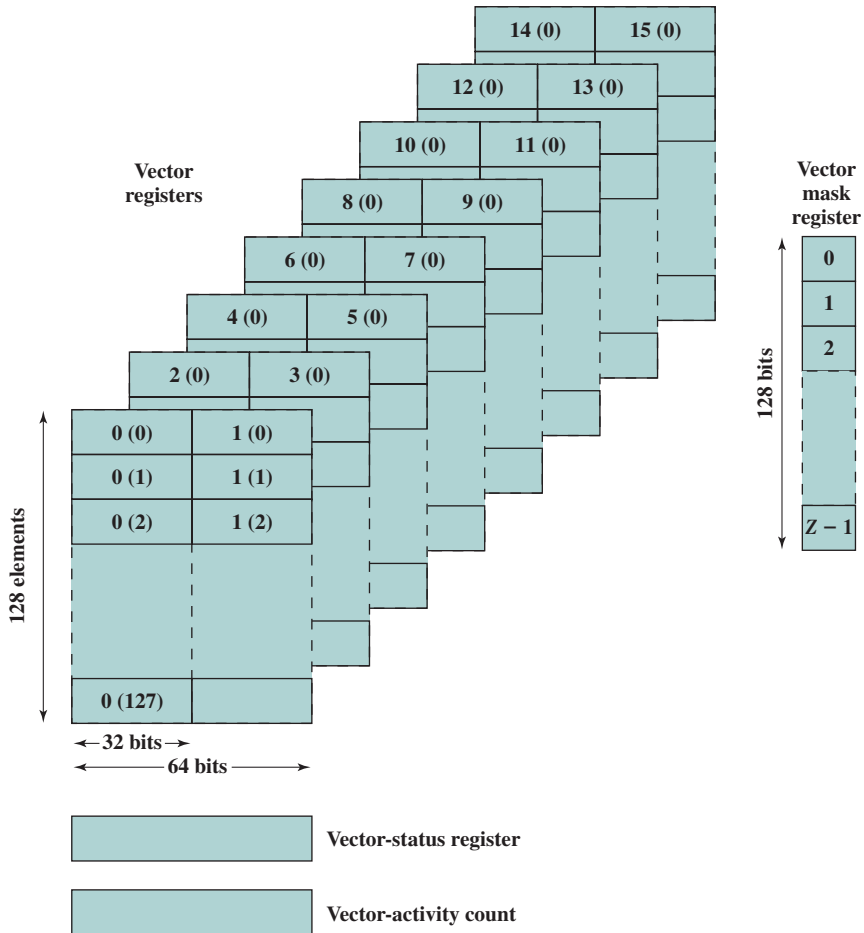
| Operation                     | Cycles   |
|-------------------------------|----------|
| AR(J) → V1(J)                 | 1        |
| V1(J) * BR(J) → V2(J)         | 1        |
| AI(J) → V3(J)                 | 1        |
| V2(J) - V3(J) * BI(J) → V2(J) | 1        |
| V2(J) → CR(J)                 | 1        |
| V1(J) * BI(J) → V4(J)         | 1        |
| V4(J) + V3(J) * BR(J) → V5(J) | 1        |
| V5(J) → CI(J)                 | 1        |
| <b>TOTAL</b>                  | <b>8</b> |

(d) Compound instruction

**Figure 17.19** Alternative Programs for Vector Calculation

The speedup that can be achieved using registers is demonstrated in Figure 17.19. The FORTRAN routine multiplies vector A by vector B to produce vector C, where each vector has a real part (AR, BR, CR) and an imaginary part (AI, BI, CI). The 3090 can perform one main-storage access per processor, or clock, cycle (either read or write); has registers that can sustain two accesses for reading and one for writing per cycle; and produces one result per cycle in its arithmetic unit. Let us assume the use of instructions that can specify two source operands and a result.<sup>4</sup> Part (a) of the figure shows that, with memory-to-memory instructions, each iteration of the computation requires a total of 18 cycles. With a pure register-to-register

<sup>4</sup>For the 370/390 architecture, the only three-operand instructions (register and storage instructions, RS) specify two operands in registers and one in memory. In part (a) of the example, we assume the existence of three-operand instructions in which all operands are in main memory. This is done for purposes of comparison and, in fact, such an instruction format could have been chosen for the vector architecture.



**Figure 17.20** Registers for the IBM 3090 Vector Facility

architecture (part (b)), this time is reduced to 12 cycles. Of course, with register-to-register operation, the vector quantities must be loaded into the vector registers prior to computation and stored in memory afterward. For large vectors, this fixed penalty is relatively small. Figure 17.19c shows that the ability to specify both storage and register operands in one instruction further reduces the time to 10 cycles per iteration. This latter type of instruction is included in the vector architecture.<sup>5</sup>

Figure 17.20 illustrates the registers that are part of the IBM 3090 vector facility. There are sixteen 32-bit vector registers. The vector registers can also be coupled to form eight 64-bit vector registers. Any register element can hold an integer or floating-point value. Thus, the vector registers may be used for 32-bit and 64-bit integer values, and 32-bit and 64-bit floating-point values.

<sup>5</sup>Compound instructions, discussed subsequently, afford a further reduction.



The architecture specifies that each register contains from 8 to 512 scalar elements. The choice of actual length involves a design trade-off. The time to do a vector operation consists essentially of the overhead for pipeline startup and register filling plus one cycle per vector element. Thus, the use of a large number of register elements reduces the relative startup time for a computation. However, this efficiency must be balanced against the added time required for saving and restoring vector registers on a process switch and the practical cost and space limits. These considerations led to the use of 128 elements per register in later 3090 implementations.

Three additional registers are needed by the vector facility. The vector-mask register contains mask bits that may be used to select which elements in the vector registers are to be processed for a particular operation. The vector-status register contains control fields, such as the vector count, that determine how many elements in the vector registers are to be processed. The vector-activity count keeps track of the time spent executing vector instructions.

**COMPOUND INSTRUCTIONS** As was discussed previously, instruction execution can be overlapped using chaining to improve performance. The designers of the IBM vector facility chose not to include this capability for several reasons. The System/370 architecture would have to be extended to handle complex interruptions (including their effect on virtual memory management), and corresponding changes would be needed in the software. A more basic issue was the cost of including the additional controls and register access paths in the vector facility for generalized chaining.

Instead, three operations are provided that combine into one instruction (one opcode) the most common sequences in vector computation, namely multiplication followed by addition, subtraction, or summation. The storage-to-register MULTIPLY-AND-ADD instruction, for example, fetches a vector from storage, multiplies it by a vector from a register, and adds the product to a third vector in a register. By use of the compound instructions MULTIPLY-AND-ADD and MULTIPLY-AND-SUBTRACT in the example of Figure 17.19, the total time for the iteration is reduced from 10 to 8 cycles.

Unlike chaining, compound instructions do not require the use of additional registers for temporary storage of intermediate results, and they require one less register access. For example, consider the following chain:

$$A \rightarrow VR1$$

$$VR1 + VR2 \rightarrow VR1$$

In this case, two stores to the vector register VR1 are required. In the IBM architecture there is a storage-to-register ADD instruction. With this instruction, only the sum is placed in VR1. The compound instruction also avoids the need to reflect in the machine-state description the concurrent execution of a number of instructions, which simplifies status saving and restoring by the operating system and the handling of interrupts.

**THE INSTRUCTION SET** Table 17.3 summarizes the arithmetic and logical operations that are defined for the vector architecture. In addition, there are memory-to-register

**Table 17.3** IBM 3090 Vector Facility: Arithmetic and Logical Instructions

| Operation               | Data Types     |       |                   |                             |                                |                                |                                |  |  |
|-------------------------|----------------|-------|-------------------|-----------------------------|--------------------------------|--------------------------------|--------------------------------|--|--|
|                         | Floating-Point |       | Binary or Logical |                             |                                |                                |                                |  |  |
|                         | Long           | Short | Binary or Logical |                             |                                |                                | Operand Locations              |  |  |
| Add                     | FL             | FS    | BI                | $V + V \rightarrow V$       | $V + S \rightarrow V$          | $Q + V \rightarrow V$          | $Q + S \rightarrow V$          |  |  |
| Subtract                | FL             | FS    | BI                | $V - V \rightarrow V$       | $V - S \rightarrow V$          | $V - V \rightarrow V$          | $Q - S \rightarrow V$          |  |  |
| Multiply                | FL             | FS    | BI                | $V \times V \rightarrow V$  | $V \times V \rightarrow V$     | $Q \times V \rightarrow V$     | $Q \times S \rightarrow V$     |  |  |
| Divide                  | FL             | FS    | —                 | $V/V \rightarrow V$         | $V/S \rightarrow V$            | $Q/V \rightarrow V$            | $Q/S \rightarrow V$            |  |  |
| Compare                 | FL             | FS    | BI                | $V \cdot V \rightarrow V$   | $V \cdot S \rightarrow V$      | $Q \cdot V \rightarrow V$      | $Q \cdot S \rightarrow V$      |  |  |
| Multiply and Add        | FL             | FS    | —                 |                             | $V + V \times S \rightarrow V$ | $V + Q \times V \rightarrow V$ | $V + Q \times S \rightarrow V$ |  |  |
| Multiply and Subtract   | FL             | FS    | —                 |                             | $V - V \times S \rightarrow V$ | $V - Q \times V \rightarrow V$ | $V - Q \times S \rightarrow V$ |  |  |
| Multiply and Accumulate | FL             | FS    | —                 | $P + \cdot V \rightarrow V$ | $P + \cdot S \rightarrow V$    |                                |                                |  |  |
| Complement              | FL             | FS    | BI                | $-V \rightarrow V$          |                                |                                |                                |  |  |
| Positive Absolute       | FL             | FS    | BI                | $ V  \rightarrow V$         |                                |                                |                                |  |  |
| Negative Absolute       | FL             | FS    | BI                | $- V  \rightarrow V$        |                                |                                |                                |  |  |
| Maximum                 | FL             | FS    | —                 |                             |                                | $Q \cdot V \rightarrow Q$      |                                |  |  |
| Maximum Absolute        | FL             | FS    | —                 |                             |                                | $Q \cdot V \rightarrow Q$      |                                |  |  |
| Minimum                 | FL             | FS    | —                 |                             |                                | $Q \cdot V \rightarrow Q$      |                                |  |  |
| Shift Left Logical      | —              | —     | LO                | $\cdot V \rightarrow Q$     |                                |                                |                                |  |  |
| Shift Right Logical     | —              | —     | LO                | $\cdot V \rightarrow Q$     |                                |                                |                                |  |  |
| And                     | —              | —     | LO                | $V \& V \rightarrow V$      | $V \& S \rightarrow V$         | $Q \& V \rightarrow V$         | $Q \& S \rightarrow V$         |  |  |
| OR                      | —              | —     | LO                | $V/V \rightarrow V$         | $V/S \rightarrow V$            | $Q/V \rightarrow V$            | $Q/S \rightarrow V$            |  |  |
| Exclusive-OR            | —              | —     | LO                | $V \oplus V \rightarrow V$  | $V \oplus S \rightarrow V$     | $Q \oplus V \rightarrow V$     | $Q \oplus S \rightarrow V$     |  |  |

|                    |                         |                                               |
|--------------------|-------------------------|-----------------------------------------------|
| <i>Explanation</i> | <b>Data types</b>       | <b>Operand locations</b>                      |
|                    | FL Long floating point  | V Vector register                             |
|                    | FS Short floating point | S Storerger                                   |
|                    | BI Binary integer       | Q Scalar (general or floating-point register) |
|                    | LO Logical              | P Partial sums in vector register             |
|                    |                         | . Special operation                           |

load and register-to-memory store instructions. Note that many of the instructions use a three-operand format. Also, many instructions have a number of variants, depending on the location of the operands. A source operand may be a vector register (V), storage (S), or a scalar register (Q). The target is always a vector register, except for comparison, the result of which goes into the vector-mask register. With all these variants, the total number of opcodes (distinct instructions) is 171. This rather large number, however, is not as expensive to implement as might be imagined. Once the machine provides the arithmetic units and the data paths to feed operands from storage, scalar registers, and vector registers to the vector pipelines, the major hardware cost has been incurred. The architecture can, with little difference in cost, provide a rich set of variants on the use of those registers and pipelines.

Most of the instructions in Table 17.3 are self-explanatory. The two summation instructions warrant further explanation. The accumulate operation adds together the elements of a single vector (ACCUMULATE) or the elements of the product of two vectors (MULTIPLY-AND-ACCUMULATE). These instructions present an interesting design problem. We would like to perform this operation as rapidly as possible, taking full advantage of the ALU pipeline. The difficulty is that the sum of two numbers put into the pipeline is not available until several cycles later. Thus, the third element in the vector cannot be added to the sum of the first two elements until those two elements have gone through the entire pipeline. To overcome this problem, the elements of the vector are added in such a way as to produce four partial sums. In particular, elements 0, 4, 8, 12, . . . , 124 are added in that order to produce partial sum 0; elements 1, 5, 9, 13, . . . , 125 to partial sum 1; elements 2, 6, 10, 14, . . . , 126 to partial sum 2; and elements 3, 7, 11, 15, . . . , 127 to partial sum 4. Each of these partial sums can proceed through the pipeline at top speed, because the delay in the pipeline is roughly four cycles. A separate vector register is used to hold the partial sums. When all elements of the original vector have been processed, the four partial sums are added together to produce the final result. The performance of this second phase is not critical, because only four vector elements are involved.

## 17.8 RECOMMENDED READING

[MILE00] is an overview of cache coherence algorithms and techniques for multiprocessors, with an emphasis on performance issues. Another survey of the issues relating to cache coherence in multiprocessors is [LILJ93]. [TOMA93] contains reprints of many of the key papers on the subject.

[UNGE02] is an excellent survey of the concepts of multithreaded processors and chip multiprocessors. [UNGE03] is a lengthy survey of both proposed and current multithreaded processors that use explicit multithreading.

A thorough treatment of clusters can be found in [BUYY99a] and [BUYY99b]. [WEYG01] is a less technical survey of clusters, with good commentary on various commercial products. [DESA05] describes IBM's blade server architecture.

Good discussions of vector computation can be found in [STON93] and [HWAN93].

- BUY99a** Buyya, R. *High-Performance Cluster Computing: Architectures and Systems*. Upper Saddle River, NJ: Prentice Hall, 1999.
- BUY99b** Buyya, R. *High-Performance Cluster Computing: Programming and Applications*. Upper Saddle River, NJ: Prentice Hall, 1999.
- DESA05** Desai, D., et al. "BladeCenter System Overview." *IBM Journal of Research and Development*, November 2005.
- LIL93** Lilja, D. "Cache Coherence in Large-Scale Shared-Memory Multiprocessors: Issues and Comparisons." *ACM Computing Surveys*, September 1993.
- MILE00** Milenkovic, A. "Achieving High Performance in Bus-Based Shared-Memory Multiprocessors." *IEEE Concurrency*, July-September 2000.
- TOMA93** Tomasevic, M., and Milutinovic, V. *The Cache Coherence Problem in Shared-Memory Multiprocessors: Hardware Solutions*. Los Alamitos, CA: IEEE Computer Society Press, 1993.
- UNGE02** Ungerer, T.; Rubic, B.; and Silc, J. "Multithreaded Processors." *The Computer Journal*, No. 3, 2002.
- UNGE03** Ungerer, T.; Rubic, B.; and Silc, J. "A Survey of Processors with Explicit Multithreading." *ACM Computing Surveys*, March, 2003.
- WEYG01** Weygant, P. *Clusters for High Availability*. Upper Saddle River, NJ: Prentice Hall, 2001.

## 17.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

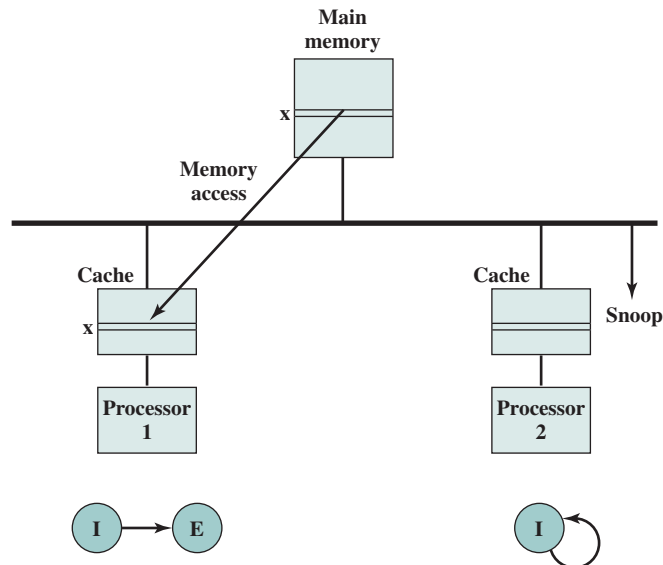
|                                                                                            |                                                                                                             |                                                                                                        |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| active standby<br>cache coherence<br>cluster<br>directory protocol<br>failback<br>failover | MESI protocol<br>multiprocessor<br>nonuniform memory access<br>(NUMA)<br>passive standby<br>snoopy protocol | symmetric multiprocessor<br>(SMP)<br>uniform memory access<br>(UMA)<br>uniprocessor<br>vector facility |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|

### Review Questions

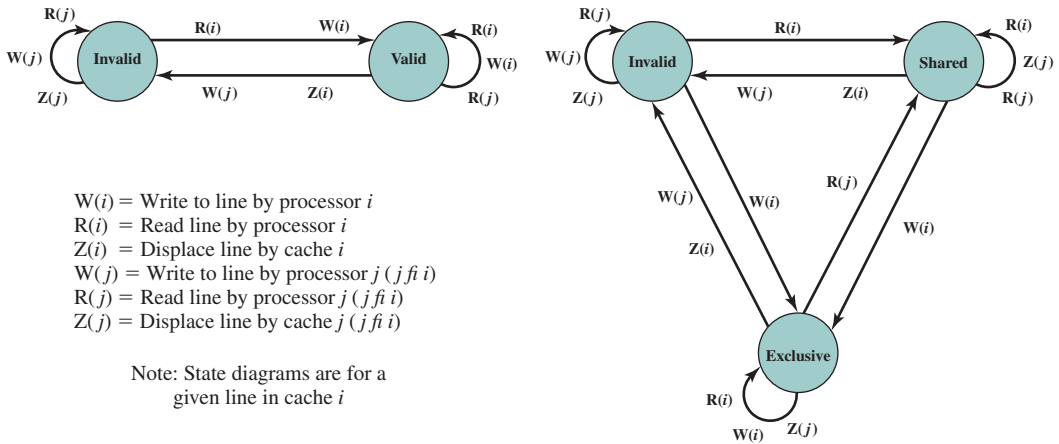
- 17.1** List and briefly define three types of computer system organization.
- 17.2** What are the chief characteristics of an SMP?
- 17.3** What are some of the potential advantages of an SMP compared with a uniprocessor?
- 17.4** What are some of the key OS design issues for an SMP?
- 17.5** What is the difference between software and hardware cache coherent schemes?
- 17.6** What is the meaning of each of the four states in the MESI protocol?
- 17.7** What are some of the key benefits of clustering?
- 17.8** What is the difference between failover and failback?
- 17.9** What are the differences among UMA, NUMA, and CC-NUMA?

## Problems

- 17.1** Let  $\alpha$  be the percentage of program code that can be executed simultaneously by  $n$  processors in a computer system. Assume that the remaining code must be executed sequentially by a single processor. Each processor has an execution rate of  $x$  MIPS.
- Derive an expression for the effective MIPS rate when using the system for exclusive execution of this program, in terms of  $n$ ,  $\alpha$ , and  $x$ .
  - If  $n = 16$  and  $x = 4$  MIPS, determine the value of  $\alpha$  that will yield a system performance of 40 MIPS.
- 17.2** A multiprocessor with eight processors has 20 attached tape drives. There are a large number of jobs submitted to the system that each require a maximum of four tape drives to complete execution. Assume that each job starts running with only three tape drives for a long period before requiring the fourth tape drive for a short period toward the end of its operation. Also assume an endless supply of such jobs.
- Assume the scheduler in the OS will not start a job unless there are four tape drives available. When a job is started, four drives are assigned immediately and are not released until the job finishes. What is the maximum number of jobs that can be in progress at once? What are the maximum and minimum number of tape drives that may be left idle as a result of this policy?
  - Suggest an alternative policy to improve tape drive utilization and at the same time avoid system deadlock. What is the maximum number of jobs that can be in progress at once? What are the bounds on the number of idling tape drives?
- 17.3** Can you foresee any problem with the write-once cache approach on bus-based multiprocessors? If so, suggest a solution.
- 17.4** Consider a situation in which two processors in an SMP configuration, over time, require access to the same line of data from main memory. Both processors have a cache and use the MESI protocol. Initially, both caches have an invalid copy of the line. Figure 17.21 depicts the consequence of a read of line  $x$  by Processor P1. If this is the start of a sequence of accesses, draw the subsequent figures for the following sequence:
- P2 reads  $x$ .
  - P1 writes to  $x$  (for clarity, label the line in P1's cache  $x'$ ).



**Figure 17.21** MESI Example: Processor 1 Reads Line  $x$

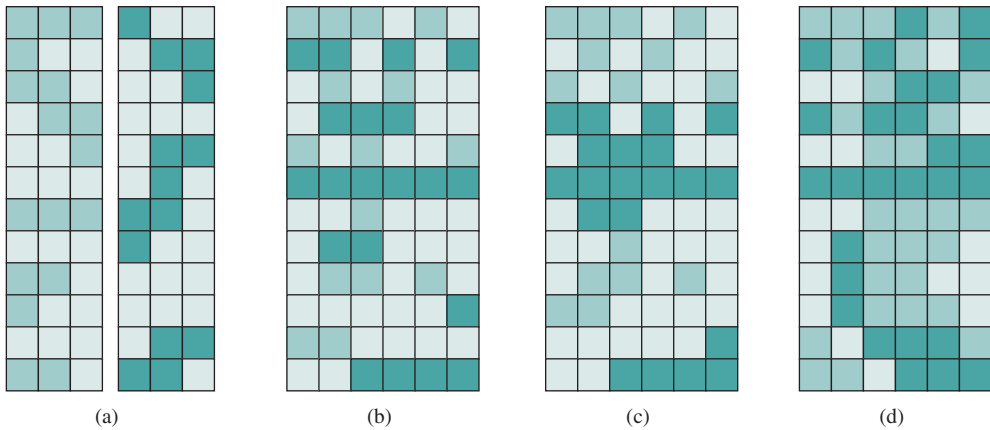


**Figure 17.22** Two Cache Coherence Protocols

3. P1 writes to  $x$  (label the line in P1's cache  $x''$ ).
  4. P2 reads  $x$ .
- 17.5** Figure 17.22 shows the state diagrams of two possible cache coherence protocols. Deduce and explain each protocol, and compare each to MESI.
- 17.6** Consider an SMP with both L1 and L2 caches using the MESI protocol. As explained in Section 17.3, one of four states is associated with each line in the L2 cache. Are all four states also needed for each line in the L1 cache? If so, why? If not, explain which state or states can be eliminated.
- 17.7** An earlier version of the IBM mainframe, the S/390 G4, used three levels of cache. As with the z990, only the first level was on the processor chip [called the processor unit (PU)]. The L2 cache was also similar to the z990. An L3 cache was on a separate chip that acted as a memory controller, and was interposed between the L2 caches and the memory cards. Table 17.4 shows the performance of a three-level cache arrangement for the IBM S/390. The purpose of this problem is to determine whether the inclusion of the third level of cache seems worthwhile. Determine the access penalty (average number of PU cycles) for a system with only an L1 cache, and normalize that value to 1.0. Then determine the normalized access penalty when both an L1 and L2 cache are used, and the access penalty when all three caches are used. Note the amount of improvement in each case and state your opinion on the value of the L3 cache.
- 17.8 a.** Consider a uniprocessor with separate data and instruction caches, with hit ratios of  $H_d$  and  $H_i$ , respectively. Access time from processor to cache is  $c$  clock cycles, and transfer time for a block between memory and cache is  $b$  clock cycles. Let  $f_i$

**Table 17.4** Typical Cache Hit Rate on S/390 SMP Configuration [MAK97]

| Memory Subsystem | Access Penalty (PU cycles) | Cache Size | Hit Rate (%) |
|------------------|----------------------------|------------|--------------|
| L1 cache         | 1                          | 32 KB      | 89           |
| L2 cache         | 5                          | 256 KB     | 5            |
| L3 cache         | 14                         | 2 MB       | 3            |
| Memory           | 32                         | 8 GB       | 3            |



**Figure 17.23** Diagram for Problem 18.9

be the fraction of memory accesses that are for instructions, and  $f_d$  is the fraction of dirty lines in the data cache among lines replaced. Assume a write-back policy and determine the effective memory access time in terms of the parameters just defined.

- b.** Now assume a bus-based SMP in which each processor has the characteristics of part (a). Every processor must handle cache invalidation in addition to memory reads and writes. This affects effective memory access time. Let  $f_{inv}$  be the fraction of data references that cause invalidation signals to be sent to other data caches. The processor sending the signal requires  $t$  clock cycles to complete the invalidation operation. Other processors are not involved in the invalidation operation. Determine the effective memory access time.

**17.9** What organizational alternative is suggested by each of the illustrations in Figure 17.23?

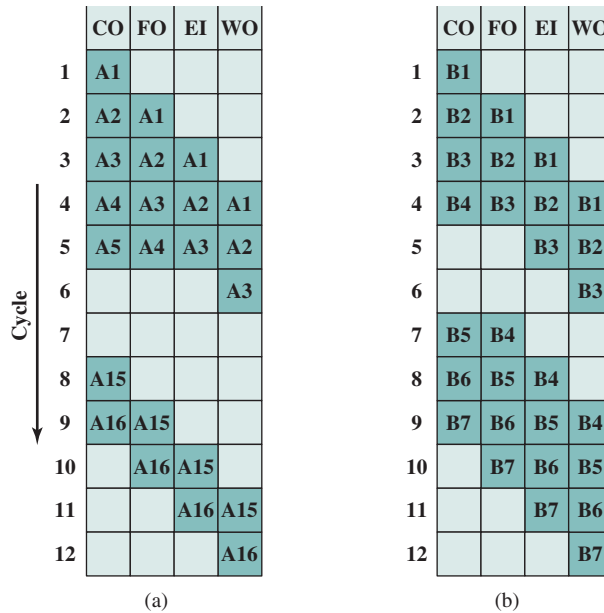
**17.10** In Figure 17.7, some of the diagrams show horizontal rows that are partially filled. In other cases, there are rows that are completely blank. These represent two different types of loss of efficiency. Explain.

**17.11** Consider the pipeline depiction in Figure 14.13b, which is redrawn in Figure 17.24a, with the fetch and decode stages ignored, to represent the execution of thread A. Figure 17.24b illustrates the execution of a separate thread B. In both cases, a simple pipelined processor is used.

- Show an instruction issue diagram, similar to Figure 17.7a, for each of the two threads.
- Assume that the two threads are to be executed in parallel on a chip multiprocessor, with each of the two processors on the chip using a simple pipeline. Show an instruction issue diagram similar to Figure 17.7k. Also show a pipeline execution diagram in the style of Figure 17.24.
- Assume a two-issue superscalar architecture. Repeat part (b) for an interleaved multithreading superscalar implementation, assuming no data dependencies. *Note:* There is no unique answer; you need to make assumptions about latency and priority.
- Repeat part (c) for a blocked multithreading superscalar implementation.
- Repeat for a four-issue SMT architecture.

**17.12** The following code segment needs to be executed 64 times for the evaluation of the vector arithmetic expression:  $D(I) = A(I) + B(I) \times C(I)$  for  $0 \leq I \leq 63$ .

```
Load R1, B(I) /R1 ← Memory (α + I)/
Load R2, C(I) /R2 ← Memory (β + I)/
```



**Figure 17.24** Two Threads of Execution

Multiply R1, R2     /R1  $\leftarrow$  (R1)  $\times$  (R2)/  
 Load R3, A(I)     /R3  $\leftarrow$  Memory ( $\gamma + I$ )/  
 Add R3, R1         /R3  $\leftarrow$  (R3) + (R1)/  
 Store D1, R3       /Memory ( $\theta + I$ )  $\leftarrow$  (R3)/

where R1, R2, and R3 are processor registers, and  $\alpha, \beta, \gamma, \theta$  are the starting main memory addresses of arrays B(I), C(I), A(I), and D(I), respectively. Assume four clock cycles for each Load or Store, two cycles for the Add, and eight cycles for the Multiplier on either a uniprocessor or a single processor in an SIMD machine.

- Calculate the total number of processor cycles needed to execute this code segment repeatedly 64 times on a SISD uniprocessor computer sequentially, ignoring all other time delays.
- Consider the use of an SIMD computer with 64 processing elements to execute the vector operations in six synchronized vector instructions over 64-component vector data and both driven by the same-speed clock. Calculate the total execution time on the SIMD machine, ignoring instruction broadcast and other delays.
- What is the speedup gain of the SIMD computer over the SISD computer?

**17.13** Produce a vectorized version of the following program:

```

DO 20 I = 1, N
 B(I, 1) = 0
 DO 10 J = 1, M
 A(I) = A(I) + B(I, J) $\times$ C(I, J)
 10 CONTINUE
 D(I) = E(I) + A(I)
 20 CONTINUE

```

**17.14** An application program is executed on a nine-computer cluster. A benchmark program took time  $T$  on this cluster. Further, it was found that 25% of  $T$  was time in



which the application was running simultaneously on all nine computers. The remaining time, the application had to run on a single computer.

- a. Calculate the effective speedup under the aforementioned condition as compared to executing the program on a single computer. Also calculate  $\alpha$ , the percentage of code that has been parallelized (programmed or compiled so as to use the cluster mode) in the preceding program.
- b. Suppose that we are able to effectively use 17 computers rather than 9 computers on the parallelized portion of the code. Calculate the effective speedup that is achieved.

**17.15** The following FORTRAN program is to be executed on a computer, and a parallel version is to be executed on a 32-computer cluster.

```

L1: DO 10 I = 1, 1024
L2: SUM(I) = 0
L3: DO 20 J = 1, I
L4: 20 SUM(I) = SUM(I) + I
L5: 10 CONTINUE

```

Suppose lines 2 and 4 each take two machine cycle times, including all processor and memory-access activities. Ignore the overhead caused by the software loop control statements (lines 1, 3, 5) and all other system overhead and resource conflicts.

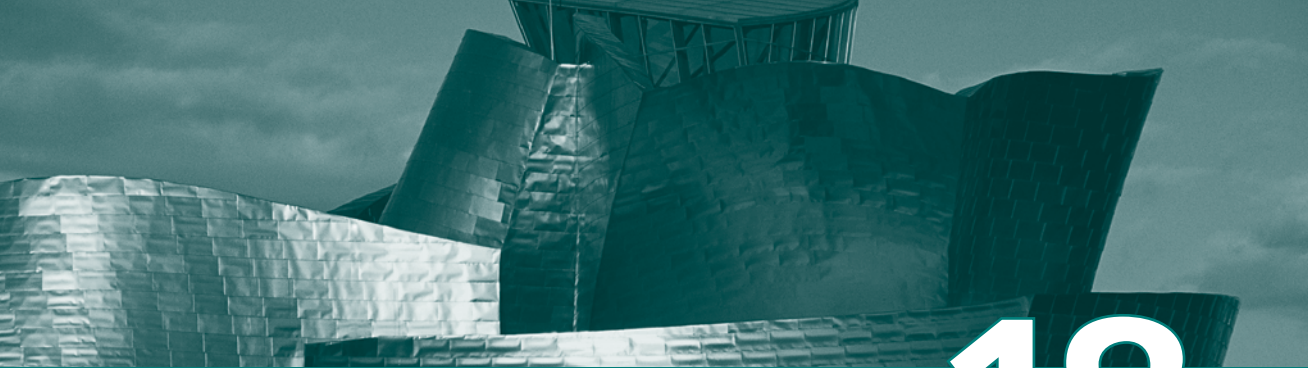
- a. What is the total execution time (in machine cycle times) of the program on a single computer?
- b. Divide the I-loop iterations among the 32 computers as follows: Computer 1 executes the first 32 iterations ( $I = 1$  to 32), processor 2 executes the next 32 iterations, and so on. What are the execution time and speedup factor compared with part (a)? (Note that the computational workload, dictated by the J-loop, is unbalanced among the computers.)
- c. Explain how to modify the parallelizing to facilitate a balanced parallel execution of all the computational workload over 32 computers. By a balanced load is meant an equal number of additions assigned to each computer with respect to both loops.
- d. What is the minimum execution time resulting from the parallel execution on 32 computers? What is the resulting speedup over a single computer?

**17.16** Consider the following two versions of a program to add two vectors:

|                                                                                                                                                                                             |                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> L1:      DO 10 I = 1, N L2:          A(I) = B(I) + C(I) L3:  10      CONTINUE L4:          SUM = 0 L5:      DO 20 J = 1, N L6:          SUM = SUM + A(J) L7:  20      CONTINUE </pre> | <pre> DOALL K = 1, M DO 10 I = L(K - 1) + 1, KL     A(I) = B(I) + C(I) 10 CONTINUE SUM(K) = 0 DO 20 J = 1, L     SUM(K) = SUM(K) + A(L(K - 1) + J) 20 CONTINUE ENDALL </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- a. The program on the left executes on a uniprocessor. Suppose each line of code L2, L4, and L6 takes one processor clock cycle to execute. For simplicity, ignore the time required for the other lines of code. Initially all arrays are already loaded in main memory and the short program fragment is in the instruction cache. How many clock cycles are required to execute this program?

- b. The program on the right is written to execute on a multiprocessor with  $M$  processors. We partition the looping operations into  $M$  sections with  $L = N/M$  elements per section. DOALL declares that all  $M$  sections are executed in parallel. The result of this program is to produce  $M$  partial sums. Assume that  $k$  clock cycles are needed for each interprocessor communication operation via the shared memory and that therefore the addition of each partial sum requires  $k$  cycles. An  $l$ -level binary adder tree can merge all the partial sums, where  $l = \log_2 M$ . How many cycles are needed to produce the final sum?
- c. Suppose  $N = 2^{20}$  elements in the array and  $M = 256$ . What is the speedup achieved by using the multiprocessor? Assume  $k = 200$ . What percentage is this of the theoretical speedup of a factor of 256?



# CHAPTER 18

## MULTICORE COMPUTERS

### **18.1 Hardware Performance Issues**

- Increase in Parallelism and Complexity
- Power Consumption

### **18.2 Software Performance Issues**

- Software on Multicore
- Application Example: Valve Game Software

### **18.3 Multicore Organization**

### **18.4 Intel x86 Multicore Organization**

- Intel Core Duo
- Intel Core i7-990X

### **18.5 ARM11 MPCore**

- Interrupt Handling
- Cache Coherency

### **18.6 IBM zEnterprise 196 Mainframe**

### **18.7 Recommended Reading**

### **18.8 Key Terms, Review Questions, and Problems**

### LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- ◆ Understand the hardware performance issues that have driven the move to multicore computers.
- ◆ Understand the software performance issues posed by the use of multithreaded multicore computers.
- ◆ Have an appreciation of the use of multicore organization on embedded systems, PCs and servers, and mainframes.

A **multicore** computer, also known as a **chip multiprocessor**, combines two or more processors (called cores) on a single piece of silicon (called a die). Typically, each core consists of all of the components of an independent processor, such as registers, ALU, pipeline hardware, and control unit, plus L1 instruction and data caches. In addition to the multiple cores, contemporary multicore chips also include L2 cache and, increasingly, L3 cache.

This chapter provides an overview of multicore systems. We begin with a look at the hardware performance factors that led to the development of multicore computers and the software challenges of exploiting the power of a multicore system. Next, we look at multicore organization. Finally, we examine three examples of multicore products, covering personal computer and workstation systems (Intel), embedded systems (ARM), and mainframes (IBM).

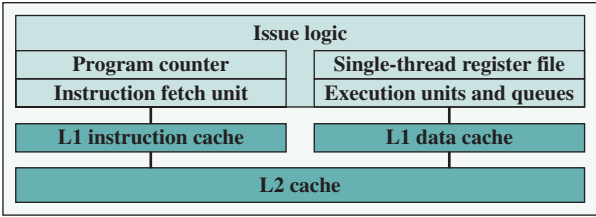
## 18.1 HARDWARE PERFORMANCE ISSUES

As we discuss in Chapter 2, microprocessor systems have experienced a steady, exponential increase in execution performance for decades. Figure 2.12 shows that this increase is due partly to refinements in the organization of the processor on the chip, and partly to the increase in the clock frequency.

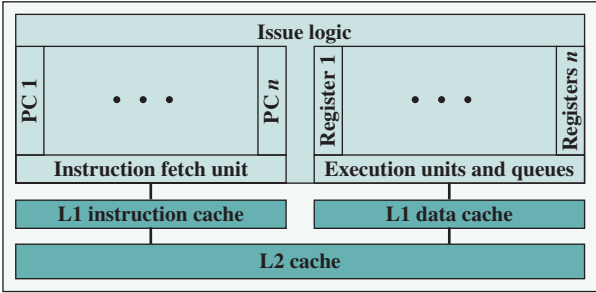
### Increase in Parallelism and Complexity

The organizational changes in processor design have primarily been focused on increasing instruction-level parallelism, so that more work could be done in each clock cycle. These changes include, in chronological order (Figure 18.1):

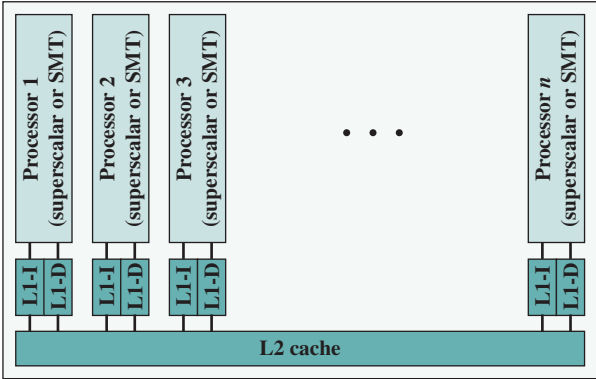
- **Pipelining:** Individual instructions are executed through a pipeline of stages so that while one instruction is executing in one stage of the pipeline, another instruction is executing in another stage of the pipeline.
- **Superscalar:** Multiple pipelines are constructed by replicating execution resources. This enables parallel execution of instructions in parallel pipelines, so long as hazards are avoided.
- **Simultaneous multithreading (SMT):** Register banks are replicated so that multiple threads can share the use of pipeline resources.



(a) Superscalar



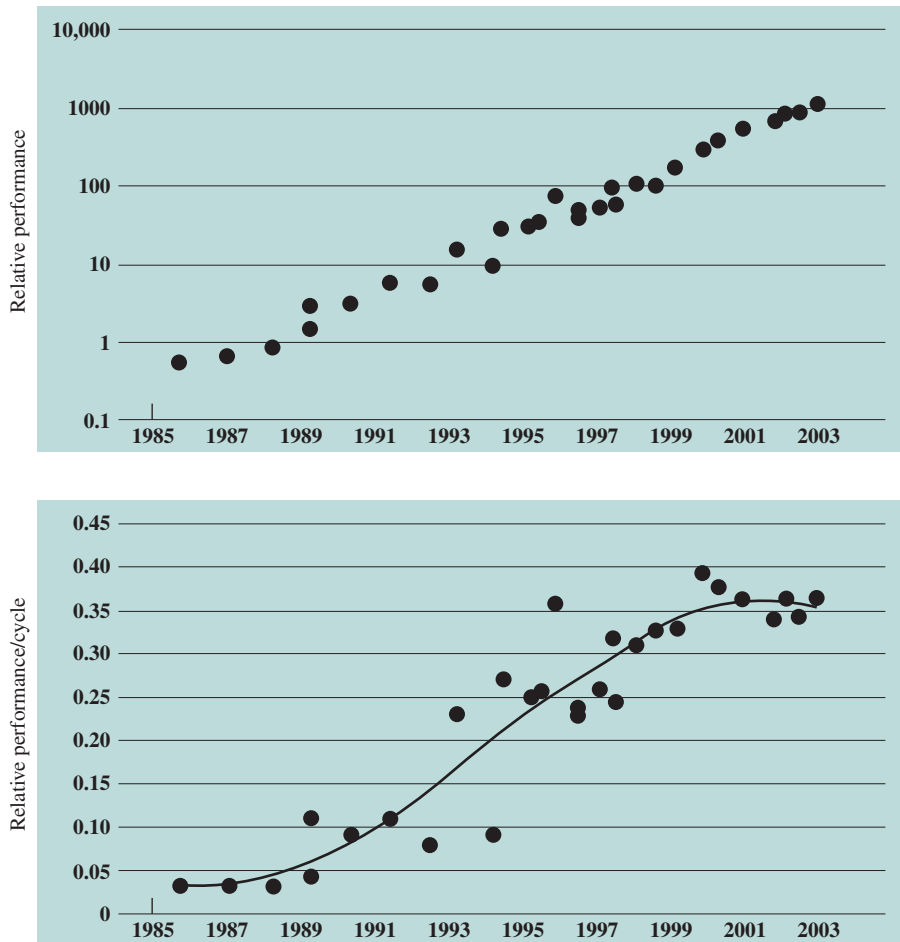
(b) Simultaneous multithreading



(c) Multicore

**Figure 18.1** Alternative Chip Organizations

For each of these innovations, designers have over the years attempted to increase the performance of the system by adding complexity. In the case of pipelining, simple three-stage pipelines were replaced by pipelines with five stages, and then many more stages, with some implementations having over a dozen stages. There is a practical limit to how far this trend can be taken, because with more stages, there is the need for more logic, more interconnections, and more control signals. With superscalar organization, increased performance can be achieved by increasing the number of parallel pipelines. Again, there are diminishing returns as the number of pipelines increases. More logic is required to manage hazards and to stage instruction resources. Eventually, a single thread of execution reaches the point where hazards and resource dependencies prevent the full use of the multiple



**Figure 18.2** Some Intel Hardware Trends

pipelines available. This same point of diminishing returns is reached with SMT, as the complexity of managing multiple threads over a set of pipelines limits the number of threads and number of pipelines that can be effectively utilized.

Figure 18.2, from [OLUK05], is instructive in this context. The upper graph shows the exponential increase in Intel processor performance over the years.<sup>1</sup> The lower graph is calculated by combining Intel's published SPEC CPU figures and processor clock frequencies to give a measure of the extent to which performance improvement is due to increased exploitation of instruction-level parallelism. There is a flat region in the late 1980s before parallelism was exploited extensively. This is followed by a steep rise as designers were able to increasingly exploit pipelining, superscalar techniques, and SMT. But, beginning about 2000, a new flat region of the curve appears, as the limits of effective exploitation of instruction-level parallelism are reached.

<sup>1</sup>The data are based on published SPEC CPU figures from Intel, normalized across varying suites.

There is a related set of problems dealing with the design and fabrication of the computer chip. The increase in complexity to deal with all of the logical issues related to very long pipelines, multiple superscalar pipelines, and multiple SMT register banks means that increasing amounts of the chip area are occupied with coordinating and signal transfer logic. This increases the difficulty of designing, fabricating, and debugging the chips. The increasingly difficult engineering challenge related to processor logic is one of the reasons that an increasing fraction of the processor chip is devoted to the simpler memory logic. Power issues, discussed next, provide another reason.

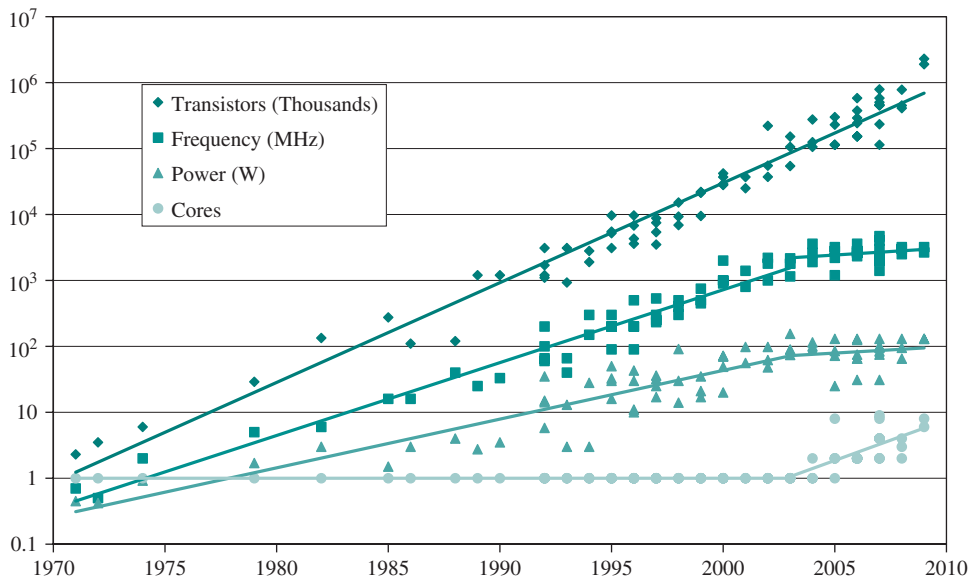
### Power Consumption

To maintain the trend of higher performance as the number of transistors per chip rise, designers have resorted to more elaborate processor designs (pipelining, superscalar, SMT) and to high clock frequencies. Unfortunately, power requirements have grown exponentially as chip density and clock frequency have risen. This is shown in the Figure 18.3, which repeats Figure 2.11.

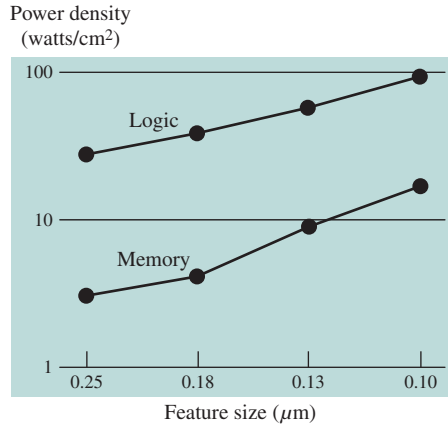
One way to control power density is to use more of the chip area for cache memory. Memory transistors are smaller and have a power density an order of magnitude lower than that of logic (see Figure 18.4). Further, as chip transistor density has increased, the percentage of chip area devoted to memory has grown, and is now well over half the chip area.

By 2015, we can expect to see microprocessor chips with about 100 billion transistors on a 300 mm<sup>2</sup> die. Assuming about 50–60% of the chip area is devoted to memory, the chip will support cache memory of about 100 MB and leave over 1 billion transistors available for logic.

How to use all those logic transistors is a key design issue. As discussed earlier in this section, there are limits to the effective use of such techniques as superscalar



**Figure 18.3** Processor Trends



**Figure 18.4** Power and Memory Considerations

and SMT. In general terms, the experience of recent decades has been encapsulated in a rule of thumb known as **Pollack's rule** [POLL99], which states that performance increase is roughly proportional to square root of increase in complexity. In other words, if you double the logic in a processor core, then it delivers only 40% more performance. In principle, the use of multiple cores has the potential to provide near-linear performance improvement with the increase in the number of cores.

Power considerations provide another motive for moving toward a multicore organization. Because the chip has such a huge amount of cache memory, it becomes unlikely that any one thread of execution can effectively use all that memory. Even with SMT, you are multithreading in a relatively limited fashion and cannot therefore fully exploit a gigantic cache, whereas a number of relatively independent threads or processes has a greater opportunity to take full advantage of the cache memory.

## 18.2 SOFTWARE PERFORMANCE ISSUES

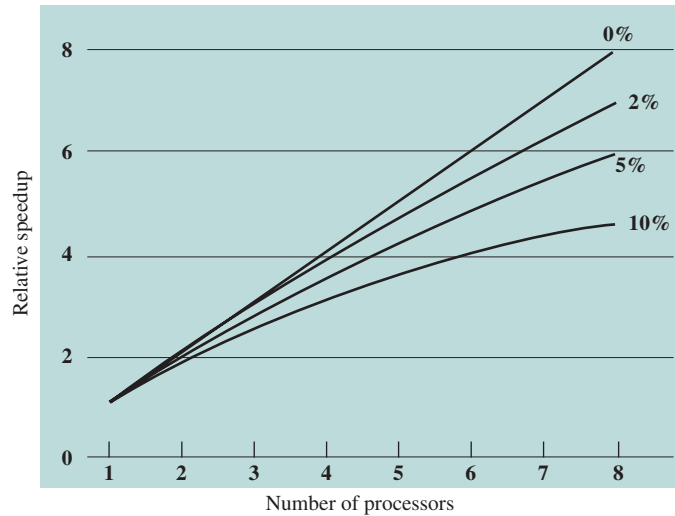
A detailed examination of the software performance issues related to multicore organization is beyond our scope. In this section, we first provide an overview of these issues, and then look at an example of an application designed to exploit multicore capabilities.

### Software on Multicore

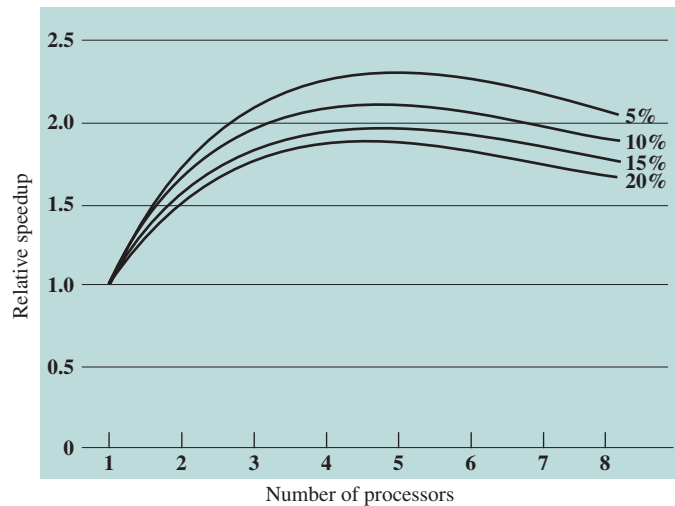
The potential performance benefits of a multicore organization depend on the ability to effectively exploit the parallel resources available to the application. Let us focus first on a single application running on a multicore system. Recall from Chapter 2 that Amdahl's law states that:

$$\begin{aligned}
 \text{Speed up} &= \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} \\
 &= \frac{1}{(1 - f) + \frac{f}{N}}
 \end{aligned} \tag{18.1}$$





(a) Speedup with 0%, 2%, 5%, and 10% sequential portions



(b) Speedup with overheads

**Figure 18.5** Performance Effect of Multiple Cores

The law assumes a program in which a fraction  $(1 - f)$  of the execution time involves code that is inherently serial and a fraction  $f$  that involves code that is infinitely parallelizable with no scheduling overhead.

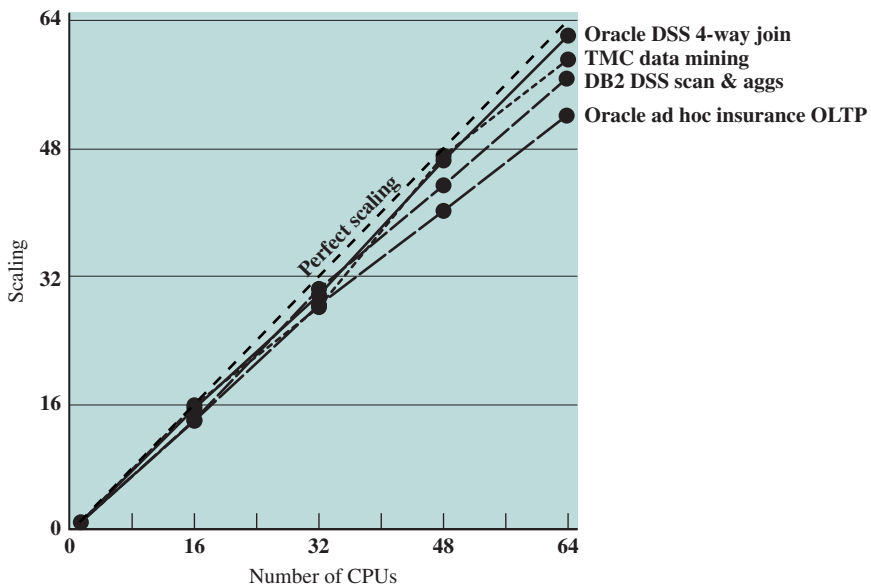
This law appears to make the prospect of a multicore organization attractive. But as Figure 18.5a shows, even a small amount of serial code has a noticeable impact. If only 10% of the code is inherently serial ( $f = 0.9$ ), running the program on a multicore system with 8 processors yields a performance gain of only a factor of 4.7. In addition, software typically incurs overhead as a result of communication and distribution of work among multiple processors and as a result of cache coherence overhead. This results in a curve where performance peaks and then begins to degrade because of the

increased burden of the overhead of using multiple processors (e.g., coordination and OS management). Figure 18.5b, from [MCDO05], is a representative example.

However, software engineers have been addressing this problem and there are numerous applications in which it is possible to effectively exploit a multicore system. [MCDO05] analyzes the effectiveness of multicore systems on a set of database applications, in which great attention was paid to reducing the serial fraction within hardware architectures, operating systems, middleware, and the database application software. Figure 18.6 shows the result. As this example shows, database management systems and database applications are one area in which multicore systems can be used effectively. Many kinds of servers can also effectively use the parallel multicore organization, because servers typically handle numerous relatively independent transactions in parallel.

In addition to general-purpose server software, a number of classes of applications benefit directly from the ability to scale throughput with the number of cores. [MCDO06] lists the following examples:

- **Multithreaded native applications:** Multithreaded applications are characterized by having a small number of highly threaded processes. Examples of threaded applications include Lotus Domino or Siebel CRM (Customer Relationship Manager).
- **Multiprocess applications:** Multiprocess applications are characterized by the presence of many single-threaded processes. Examples of multi-process applications include the Oracle database, SAP, and PeopleSoft.
- **Java applications:** Java applications embrace threading in a fundamental way. Not only does the Java language greatly facilitate multithreaded applications, but the Java Virtual Machine is a multithreaded process that



**Figure 18.6** Scaling of Database Workloads on Multiple-Processor Hardware

provides scheduling and memory management for Java applications. Java applications that can benefit directly from multicore resources include application servers such as Sun's Java Application Server, BEA's Weblogic, IBM's Websphere, and the open-source Tomcat application server. All applications that use a Java 2 Platform, Enterprise Edition (J2EE platform) application server can immediately benefit from multicore technology.

- **Multi-instance applications:** Even if an individual application does not scale to take advantage of a large number of threads, it is still possible to gain from multicore architecture by running multiple instances of the application in parallel. If multiple application instances require some degree of isolation, virtualization technology (for the hardware of the operating system) can be used to provide each of them with its own separate and secure environment.

### Application Example: Valve Game Software

Valve is an entertainment and technology company that has developed a number of popular games, as well as the Source engine, one of the most widely played game engines available. Source is an animation engine used by Valve for its games and licensed for other game developers.

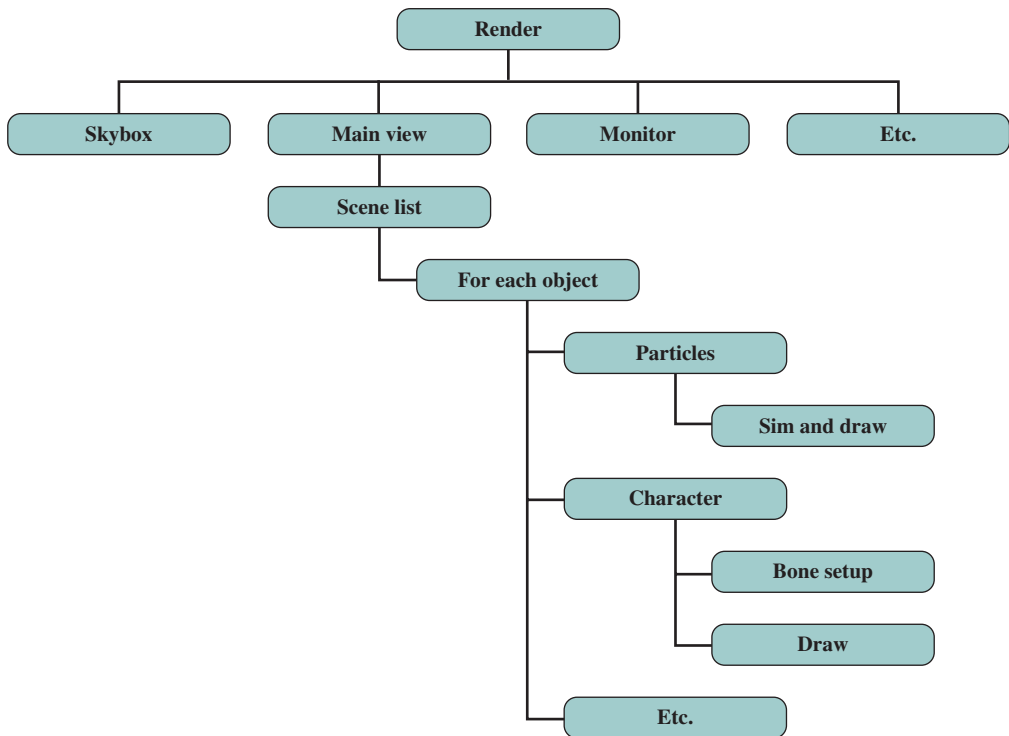
In recent years, Valve has reprogrammed the Source engine software to use multithreading to exploit the power of multicore processor chips from Intel and AMD [REIM06]. The revised Source engine code provides more powerful support for Valve games such as Half Life 2.

From Valve's perspective, threading granularity options are defined as follows [HARR06]:

- **Coarse threading:** Individual modules, called systems, are assigned to individual processors. In the Source engine case, this would mean putting rendering on one processor, AI (artificial intelligence) on another, physics on another, and so on. This is straightforward. In essence, each major module is single threaded and the principal coordination involves synchronizing all the threads with a timeline thread.
- **Fine-grained threading:** Many similar or identical tasks are spread across multiple processors. For example, a loop that iterates over an array of data can be split up into a number of smaller parallel loops in individual threads that can be scheduled in parallel.
- **Hybrid threading:** This involves the selective use of fine-grain threading for some systems and single threading for other systems.

Valve found that through coarse threading, it could achieve up to twice the performance across two processors compared to executing on a single processor. But this performance gain could only be achieved with contrived cases. For real-world gameplay, the improvement was on the order of a factor of 1.2. Valve also found that effective use of fine-grain threading was difficult. The time per work unit can be variable, and managing the timeline of outcomes and consequences involved complex programming.

Valve found that a hybrid threading approach was the most promising and would scale the best as multicore systems with eight or sixteen processors became



**Figure 18.7** Hybrid Threading for Rendering Module

available. Valve identified systems that operate very effectively when assigned to a single processor permanently. An example is sound mixing, which has little user interaction, is not constrained by the frame configuration of windows, and works on its own set of data. Other modules, such as scene rendering, can be organized into a number of threads so that the module can execute on a single processor but achieve greater performance as it is spread out over more and more processors.

Figure 18.7 illustrates the thread structure for the rendering module. In this hierarchical structure, higher-level threads spawn lower-level threads as needed. The rendering module relies on a critical part of the Source engine, the world list, which is a database representation of the visual elements in the game's world. The first task is to determine what are the areas of the world that need to be rendered. The next task is to determine what objects are in the scene as viewed from multiple angles. Then comes the processor-intensive work. The rendering module has to work out the rendering of each object from multiple points of view, such as the player's view, the view of TV monitors, and the point of view of reflections in water.

Some of the key elements of the threading strategy for the rendering module are listed in [LEON07] and include the following:

- Construct scene-rendering lists for multiple scenes in parallel (e.g., the world and its reflection in water).
- Overlap graphics simulation.

- Compute character bone transformations for all characters in all scenes in parallel.
- Allow multiple threads to draw in parallel.

The designers found that simply locking key databases, such as the world list, for a thread was too inefficient. Over 95% of the time, a thread is trying to read from a data set, and only 5% of the time at most is spent in writing to a data set. Thus, a concurrency mechanism known as the single-writer-multiple-readers model works effectively.

## 18.3 MULTICORE ORGANIZATION

At a top level of description, the main variables in a multicore organization are as follows:

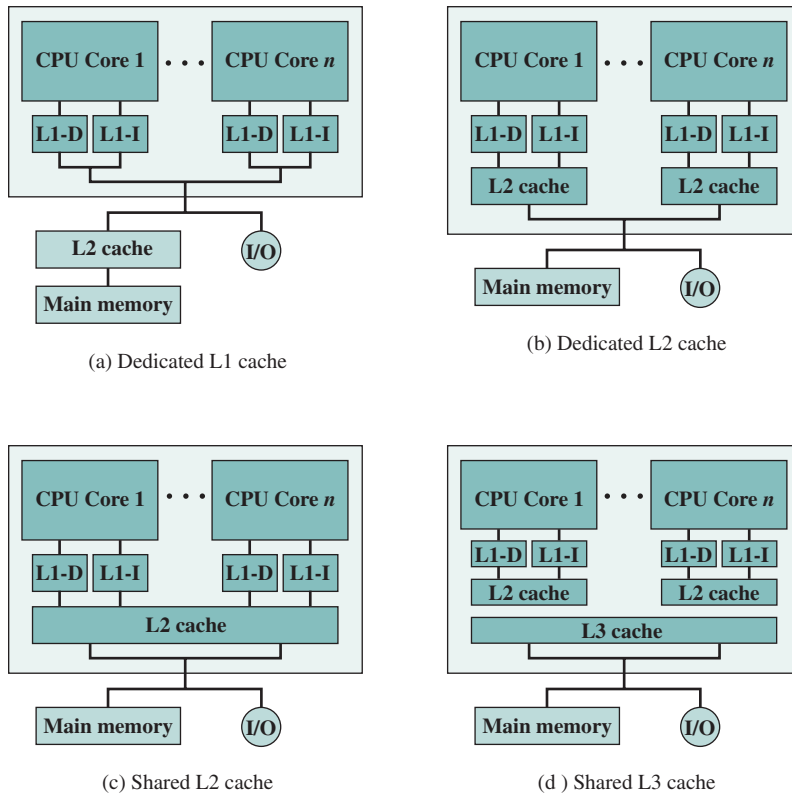
- The number of core processors on the chip
- The number of levels of cache memory
- The amount of cache memory that is shared

Figure 18.8 shows four general organizations for multicore systems. Figure 18.8a is an organization found in some of the earlier multicore computer chips and is still seen in embedded chips. In this organization, the only on-chip cache is L1 cache, with each core having its own dedicated L1 cache. Almost invariably, the L1 cache is divided into instruction and data caches. An example of this organization is the ARM11 MPCore.

The organization of Figure 18.8b is also one in which there is no on-chip cache sharing. In this, there is enough area available on the chip to allow for L2 cache. An example of this organization is the AMD Opteron. Figure 18.8c shows a similar allocation of chip space to memory, but with the use of a shared L2 cache. The Intel Core Duo has this organization. Finally, as the amount of cache memory available on the chip continues to grow, performance considerations dictate splitting off a separate, shared L3 cache, with dedicated L1 and L2 caches for each core processor. The Intel Core i7 is an example of this organization.

The use of a shared L2 cache on the chip has several advantages over exclusive reliance on dedicated caches:

1. Constructive interference can reduce overall miss rates. That is, if a thread on one core accesses a main memory location, this brings the frame containing the referenced location into the shared cache. If a thread on another core soon thereafter accesses the same memory block, the memory locations will already be available in the shared on-chip cache.
2. A related advantage is that data shared by multiple cores is not replicated at the shared cache level.
3. With proper frame replacement algorithms, the amount of shared cache allocated to each core is dynamic, so that threads that have a less locality can employ more cache.



**Figure 18.8** Multicore Organization Alternatives

4. Interprocessor communication is easy to implement, via shared memory locations.
5. The use of a shared L2 cache confines the cache coherency problem to the L1 cache level, which may provide some additional performance advantage.

A potential advantage to having only dedicated L2 caches on the chip is that each core enjoys more rapid access to its private L2 cache. This is advantageous for threads that exhibit strong locality.

As both the amount of memory available and the number of cores grow, the use of a shared L3 cache combined with either a shared L2 cache or dedicated per-core L2 caches seems likely to provide better performance than simply a massive shared L2 cache.

Another organizational design decision in a multicore system is whether the individual cores will be superscalar or will implement simultaneous multithreading (SMT). For example, the Intel Core Duo uses superscalar cores, whereas the Intel Core i7 uses SMT cores. SMT has the effect of scaling up the number of hardware-level threads that the multicore system supports. Thus, a multicore system with four cores and SMT that supports four simultaneous threads in each core appears the same to the application level as a multicore system with 16 cores. As software is developed to more fully exploit parallel resources, an SMT approach appears to be more attractive than a superscalar approach.

## 18.4 INTEL x86 MULTICORE ORGANIZATION

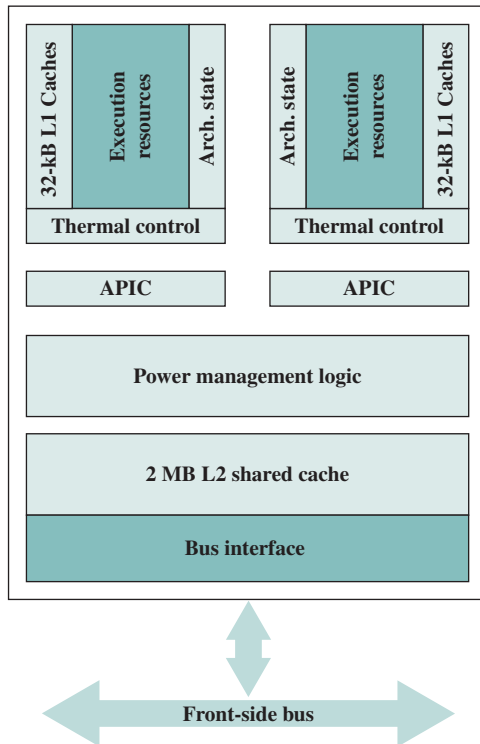
Intel has introduced a number of multicore products in recent years. In this section, we look at two examples: the Intel Core Duo and the Intel Core i7-990X.

### Intel Core Duo

The Intel Core Duo, introduced in 2006, implements two x86 superscalar processors with a shared L2 cache (Figure 18.8c).

The general structure of the Intel Core Duo is shown in Figure 18.9. Let us consider the key elements starting from the top of the figure. As is common in multicore systems, each core has its own dedicated **L1 cache**. In this case, each core has a 32-kB instruction cache and a 32-kB data cache.

Each core has an independent **thermal control unit**. With the high transistor density of today's chips, thermal management is a fundamental capability, especially for laptop and mobile systems. The Core Duo thermal control unit is designed to manage chip heat dissipation to maximize processor performance within thermal constraints. Thermal management also improves ergonomics with a cooler system and lower fan acoustic noise. In essence, the thermal management unit monitors digital sensors for high-accuracy die temperature measurements. Each core can be defined as an independent thermal zone. The maximum temperature for each



**Figure 18.9** Intel Core Duo Block Diagram

thermal zone is reported separately via dedicated registers that can be polled by software. If the temperature in a core exceeds a threshold, the thermal control unit reduces the clock rate for that core to reduce heat generation.

The next key element of the Core Duo organization is the **Advanced Programmable Interrupt Controller** (APIC). The APIC performs a number of functions, including the following:

1. The APIC can provide interprocessor interrupts, which allow any process to interrupt any other processor or set of processors. In the case of the Core Duo, a thread in one core can generate an interrupt, which is accepted by the local APIC, routed to the APIC of the other core, and communicated as an interrupt to the other core.
2. The APIC accepts I/O interrupts and routes these to the appropriate core.
3. Each APIC includes a timer, which can be set by the OS to generate an interrupt to the local core.

The **power management logic** is responsible for reducing power consumption when possible, thus increasing battery life for mobile platforms, such as laptops. In essence, the power management logic monitors thermal conditions and CPU activity and adjusts voltage levels and power consumption appropriately. It includes an advanced power-gating capability that allows for an ultra fine-grained logic control that turns on individual processor logic subsystems only if and when they are needed.

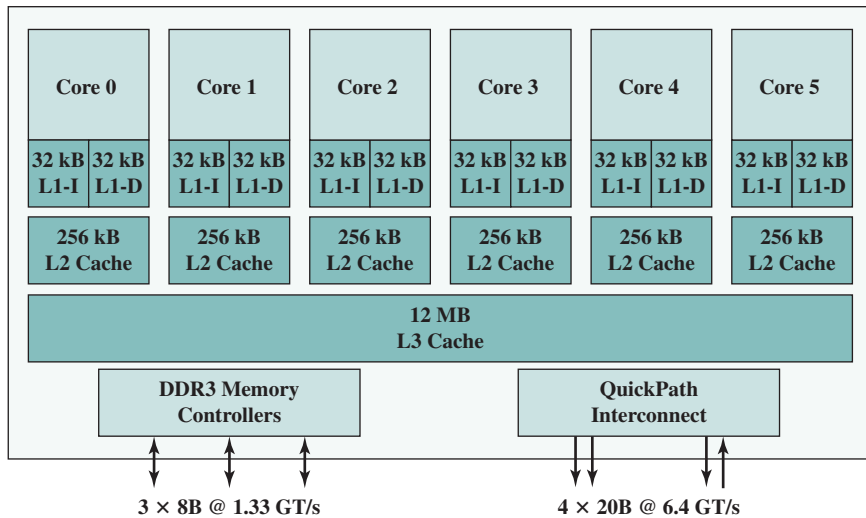
The Core Duo chip includes a shared 2-MB **L2 cache**. The cache logic allows for a dynamic allocation of cache space based on current core needs, so that one core can be assigned up to 100% of the L2 cache. The L2 cache includes logic to support the MESI cache coherence protocol for the attached L1 caches. The key point to consider is when a cache write is done at the L1 level. A cache line gets the M state when a processor writes to it; if the line is not in E or M-state prior to writing it, the cache sends a Read-For-Ownership (RFO) request that ensures that the line exists in the L1 cache and is in the I state in the other L1 cache. The Intel Core Duo extends this protocol to take into account the case when there are multiple Core Duo chips organized as a symmetric multiprocessor (SMP) system. The L2 cache controller allow the system to distinguish between a situation in which data are shared by the two local cores, but not with the rest of the world, and a situation in which the data are shared by one or more caches on the die as well as by an agent on the external bus (can be another processor). When a core issues an RFO, if the line is shared only by the other cache within the local die, we can resolve the RFO internally very fast, without going to the external bus at all. Only if the line is shared with another agent on the external bus do we need to issue the RFO externally.

The **bus interface** connects to the external bus, known as the Front Side Bus, which connects to main memory, I/O controllers, and other processor chips.

## Intel Core i7-990X

The Intel Core i7-990X, introduced in November of 2008, implements four x86 SMT processors, each with a dedicated L2 cache, and with a shared L3 cache (Figure 18.8d).





**Figure 18.10** Intel Core i7-990X Block Diagram

The general structure of the Intel Core i7-990X is shown in Figure 18.10. Each core has its own **dedicated L2 cache** and the four cores share a 12-MB **L3 cache**. One mechanism Intel uses to make its caches more effective is prefetching, in which the hardware examines memory access patterns and attempts to fill the caches speculatively with data that's likely to be requested soon. It is interesting to compare the performance of this three-level on chip cache organization with a comparable two-level organization from Intel. Table 18.1 shows the cache access latency, in terms of clock cycles for two Intel multicore systems running at the same clock frequency. The Core 2 Quad has a shared L2 cache, similar to the Core Duo. The Core i7 improves on L2 cache performance with the use of the dedicated L2 caches, and provides a relatively high-speed access to the L3 cache.

The Core i7-990X chip supports two forms of external communications to other chips. The **DDR3 memory controller** brings the memory controller for the DDR main memory<sup>2</sup> onto the chip. The interface supports three channels that are 8 bytes wide for a total bus width of 192 bits, for an aggregate data rate of up to 32 GB/s. With the memory controller on the chip, the Front Side Bus is eliminated.

**Table 18.1** Cache Latency (in clock cycles)

| CPU         | Clock Frequency | L1 Cache | L2 Cache  | L3 Cache  |
|-------------|-----------------|----------|-----------|-----------|
| Core 2 Quad | 2.66 GHz        | 3 cycles | 15 cycles | —         |
| Core i7     | 2.66 GHz        | 4 cycles | 11 cycles | 39 cycles |

<sup>2</sup>The DDR synchronous RAM memory is discussed in Chapter 5.

The **QuickPath Interconnect (QPI)** is a cache-coherent, point-to-point link based electrical interconnect specification for Intel processors and chipsets. It enables high-speed communications among connected processor chips. The QPI link operates at 6.4 GT/s (transfers per second). At 16 bits per transfer, that adds up to 12.8 GB/s, and since QPI links involve dedicated bidirectional pairs, the total bandwidth is 25.6 GB/s. Section 3.5 covers QPI in some detail.

## 18.5 ARM11 MPCORE

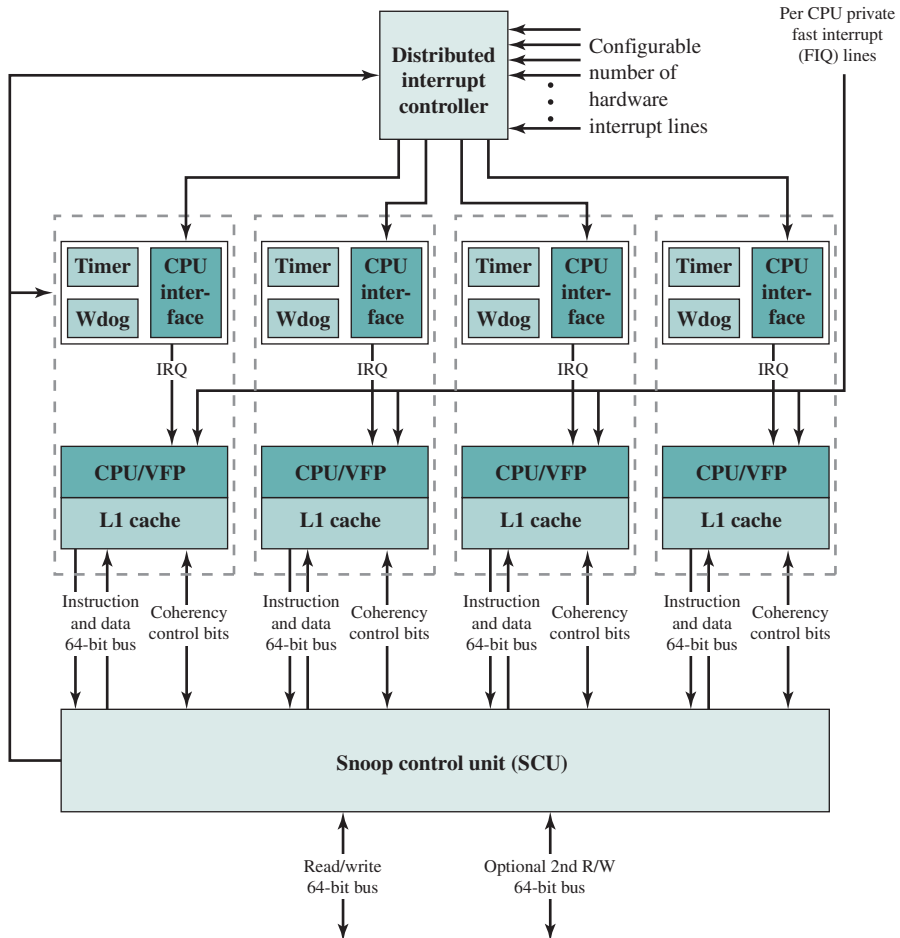
The ARM11 MPCore is a multicore product based on the ARM11 processor family. The ARM11 MPCore can be configured with up to four processors, each with its own L1 instruction and data caches, per chip. Table 18.2 lists the configurable options for the system, including the default values.

Figure 18.11 presents a block diagram of the ARM11 MPCore. The key elements of the system are as follows:

- **Distributed interrupt controller (DIC):** Handles interrupt detection and interrupt prioritization. The DIC distributes interrupts to individual processors.
- **Timer:** Each CPU has its own private timer that can generate interrupts.
- **Watchdog:** Issues warning alerts in the event of software failures. If the watchdog is enabled, it is set to a predetermined value and counts down to 0. It is periodically reset. If the watchdog value reaches zero, an alert is issued.
- **CPU interface:** Handles interrupt acknowledgment, interrupt masking, and interrupt completion acknowledgement.
- **CPU:** A single ARM11 processor. Individual CPUs are referred to as **MP11 CPUs**.
- **Vector floating-point (VFP) unit:** A coprocessor that implements floating-point operations in hardware.
- **L1 cache:** Each CPU has its own dedicated L1 data cache and L1 instruction cache.
- **Snoop control unit (SCU):** Responsible for maintaining coherency among L1 data caches.

**Table 18.2** ARM11 MPCore Configurable Options

| Feature                                               | Range of Options                  | Default Value |
|-------------------------------------------------------|-----------------------------------|---------------|
| Processors                                            | 1 to 4                            | 4             |
| Instruction cache size per processor                  | 16 kB, 32 kB, or 64 kB            | 32 kB         |
| Data cache size per processor                         | 16 kB, 32 kB, or 64 kB            | 32 kB         |
| Master ports                                          | 1 or 2                            | 2             |
| Width of interrupt bus                                | 0 to 224 by increments of 32 pins | 32 pins       |
| Vector floating point (VFP) coprocessor per processor | Included or not                   | Included      |



**Figure 18.11** ARM11 MPCore Processor Block Diagram

## Interrupt Handling

The Distributed Interrupt Controller (DIC) collates interrupts from a large number of sources. It provides

- Masking of interrupts
- Prioritization of the interrupts
- Distribution of the interrupts to the target MP11 CPUs
- Tracking the status of interrupts
- Generation of interrupts by software

The DIC is a single functional unit that is placed in the system alongside MP11 CPUs. This enables the number of interrupts supported in the system to

be independent of the MP11 CPU design. The DIC is memory mapped; that is, control registers for the DIC are defined relative to a main memory base address. The DIC is accessed by the MP11 CPUs using a private interface through the SCU.

The DIC is designed to satisfy two functional requirements:

- Provide a means of routing an interrupt request to a single CPU or multiple CPUs, as required.
- Provide a means of interprocessor communication so that a thread on one CPU can cause activity by a thread on another CPU.

As an example that makes use of both requirements, consider a multithreaded application that has threads running on multiple processors. Suppose the application allocates some virtual memory. To maintain consistency, the operating system must update memory translation tables on all processors. The OS could update the tables on the processor where the virtual memory allocation took place, and then issue an interrupt to all the other processors running this application. The other processors could then use this interrupt's ID to determine that they need to update their memory translation tables.

The DIC can route an interrupt to one or more CPUs in the following three ways:

- An interrupt can be directed to a specific processor only.
- An interrupt can be directed to a defined group of processors. The MPCore views the first processor to accept the interrupt, typically the least loaded, as being best positioned to handle the interrupt.
- An interrupt can be directed to all processors.

From the point of view of software running on a particular CPU, the OS can generate an interrupt to all but self, to self, or to specific other CPUs. For communication between threads running on different CPUs, the interrupt mechanism is typically combined with shared memory for message passing. Thus, when a thread is interrupted by an interprocessor communication interrupt, it reads from the appropriate block of shared memory to retrieve a message from the thread that triggered the interrupt. A total of 16 interrupt IDs per CPU are available for interprocessor communication.

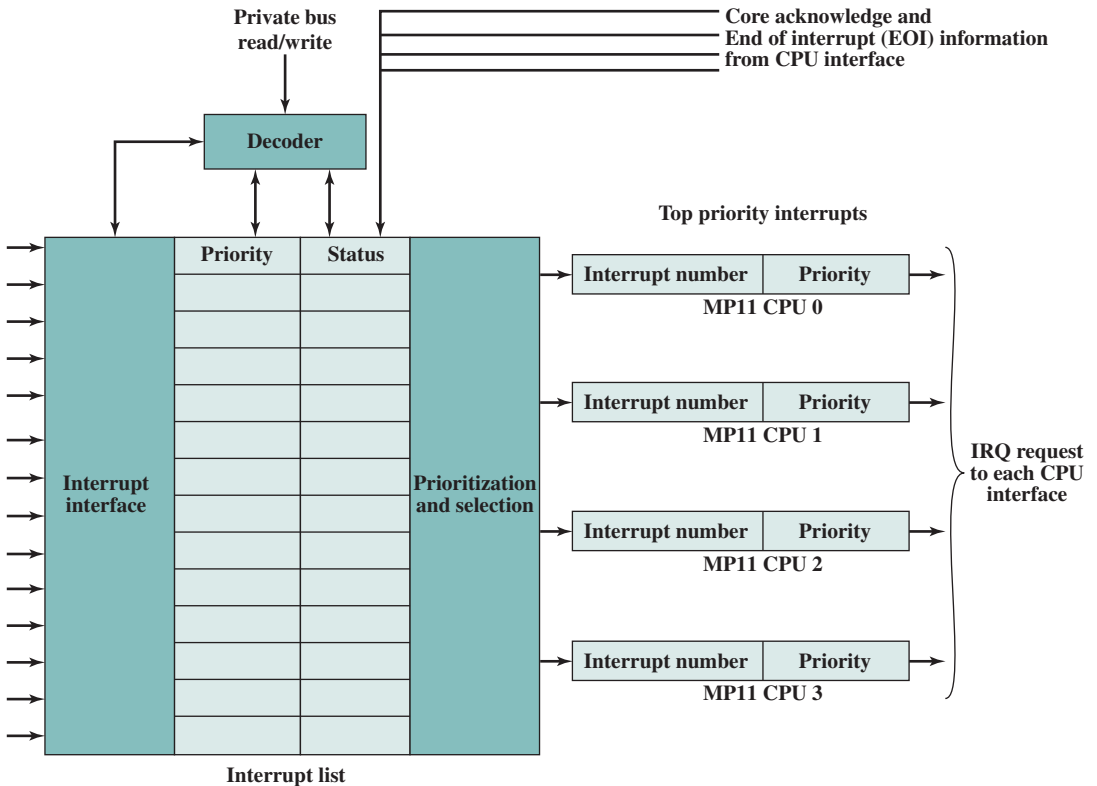
From the point of view of an MP11 CPU, an interrupt can be

- **Inactive:** An Inactive interrupt is one that is nonasserted, or which in a multi-processing environment has been completely processed by that CPU but can still be either Pending or Active in some of the CPUs to which it is targeted, and so might not have been cleared at the interrupt source.
- **Pending:** A Pending interrupt is one that has been asserted, and for which processing has not started on that CPU.
- **Active:** An Active interrupt is one that has been started on that CPU, but processing is not complete. An Active interrupt can be pre-empted when a new interrupt of higher priority interrupts MP11 CPU interrupt processing.

Interrupts come from the following sources:

- **Interprocessor interrupts (IPIs):** Each CPU has private interrupts, ID0-ID15, that can only be triggered by software. The priority of an IPI depends on the receiving CPU, not the sending CPU.
- **Private timer and/or watchdog interrupts:** These use interrupt IDs 29 and 30.
- **Legacy FIQ line:** In legacy IRQ mode, the legacy FIQ pin, on a per CPU basis, bypasses the Interrupt Distributor logic and directly drives interrupt requests into the CPU.
- **Hardware interrupts:** Hardware interrupts are triggered by programmable events on associated interrupt input lines. CPUs can support up to 224 interrupt input lines. Hardware interrupts start at ID32.

Figure 18.12 is a block diagram of the DIC. The DIC is configurable to support between 0 and 255 hardware interrupt inputs. The DIC maintains a list of interrupts, showing their priority and status. The Interrupt Distributor transmits to each CPU Interface the highest Pending interrupt for that interface. It receives back the



**Figure 18.12** Interrupt Distributor Block Diagram

information that the interrupt has been acknowledged, and can then change the status of the corresponding interrupt. The CPU Interface also transmits End of Interrupt Information (EOI), which enables the Interrupt Distributor to update the status of this interrupt from Active to Inactive.

## Cache Coherency

The MPCore's Snoop Control Unit (SCU) is designed to resolve most of the traditional bottlenecks related to access to shared data and the scalability limitation introduced by coherence traffic.

The L1 cache coherency scheme is based on the MESI protocol described in Chapter 17. The SCU monitors operations with shared data to optimize MESI state migration. The SCU introduces three types of optimization: direct data intervention, duplicated tag RAMs, and migratory lines.

**Direct data intervention (DDI)** enables copying clean data from one CPU L1 data cache to another CPU L1 data cache without accessing external memory. This reduces read after read activity from the Level 1 cache to the Level 2 cache. Thus, a local L1 cache miss is resolved in a remote L1 cache rather than from access to the shared L2 cache.

Recall that main memory location of each line within a cache is identified by a tag for that line. The tags can be implemented as a separate block of RAM of the same length as the number of lines in the cache. In the SCU, **duplicated tag RAMs** are duplicated versions of L1 tag RAMs used by the SCU to check for data availability before sending coherency commands to the relevant CPUs. Coherency commands are sent only to CPUs that must update their coherent data cache. This reduces the power consumption and performance impact from snooping into and manipulating each processor's cache on each memory update. Having tag data available locally lets the SCU limit cache manipulations to processors that have cache lines in common.

The **migratory lines** feature enables moving dirty data from one CPU to another without writing to L2 and reading the data back in from external memory. The operation can be described as follows. In a typical MESI protocol, one processor has a modified line and another processor attempts to read that line, the following actions occur:

1. The line contents are transferred from the modified line to the processor that initiated the read.
2. The line contents are written back to main memory.
3. The line is put in the shared state in both caches.

The MPCore SCU handles this situation differently. The SCU monitors the system for a migratory line. If one processor has a modified line, and another processor reads then writes to it, the SCU assumes such a location will experience this same operation in the future. As this operation starts again, the SCU will automatically move the cache line directly to an invalid state rather than expending energy moving it first into the shared state. This optimization also causes the processor to transfer the cache line directly to the other processor without intervening external memory operations.

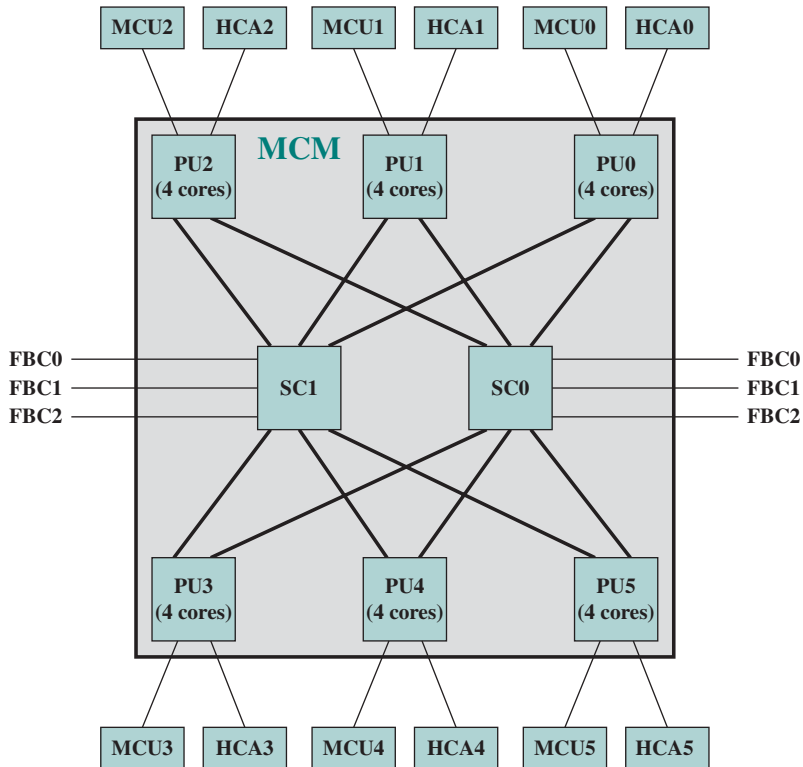
## 18.6 IBM zENTERPRISE 196 MAINFRAME

In this section, we look at a mainframe computer organization that uses multicore processor chips. The example we use is the IBM zEnterprise 196 mainframe computer [CURR11, WHIT11], which began shipping in late 2010. Section 7.8 provides a general overview of the z196, together with a discussion of its I/O structure.

### Organization

The principal building block of the mainframe is the multichip module (MCM), a glass ceramic module that houses 8 chips. The key components of the configuration are shown in Figure 18.13:

- **Processor unit (PU):** There are six 5.2-GHz processor PU chips, each containing four processor cores plus three levels of cache. The PUs have external



FBC = fabric book connectivity  
HCA = host channel adapter  
MCM = multichip module

MCU = memory control unit  
PU = processor unit  
SC = storage control

**Figure 18.13** IBM z196 Processor Node Structure

connections to main memory via memory control units and to I/O via host channel adapters. Thus, each MCM includes 24 core processors.

- **Storage control (SC):** The two SC chips contain an additional level of cache plus interconnection logic for connecting to three other MCMs.

The microprocessor core features a wide superscalar, out-of-order pipeline that can decode three z/Architecture CISC instructions per clock cycle and execute up to five operations per cycle. The instruction execution path is predicted by branch direction and target prediction logic. Each core has six execution units: two integer units, one floating-point unit, two load/store units, and one decimal unit.

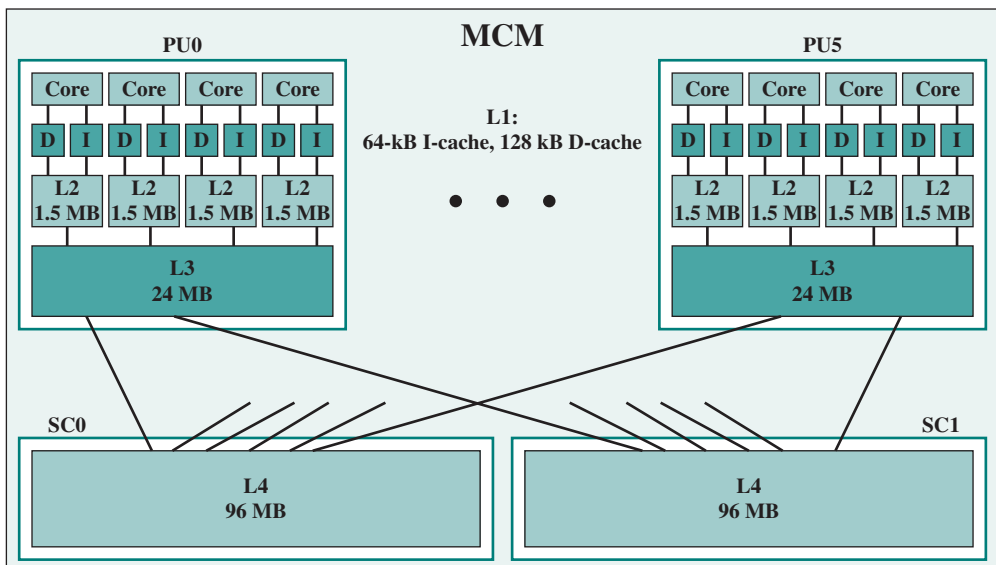
### Cache Structure

The z196 incorporates a four-level cache structure, which IBM states is the industry's first four-level cache. We look at each level in turn (Figure 18.14).

Each core has a dedicated 192-kB **level 1 cache**, divided into a 128-kB data cache and a 64-kB instruction cache. The L1 cache is designed as a store-through cache to L2, that is, altered data are also stored to the next level of memory. These caches are 8-way set associative.

Each core also has a dedicated 1.5-MB **level 2 cache**, which is also a store-through to L3. The L2 cache is 12-way set associative.

Each 4-core processor unit chip includes a 24-MB **level 3 cache** shared by all four processors. Because L1 and L2 caches are store-through, the L3 cache must process every store generated by the four cores on its chip. This feature maintains data availability during a core failure. The L3 cache is 12-way set associative. The z196 implements embedded DRAM (eDRAM) as L3 cache memory on the chip.



**Figure 18.14** IBM z196 Cache Hierarchy



While this eDRAM memory is slower than static RAM (SRAM) normally used to implement cache memory, you can put a lot of it onto a given area. For many workloads, having more memory closer to the chip is more important than having fast memory.

Finally, all 6 PUs on an MCM share a 192-MB **level 4 cache**, which is split into one 96-MB cache on each SC. The principal motivation for incorporating a level 4 cache is that the very high clock speed of the core processors results in a significant mismatch with main memory speed. The fourth cache layer is needed to keep the cores running efficiently. The large shared L3 and L4 caches are suited to transaction-processing workloads exhibiting a high degree of data sharing and task swapping. The L4 cache is 24-way set associative. The SC chip, which houses the L4 cache, also acts as an L4 cache cross-point switch for L4-to-L4 traffic to up to three remote books<sup>3</sup> by three bidirectional data buses. L4 is the coherence manager, meaning that all memory fetches must be in the L4 cache before that data can be used by the processor.

All four caches use a line size of 256 bytes.

The z196 is an interesting study in design trade-offs and the difficulty in exploiting the increasingly powerful processors available with current technology. The large L4 cache is intended to drive the need for access to main memory down to the bare minimum. However, the distance to the off-chip L4 cache costs a number of instruction cycles. Thus, the on-chip area devoted to cache is as large as possible, even to the point of having fewer cores than possible on the chip. The L1 caches are small, to minimize distance from the processor and ensure that access can occur in one cycle. The L2 cache is dedicated to a single core, in an attempt to maximize the amount of access that can occur without resort to a shared cache. The L3 cache is shared by all four processors on a chip and is as large as possible, to minimize the need to go to the L4 cache.

Because all of the books of the zEnterprise 196 share the workload, the four L4 caches on the four books form a single pool of L4 cache memory. Thus, access to L4 means not only going off-chip but perhaps off-book, further increasing access delay. This means relatively large distances exist between the higher-level caches in the processors and the L4 cache content.

To overcome the delays that are inherent to the book design and to save cycles to access the off-book L4 content, the designers try to keep instructions and data as close to the processors as possible by directing as much work of a given logical partition workload on the processors located in the same book as the L4 cache. This is achieved by having the system resource manager/scheduler and the z/OS dispatcher work together to keep as much work as possible within the boundaries of as few processors and L4 cache space (which is best within a book boundary) as can be achieved without affecting throughput and response times. Preventing the resource manager/scheduler and the dispatcher from scheduling and dispatching a workload on any processor available, and keeping the workload in as small a portion, contributes to overcoming latency in a high-frequency processor design such as the z196.

---

<sup>3</sup>Recall from Chapter 7 that a z196 book consists of an MCM, memory cards, and I/O cage connections.

## 18.7 RECOMMENDED READING

Two books that provide good coverage of the issues in this chapter are [OLUK07] and [BAER10]. [GOCH06] and [MEND06] describe the Intel Core Duo. [FOG08b] provides a detailed description of the Core Duo pipeline architecture.

[ARM08b] provides thorough coverage of the ARM Cortex-A8 pipeline. [HIRA07] and [GOOD05] are good overview articles.

**ARM08b** ARM Limited. *ARM11 MPCore Processor Technical Reference Manual*. ARM DDI 0360E, 2008. [www.arm.com](http://www.arm.com)

**BAER10** Baer, J. *Microprocessor Architecture: From Simple Pipelines to Chip Multiprocessors*. New York: Cambridge University Press, 2010.

**FOG08b** Fog, A. *The Microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. <http://www.agner.org/optimize/>

**GOCH06** Gochman, S., et al. "Introduction to Intel Core Duo Processor Architecture." *Intel Technology Journal*, May 2006.

**GOOD05** Goodacre, J., and Sloss, A. "Parallelism and the ARM Instruction Set Architecture." *Computer*, July 2005.

**HIRA07** Hirata, K., and Goodacre, J. "ARM MPCore: The Streamlined and Scalable ARM11 processor core." *Proceedings, 2007 Conference on Asia South Pacific Design Automation*, 2007.

**MEND06** Mendelson, A., et al. "CMP Implementation in Systems Based on the Intel Core Duo Processor." *Intel Technology Journal*, May 2006.

**OLUK07** Olukotun, K.; Hammond, L.; and Laudon, J. *Chip Multiprocessor Architecture: Techniques to Improve Throughput and Latency*. San Rafael, CA: Morgan & Claypool, 2007.

## 18.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

|                                     |                                                |             |
|-------------------------------------|------------------------------------------------|-------------|
| Amdahl's law<br>chip multiprocessor | multicore<br>simultaneous multithreading (SMT) | superscalar |
|-------------------------------------|------------------------------------------------|-------------|

### Review Questions

- 18.1** Summarize the differences among simple instruction pipelining, superscalar, and simultaneous multithreading.
- 18.2** Give several reasons for the choice by designers to move to a multicore organization rather than increase parallelism within a single processor.
- 18.3** Why is there a trend toward giving an increasing fraction of chip area to cache memory?
- 18.4** List some examples of applications that benefit directly from the ability to scale throughput with the number of cores.

- 18.5** At a top level, what are the main design variables in a multicore organization?
- 18.6** List some advantages of a shared L2 cache among cores compared to separate dedicated L2 caches for each core.

## Problems

- 18.1** Consider the following problem. A designer has a chip available and must decide what fraction of the chip will be devoted to cache memory (L1, L2, L3). The remainder of the chip can be devoted to a single complex superscalar and/or SMT core or multiple somewhat simpler cores. Define the following parameters:

$n$  = maximum number of cores that can be contained on the chip

$k$  = actual number of cores implemented ( $1 \leq k \leq n$ , where  $r = n/k$  is an integer)

$perf(r)$  = sequential performance gain by using the resources equivalent to  $r$  cores to form a single processor, where  $perf(1) = 1$ .

$f$  = fraction of software that is parallelizable across multiple cores.

Thus, if we construct a chip with  $n$  cores, we expect each core to provide sequential performance of 1 and for the  $n$  cores to be able to exploit parallelism up to a degree of  $n$  parallel threads. Similarly, if the chip has  $k$  cores, then each core should exhibit a performance of  $perf(r)$  and the chip is able to exploit parallelism up to a degree of  $k$  parallel threads. We can modify Amdahl's law (Equation 18.1) to reflect this situation as follows:

$$\text{Speedup} = \frac{1}{\frac{1-f}{perf(r)} + \frac{f \times r}{perf(r) \times n}}$$

- a. Justify this modification of Amdahl's law.
  - b. Using Pollack's rule, we set  $perf(r) = \sqrt{r}$ . Let  $n = 16$ . We want to plot speedup as a function of  $r$  for  $f = 0.5; f = 0.9; f = 0.975; f = 0.99; f = 0.999$ . The results are available in a document at this book's Premium Content site (multicore-performance.pdf). What conclusions can you draw?
  - c. Repeat part (b) for  $n = 256$ .
- 18.2** The technical reference manual for the ARM11 MPCore says that the Distributed Interrupt Controller is memory mapped. That is, the core processors use memory mapped I/O to communicate with the DIC. Recall from Chapter 7 that with memory-mapped I/O, there is a single address space for memory locations and I/O devices. The processor treats the status and data registers of I/O modules as memory locations and uses the same machine instructions to access both memory and I/O devices. Based on this information, what path through the block diagram of Figure 18.11 is used for the core processors to communicate with the DIC?
- 18.3** In this question we analyze the performance of the following C program on a multi-threaded architecture. You should assume that arrays A, B, and C do not overlap in memory.

```
for (i=0; i<328; i++) {
 A[i] = A[i]*B[i];
 C[i] = C[i]+A[i];
}
```

Our machine is a single-issue, in-order processor. It switches to a different thread every cycle using fixed round robin scheduling. Each of the  $N$  threads executes one instruction every  $N$  cycles. We allocate the code to the threads such that every thread executes every  $N$ th iteration of the original C code.

Integer instructions take 1 cycle to execute, floating-point instructions take 4 cycles and memory instructions take 3 cycles. All execution units are fully pipelined. If an instruction cannot issue because its data is not yet available, it inserts a bubble into the pipeline, and retries after 1 cycles.

Below is our program in assembly code for this machine for a single thread executing the entire loop.

```

loop: ld f1, 0 (r1) ;f1 = A[i]
 ld f2, 0 (r2) ;f2 = B[i]
 fmul f4, f2, f1 ;f4 = f1*f2
 st f4 0(r1) ;A[i] = f4
 ld f3, 0(r3) ;f3 = C[i]
 fadd f5, f4, f3 ;f5 = f4 + f3
 st f5 0(r3) ;C[i] = f5
 add r1, r1, 4 ;i++
 add r2, r2, 4
 add r3, r3, 4
 add r4, r4, -1
 bnez r4, loop ;loop

```

- a. We allocate the assembly code of the loop to  $N$  threads such that every thread executes every  $N$ th iteration of the original loop. Write the assembly code that one of the  $N$  threads would execute on this multithreaded machine.
- b. What is the minimum number of threads this machine needs to remain fully utilized issuing an instruction every cycle for our program?
- c. Could we reach peak performance running this program using fewer threads by rearranging the instructions? Explain briefly.
- d. What will be the peak performance in flops/cycle for this program?

*This page intentionally left blank*

# APPENDIX A

---

## PROJECTS FOR TEACHING COMPUTER ORGANIZATION AND ARCHITECTURE

**A.1 Interactive Simulations**

**A.2 Research Projects**

**A.3 Simulation Projects**

SimpleScalar

SMPCache

**A.4 Assembly Language Projects**

**A.5 Reading/Report Assignments**

**A.6 Writing Assignments**

**A.7 Test Bank**

Many instructors believe that research or implementation projects are crucial to the clear understanding of the concepts of computer organization and architecture. Without projects, it may be difficult for students to grasp some of the basic concepts and interactions among components. Projects reinforce the concepts introduced in the book, give students a greater appreciation of the inner workings of processors and computer systems, and can motivate students and give them confidence that they have mastered the material.

In this text, I have tried to present the concepts of computer organization and architecture as clearly as possible and have provided numerous homework problems to reinforce those concepts. Many instructors will wish to supplement this material with projects. This appendix provides some guidance in that regard and describes support material available in the **Instructor's Resource Center (IRC)** for this book, accessible by instructors online from Prentice Hall. The support material covers six types of projects and other student exercises:

- Interactive simulations
- Research projects
- Simulation projects
- Assembly language projects
- Reading/report assignments
- Writing assignments
- Test bank

## A.1 INTERACTIVE SIMULATIONS

Interactive simulations provide a powerful tool for understanding the complex design features of a modern computer system. Today's students want to be able to visualize the various complex computer systems mechanisms on their own computer screen. A total of 20 simulations are used to illustrate key functions and algorithms in computer organization and architecture design. Table A.1 lists the simulations by chapter. At the relevant point in the book, an icon indicates that a relevant interactive simulation is available online for student use.

Because the simulations enable the user to set initial conditions, they can serve as the basis for student assignments. The IRC for this book includes a set of assignments, one set for each of the interactive simulations. Each assignment includes a several specific problems that can be assigned to students.

The interactive simulations were developed under the direction of Professor Israel Koren, at the University of Massachusetts Department of Electrical and Computer Engineering. Aswin Sreedhar of the University of Massachusetts developed the interactive simulation assignments. For access to the animations, click on the rotating globe at this book's web site at <http://williamstallings.com/ComputerOrganization>.

**Table A.1** Computer Organization and Architecture—Interactive Simulations by Chapter

| <b>Chapter 4—Cache Memory</b>                                              |                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cache Simulator                                                            | Emulates small-sized caches based on a user-input cache model and displays the cache contents at the end of the simulation cycle based on an input sequence which is entered by the user, or randomly generated if so selected. |
| Cache Time Analysis                                                        | Demonstrates Average Memory Access Time analysis for the cache parameters you specify.                                                                                                                                          |
| Multitask Cache Demonstrator                                               | Models cache on a system that supports multitasking.                                                                                                                                                                            |
| Selective Victim Cache Simulator                                           | Compares three different cache policies.                                                                                                                                                                                        |
| <b>Chapter 5—Internal Memory</b>                                           |                                                                                                                                                                                                                                 |
| Interleaved Memory Simulator                                               | Demonstrates the effect of interleaving memory.                                                                                                                                                                                 |
| <b>Chapter 6—External Memory</b>                                           |                                                                                                                                                                                                                                 |
| RAID                                                                       | Determine storage efficiency and reliability.                                                                                                                                                                                   |
| <b>Chapter 7—Input/Output</b>                                              |                                                                                                                                                                                                                                 |
| I/O System Design Tool                                                     | Evaluates comparative cost and performance of different I/O systems.                                                                                                                                                            |
| <b>Chapter 8—OS Support</b>                                                |                                                                                                                                                                                                                                 |
| Page Replacement Algorithms                                                | Compares LRU, FIFO, and Optimal.                                                                                                                                                                                                |
| More Page Replacement Algorithms                                           | Compares a number of policies.                                                                                                                                                                                                  |
| <b>Chapter 14—CPU Structure and Function</b>                               |                                                                                                                                                                                                                                 |
| Reservation Table Analyzer                                                 | Evaluates reservation tables, which are a way of representing the task flow pattern of a pipelined system.                                                                                                                      |
| Branch Prediction                                                          | Demonstrates three different branch prediction schemes.                                                                                                                                                                         |
| Branch Target Buffer                                                       | Combined branch predictor/branch target buffer simulator.                                                                                                                                                                       |
| <b>Chapter 15—Reduced Instruction Set Computers</b>                        |                                                                                                                                                                                                                                 |
| MIPS 5-Stage Pipeline                                                      | Simulates the pipeline.                                                                                                                                                                                                         |
| Loop Unrolling                                                             | Simulates the loop unrolling software technique for exploiting instruction-level parallelism.                                                                                                                                   |
| <b>Chapter 16—Instruction-Level Parallelism and Superscalar Processors</b> |                                                                                                                                                                                                                                 |
| Pipeline with Static vs. Dynamic Scheduling                                | A more complex simulation of the MIPS pipeline.                                                                                                                                                                                 |
| Reorder Buffer Simulator                                                   | Simulates instruction reordering in a RISC pipeline.                                                                                                                                                                            |
| Scoreboarding Technique for Dynamic Scheduling                             | Simulation of an instruction scheduling technique used in a number of processors.                                                                                                                                               |
| Tomasulo's Algorithm                                                       | Simulation of another instruction scheduling technique.                                                                                                                                                                         |
| Alternative Simulation of Tomasulo's Algorithm                             | Another simulation of Tomasulo's algorithm.                                                                                                                                                                                     |
| <b>Chapter 17—Parallel Processing</b>                                      |                                                                                                                                                                                                                                 |
| Vector Processor Simulation                                                | Demonstrates execution of vector processing instructions.                                                                                                                                                                       |



## A.2 RESEARCH PROJECTS

An effective way of reinforcing basic concepts from the course and for teaching students research skills is to assign a research project. Such a project could involve a literature search as well as a Web search of vendor products, research lab activities, and standardization efforts. Projects could be assigned to teams or, for smaller projects, to individuals. In any case, it is best to require some sort of project proposal early in the term, giving the instructor time to evaluate the proposal for appropriate topic and appropriate level of effort. Student handouts for research projects should include

- A format for the proposal
- A format for the final report
- A schedule with intermediate and final deadlines
- A list of possible project topics

The students can select one of the listed topics or devise their own comparable project. The IRC includes a suggested format for the proposal and final report as well as a list of possible research topics.

## A.3 SIMULATION PROJECTS

An excellent way to obtain a grasp of the internal operation of a processor and to study and appreciate some of the design trade-offs and performance implications is by simulating key elements of the processor. Two useful tools that are useful for this purpose are SimpleScalar and SMPCache.

Compared with actual hardware implementation, simulation provides two advantages for both research and educational use:

- With simulation, it is easy to modify various elements of an organization, to vary the performance characteristics of various components, and then to analyze the effects of such modifications.
- Simulation provides for detailed performance statistics collection, which can be used to understand performance trade-offs.

### SimpleScalar

SimpleScalar [BURG97, MANJ01a, MANJ01b] is a set of tools that can be used to simulate real programs on a range of modern processors and systems. The tool set includes compiler, assembler, linker, and simulation and visualization tools. SimpleScalar provides processor simulators that range from an extremely fast functional simulator to a detailed out-of-order issue, superscalar processor simulator that supports nonblocking caches and speculative execution. The instruction set architecture and organizational parameters may be modified to create a variety of experiments.

The IRC for this book includes a concise introduction to SimpleScalar for students, with instructions on how to load and get started with SimpleScalar. The manual also includes some suggested project assignments.

SimpleScalar is a portable software package that runs on most UNIX platforms. The SimpleScalar software can be downloaded from the SimpleScalar Web site. It is available at no cost for noncommercial use.

### SMPCache

SMPCache is a trace-driven simulator for the analysis and teaching of cache memory systems on symmetric multiprocessors [RODR01]. The simulation is based on a model built according to the architectural basic principles of these systems. The simulator has a full graphic and friendly interface. Some of the parameters that they can be studied with the simulator are: program locality; influence of the number of processors, cache coherence protocols, schemes for bus arbitration, mapping, replacement policies, cache size (blocks in cache), number of cache sets (for set associative caches), number of words by block (memory block size).

The IRC for this book includes a concise introduction to SMPCache for students, with instructions on how to load and get started with SMPCache. The manual also includes some suggested project assignments.

SMPCache is a portable software package that runs on PC systems with Windows. The SMPCache software can be downloaded from the SMPCache Web site. It is available at no cost for noncommercial use.

## A.4 ASSEMBLY LANGUAGE PROJECTS

Assembly language programming is often used to teach students low-level hardware components and computer architecture basics. CodeBlue is a simplified assembly language program developed at the U. S. Air Force Academy. The goal of the work was to develop and teach assembly language concepts using a visual simulator that students can learn in a single class. The developers also wanted students to find the language motivational and fun to use. The CodeBlue language is much simpler than most simplified architecture instruction sets such as the SC123. Still it allows students to develop interesting assembly level programs that compete in tournaments, similar to the far more complex SPIMbot simulator. Most important, through CodeBlue programming, students learn fundamental computer architecture concepts such as instructions and data co-residence in memory, control structure implementation, and addressing modes.

To provide a basis for projects, the developers have built a visual development environment that allows students to create a program, see its representation in memory, step through the program's execution, and simulate a battle of competing programs in a visual memory environment.

Projects can be built around the concept of a Core War tournament. Core War is a programming game introduced to the public in the early 1980s, which was popular for a period of 15 years or so. Core War has four main components: a memory array of 8000 addresses, a simplified assembly language Redcode, an executive program called MARS (an acronym for Memory Array Redcode Simulator) and the set of contending battle programs. Two battle programs are entered into the memory array at randomly chosen positions; neither program knows where the

other one is. MARS executes the programs in a simple version of time-sharing. The two programs take turns: a single instruction of the first program is executed, then a single instruction of the second, and so on. What a battle program does during the execution cycles allotted to it is entirely up to the programmer. The aim is to destroy the other program by ruining its instructions. The CodeBlue environment substitutes CodeBlue for Redcode and provides its own interactive execution interface.

The IRC includes the CodeBlue environment, a user's manual for students, other supporting material, and suggested assignments.

## A.5 READING/REPORT ASSIGNMENTS

Another excellent way to reinforce concepts from the course and to give students research experience is to assign papers from the literature to be read and analyzed. The IRC includes a suggested list of papers to be assigned, organized by chapter. The Premium Content Web site provides a copy of each of the papers. The IRC also includes a suggested assignment wording.

## A.6 WRITING ASSIGNMENTS

Writing assignments can have a powerful multiplier effect in the learning process in a technical discipline such as computer organization and architecture. Adherents of the Writing Across the Curriculum (WAC) movement (<http://wac.colostate.edu/>) report substantial benefits of writing assignments in facilitating learning. Writing assignments lead to more detailed and complete thinking about a particular topic. In addition, writing assignments help to overcome the tendency of students to pursue a subject with a minimum of personal engagement, just learning facts and problem-solving techniques without obtaining a deep understanding of the subject matter.

The IRC contains a number of suggested writing assignments, organized by chapter. Instructors may ultimately find that this is the most important part of their approach to teaching the material. I would greatly appreciate any feedback on this area and any suggestions for additional writing assignments.

## A.7 TEST BANK

A test bank for the book is available at the IRC site for this book. For each chapter, the test bank includes true/false, multiple choice, and fill-in-the-blank questions. The test bank is an effective way to assess student comprehension of the material.

# APPENDIX B

---

## ASSEMBLY LANGUAGE AND RELATED TOPICS

### **B.1 Assembly Language**

Assembly Language Elements

Type of Assembly Language Statements

Example: Greatest Common Divisor Program

### **B.2 Assemblers**

Two-Pass Assembler

One-Pass Assembler

Example: Prime Number Program

### **B.3 Loading and Linking**

Relocation

Loading

Linking

### **B.4 Recommended Reading**

### **B.5 Key Terms, Review Questions, and Problems**

The topic of assembly language was briefly introduced in Chapter 13. This appendix provides more detail and also covers a number of related topics. There are a number of reasons why it is worthwhile to study assembly language programming (as compared with programming in a higher-level language), including the following:

1. It clarifies the execution of instructions.
2. It shows how data is represented in memory.
3. It shows how a program interacts with the operating system, processor, and the I/O system.
4. It clarifies how a program accesses external devices.
5. Understanding assembly language programmers makes students better high-level language (HLL) programmers, by giving them a better idea of the target language that the HLL must be translated into.

We begin this chapter with a study of the basic elements of an assembly language, using the x86 architecture for our examples.<sup>1</sup> Next, we look at the operation of the assembler. This is followed by a discussion of linkers and loaders.

Table B.1 defines some of the key terms used in this appendix.

## B.1 ASSEMBLY LANGUAGE

Assembly language is a programming language that is one step away from machine language. Typically, each assembly language instruction is translated into one machine instruction by the assembler. Assembly language is hardware dependent, with a different assembly language for each type of processor. In particular, assembly language instructions can make reference to specific registers in the processor, include all of the opcodes of the processor, and reflect the bit length of the various registers of the processor and operands of the machine language. An assembly language programmer must therefore understand the computer's architecture.

Programmers rarely use assembly language for applications or even systems programs. HLLs provide an expressive power and conciseness that greatly eases the programmer's tasks. The disadvantages of using an assembly language rather than an HLL include the following [FOG08a]:

1. **Development time.** Writing code in assembly language takes much longer than writing in a high-level language.
2. **Reliability and security.** It is easy to make errors in assembly code. The assembler is not checking if the calling conventions and register save conventions are obeyed. Nobody is checking for you if the number of PUSH and POP instructions is the same in all possible branches and paths. There are so many possibilities for hidden errors in assembly code that it affects the reliability and security of the project unless you have a very systematic approach to testing and verifying.

---

<sup>1</sup>There are a number of assemblers for the x86 architecture. Our examples use NASM (Netwide Assembler), an open source assembler. A copy of the NASM manual is at this book's Premium Content site.

**Table B.1** Key Terms for this Appendix**Assembler**

A program that translates assembly language into machine code.

**Assembly Language**

A symbolic representation of the machine language of a specific processor, augmented by additional types of statements that facilitate program writing and that provide instructions to the assembler.

**Compiler**

A program that converts another program from some source language (or programming language) to machine language (object code). Some compilers output assembly language which is then converted to machine language by a separate assembler. A compiler is distinguished from an assembler by the fact that each input statement does not, in general, correspond to a single machine instruction or fixed sequence of instructions. A compiler may support such features as automatic allocation of variables, arbitrary arithmetic expressions, control structures such as FOR and WHILE loops, variable scope, input/output operations, higher-order functions and portability of source code.

**Executable Code**

The machine code generated by a source code language processor such as an assembler or compiler. This is software in a form that can be run in the computer.

**Instruction Set**

The collection of all possible instructions for a particular computer; that is, the collection of machine language instructions that a particular processor understands.

**Linker**

A utility program that combines one or more files containing object code from separately compiled program modules into a single file containing loadable or executable code.

**Loader**

A program routine that copies an executable program into memory for execution.

**Machine Language, or Machine Code**

The binary representation of a computer program which is actually read and interpreted by the computer. A program in machine code consists of a sequence of machine instructions (possibly interspersed with data). Instructions are binary strings which may be either all the same size (e.g., one 32-bit word for many modern RISC microprocessors) or of different sizes.

**Object Code**

The machine language representation of programming source code. Object code is created by a compiler or assembler and is then turned into executable code by the linker.

3. **Debugging and verifying.** Assembly code is more difficult to debug and verify because there are more possibilities for errors than in high-level code.
4. **Maintainability.** Assembly code is more difficult to modify and maintain because the language allows unstructured spaghetti code and all kinds of tricks that are difficult for others to understand. Thorough documentation and a consistent programming style are needed.
5. **Portability.** Assembly code is platform-specific. Porting to a different platform is difficult.

6. **System code can use intrinsic functions instead of assembly.** The best modern C++ compilers have intrinsic functions for accessing system control registers and other system instructions. Assembly code is no longer needed for device drivers and other system code when intrinsic functions are available.
7. **Application code can use intrinsic functions or vector classes instead of assembly.** The best modern C++ compilers have intrinsic functions for vector operations and other special instructions that previously required assembly programming.
8. **Compilers have been improved a lot in recent years.** The best compilers are now quite good. It takes a lot of expertise and experience to optimize better than the best C++ compiler.

Yet there are still some advantages to the occasional use of assembly language, including the following [FOG08a]:

1. **Debugging and verifying.** Looking at compiler-generated assembly code or the disassembly window in a debugger is useful for finding errors and for checking how well a compiler optimizes a particular piece of code.
2. **Making compilers.** Understanding assembly coding techniques is necessary for making compilers, debuggers and other development tools.
3. **Embedded systems.** Small embedded systems have fewer resources than PCs and mainframes. Assembly programming can be necessary for optimizing code for speed or size in small embedded systems.
4. **Hardware drivers and system code.** Accessing hardware, system control registers, and so on may sometimes be difficult or impossible with high level code.
5. **Accessing instructions that are not accessible from high-level language.** Certain assembly instructions have no high-level language equivalent.
6. **Self-modifying code.** Self-modifying code is generally not profitable because it interferes with efficient code caching. It may, however, be advantageous, for example, to include a small compiler in math programs where a user-defined function has to be calculated many times.
7. **Optimizing code for size.** Storage space and memory is so cheap nowadays that it is not worth the effort to use assembly language for reducing code size. However, cache size is still such a critical resource that it may be useful in some cases to optimize a critical piece of code for size in order to make it fit into the code cache.
8. **Optimizing code for speed.** Modern C++ compilers generally optimize code quite well in most cases. But there are still cases where compilers perform poorly and where dramatic increases in speed can be achieved by careful assembly programming.
9. **Function libraries.** The total benefit of optimizing code is higher in function libraries that are used by many programmers.
10. **Making function libraries compatible with multiple compilers and operating systems.** It is possible to make library functions with multiple entries that are compatible with different compilers and different operating systems. This requires assembly programming.

The terms *assembly language* and *machine language* are sometimes, erroneously, used synonymously. Machine language consists of instructions directly executable by the processor. Each machine language instruction is a binary string containing an opcode, operand references, and perhaps other bits related to execution, such as flags. For convenience, instead of writing an instruction as a bit string, it can be written symbolically, with names for opcodes and registers. An assembly language makes much greater use of symbolic names, including assigning names to specific main memory locations and specific instruction locations. Assembly language also includes statements that are not directly executable but serve as instructions to the assembler that produces machine code from an assembly language program.

### Assembly Language Elements

A statement in a typical assembly language has the form shown in Figure B.1. It consists of four elements: label, mnemonic, operand, and comment.

**LABEL** If a label is present, the assembler defines the label as equivalent to the address into which the first byte of the object code generated for that instruction will be loaded. The programmer may subsequently use the label as an address or as data in another instruction's address field. The assembler replaces the label with the assigned value when creating an object program. Labels are most frequently used in branch instructions.

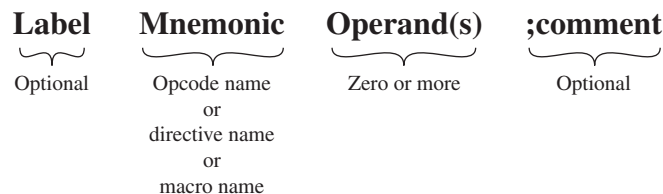
As an example, here is a program fragment:

```
L2: SUB EAX, EDX ;subtract contents of register EDX from
 ;contents of EAX and store result in EAX
 JG L2 ;jump to L2 if result of subtraction is
 ;positive
```

The program will continue to loop back to location L2 until the result is zero or negative. Thus, when the `kg` instruction is executed, if the result is positive, the processor places the address equivalent to the label L2 in the program counter.

Reasons for using a label include the following:

1. A label makes a program location easier to find and remember.
2. The label can easily be moved to correct a program. The assembler will automatically change the address in all instructions that use the label when the program is reassembled.
3. The programmer does not have to calculate relative or absolute memory addresses, but just uses labels as needed.



**Figure B.1** Assembly-Language Statement Structure

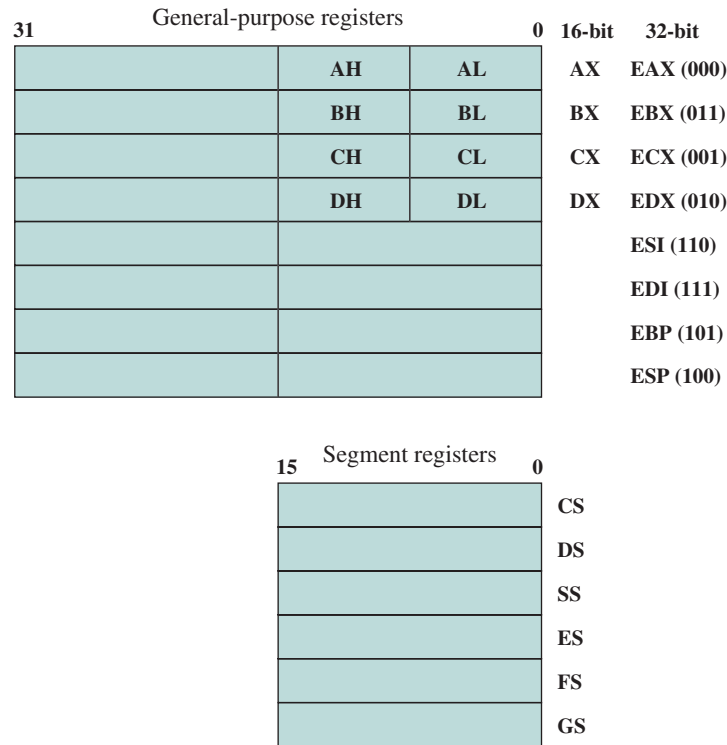


**MNEMONIC** The mnemonic is the name of the operation or function of the assembly language statement. As discussed subsequently, a statement can correspond to a machine instruction, an assembler directive, or a macro. In the case of a machine instruction, a mnemonic is the symbolic name associated with a particular opcode.

Table 12.8 lists the mnemonic, or instruction name, of many of the x86 instructions. Appendix A of [CART06] lists the x86 instructions, together with the operands for each and the effect of the instruction on the condition codes. Appendix B of the NASM manual provides a more detailed description of each x86 instruction. Both documents are available at this book’s Premium Content site.

**OPERAND(S)** An assembly language statement includes zero or more operands. Each operand identifies an immediate value, a register value, or a memory location. Typically, the assembly language provides conventions for distinguishing among the three types of operand references, as well as conventions for indicating addressing mode.

For the x86 architecture, an assembly language statement may refer to a register operand by name. Figure B.2 illustrates the general-purpose x86 registers, with their symbolic name and their bit encoding. The assembler will translate the symbolic name into the binary identifier for the register.



**Figure B.2** Intel x86 Program Execution Registers

As discussed in Section 11.2, the x86 architecture has a rich set of addressing modes, each of which must be expressed symbolically in the assembly language. Here we cite a few of the common examples. For **register addressing**, the name of the register is used in the instruction. For example, `MOV ECX, EBX` copies the contents of register EBX into register ECX. Immediate addressing indicates that the value is encoded in the instruction. For example, `MOV EAX, 100H` copies the hexadecimal value 100 into register EAX. The immediate value can be expressed as a binary number with the suffix B or a decimal number with no suffix. Thus, equivalent statements to the preceding one are `MOV EAX, 100000000B` and `MOV EAX, 256`. **Direct addressing** refers to a memory location and is expressed as a displacement from the DS segment register. This is best explained by example. Assume that the 16-bit data segment register DS contains the value 1000H. Then the following sequence occurs:

```
MOV AX, 1234H
MOV [3518H], AX
```

First the 16-bit register AX is initialized to 1234H. Then, in line two, the contents of AX are moved to the logical address DS:3518H. This address is formed by shifting the contents of DS left 4 bits and adding 3518H to form the 32-bit logical address 13518H.

**COMMENT** All assembly languages allow the placement of comments in the program. A comment can either occur at the right-hand end of an assembly statement or can occupy an entire text line. In either case, the comment begins with a special character that signals to the assembler that the rest of the line is a comment and is to be ignored by the assembler. Typically, assembly languages for the x86 architecture use a semicolon (;) for the special character.

## Type of Assembly Language Statements

Assembly language statements are one of four types: instruction, directive, macro definition, and comment. A comment statement is simply a statement that consists entirely of a comment. The remaining types are briefly described in this section.

**INSTRUCTIONS** The bulk of the noncomment statements in an assembly language program are symbolic representations of machine language instructions. Almost invariably, there is a one-to-one relationship between an assembly language instruction and a machine instruction. The assembler resolves any symbolic references and translates the assembly language instruction into the binary string that comprises the machine instruction.

**DIRECTIVES** Directives, also called **pseudo-instructions**, are assembly language statements that are not directly translated into machine language instructions. Instead, directives are instruction to the assembler to perform specified actions doing the assembly process. Examples include the following:

- Define constants
- Designate areas of memory for data storage
- Initialize areas of memory

- Place tables or other fixed data in memory
- Allow references to other programs

Table B.2 lists some of the NASM directives. As an example, consider the following sequence of statements:

**Table B.2** Some NASM Assembly-Language Directives

**(a) Letters for RESx and Dx Directives**

| Unit                  | Letter |
|-----------------------|--------|
| byte                  | B      |
| word (2 bytes)        | W      |
| double word (4 bytes) | D      |
| quad word (8 bytes)   | Q      |
| ten bytes             | T      |

**(b) Directives**

| Name                                     | Description                                  | Example                                                          |
|------------------------------------------|----------------------------------------------|------------------------------------------------------------------|
| DB, DW,<br>DD, DQ,<br>DT                 | Initialize locations                         | L6 DD 1A92H<br>;doubleword at L6 initialized to 1A92H            |
| RESB,<br>RESW,<br>RESD,<br>RESQ,<br>REST | Reserve<br>uninitialized<br>locations        | BUFFER RESB 64<br>;reserve 64 bytes starting at BUFFER           |
| INCBIN                                   | Include binary file<br>in output             | INCBIN "file.dat" ; include this file                            |
| EQU                                      | Define a symbol to a<br>given constant value | MSGLEN EQU 25<br>;the constant MSGLEN equals decimal 25          |
| TIMES                                    | Repeat instruction<br>multiple times         | ZEROBUF TIMES 64 DB 0<br>;initialize 64-byte buffer to all zeros |

```
L2 DB "A" ;byte initialized to ASCII code for A (65)
MOV AL, [L1] ;copy byte at L1 into AL
MOV EAX, L1 ;store address of byte at L1 in EAX
MOV [L1], AH ;copy contents of AH into byte at L1
```

If a plain label is used, it is interpreted as the address (or offset) of the data. If the label is placed inside square brackets, it is interpreted as the data at the address.

**MACRO DEFINITIONS** A macro definition is similar to a subroutine in several ways. A subroutine is a section of a program that is written once, and can be used multiple times by calling the subroutine from any point in the program. When a program is compiled or assembled, the subroutine is loaded only once. A call to the subroutine transfers control to the subroutine and a return instruction in the subroutine returns

control to the point of the call. Similarly, a macro definition is a section of code that the programmer writes once, and then can use many times. The main difference is that when the assembler encounters a macro call, it replaces the macro call with the macro itself. This process is called **macro expansion**. So, if a macro is defined in an assembly language program and invoked 10 times, then 10 instances of the macro will appear in the assembled code. In essence, subroutines are handled by the hardware at run time, whereas macros are handled by the assembler at assembly time. Macros provide the same advantage as subroutines in terms of modular programming, but without the runtime overhead of a subroutine call and return. The tradeoff is that the macro approach uses more space in the object code.

In NASM and many other assemblers, a distinction is made between a single-line macro and a multi-line macro. In NASM, single-line macros are defined using the `%DEFINE` directive. Here is an example in which multiple single-line macros are expanded. First, we define two macros:

```
%DEFINE B(X) = 2*X
%DEFINE A(X) = 1 + B(X)
```

At some point in the assembly language program, the following statement appears:

```
MOV AX, A(8)
```

The assembler expands this statement to:

```
MOV AX, 1+2*8
```

which assembles to a machine instruction to move the immediate value 17 to register AX.

Multiline macros are defined using the mnemonic `&MACRO`. Here is an example of a multiline macro definition:

```
%MACRO PROLOGUE 1
 PUSH EBP ;push contents of EBP onto stack
 ;pointed to by ESP and
 ;decrement contents of ESP by 4
 MOV EBP, ESP ;copy contents of ESP to EBP
 SUB ESP, %1 ;subtract first parameter value from ESP
```

The number 1 after the macro name in the `%MACRO` line defines the number of parameters the macro expects to receive. The use of `%1` inside the macro definition refers to the first parameter to the macro call.

The macro call

```
MYFUNC: PROLOGUE 12
```

expands to the following lines of code:

```
MYFUNC: PUSH EBP
 MOV EBP, ESP
 SUB ESP, 12
```

### Example: Greatest Common Divisor Program

As an example of the use of assembly language, we look at a program to compute the greatest common divisor of two integers. We define the greatest common divisor of the integers  $a$  and  $b$  as follows:

$$\text{gcd}(a,b) = \max[k, \text{such that } k \text{ divides } a \text{ and } k \text{ divides } b]$$

where we say that  $k$  divides  $a$  if there is no remainder. Euclid's algorithm for the greatest common divisor is based on the following theorem. For any nonnegative integers  $a$  and  $b$ ,

$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$$

Here is a C language program that implements Euclid's algorithm:

```
unsigned int gcd (unsigned int a, unsigned int b)
{
 if (a == 0 && b == 0)
 b = 1;
 else if (b == 0)
 b = a;
 else if (a != 0)
 while (a != b)
 if (a < b)
 b -= a;
 else
 a -= b;
 return b;
}
```

Figure B.3 shows two assembly language versions of the preceding program. The program on the left was done by a C compiler; the program on the right was programmed by hand. The latter program uses a number of programmer's tricks to produce a tighter, more efficient implementation.

## B.2 ASSEMBLERS

The assembler is a software utility that takes an assembly program as input and produces object code as output. The object code is a binary file. The assembler views this file as a block of memory starting at relative location 0.

There are two general approaches to assemblers: the two-pass assembler and the one-pass assembler.

### Two-Pass Assembler

We look first at the two-pass assembler, which is more common and somewhat easier to understand. The assembler makes two passes through the source code (Figure B.4):

|      |      |          |      |      |          |
|------|------|----------|------|------|----------|
| gcd: | mov  | ebx, eax | gcd: | neg  | eax      |
|      | mov  | eax, edx |      | je   | L3       |
|      | test | ebx, ebx | L1:  | neg  | eax      |
|      | jne  | L1       |      | xchg | eax, edx |
|      | test | edx, edx | L2:  | sub  | eax, edx |
|      | jne  | L1       |      | jg   | L2       |
|      | mov  | eax, 1   |      | jne  | L1       |
|      | ret  |          | L3:  | add  | eax, edx |
| L1:  | test | eax, eax |      | jne  | L4       |
|      | jne  | L2       |      | inc  | eax      |
|      | mov  | eax, ebx | L4:  | ret  |          |
|      | ret  |          |      |      |          |
| L2:  | test | ebx, ebx |      |      |          |
|      | je   | L5       |      |      |          |
| L3:  | cmp  | ebx, eax |      |      |          |
|      | je   | L5       |      |      |          |
|      | jae  | L4       |      |      |          |
|      | sub  | eax, ebx |      |      |          |
|      | jmp  | L3       |      |      |          |
| L4:  | sub  | ebx, eax |      |      |          |
|      | jmp  | L3       |      |      |          |
| L5:  | ret  |          |      |      |          |

(a) Compiled program

(b) Written directly in assembly language

**Figure B.3** Assembly Programs for Greatest Common Divisor

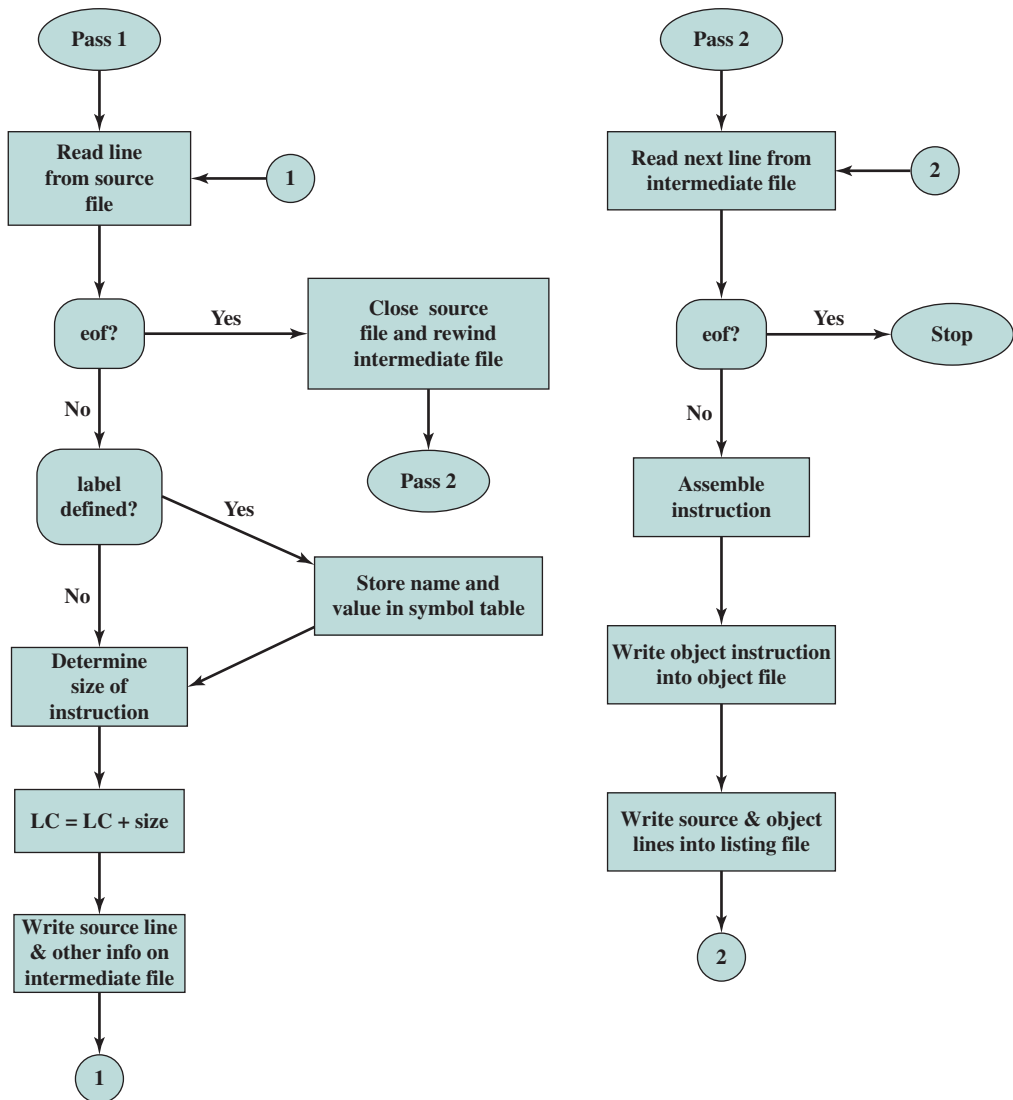
**FIRST PASS** In the first pass, the assembler is only concerned with label definitions. The first pass is used to construct a **symbol table** that contains a list of all labels and their associated **location counter (LC)** values. The first byte of the object code will have the LC value of 0. The first pass examines each assembly statement. Although the assembler is not yet ready to translate instructions, it must examine each instruction sufficiently to determine the length of the corresponding machine instruction and therefore how much to increment the LC. This may require not only examining the opcode but also looking at the operands and the addressing modes.

Directives such as **DQ** and **REST** (see Table B.2) cause the location counter to be adjusted according to how much storage is specified.

When assembler encounters a statement with a label, it places the label into the symbol table, along with the current LC value. The assembler continues until it has read all of the assembly language statements.

**SECOND PASS** The second pass reads the program again from the beginning. Each instruction is translated into the appropriate binary machine code. Translation includes the following operations:

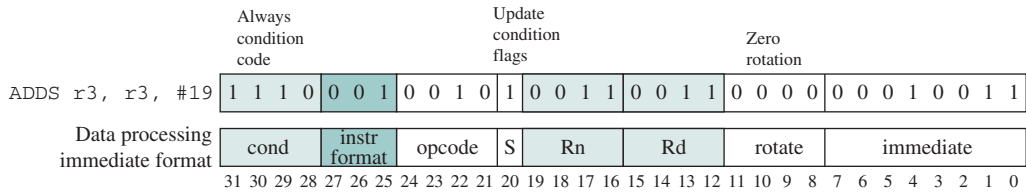
1. Translate the mnemonic into a binary opcode.
2. Use the opcode to determine the format of the instruction and the location and length of the various fields in the instruction.
3. Translate each operand name into the appropriate register or memory code.



**Figure B.4** Flowchart of Two-Pass Assembler

4. Translate each immediate value into a binary string.
5. Translate any references to labels into the appropriate LC value using the symbol table.
6. Set any other bits in the instruction that are needed, including addressing mode indicators, condition code bits, and so on.

A simple example, using the ARM assembly language, is shown in Figure B.5. The ARM assembly language instruction `ADDs r3, r3, #19` is translated into the binary machine instruction `1110 0010 0101 0011 0011 0000 0001 0011`.



**Figure B.5** Translating an ARM Assembly Instruction into a Binary Machine Instruction

**ZEROth PASS** Most assembly language includes the ability to define macros. When macros are present there is an additional pass that the assembler must make before the first pass. Typically, the assembly language requires that all macro definitions must appear at the beginning of the program.

The assembler begins this “zeroth pass” by reading all macro definitions. Once all the macros are recognized, the assembler goes through the source code and expands the macros with their associated parameters whenever a macro call is encountered. The macro processing pass generates a new version of the source code with all of the macro expansions in place and all of the macro definitions removed.

### One-Pass Assembler

It is possible to implement an assembler that makes only a single pass through the source code (not counting the macro processing pass). The main difficulty in trying to assemble a program in one pass involves forward references to labels. Instruction operands may be symbols that have not yet been defined in the source program. Therefore, the assembler does not know what relative address to insert in the translated instruction.

In essence, the process of resolving forward references works as follows. When the assembler encounters an instruction operand that is a symbol that is not yet defined, the assembler does the following:

1. It leaves the instruction operand field empty (all zeros) in the assembled binary instruction.
2. The symbol used as an operand is entered in the symbol table. The table entry is flagged to indicate that the symbol is undefined.
3. The address of the operand field in the instruction that refers to the undefined symbol is added to a list of forward references associated with the symbol table entry.

When the symbol definition is encountered so that a LC value can be associated with it, the assembler inserts the LC value in the appropriate entry in the symbol table. If there is a forward reference list associated with the symbol, then the assembler inserts the proper address into any instruction previously generated that is on the forward reference list.

### Example: Prime Number Program

We now look at an example that includes directives. This example looks at a program that finds prime numbers. Recall that prime numbers are evenly divisible by only 1



```

unsigned guess; /* current guess for prime */
unsigned factor; /* possible factor of guess */
unsigned limit; /* find primes up to this value */

printf ("Find primes up to : ");
scanf ("%u", &limit);
printf ("2\n"); /* treat first two primes as */
printf ("3\n"); /* special case */
guess = 5; /* initial guess */
while (guess <= limit) { /* look for a factor of guess */
 factor = 3;
 while (factor * factor < guess && guess % factor != 0)
 factor += 2;
 if (guess % factor != 0)
 printf ("%d\n", guess);
 guess += 2; /* only look at odd numbers */
}

```

**Figure B.6** C Program for Testing Primality

and themselves. There is no formula for doing this. The basic method this program uses is to find the factors of all odd numbers below a given limit. If no factor can be found for an odd number, it is prime. Figure B.6 shows the basic algorithm written in C. Figure B.7 shows the same algorithm written in NASM assembly language.

## B.3 LOADING AND LINKING

The first step in the creation of an active process is to load a program into main memory and create a process image (Figure B.8). Figure B.9 depicts a scenario typical for most systems. The application consists of a number of compiled or assembled modules in object-code form. These are linked to resolve any references between modules. At the same time, references to library routines are resolved. The library routines themselves may be incorporated into the program or referenced as shared code that must be supplied by the operating system at run time. In this section, we summarize the key features of linkers and loaders. First, we discuss the concept of relocation. Then, for clarity in the presentation, we describe the loading task when a single program module is involved; no linking is required. We can then look at the linking and loading functions as a whole.

### Relocation

In a multiprogramming system, the available main memory is generally shared among a number of processes. Typically, it is not possible for the programmer to know in advance which other programs will be resident in main memory at the time of execution of his or her program. In addition, we would like to be able to swap active processes in and out of main memory to maximize processor utilization by providing a large pool of ready processes to execute. Once a program has been swapped out to disk, it would be quite limiting to declare that when it is next

```

#include "asm_io.inc"
segment .data
Message db "Find primes up to: ", 0

segment .bss
Limit resd 1 ; find primes up to this limit
Guess resd 1 ; the current guess for prime

segment .text
global _asm_main
_asm_main:
 enter 0,0 ; setup routine
 pusha

 mov eax, Message
 call print_string
 call read_int ; scanf("%u", & limit);
 mov [Limit], eax
 mov eax, 2 ; printf("2\n");
 call print_int
 call print_nl
 mov eax, 3 ; printf("3\n");
 call print_int
 call print_nl

 mov dword [Guess], 5 ; Guess = 5;
while_limit: ; while (Guess <= Limit)
 mov eax, [Guess]
 cmp eax, [Limit]
 jnbe end_while_limit ; use jnbe since numbers are unsigned

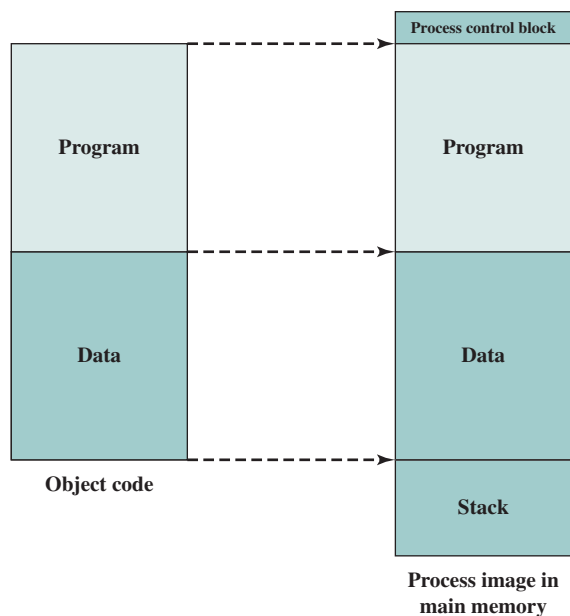
 mov ebx, 3 ; ebx is factor = 3;
while_factor:
 mov eax, ebx
 mul eax ; edx:eax = eax*eax
 jo end_while_factor ; if answer won't fit in eax alone
 cmp eax, [Guess]
 jnb end_while_factor ; if !(factor*factor < guess)
 mov eax, [Guess]
 mov edx, 0
 div ebx ; edx = edx:eax% ebx
 cmp edx, 0
 je end_while_factor ; if !(guess% factor != 0)

 add ebx, 2; factor += 2;
 jmp while_factor
end_while_factor:
 je end_if ; if !(guess% factor != 0)
 mov eax, [Guess]
 call print_int
 call print_nl
end_if:
 add dword [Guess], 2 ; guess += 2
 jmp while_limit
end_while_limit:

 popa
 mov eax, 0 ; return back to C
 leave
 ret

```

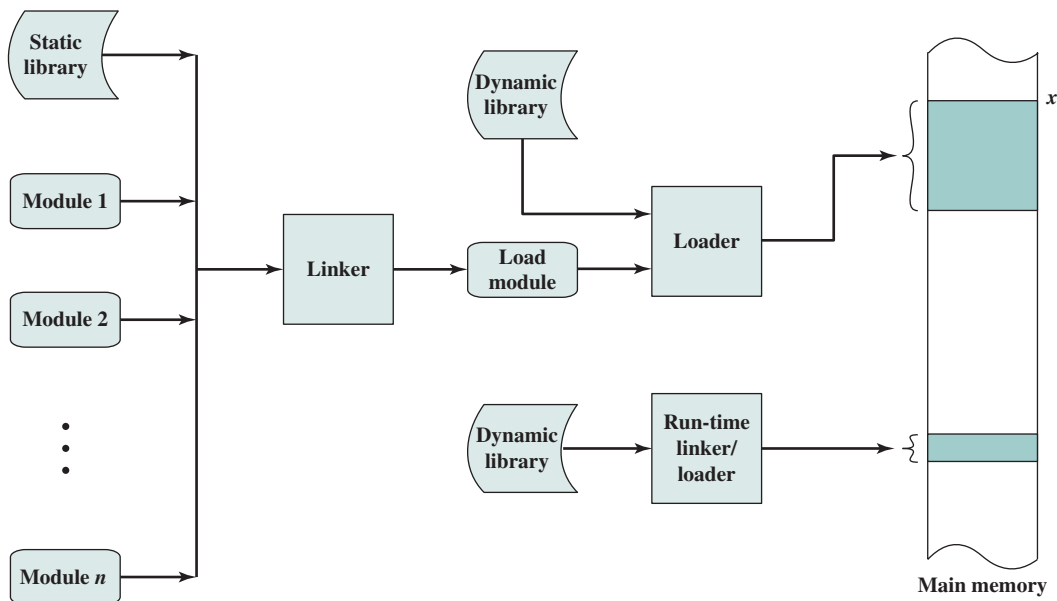
**Figure B.7** Assembly Program for Testing Primality



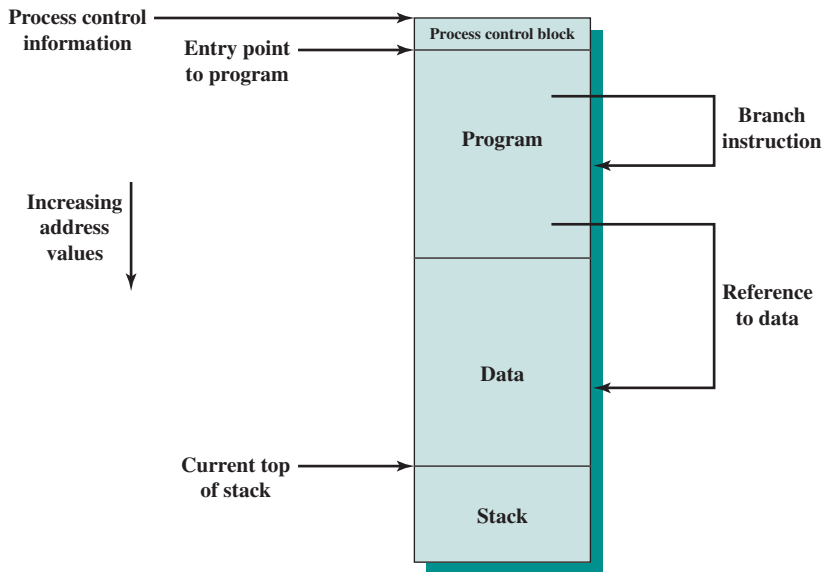
**Figure B.8** The Loading Function

swapped back in, it must be placed in the same main memory region as before. Instead, we may need to **relocate** the process to a different area of memory.

Thus, we cannot know ahead of time where a program will be placed, and we must allow that the program may be moved about in main memory due to swapping.



**Figure B.9** A Linking and Loading Scenario



**Figure B.10** Addressing Requirements for a Process

These facts raise some technical concerns related to addressing, as illustrated in Figure B.10. The figure depicts a process image. For simplicity, let us assume that the process image occupies a contiguous region of main memory. Clearly, the operating system will need to know the location of process control information and of the execution stack, as well as the entry point to begin execution of the program for this process. Because the operating system is managing memory and is responsible for bringing this process into main memory, these addresses are easy to come by. In addition, however, the processor must deal with memory references within the program. Branch instructions contain an address to reference the instruction to be executed next. Data reference instructions contain the address of the byte or word of data referenced. Somehow, the processor hardware and operating system software must be able to translate the memory references found in the code of the program into actual physical memory addresses, reflecting the current location of the program in main memory.

## Loading

In Figure B.9, the loader places the load module in main memory starting at location  $x$ . In loading the program, the addressing requirement illustrated in Figure B.10 must be satisfied. In general, three approaches can be taken:

- Absolute loading
- Relocatable loading
- Dynamic run-time loading

**ABSOLUTE LOADING** An absolute loader requires that a given load module always be loaded into the same location in main memory. Thus, in the load module

presented to the loader, all address references must be to specific, or absolute, main memory addresses. For example, if  $x$  in Figure B.9 is location 1024, then the first word in a load module destined for that region of memory has address 1024.

The assignment of specific address values to memory references within a program can be done either by the programmer or at compile or assembly time (Table B.3a). There are several disadvantages to the former approach. First, every programmer would have to know the intended assignment strategy for placing modules into main memory. Second, if any modifications are made to the program that involve insertions or deletions in the body of the module, then all of the addresses will have to be altered. Accordingly, it is preferable to allow memory references within programs to be expressed symbolically and then resolve those symbolic references at the time of compilation or assembly. This is illustrated in Figure B.11. Every reference to an instruction or item of data is initially represented by a symbol. In preparing the module for input to an absolute loader, the assembler or compiler will convert all of these references to specific addresses (in this example, for a module to be loaded starting at location 1024), as shown in Figure B.11b.

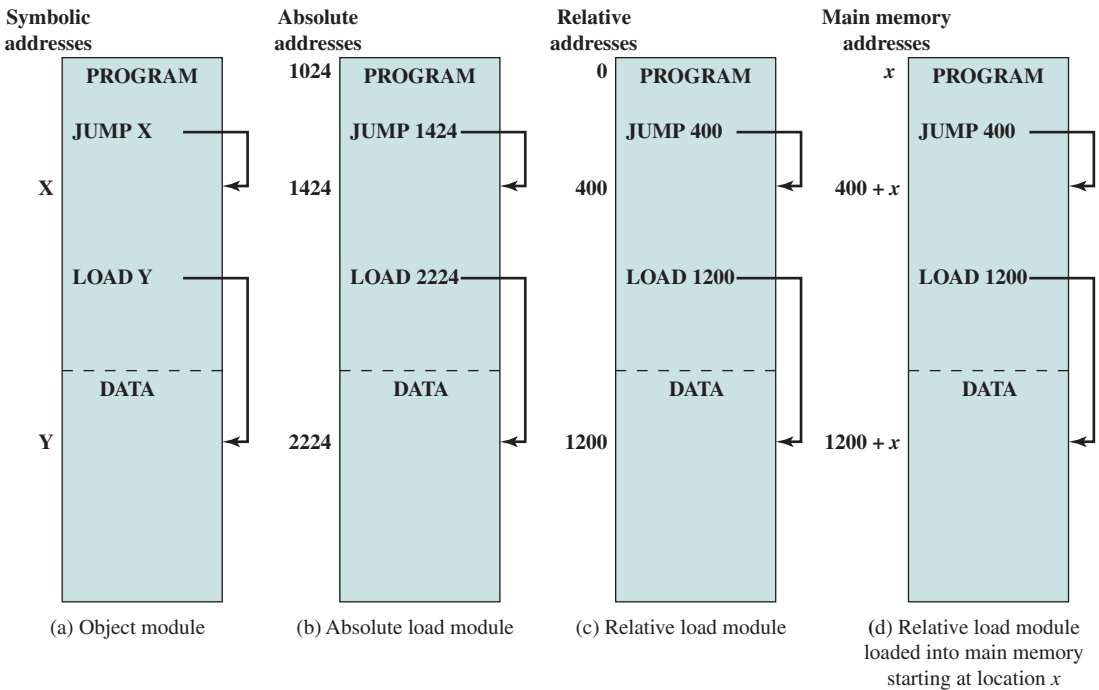
**Table B.3** Address Binding

**(a) Loader**

| Binding Time             | Function                                                                                                                                 |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Programming time         | All actual physical addresses are directly specified by the programmer in the program itself.                                            |
| Compile or assembly time | The program contains symbolic address references, and these are converted to actual physical addresses by the compiler or assembler.     |
| Load time                | The compiler or assembler produces relative addresses. The loader translates these to absolute addresses at the time of program loading. |
| Run time                 | The loaded program retains relative addresses. These are converted dynamically to absolute addresses by processor hardware.              |

**(b) Linker**

| Linkage Time             | Function                                                                                                                                                                                                                                  |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programming time         | No external program or data references are allowed. The programmer must place into the program the source code for all subprograms that are referenced.                                                                                   |
| Compile or assembly time | The assembler must fetch the source code of every subroutine that is referenced and assemble them as a unit.                                                                                                                              |
| Load module creation     | All object modules have been assembled using relative addresses. These modules are linked together and all references are restated relative to the origin of the final load module.                                                       |
| Load time                | External references are not resolved until the load module is to be loaded into main memory. At that time, referenced dynamic link modules are appended to the load module, and the entire package is loaded into main or virtual memory. |
| Run time                 | External references are not resolved until the external call is executed by the processor. At that time, the process is interrupted and the desired module is linked to the calling program.                                              |



**Figure B.11** Absolute and Relocatable Load Modules

**RELOCATABLE LOADING** The disadvantage of binding memory references to specific addresses prior to loading is that the resulting load module can only be placed in one region of main memory. However, when many programs share main memory, it may not be desirable to decide ahead of time into which region of memory a particular module should be loaded. It is better to make that decision at load time. Thus we need a load module that can be located anywhere in main memory.

To satisfy this new requirement, the assembler or compiler produces not actual main memory addresses (absolute addresses) but addresses that are relative to some known point, such as the start of the program. This technique is illustrated in Figure B.11c. The start of the load module is assigned the relative address 0, and all other memory references within the module are expressed relative to the beginning of the module.

With all memory references expressed in relative format, it becomes a simple task for the loader to place the module in the desired location. If the module is to be loaded beginning at location  $x$ , then the loader must simply add  $x$  to each memory reference as it loads the module into memory. To assist in this task, the load module must include information that tells the loader where the address references are and how they are to be interpreted (usually relative to the program origin, but also possibly relative to some other point in the program, such as the current location). This set of information is prepared by the compiler or assembler and is usually referred to as the relocation dictionary.

**DYNAMIC RUN-TIME LOADING** Relocatable loaders are common and provide obvious benefits relative to absolute loaders. However, in a multiprogramming environment, even one that does not depend on virtual memory, the relocatable loading scheme is inadequate. We have referred to the need to swap process images in and out of main memory to maximize the utilization of the processor. To maximize main memory utilization, we would like to be able to swap the process image back into different locations at different times. Thus, a program, once loaded, may be swapped out to disk and then swapped back in at a different location. This would be impossible if memory references had been bound to absolute addresses at the initial load time.

The alternative is to defer the calculation of an absolute address until it is actually needed at run time. For this purpose, the load module is loaded into main memory with all memory references in relative form (Figure B.11c). It is not until an instruction is actually executed that the absolute address is calculated. To assure that this function does not degrade performance, it must be done by special processor hardware rather than software. This hardware is described in Chapter 8.

Dynamic address calculation provides complete flexibility. A program can be loaded into any region of main memory. Subsequently, the execution of the program can be interrupted and the program can be swapped out of main memory, to be later swapped back in at a different location.

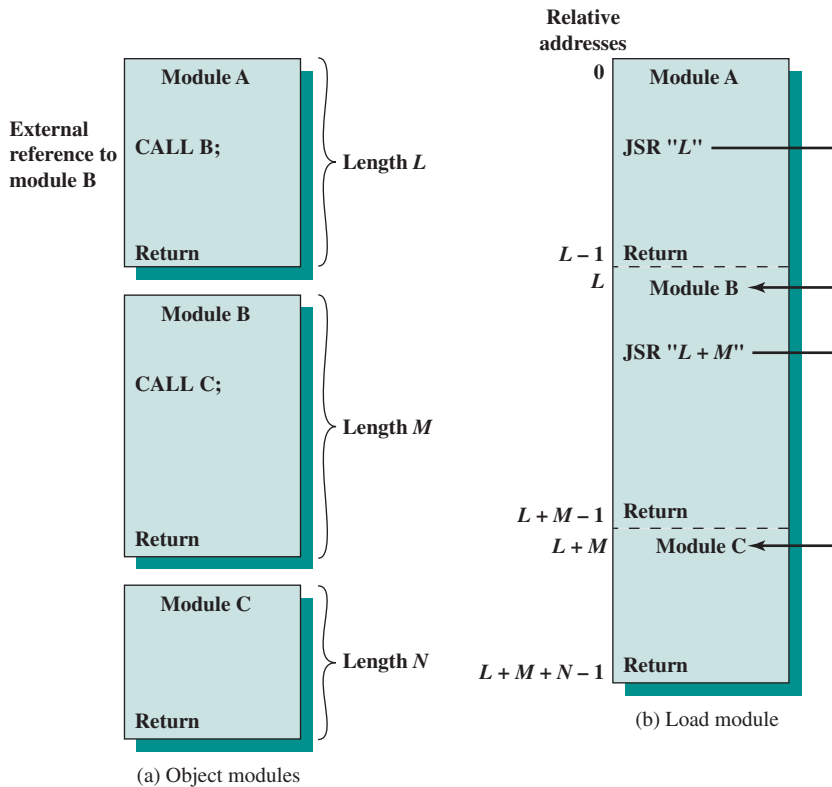
## Linking

The function of a linker is to take as input a collection of object modules and produce a load module, consisting of an integrated set of program and data modules, to be passed to the loader. In each object module, there may be address references to locations in other modules. Each such reference can only be expressed symbolically in an unlinked object module. The linker creates a single load module that is the contiguous joining of all of the object modules. Each intramodule reference must be changed from a symbolic address to a reference to a location within the overall load module. For example, module A in Figure B.12a contains a procedure invocation of module B. When these modules are combined in the load module, this symbolic reference to module B is changed to a specific reference to the location of the entry point of B within the load module.

**LINKAGE EDITOR** The nature of this address linkage will depend on the type of load module to be created and when the linkage occurs (Table B.3b). If, as is usually the case, a relocatable load module is desired, then linkage is usually done in the following fashion. Each compiled or assembled object module is created with references relative to the beginning of the object module. All of these modules are put together into a single relocatable load module with all references relative to the origin of the load module. This module can be used as input for relocatable loading or dynamic run-time loading.

A linker that produces a relocatable load module is often referred to as a linkage editor. Figure B.12 illustrates the linkage editor function.

**DYNAMIC LINKER** As with loading, it is possible to defer some linkage functions. The term *dynamic linking* is used to refer to the practice of deferring the linkage of some external modules until after the load module has been created. Thus, the load



**Figure B.12** The Linking Function

module contains unresolved references to other programs. These references can be resolved either at load time or run time.

For **load-time dynamic linking** (involving upper dynamic library in Figure B.9), the following steps occur. The load module (application module) to be loaded is read into memory. Any reference to an external module (target module) causes the loader to find the target module, load it, and alter the reference to a relative address in memory from the beginning of the application module. There are several advantages to this approach over what might be called static linking:

- It becomes easier to incorporate changed or upgraded versions of the target module, which may be an operating system utility or some other general-purpose routine. With static linking, a change to such a supporting module would require the relinking of the entire application module. Not only is this inefficient, but it may be impossible in some circumstances. For example, in the personal computer field, most commercial software is released in load module form; source and object versions are not released.
- Having target code in a dynamic link file paves the way for automatic code sharing. The operating system can recognize that more than one application is using the same target code because it loaded and linked that code. It can use that information to load a single copy of the target code and link it to both applications, rather than having to load one copy for each application.



- It becomes easier for independent software developers to extend the functionality of a widely used operating system such as Linux. A developer can come up with a new function that may be useful to a variety of applications and package it as a dynamic link module.

With **run-time dynamic linking** (involving lower dynamic library in Figure B.9), some of the linking is postponed until execution time. External references to target modules remain in the loaded program. When a call is made to the absent module, the operating system locates the module, loads it, and links it to the calling module. Such modules are typically shareable. In the Windows environment, these are called dynamic-link libraries (DLLs). Thus, if one process is already making use of a dynamically linked shared module, then that module is in main memory and a new process can simply link to the already-loaded module.

The use of DLLs can lead to a problem commonly referred to as **DLL hell**. DLL occurs if two or more processes are sharing a DLL module but expect different versions of the module. For example, an application or system function might be re-installed and bring in with it an older version of a DLL file.

We have seen that dynamic loading allows an entire load module to be moved around; however, the structure of the module is static, being unchanged throughout the execution of the process and from one execution to the next. However, in some cases, it is not possible to determine prior to execution which object modules will be required. This situation is typified by transaction-processing applications, such as an airline reservation system or a banking application. The nature of the transaction dictates which program modules are required, and they are loaded as appropriate and linked with the main program. The advantage of the use of such a dynamic linker is that it is not necessary to allocate memory for program units unless those units are referenced. This capability is used in support of segmentation systems.

One additional refinement is possible: An application need not know the names of all the modules or entry points that may be called. For example, a charting program may be written to work with a variety of plotters, each of which is driven by a different driver package. The application can learn the name of the plotter that is currently installed on the system from another process or by looking it up in a configuration file. This allows the user of the application to install a new plotter that did not exist at the time the application was written.

## B.4 RECOMMENDED READING

[SALO93] covers the design and implementation of assemblers and loaders.

The topics of linking and loading are covered in many books on program development, computer architecture, and operating systems. A particularly detailed treatment is [BECK97]. [CLAR98] also contains a good discussion. A thorough practical discussion of this topic, with numerous OS examples, is [LEVI00].

[BART03] is an excellent treatment for learning assembly language for x86 processors; suitable for self-study. [CART06] covers assembly language for x86 machines. For the serious x86 programmer, [FOG08a] is highly useful. [KNAG04] is a thorough treatment of ARM assembly language.

- BART03** Bartlett, J. *Programming from the Ground Up*. 2003. Available at this book's Premium Content site.
- BECK97** Beck, L. *System Software*. Reading, MA: Addison-Wesley, 1997.
- CART06** Carter, P. *PC Assembly Language*. July 23, 2006. Available at this book's Web site.
- CLAR98** Clarke, D., and Merusi, D. *System Software Programming: The Way Things Work*. Upper Saddle River, NJ: Prentice Hall, 1998.
- FOG08a** Fog, A. *Optimizing Subroutines in Assembly Language: An Optimization Guide for x86 Platforms*. Copenhagen University College of Engineering, 2008. <http://www.agner.org/optimize/>
- KNAG04** Knaggs, P., and Welsh, S. *ARM: Assembly Language Programming*. Bournemouth University School of Design, Engineering & Computing. August 31, 2004. Available at this book's Premium Content site.
- LEVI00** Levine, J. *Linkers and Loaders*. San Francisco: Morgan Kaufmann, 2000.
- SALO93** Salomon, D. *Assemblers and Loaders*. Ellis Horwood Ltd, 1993. Available at this book's Premium Content site.

## B.5 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

|                                                                                         |                                                                                     |                                                                                                           |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| assembler<br>assembly language<br>comment<br>directive<br>dynamic linker<br>instruction | label<br>linkage editor<br>linking<br>load-time dynamic linking<br>loading<br>macro | mnemonic<br>one-pass assembler<br>operand<br>relocation<br>run-time dynamic linking<br>two-pass assembler |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|

### Review Questions

- B.1** List some reasons why it is worthwhile to study assembly language programming.
- B.2** What is an assembly language?
- B.3** List some disadvantages of assembly language compared to high-level languages.
- B.4** List some advantages of assembly language compared to high-level languages.
- B.5** What are the typical elements of an assembly language statement?
- B.6** List and briefly define four different kinds of assembly language statements.
- B.7** What is the difference between a one-pass assembler and a two-pass assembler?

### Problems

- B.1** Core War is a programming game introduced to the public in the early 1980s [DEWD84], which was popular for a period of 15 years or so. Core War has four main components: a memory array of 8000 addresses, a simplified assembly language Redcode, an executive program called MARS (an acronym for Memory Array Redcode

Simulator) and the set of contending battle programs. Two battle programs are entered into the memory array at randomly chosen positions; neither program knows where the other one is. MARS executes the programs in a simple version of time-sharing. The two programs take turns: a single instruction of the first program is executed, then a single instruction of the second, and so on. What a battle program does during the execution cycles allotted to it is entirely up to the programmer. The aim is to destroy the other program by ruining its instructions. In this problem and the next several, we use an even simpler language, called CodeBlue, to explore some Core War concepts.

CodeBlue contains only five assembly language statements and uses three addressing modes (Table B.4). Addresses wrap around, so that for the last location in memory, the relative address of +1 refers to the first location in memory. For example, `ADD #4, 6` adds 4 to the contents of relative location 6 and stores the results in location 6; `JUMP @5` transfers execution to the memory address contained in the location five slots past the location of the current `JUMP` instruction.

- a. The program Imp is the single instruction `COPY 0, 1`. What does it do?
- b. The program Dwarf is the following sequence of instructions:

```
ADD #4, 3
COPY 2, @2
JUMP -2
DATA 0
```

What does it do?

- c. Rewrite Dwarf using symbols, so that it looks more like a typical assembly language program.
- B.2** What happens if we pit Imp against Dwarf?
- B.3** Write a “carpet bombing” program in CodeBlue that zeros out all of memory (with the possible exception of the program locations).
- B.4** How would the following program fare against Imp?

**Table B.4** CodeBlue Assembly Language

**(a) Instruction Set**

| Format       | Meaning                          |
|--------------|----------------------------------|
| DATA <value> | <value> set at current location  |
| COPY A, B    | copies source A to destination B |
| ADD A, B     | adds A to B, putting result in B |
| JUMP A       | transfer execution to A          |
| JUMPZ A, B   | if B = 0, transfer to A          |

**(b) Addressing Modes**

| Mode     | Format              | Meaning                                                                                                                                                |
|----------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Literal  | # followed by value | This is an immediate mode, the operand value is in the instruction.                                                                                    |
| Relative | Value               | The value represents an offset from the current location, which contains the operand.                                                                  |
| Indirect | @ followed by value | The value represents an offset from the current location; the offset location contains the relative address of the location that contains the operand. |

```

Loop COPY #0, -1
 JUMP -1

```

*Hint:* Remember that instruction execution alternates between the two opposing programs.

- B.5 a.** What is the value of the C status flag after the following sequence:

```

mov al, 3
add al, 4

```

- b.** What is the value of the C status flag after the following sequence:

```

mov al, 3
sub al, 4

```

- B.6** Consider the following NAMS instruction:

```

cmp vleft, vright

```

For signed integers, there are three status flags that are relevant. If  $vleft = vright$ , then ZF is set. If  $vleft > vright$ , ZF is unset (set to 0) and SF = OF. If  $vleft < vright$ , ZF is unset and SF  $\neq$  OF. Why does SF = OF if  $vleft > vright$ ?

- B.7** Consider the following NASM code fragment:

```

mov al, 0
cmp al, al
je next

```

Write an equivalent program consisting of a single instruction.

- B.8** Consider the following C program:

```

/* a simple C program to average 3 integers */
main ()
{
 int avg;
 int i1 = 20;
 int i2 = 13;
 int i3 = 82;
 avg = (i1 + i2 + i3)/3;
}

```

Write an NASM version of this program.

- B.9** Consider the following C code fragment:

```

if (EAX == 0) EBX = 1;
else EBX = 2;

```

Write an equivalent NASM code fragment.

- B.10** The initialize data directives can be used to initialize multiple locations. For example,

```

db 0x55, 0x56, 0x57

```

reserves three bytes and initializes their values.

NASM supports the special token \$ to allow calculations to involve the current assembly position. That is, \$ evaluates to the assembly position at the beginning of the line containing the expression. With the preceding two facts in mind, consider the following sequence of directives:

```
message db 'hello, world'
msglen equ $-message
```

What value is assigned to the symbol msglen?

- B.11** Assume the three symbolic variables V1, V2, V3 contain integer values. Write an NASM code fragment that moves the smallest value into integer ax. Use only the instructions `mov`, `cmp`, and `jbe`.
- B.12** Describe the effect of this instruction: `cmp eax, 1`  
Assume that the immediately preceding instruction updated the contents of `eax`.
- B.13** The `xchg` instruction can be used to exchange the contents of two registers. Suppose that the x86 instruction set did not support this instruction.
- Implement `xchg ax, bx` using only push and pop instructions.
  - Implement `xchg ax, bx` using only the `xor` instruction (do not involve other registers).
- B.14** In the following program, assume that `a`, `b`, `x`, `y` are symbols for main memory locations. What does the program do? You can answer the question by writing the equivalent logic in C.

```
 mov eax,a
 mov ebx,b
 xor eax,x
 xor ebx,y
 or eax,ebx
 jnz L2
L1:
 jmp L3
L2:
 ;another sequence of instructions...
L3:
```

- B.15** Section B.1 includes a C program that calculates the greatest common divisor of two integers.
- Describe the algorithm in words and show how the program does implement the Euclid algorithm approach to calculating the greatest common divisor.
  - Add comments to the assembly program of Figure B.3a to clarify that it implements the same logic as the C program.
  - Repeat part (b) for the program of Figure B.3b.
- B.16**
- A 2-pass assembler can handle future symbols and an instruction can therefore use a future symbol as an operand. This is not always true for directives. The `EQU` directive, for example, cannot use a future symbol. The directive “`A EQU B+1`” is easy to execute if `B` is previously defined, but impossible if `B` is a future symbol. What’s the reason for this?
  - Suggest a way for the assembler to eliminate this limitation such that any source line could use future symbols.
- B.17** Consider a symbol directive `MAX` of the following form:
- ```
symbol MAX list of expressions
```
- The label is mandatory and is assigned the value of the largest expression in the operand field. Example:

```
MSGLEN MAX A, B, C ;where A, B, C are defined symbols
```

How is `MAX` executed by the Assembler and in what pass?

GLOSSARY

absolute address An address in a computer language that identifies a storage location or a device without the use of any intermediate reference.

accumulator The name of the CPU register in a single-address instruction format. The accumulator, or AC, is implicitly one of the two operands for the instruction.

address bus That portion of a system bus used for the transfer of an address. Typically, the address identifies a main memory location or an I/O device.

address space The range of addresses (memory, I/O) that can be referenced.

arithmetic and logic unit (ALU) A part of a computer that performs arithmetic operations, logic operations, and related operations.

ASCII American Standard Code for Information Interchange. ASCII is a 7-bit code used to represent numeric, alphabetic, and special printable characters. It also includes codes for *control characters*, which are not printed or displayed but specify some control function.

assembly language A computer-oriented language whose instructions are usually in one-to-one correspondence with computer instructions and that may provide facilities such as the use of macro-instructions. Synonymous with *computer-dependent language*.

associative memory A memory whose storage locations are identified by their contents, or by a part of their contents, rather than by their names or positions.

asynchronous timing A technique in which the occurrence of one event on a bus follows and depends on the occurrence of a previous event.

autoindexing A form of indexed addressing in which the index register is automatically incremented or decremented with each memory reference.

base In the numeration system commonly used in scientific papers, the number that is raised to the power denoted by the exponent and then multiplied by the mantissa to determine the real number represented (e.g., the number 10 in the expression $2.7 \times 10^2 = 270$).

base address A numeric value that is used as a reference in the calculation of addresses in the execution of a computer program.

binary operator An operator that represents an operation on two and only two operands.

bit In the pure binary numeration system, either of the digits 0 and 1.

block multiplexor channel A multiplexer channel that interleaves blocks of data. See also *byte multiplexor channel*. Contrast with *selector channel*.

branch prediction A mechanism used by the processor to predict the outcome of a program branch prior to its execution.

buffer Storage used to compensate for a difference in rate of flow of data, or time of occurrence of events, when transferring data from one device to another.

bus A shared communications path consisting of one or a collection of lines. In some computer systems, CPU, memory, and I/O components are connected by a common bus. Since the lines are shared by all components, only one component at a time can successfully transmit.

bus arbitration The process of determining which competing bus master will be permitted access to the bus.

bus master A device attached to a bus that is capable of initiating and controlling communication on the bus.

byte A sequence of eight bits. Also referred to as an *octet*.

byte multiplexor channel A multiplexer channel that interleaves bytes of data. See also *block multiplexor channel*. Contrast with *selector channel*.

cache A relatively small fast memory interposed between a larger, slower memory and the logic that accesses the larger memory. The cache holds recently accessed data, and is designed to speed up subsequent access to the same data.

cache coherence protocol A mechanism to maintain data validity among multiple caches so that every data access will always acquire the most recent version of the contents of a main memory word.

cache line A block of data associated with a cache tag and the unit of transfer between cache and memory.

cache memory A special buffer storage, smaller and faster than main storage, that is used to hold a copy of instructions and data in main storage that are likely to be needed next by the processor and that have been obtained automatically from main storage.

CD-ROM Compact Disk Read-Only Memory. A nonerasable disk used for storing computer data. The standard system uses 12-cm disks and can hold more than 550 Mbytes.

central processing unit (CPU) That portion of a computer that fetches and executes instructions. It consists of an Arithmetic and Logic Unit (ALU), a control unit, and registers. Often simply referred to as a *processor*.

cluster A group of interconnected, whole computers working together as a unified computing resource that can create the illusion of being one machine. The term *whole computer* means a system that can run on its own, apart from the cluster.

combinational circuit A logic device whose output values, at any given instant, depend only upon the input values at that time. A combinational circuit is a special case of a sequential circuit that does not have a storage capability. Synonymous with *combinatorial circuit*.

compact disk (CD) A nonerasable disk that stores digitized audio information.

computer architecture Those attributes of a system visible to a programmer or, put another way, those attributes that have a direct impact on the logical execution of a program. Examples of architectural attributes include the instruction set, the number of bits used to represent various data types (e.g., numbers, characters), I/O mechanisms, and techniques for addressing memory.

computer instruction An instruction that can be recognized by the processing unit of the computer for which it is designed. Synonymous with *machine instruction*.

computer instruction set A complete set of the operators of the instructions of a computer together with a description of the types of meanings that can be attributed to their operands. Synonymous with *machine instruction set*.

computer organization Refers to the operational units and their interconnections that realize the architectural specifications. Organizational attributes include those hardware details transparent to the programmer, such as control signals; interfaces between the computer and peripherals; and the memory technology used.

conditional jump A jump that takes place only when the instruction that specifies it is executed and specified conditions are satisfied. Contrast with *unconditional jump*.

condition code A code that reflects the result of a previous operation (e.g., arithmetic). A CPU may include one or more condition codes, which may be stored separately within the CPU or as part of a larger control register. Also known as a *flag*.

control bus That portion of a system bus used for the transfer of control signals.

control registers CPU registers employed to control CPU operation. Most of these registers are not user visible.

control storage A portion of storage that contains microcode.

control unit That part of the CPU that controls CPU operations, including ALU operations, the movement of data within the CPU, and the exchange of data and control signals across external interfaces (e.g., the system bus).

daisy chain A method of device interconnection for determining interrupt priority by connecting the interrupt sources serially.

data bus That portion of a system bus used for the transfer of data.

data communication Data transfer between devices. The term generally excludes I/O.

decoder A device that has a number of input lines of which any number may carry signals and a number of output lines of which not more than one may carry a signal, there being a one-to-one correspondence between the outputs and the combinations of input signals.

demand paging The transfer of a page from auxiliary storage to real storage at the moment of need.

direct access The capability to obtain data from a storage device or to enter data into a storage device in a sequence independent of their relative position, by means of addresses that indicate the physical location of the data.

direct address An address that designates the storage location of an item of data to be treated as operand. Synonymous with *one-level address*.

direct memory access (DMA) A form of I/O in which a special module, called a *DMA module*, controls the exchange of data between main memory and an I/O module. The CPU sends a request for the transfer of a block of data to the DMA module and is interrupted only after the entire block has been transferred.

disabled interrupt A condition, usually created by the CPU, during which the CPU will ignore interrupt request signals of a specified class.

diskette A flexible magnetic disk enclosed in a protective container. Synonymous with *flexible disk*.

disk pack An assembly of magnetic disks that can be removed as a whole from a disk drive, together with a container from which the assembly must be separated when operating.

disk stripping A type of disk array mapping in which logically contiguous blocks of data, or strips, are mapped round-robin to consecutive array members. A set of logically consecutive strips that maps exactly one strip to each array member is referred to as a stripe.

dynamic RAM A RAM whose cells are implemented using capacitors. A dynamic RAM will gradually lose its data unless it is periodically refreshed.

emulation The imitation of all or part of one system by another, primarily by hardware, so that the imitating system accepts the same data, executes the same programs, and achieves the same results as the imitated system.

enabled interrupt A condition, usually created by the CPU, during which the CPU will respond to interrupt request signals of a specified class.

erasable optical disk A disk that uses optical technology but that can be easily erased and rewritten. Both 3.25-inch and 5.25-inch disks are in use. A typical capacity is 650 Mbytes.

error-correcting code A code in which each character or signal conforms to specific rules of construction so that deviations from these rules indicate the presence of an error and in which some or all of the detected errors can be corrected automatically.

error-detecting code A code in which each character or signal conforms to specific rules of construction so that deviations from these rules indicate the presence of an error.

execute cycle That portion of the instruction cycle during which the CPU performs the operation specified by the instruction opcode.

fetch cycle That portion of the instruction cycle during which the CPU fetches from memory the instruction to be executed.

firmware Microcode stored in read-only memory.

fixed-point representation system A radix numeration system in which the radix point is implicitly fixed in the series of digit places by some convention upon which agreement has been reached.

flip-flop A circuit or device containing active elements, capable of assuming either one of two stable states at a given time. Synonymous with *bistable circuit*, *toggle*.

floating-point representation system A numeration system in which a real number is represented by a pair of distinct numerals, the real number being the product of the fixed-point part, one of the numerals, and a value obtained by raising the implicit floating-point base to a power denoted by the exponent in the floating-point representation, indicated by the second numeral.

G Prefix meaning 2^{30} .

gate An electronic circuit that produces an output signal that is a simple Boolean operation on its input signals.

general-purpose register A register, usually explicitly addressable, within a set of registers, that can be used for different purposes, for example, as an accumulator, as an index register, or as a special handler of data.

global variable A variable defined in one portion of a computer program and used in at least one other portion of that computer program.

high-performance computing (HPC) A research area dealing with supercomputers and the software that runs on supercomputers. The emphasis is on scientific applications, which may involve heavy use of vector and matrix computation, and parallel algorithms.

immediate address The contents of an address part that contains the value of an operand rather than an address. Synonymous with *zero-level address*.

indexed address An address that is modified by the content of an index register prior to or during the execution of a computer instruction.

indexing A technique of address modification by means of index registers.

index register A register whose contents can be used to modify an operand address during the execution of computer instructions; it can also be used as a counter. An index register may be used to control the execution of a loop, to control the use of an array, as a switch, for table lookup, or as a pointer.

indirect address An address of a storage location that contains an address.

indirect cycle That portion of the instruction cycle during which the CPU performs a memory access to convert an indirect address into a direct address.

input-output (I/O) Pertaining to either input or output, or both. Refers to the movement of data between a computer and a directly attached peripheral.

instruction address register A special-purpose register used to hold the address of the next instruction to be executed.

instruction cycle The processing performed by a CPU to execute a single instruction.

instruction format The layout of a computer instruction as a sequence of bits. The format divides the instruction into fields, corresponding to the constituent elements of the instruction (e.g., opcode, operands).

instruction issue The process of initiating instruction execution in the processor's functional units. This occurs when an instruction moves from the decode stage of the pipeline to the first execute stage of the pipeline.

instruction register A register that is used to hold an instruction for interpretation.

integrated circuit (IC) A tiny piece of solid material, such as silicon, upon which is etched or imprinted a collection of electronic components and their interconnections.

interrupt A suspension of a process, such as the execution of a computer program, caused by an event external to that process, and performed in such a way that the process can be resumed. Synonymous with *interruption*.

interrupt cycle That portion of the instruction cycle during which the CPU checks for interrupts. If an enabled interrupt is pending, the CPU saves the current program state and resumes processing at an interrupt-handler routine.

interrupt-driven I/O A form of I/O. The CPU issues an I/O command, continues to execute subsequent instructions, and is interrupted by the I/O module when the latter has completed its work.

I/O channel A relatively complex I/O module that relieves the CPU of the details of I/O operations. An I/O channel will execute a sequence of I/O commands from main memory without the need for CPU involvement.

I/O controller A relatively simple I/O module that requires detailed control from the CPU or an I/O channel. Synonymous with *device controller*.

I/O module One of the major component types of a computer. It is responsible for the control of one or more external devices (peripherals) and for the exchange of data between those devices and main memory and/or CPU registers.

I/O processor An I/O module with its own processor, capable of executing its own specialized I/O instructions or, in some cases, general-purpose machine instructions.

isolated I/O A method of addressing I/O modules and external devices. The I/O address space is treated separately from main memory address space. Specific I/O machine instructions must be used. Compare *memory-mapped I/O*.

k Prefix meaning $2^{10} = 1024$. Thus, 2 kb = 2048 bits.

local variable A variable that is defined and used only in one specified portion of a computer program.

locality of reference The tendency of a processor to access the same set of memory locations repetitively over a short period of time.

M Prefix meaning $2^{20} = 1,048,576$. Thus, 2 Mb = 2,097,152 bits.

magnetic disk A flat circular plate with a magnetizable surface layer, on one or both sides of which data can be stored.

magnetic tape A tape with a magnetizable surface layer on which data can be stored by magnetic recording.

mainframe A term originally referring to the cabinet containing the central processor unit or “main frame” of a large batch machine. After the emergence of smaller minicomputer designs in the early 1970s, the traditional larger machines were described as mainframe computers, mainframes. Typical characteristics of a mainframe are that it supports a large database, has elaborate I/O hardware, and is used in a central data processing facility.

main memory Program-addressable storage from which instructions and other data can be loaded directly into registers for subsequent execution or processing.

memory address register (MAR) A register, in a processing unit, that contains the address of the storage location being accessed.

memory buffer register (MBR) A register that contains data read from memory or data to be written to memory.

memory cycle time The inverse of the rate at which memory can be accessed. It is the minimum time between the response to one access request (read or write) and the response to the next access request.

memory-mapped I/O A method of addressing I/O modules and external devices. A single address space is used for both main memory and I/O addresses, and the same machine instructions are used both for memory read/write and for I/O.

microcomputer A computer system whose processing unit is a microprocessor. A basic microcomputer includes a microprocessor, storage, and an input/output facility, which may or may not be on one chip.

microinstruction An instruction that controls data flow and sequencing in a processor at a more fundamental level than machine instructions. Individual machine instructions and perhaps other functions may be implemented by microprograms.

micro-operation An elementary CPU operation, performed during one clock pulse.

microprocessor A processor whose elements have been miniaturized into one or a few integrated circuits.

microprogram A sequence of microinstructions that are in special storage where they can be dynamically accessed to perform various functions.

microprogrammed CPU A CPU whose control unit is implemented using microprogramming.

microprogramming language An instruction set used to specify microprograms.

multiplexer A combinational circuit that connects multiple inputs to a single output. At any time, only one of the inputs is selected to be passed to the output.

multiplexor channel A channel designed to operate with a number of I/O devices simultaneously. Several I/O devices can transfer records at the same time by interleaving items of data. See also *byte multiplexor channel*, *block multiplexor channel*.

multiprocessor A computer that has two or more processors that have common access to a main storage.

multiprogramming A mode of operation that provides for the interleaved execution of two or more computer programs by a single processor.

multitasking A mode of operation that provides for the concurrent performance or interleaved execution of two or more computer tasks. The same as multiprogramming, using different terminology.

nonuniform memory access (NUMA) multiprocessor A shared-memory multiprocessor in which the access time from a given processor to a word in memory varies with the location of the memory word.

nonvolatile memory Memory whose contents are stable and do not require a constant power source.

nucleus That portion of an operating system that contains its basic and most frequently used functions. Often, the nucleus remains resident in main memory.

ones complement representation Used to represent binary integers. A positive integer is represented as in sign magnitude. A negative integer is represented by reversing each bit in the representation of a positive integer of the same magnitude.

opcode Abbreviated form for *operation code*.

operand An entity on which an operation is performed.

operating system Software that controls the execution of programs and that provides services such as resource allocation, scheduling, input/output control, and data management.

operation code A code used to represent the operations of a computer. Usually abbreviated to opcode.

orthogonality A principle by which two variables or dimensions are independent of one another. In the context of an instruction set, the term is generally used to indicate that other elements of an instruction (address mode, number of operands, length of operand) are independent of (not determined by) opcode.

page In a virtual storage system, a fixed-length block that has a virtual address and that is transferred as a unit between real storage and auxiliary storage.

page fault Occurs when the page containing a referenced word is not in main memory. This causes an interrupt and requires the operating system to bring in the needed page.

page frame An area of main storage used to hold a page.

parity Bit A binary digit appended to a group of binary digits to make the sum of all the digits either always odd (odd parity) or always even (even parity).

peripheral equipment In a computer system, with respect to a particular processing unit, any equipment that provides the processing unit with outside communication. Synonymous with *peripheral device*.

pipeline A processor organization in which the processor consists of a number of stages, allowing multiple instructions to be executed concurrently.

predicated execution A mechanism that supports the conditional execution of individual instructions. This makes it possible to execute speculatively both branches of a branch instruction and retain the results of the branch that is ultimately taken.

process A program in execution. A process is controlled and scheduled by the operating system.

process control block The manifestation of a process in an operating system. It is a data structure containing information about the characteristics and state of the process.

processor In a computer, a functional unit that interprets and executes instructions. A processor consists of at least an instruction control unit and an arithmetic unit.

processor cycle time The time required for the shortest well-defined CPU micro-operation. It is the basic unit of time for measuring all CPU actions. Synonymous with *machine cycle time*.

program counter Instruction address register.

programmable logic array (PLA) An array of gates whose interconnections can be programmed to perform a specific logical function.

programmable read-only memory (PROM) Semiconductor memory whose contents may be set only once. The writing process is performed electrically and may be performed by the user at a time later than original chip fabrication.

programmed I/O A form of I/O in which the CPU issues an I/O command to an I/O module and must then wait for the operation to be complete before proceeding.

program status word (PSW) An area in storage used to indicate the order in which instructions are executed, and to hold and indicate the status of the computer system. Synonymous with *processor status word*.

random-access memory (RAM) Memory in which each addressable location has a unique addressing mechanism. The time to access a given location is independent of the sequence of prior access.

read-only memory (ROM) Semiconductor memory whose contents cannot be altered, except by destroying the storage unit. Nonerasable memory.

redundant array of independent disks (RAID) A disk array in which part of the physical storage capacity is used to store redundant information about user data stored on the remainder of the storage capacity. The redundant information enables regeneration of user data in the event that one of the array's member disks or the access path to it fails.

registers High-speed memory internal to the CPU. Some registers are user visible; that is, available to the programmer via the machine instruction set. Other registers are used only by the CPU, for control purposes.

scalar A quantity characterized by a single value.

secondary memory Memory located outside the computer system itself; that is, it cannot be processed directly by the processor. It must first be copied into main memory. Examples include disk and tape.

selector channel An I/O channel designed to operate with only one I/O device at a time. Once the I/O device is selected, a complete record is transferred one byte at a time. Contrast with *block multiplexor channel*, *multiplexor channel*.

semiconductor A solid crystalline substance, such as silicon or germanium, whose electrical conductivity is intermediate between insulators and good conductors. Used to fabricate transistors and solid-state components.

sequential circuit A digital logic circuit whose output depends on the current input plus the state of the circuit. Sequential circuits thus possess the attribute of memory.

sign-magnitude representation Used to represent binary integers. In an N -bit word, the leftmost bit is the sign (0 = positive, 1 = negative) and the remaining $N - 1$ bits comprise the magnitude of the number.

solid-state component A component whose operation depends on the control of electric or magnetic phenomena in solids (e.g., transistor crystal diode, ferrite core).

speculative execution The execution of instructions along one path of a branch. If it later turns out that this branch was not taken, then the results of the speculative execution are discarded.

stack An ordered list in which items are appended to and deleted from the same end of the list, known as the top. That is, the next item appended to the list is put on the top, and the next item to be removed from the list is the item that has been in the list the shortest time. This method is characterized as last-in-first-out.

static RAM A RAM whose cells are implemented using flip-flops. A static RAM will hold its data as long as power is supplied to it; no periodic refresh is required.

superpipelined processor A processor design in which the instruction pipeline consists of many very small stages, so that more than one pipeline stage can be executed during one clock cycle and so that a large number of instructions may be in the pipeline at the same time.

superscalar processor A processor design that includes multiple-instruction pipelines, so that more than one instruction can be executing in the same pipeline stage simultaneously.

symmetric multiprocessing (SMP) A form of multiprocessing that allows the operating system to execute on any available processor or on several available processors simultaneously.

synchronous timing A technique in which the occurrence of events on a bus is determined by a clock. The clock defines equal-width time slots, and events begin only at the beginning of a time slot.

system bus A bus used to interconnect major computer components (CPU, memory, I/O).

truth table A table that describes a logic function by listing all possible combinations of input values and indicating, for each combination, the output value.

twos complement representation Used to represent binary integers. A positive integer is represented as in sign magnitude. A negative number is represented by taking the Boolean complement of each bit of the corresponding positive number, then adding 1 to the resulting bit pattern viewed as an unsigned integer.

unary operator An operator that represents an operation on one and only one operand.

unconditional jump A jump that takes place whenever the instruction that specified it is executed.

uniprocessing Sequential execution of instructions by a processing unit, or independent use of a processing unit in a multiprocessing system.

user-visible registers CPU registers that may be referenced by the programmer. The instruction-set format allows one or more registers to be specified as operands or addresses of operands.

vector A quantity usually characterized by an ordered set of scalars.

very long instruction word (VLIW) Refers to the use of instructions that contain multiple operations. In effect, multiple instructions are contained in a single word. Typically, a VLIW is constructed by the compiler, which places operations that may be executed in parallel in the same word.

virtual storage The storage space that may be regarded as addressable main storage by the user of a computer system in which virtual addresses are mapped into real addresses. The size of virtual storage is limited by the addressing scheme of the computer system and by the amount of auxiliary storage available, and not by the actual number of main storage locations.

volatile memory A memory in which a constant electrical power source is required to maintain the contents of memory. If the power is switched off, the stored information is lost.

word An ordered set of bytes or bits that is the normal unit in which information may be stored, transmitted, or operated on within a given computer. Typically, if a processor has a fixed-length instruction set, then the instruction length equals the word length.

REFERENCES

ABBREVIATIONS

ACM Association for Computing Machinery
IBM International Business Machines Corporation
IEEE Institute of Electrical and Electronics Engineers

- ACOS86** Acosta, R.; Kjelstrup, J.; and Torng, H. "An Instruction Issuing Approach to Enhancing Performance in Multiple Functional Unit Processors." *IEEE Transactions on Computers*, September 1986.
- ADAM91** Adamek, J. *Foundations of Coding*. New York: Wiley, 1991.
- AGAR89** Agarwal, A. *Analysis of Cache Performance for Operating Systems and Multiprogramming*. Boston: Kluwer Academic Publishers, 1989.
- AGER87** Agerwala, T., and Cocke, J. *High Performance Reduced Instruction Set Processors*. Technical Report RC12434 (#55845). Yorktown, NY: IBM Thomas J. Watson Research Center, January 1987.
- AMDA67** Amdahl, G. "Validity of the Single-Processor Approach to Achieving Large-Scale Computing Capability." *Proceedings, of the AFIPS Conference*, 1967.
- ANDE67a** Anderson, D.; Sparacio, F.; and Tomasulo, F. "The IBM System/360 Model 91: Machine Philosophy and Instruction Handling." *IBM Journal of Research and Development*, January 1967.
- ANDE67b** Anderson, S., et al. "The IBM System/360 Model 91: Floating-Point Execution Unit." *IBM Journal of Research and Development*, January 1967. Reprinted in [SWAR90, Volume 1].
- ANTH08** Anthes, G. "What's Next for the x86?" *ComputerWorld*, June 16, 2008.
- ARM08a** ARM Limited. *Cortex-A8 Technical Reference Manual*. ARM DDI 0344E, 2008, www.arm.com
- ARM08b** ARM Limited. *ARM11 MPCore Processor Technical Reference Manual*. ARM DDI 0360E, 2008, www.arm.com
- ASH90** Ash, R. *Information Theory*. New York: Dover, 1990.
- ATKI96** Atkins, M. "PC Software Performance Tuning." *IEEE Computer*, August 1996.
- AZIM92** Azimi, M.; Prasad, B.; and Bhat, K. "Two Level Cache Architectures." *Proceedings COMPCON '92*, February 1992.
- BACO94** Bacon, F.; Graham, S.; and Sharp, O. "Compiler Transformations for High-Performance Computing." *ACM Computing Surveys*, December 1994.
- BAER10** Baer, J. *Microprocessor Architecture: From Simple Pipelines to Chip Multiprocessors*. New York: Cambridge University Press, 2010.
- BAIL93** Bailey, D. "RISC Microprocessors and Scientific Computing." *Proceedings, Supercomputing '93*, 1993.
- BASH81** Bashe, C.; Bucholtz, W.; Hawkins, G.; Ingram, J.; and Rochester, N. "The Architecture of IBM's Early Computers." *IBM Journal of Research and Development*, September 1981.
- BASH91** Bashteen, A.; Lui, I.; and Mullan, J. "A Superpipeline Approach to the MIPS Architecture." *Proceedings, COMPCON Spring '91*, February 1991.
- BELL70** Bell, C.; Cady, R.; McFarland, H.; Delagi, B.; O'Loughlin, J.; and Noonan, R. "A New Architecture for Minicomputers—The DEC PDP-11." *Proceedings, Spring Joint Computer Conference*, 1970.
- BELL71** Bell, C., and Newell, A. *Computer Structures: Readings and Examples*. New York: McGraw-Hill, 1971.

- BELL74** Bell, J.; Casasent, D.; and Bell, C. "An Investigation into Alternative Cache Organizations." *IEEE Transactions on Computers*, April 1974.
- BELL78a** Bell, C.; Mudge, J.; and McNamara, J. *Computer Engineering: A DEC View of Hardware Systems Design*. Bedford, MA: Digital Press, 1978.
- BELL78b** Bell, C.; Newell, A.; and Siewiorek, D. "Structural Levels of the PDP-8." In [BELL78a].
- BELL78c** Bell, C.; Kotok, A.; Hastings, T.; and Hill, R. "The Evolution of the DEC System-10." *Communications of the ACM*, January 1978.
- BENH92** Benham, J. "A Geometric Approach to Presenting Computer Representations of Integers." *SIGCSE Bulletin*, December 1992.
- BETK97** Betker, M.; Fernando, J.; and Whalen, S. "The History of the Microprocessor." *Bell Labs Technical Journal*, Autumn 1997.
- BEZ03** Bez, R., et al. "Introduction to Flash Memory." *Proceedings of the IEEE*, April 2003.
- BLAA97** Blaauw, G., and Brooks, F. *Computer Architecture: Concepts and Evolution*. Reading, MA: Addison-Wesley, 1997.
- BLAH83** Blahut, R. *Theory and Practice of Error Control Codes*. Reading, MA: Addison-Wesley, 1983.
- BOHR98** Bohr, M. "Silicon Trends and Limits for Advanced Microprocessors." *Communications of the ACM*, March 1998.
- BORK03** Borkar, S. "Getting Gigascale Chips: Challenges and Opportunities in Continuing Moore's Law." *ACM Queue*, October 2003.
- BORK07** Borkar, S. "Thousand Core Chips—A Technology Perspective." *Proceedings, ACM/IEEE Design Automation Conference*, 2007.
- BRAD91a** Bradlee, D.; Eggers, S.; and Henry, R. "The Effect on RISC Performance of Register Set Size and Structure Versus Code Generation Strategy." *Proceedings, 18th Annual International Symposium on Computer Architecture*, May 1991.
- BRAD91b** Bradlee, D.; Eggers, S.; and Henry, R. "Integrating Register Allocation and Instruction Scheduling for RISCs." *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1991.
- BREW97** Brewer, E. "Clustering: Multiply and Conquer." *Data Communications*, July 1997.
- BREY09** Brey, B. *The Intel Microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and Core2 with 64-bit Extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.
- BROW96** Brown, S., and Rose, S. "Architecture of FPGAs and CPLDs: A Tutorial." *IEEE Design and Test of Computers*, Vol. 13, No. 2, 1996.
- BURK46** Burks, A.; Goldstine, H.; and von Neumann, J. *Preliminary Discussion of the Logical Design of an Electronic Computer Instrument*. Report prepared for U.S. Army Ordnance Department, 1946, reprinted in [BELL71].
- BUYY99a** Buyya, R. *High Performance Cluster Computing: Architectures and Systems*. Upper Saddle River, NJ: Prentice Hall, 1999.
- BUYY99b** Buyya, R. *High Performance Cluster Computing: Programming and Applications*. Upper Saddle River, NJ: Prentice Hall, 1999.
- CANT01** Cantin, J., and Hill, H. "Cache Performance for Selected SPEC CPU2000 Benchmarks." *Computer Architecture News*, September 2001.
- CART06** Carter, P. *PC Assembly Language*, July 23, 2006. Available at this book's Web site.
- CEKL97** Cekleov, M., and Dubois, M. "Virtual-Address Caches, Part 1: Problems and Solutions in Uniprocessors." *IEEE Micro*, September/October 1997.
- CHAI82** Chaitin, G. "Register Allocation and Spilling via Graph Coloring." *Proceedings, SIGPLAN Symposium on Compiler Construction*, June 1982.
- CHEN94** Chen, P.; Lee, E.; Gibson, G.; Katz, R.; and Patterson, D. "RAID: High-Performance, Reliable Secondary Storage." *ACM Computing Surveys*, June 1994.

- CHEN96** Chen, S., and Towsley, D. "A Performance Evaluation of RAID Architectures." *IEEE Transactions on Computers*, October 1996.
- CHOW86** Chow, F.; Himmelstein, M.; Killian, E.; and Weber, L. "Engineering a RISC Compiler System." *Proceedings, COMPCON Spring '86*, March 1986.
- CHOW87** Chow, F.; Correll, S.; Himmelstein, M.; Killian, E.; and Weber, L. "How Many Addressing Modes Are Enough?" *Proceedings, Second International Conference on Architectural Support for Programming Languages and Operating Systems*, October 1987.
- CHOW90** Chow, F., and Hennessy, J. "The Priority-Based Coloring Approach to Register Allocation." *ACM Transactions on Programming Languages*, October 1990.
- CLAR85** Clark, D., and Emer, J. "Performance of the VAX-11/780 Translation Buffer: Simulation and Measurement." *ACM Transactions on Computer Systems*, February 1985.
- CLEM00** Clemenwts, A. "The Undergraduate Curriculum in Computer Architecture." *IEEE Micro*, May/June 2000.
- COHE81** Cohen, D. "On Holy Wars and a Plea for Peace." *Computer*, October 1981.
- COLW85a** Colwell, R.; Hitchcock, C.; Jensen, E.; Brinkley-Sprunt, H.; and Kollar, C. "Computers, Complexity, and Controversy." *Computer*, September 1985.
- COLW85b** Colwell, R.; Hitchcock, C.; Jensen, E.; and Sprunt, H. "More Controversy About 'Computers, Complexity, and Controversy.'" *Computer*, December 1985.
- COOK82** Cook, R., and Dande, N. "An Experiment to Improve Operand Addressing." *Proceedings, Symposium on Architecture Support for Programming Languages and Operating Systems*, March 1982.
- COON81** Coonen J. "Underflow and Denormalized Numbers." *IEEE Computer*, March 1981.
- COUT86** Coutant, D.; Hammond, C.; and Kelley, J. "Compilers for the New Generation of Hewlett-Packard Computers." *Proceedings, COMPCON Spring '86*, March 1986.
- CRA97** Cragon, H. "An Evaluation of Code Space Requirements and Performance of Various Architectures." *Computer Architecture News*, February 1979.
- CRA92** Cragon, H. *Branch Strategy Taxonomy and Performance Models*. Los Alamitos, CA: IEEE Computer Society Press, 1992.
- CRAW90** Crawford, J. "The i486 CPU: Executing Instructions in One Clock Cycle." *IEEE Micro*, February 1990.
- CRIS97** Crisp, R. "Direct RAMBUS Technology: The New Main Memory Standard." *IEEE Micro*, November/December 1997.
- CUPP01** Cuppu, V., et al. "High Performance DRAMS in Workstation Environments." *IEEE Transactions on Computers*, November 2001.
- CURR11** Curran, B., et al. "The zEnterprise 196 System and Microprocessor." *IEEE Micro*, March/April 2011.
- DATT93** Dattatreya, G. "A Systematic Approach to Teaching Binary Arithmetic in a First Course." *IEEE Transactions on Education*, February 1993.
- DAVI87** Davidson, J., and Vaughan, R. "The Effect of Instruction Set Complexity on Program Size and Memory Performance." *Proceedings, Second International Conference on Architectural Support for Programming Languages and Operating Systems*, October 1987.
- DENN68** Denning, P. "The Working Set Model for Program Behavior." *Communications of the ACM*, May 1968.
- DERO87** DeRosa, J., and Levy, H. "An Evaluation of Branch Architectures." *Proceedings, Fourteenth International Symposium on Computer Architecture*, 1987.
- DESA05** Desai, D., et al. "BladeCenter System Overview." *IBM Journal of Research and Development*, November 2005.
- DEWA90** Dewar, R., and Smosna, M. *Microprocessors: A Programmer's View*. New York: McGraw-Hill, 1990.
- DOWD98** Dowd, K., and Severance, C. *High Performance Computing*. Sebastopol, CA: O'Reilly, 1998.

- DUBE91** Dubey, P., and Flynn, M. "Branch Strategies: Modeling and Optimization." *IEEE Transactions on Computers*, October 1991.
- ECKE90** Eckert, R. "Communication Between Computers and Peripheral Devices—An Analogy." *ACM SIGCSE Bulletin*, September 1990.
- EISC07** Eischen, C. "RAID 6 Covers More Bases." *Network World*, April 9, 2007.
- ELAY85** El-Ayat, K., and Agarwal, R. "The Intel 80386—Architecture and Implementation." *IEEE Micro*, December 1985.
- ERCE04** Ercegovac, M., and Lang, T. *Digital Arithmetic*. San Francisco: Morgan Kaufmann, 2004.
- EVEN00a** Even, G., and Paul, W. "On the Design of IEEE Compliant Floating-Point Units." *IEEE Transactions on Computers*, May 2000.
- EVEN00b** Even, G., and Seidel, P. "A Comparison of Three Rounding Algorithms for IEEE Floating-Point Multiplication." *IEEE Transactions on Computers*, July 2000.
- EVER98** Evers, M., et al. "An Analysis of Correlation and Predictability: What Makes Two-Level Branch Predictors Work." *Proceedings, 25th Annual International Symposium on Microarchitecture*, July 1998.
- EVER01** Evers, M., and Yeh, T. "Understanding Branches and Designing Branch Predictors for High-Performance Microprocessors." *Proceedings of the IEEE*, November 2001.
- FARH04** Farhat, H. *Digital Design and Computer Organization*. Boca Raton, FL: CRC Press, 2004.
- FARM92** Farmwald, M., and Mooring, D. "A Fast Path to One Memory." *IEEE Spectrum*, October 1992.
- FATA08** Fatahalian, K., and Houston, M. "A Closer Look at GPUs." *Communications of the ACM*, October 2008.
- FLEM86** Fleming, P., and Wallace, J. "How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results." *Communications of the ACM*, March 1986.
- FLYN72** Flynn, M. "Some Computer Organizations and Their Effectiveness." *IEEE Transactions on Computers*, September 1972.
- FLYN85** Flynn, M.; Johnson, J.; and Wakefield, S. "On Instruction Sets and Their Formats." *IEEE Transactions on Computers*, March 1985.
- FLYN87** Flynn, M.; Mitchell, C.; and Mulder, J. "And Now a Case for More Complex Instruction Sets." *Computer*, September 1987.
- FLYN01** Flynn, M., and Oberman, S. *Advanced Computer Arithmetic Design*. New York: Wiley, 2001.
- FOG08a** Fog, A. *Optimizing Subroutines in Assembly Language: An Optimization Guide for x86 Platforms*. Copenhagen University College of Engineering, 2008, <http://www.agner.org/optimize/>
- FOG08b** Fog, A. *The Microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008, <http://www.agner.org/optimize/>
- FRAI83** Frailey, D. "Word Length of a Computer Architecture: Definitions and Applications." *Computer Architecture News*, June 1983.
- FRIE96** Friedman, M. "RAID Keeps Going and Going and..." *IEEE Spectrum*, April 1996.
- FULL11** Fuller, S., and Millet, L., eds. *The Future of Computing Performance: Game Over or Next Level?* Washington, DC: National Academies Press, 2011, www.nap.edu
- FURB00** Furber, S. *ARM System-on-Chip Architecture*. Reading, MA: Addison-Wesley, 2000.
- FUTR01** Futral, W. *InfiniBand Architecture: Development and Deployment*. Hillsboro, OR: Intel Press, 2001.
- GENU04** Genu, P. *A Cache Primer*. Application Note AN2663. Freescale Semiconductor, Inc., 2004 (available in Premium Content Document section).
- GHA198** Ghai, S.; Joyner, J.; and John, L. *Investigating the Effectiveness of a Third Level Cache*. Technical Report TR-980501-01, Laboratory for Computer Architecture, University of Texas at Austin, 1998, <http://lca.ece.utexas.edu/pubs-by-type.html>

- GIBB04** Gibbs, W. "A Split at the Core." *Scientific American*, November 2004.
- GIFF87** Gifford, D., and Spector, A. "Case Study: IBM's System/360-370 Architecture." *Communications of the ACM*, April 1987.
- GOCH06** Gochman, S., et al. "Introduction to Intel Core Duo Processor Architecture." *Intel Technology Journal*, May 2006.
- GOLD54** Goldstine, H.; Pomerene, J.; and Smith, C. *Final Progress Report on the Physical Realization of an Electronic Computing Instrument*. Princeton: The Institute for Advanced Study Electronic Computer Project, 1954.
- GOLD91** Goldberg, D. "What Every Computer Scientist Should Know About Floating-Point Arithmetic." *ACM Computing Surveys*, March 1991.
- GOOD83** Goodman, J. "Using Cache Memory to Reduce Processor-Memory Bandwidth." *Proceedings, 10th Annual International Symposium on Computer Architecture*, 1983. Reprinted in [HILL00].
- GOOD05** Goodacre, J., and Sloss, A. "Parallelism and the ARM Instruction Set Architecture." *Computer*, July 2005.
- GREG98** Gregg, J. *Ones and Zeros: Understanding Boolean Algebra, Digital Circuits, and the Logic of Sets*. New York: Wiley, 1998.
- GRIM05** Grimheden, M., and Tornngren, M. "What Is Embedded Systems and How Should It Be Taught?—Results from a Didactic Analysis." *ACM Transactions on Embedded Computing Systems*, August 2005.
- GSOE08** Gsoedl, J. "Solid State: New Frontier in Storage." *Storage*, July 2008.
- GUST88** Gustafson, J. "Reevaluating Amdahl's Law." *Communications of the ACM*, May 1988.
- HAMM97** Hammond, L.; Nayfay, B.; and Olukotun, K. "A Single-Chip Multiprocessor." *Computer*, September 1997.
- HAND98** Handy, J. *The Cache Memory Book*. San Diego: Academic Press, 1998.
- HARR06** Harris, W. "Multi-Core in the Source Engine." *bit-tech.net technical paper*, November 2, 2006, bit-tech.net/gaming/2006/11/02/Multi_core_in_the_Source_Engin/1
- HAYE98** Hayes, J. *Computer Architecture and Organization*. New York: McGraw-Hill, 1998.
- HEAT84** Heath, J. "Re-Evaluation of RISC 1." *Computer Architecture News*, March 1984.
- HENN82** Hennessy, J., et al. "Hardware/Software Tradeoffs for Increased Performance." *Proceedings, Symposium on Architectural Support for Programming Languages and Operating Systems*, March 1982.
- HENN84** Hennessy, J. "VLSI Processor Architecture." *IEEE Transactions on Computers*, December 1984.
- HENN91** Hennessy, J., and Jouppi, N. "Computer Technology and Architecture: An Evolving Interaction." *Computer*, September 1991.
- HENN06** Henning, J. "SPEC CPU2006 Benchmark Descriptions." *Computer Architecture News*, September 2006.
- HENN07** Henning, J. "SPEC CPU Suite Growth: An Historical Perspective." *Computer Architecture News*, March 2007.
- HIDA90** Hidaka, H.; Matsuda, Y.; Asakura, M.; and Kazuyasu, F. "The Cache DRAM Architecture: A DRAM with an on-Chip Cache Memory." *IEEE Micro*, April 1990.
- HIGB90** Higbie, L. "Quick and Easy Cache Performance Analysis." *Computer Architecture News*, June 1990.
- HILL89** Hill, M. "Evaluating Associativity in CPU Caches." *IEEE Transactions on Computers*, December 1989.
- HILL00** Hill, M.; Jouppi, N.; and Sohi, G. *Readings in Computer Architecture*. San Francisco: Morgan Kaufmann, 2000.
- HINT01** Hinton, G., et al. "The Microarchitecture of the Pentium 4 Processor." *Intel Technology Journal*, Q1, 2001, <http://developer.intel.com/technology/itj/>

- HIRA07** Hirata, K., and Goodacre, J. "ARM MPCore: The Streamlined and Scalable ARM11 Processor Core." *Proceedings, 2007 Conference on Asia South Pacific Design Automation*, 2007.
- HUCK83** Huck, T. *Comparative Analysis of Computer Architectures*. Stanford University Technical Report No. 83-243, May 1983.
- HUGU91** Huguet, M., and Lang, T. "Architectural Support for Reduced Register Saving/Restoring in Single-Window Register Files." *ACM Transactions on Computer Systems*, February 1991.
- HUTC96** Hutcheson, G., and Hutcheson, J. "Technology and Economics in the Semiconductor Industry." *Scientific American*, January 1996.
- HWAN93** Hwang, K. *Advanced Computer Architecture*. New York: McGraw-Hill, 1993.
- HWAN99** Hwang, K., et al. "Designing SSI Clusters with Hierarchical Checkpointing and Single I/O Space." *IEEE Concurrency*, January–March 1999.
- IBM01** International Business Machines, Inc. *64 Mb Synchronous DRAM*. IBM Data Sheet 364164, January 2001.
- INTE98** Intel Corp. *Pentium Pro and Pentium II Processors and Related Products*. Aurora, CO, 1998.
- INTE01a** Intel Corp. *Intel Pentium 4 Processor Optimization Reference Manual*. Document 248966-04, 2001, <http://developer.intel.com/design/Pentium4/documentation.htm>
- INTE01b** Intel Corp. *Desktop Performance and Optimization for Intel Pentium 4 Processor*. Document 248966-04, 2001, <http://developer.intel.com/design/Pentium4/documentation.htm>
- INTE04a** Intel Corp. *IA-32 Intel Architecture Software Developer's Manual (4 volumes)*. Document 253665 through 253668. 2004, <http://developer.intel.com/design/Pentium4/documentation.htm>
- INTE04b** Intel Research and Development. *Architecting the Era of Tera*. Intel White Paper, February 2004, <http://www.intel.com/labs/teraera/index.htm>
- INTE04c** Intel Corp. *Endianness White Paper*, November 15, 2004.
- INTE11** Intel Corp. *Intel® 64 and IA-32 Intel Architectures Software Developer's Manual (3 volumes)*. Denver, CO, 2011.
- JACO08** Jacob, B.; Ng, S.; and Wang, D. *Memory Systems: Cache, DRAM, Disk*. Boston: Morgan Kaufmann, 2008.
- JAME90** James, D. "Multiplexed Buses: The Endian Wars Continue." *IEEE Micro*, June 1990.
- JOHN91** Johnson, M. *Superscalar Microprocessor Design*. Englewood Cliffs, NJ: Prentice Hall, 1991.
- JOHN08** John, E., and Rubio, J. *Unique Chips and Systems*. Boca Raton, FL: CRC Press, 2008.
- JOUP88** Jouppi, N. "Superscalar versus Superpipelined Machines." *Computer Architecture News*, June 1988.
- JOUP89a** Jouppi, N., and Wall, D. "Available Instruction-Level Parallelism for Superscalar and Superpipelined Machines." *Proceedings, Third International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1989.
- JOUP89b** Jouppi, N. "The Nonuniform Distribution of Instruction-Level and Machine Parallelism and Its Effect on Performance." *IEEE Transactions on Computers*, December 1989.
- KAEL91** Kaeli, D., and Emma, P. "Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns." *Proceedings, 18th Annual International Symposium on Computer Architecture*, May 1991.
- KAGA01** Kagan, M. "InfiniBand: Thinking Outside the Box Design." *Communications System Design*, September 2001, www.csdmag.com
- KALL04** Kalla, R.; Sinharoy, B.; and Tendler, J. "IBM Power5 Chip: A Dual-Core Multithreaded Processor." *IEEE Micro*, March–April 2004.
- KANE92** Kane, G., and Heinrich, J. *MIPS RISC Architecture*. Englewood Cliffs, NJ: Prentice Hall, 1992.

- KAPP00** Kapp, C. "Managing Cluster Computers." *Dr. Dobb's Journal*, July 2000.
- KATE83** Katevenis, M. *Reduced Instruction Set Computer Architectures for VLSI*. PhD dissertation, Computer Science Department, University of California at Berkeley, October 1983. Reprinted by MIT Press, Cambridge, MA, 1985.
- KATZ89** Katz, R.; Gibson, G.; and Patterson, D. "Disk System Architecture for High Performance Computing." *Proceedings of the IEEE*, December 1989.
- KEET01** Keeth, B., and Baker, R. *DRAM Circuit Design: A Tutorial*. Piscataway, NJ: IEEE Press, 2001.
- KNAG04** Knaggs, P., and Welsh, S. *ARM: Assembly Language Programming*. Bournemouth University, School of Design, Engineering, and Computing, August 31, 2004, www.freetchbooks.com/arm-assembly-language-programming-t729.html
- KNUT71** Knuth, D. "An Empirical Study of FORTRAN Programs." *Software Practice and Experience*, Vol. 1, 1971.
- KNUT98** Knuth, D. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1998.
- KOLB05** Kolbehdari, M., et al. "The Emergence of PCI Express* in the Next Generation of Mobile Platforms." *Intel Technology Journal*, February 2005.
- KOOP96** Koopman, P. "Embedded System Design Issues (the Rest of the Story)." *Proceedings, 1996 International Conference on Computer Design*, 1996.
- KUCK77** Kuck, D.; Parker, D.; and Sameh, A. "An Analysis of Rounding Methods in Floating-Point Arithmetic." *IEEE Transactions on Computers*, July 1977.
- KUGA91** Kuga, M.; Murakami, K.; and Tomita, S. "DSNS (Dynamically-hazard resolved, Statically-code-scheduled, Nonuniform Superscalar): Yet Another Superscalar Processor Architecture." *Computer Architecture News*, June 1991.
- LEE91** Lee, R.; Kwok, A.; and Briggs, F. "The Floating Point Performance of a Superscalar SPARC Processor." *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1991.
- LEON07** Leonard, T. "Dragged Kicking and Screaming: Source Multicore." *Proceedings, Game Developers Conference 2007*, March 2007.
- LEON08** Leong, P. "Recent Trends in FPGA Architectures and Applications." *Proceedings, 4th IEEE International Symposium on Electronic Design, Test, and Applications*, 2008.
- LILJ88** Lilja, D. "Reducing the Branch Penalty in Pipelined Processors." *Computer*, July 1988.
- LILJ93** Lilja, D. "Cache Coherence in Large-Scale Shared-Memory Multiprocessors: Issues and Comparisons." *ACM Computing Surveys*, September 1993.
- LITT61** Little, J. "A Proof for the Queuing Formula: $L = \lambda W$." *Operations Research*, May–June, 1961.
- LITT11** Little, J. "Little's Law as Viewed on Its 50th Anniversary." *Operations Research*, May–June, 2011.
- LOVE96** Lovett, T., and Clapp, R. "Implementation and Performance of a CC-NUMA System." *Proceedings, 23rd Annual International Symposium on Computer Architecture*, May 1996.
- LUND77** Lunde, A. "Empirical Evaluation of Some Features of Instruction Set Processor Architectures." *Communications of the ACM*, March 1977.
- MACD84** MacDougall, M. "Instruction-level Program and Process Modeling." *IEEE Computer*, July 1984.
- MADD09** Maddox, R., et al. *Weaving High Performance Multiprocessor Fabric: Architectural Insights to the Intel QuickPath Interconnect*. Hillsboro, OR: Intel Press, 2009.
- MAK04** Mak, P., et al. "Processor Subsystem Interconnect for a Large Symmetric Multiprocessing System." *IBM Journal of Research and Development*, May/July 2004.
- MAK97** Mak, P., et al. "Shared-Cache Clusters in a System with a Fully Shared Memory." *IBM Journal of Research and Development*, July/September 1997.

- MANO08** Mano, M., and Kime, C. *Logic and Computer Design Fundamentals*. Upper Saddle River, NJ: Prentice Hall, 2008.
- MANS97** Mansuripur, M., and Sincerbox, G. "Principles and Techniques of Optical Data Storage." *Proceedings of the IEEE*, November 1997.
- MARR02** Marr, D., et al. "Hyper-Threading Technology Architecture and Microarchitecture." *Intel Technology Journal*, First Quarter, 2002.
- MASH95** Mashey, J. "CISC vs. RISC (or what is RISC really)." *USENET comp.arch newsgroup*, article 46782, February 1995.
- MAYB84** Mayberry, W., and Efland, G. "Cache Boosts Multiprocessor Performance." *Computer Design*, November 1984.
- MAZI10** Mazidi, M.; Mazidi, J.; and Causey, D. *The x86 PC: Assembly Language, Design and Interfacing*. Upper Saddle River, NJ: Prentice Hall, 2010.
- MCDO05** McDougall, R. "Extreme Software Scaling." *ACM Queue*, September 2005.
- MCDO06** McDougall, R., and Laudon, J. "Multi-Core Microprocessors Are Here." *login*, October 2006.
- MCEL85** McEliece, R. "The Reliability of Computer Memories." *Scientific American*, January 1985.
- MEND06** Mendelson, A., et al. "CMP Implementation in Systems Based on the Intel Core Duo Processor." *Intel Technology Journal*, May 2006.
- MILE00** Milenkovic, A. "Achieving High Performance in Bus-Based Shared-Memory Multiprocessors." *IEEE Concurrency*, July–September 2000.
- MIRA92** Mirapuri, S.; Woodacre, M.; and Vasseghi, N. "The MIPS R4000 Processor." *IEEE Micro*, April 1992.
- MOOR65** Moore, G. "Cramming More Components Onto Integrated Circuits." *Electronics Magazine*, April 19, 1965.
- MORS78** Morse, S.; Pohlman, W.; and Ravenel, B. "The Intel 8086 Microprocessor: A 16-bit Evolution of the 8080." *Computer*, June 1978.
- MOSH01** Moshovos, A., and Sohi, G. "Microarchitectural Innovations: Boosting Microprocessor Performance Beyond Semiconductor Technology Scaling." *Proceedings of the IEEE*, November 2001.
- MULL10** Muller, J., et al. *Handbook of Floating-Point Arithmetic*. Boston: Birkhauser, 2010.
- MYER78** Myers, G. "The Evaluation of Expressions in a Storage-to-Storage Architecture." *Computer Architecture News*, June 1978.
- NOVI93** Novitsky, J.; Azimi, M.; and Ghaznavi, R. "Optimizing Systems Performance Based on Pentium Processors." *Proceedings COMPCON '92*, February 1993.
- NOWE07** Nowell, M.; Vusirikala, V.; and Hays, R. "Overview of Requirements and Applications for 40 Gigabit and 100 Gigabit Ethernet." *Ethernet Alliance White Paper*, August 2007.
- OBER97a** Oberman, S., and Flynn, M. "Design Issues in Division and Other Floating-Point Operations." *IEEE Transactions on Computers*, February 1997.
- OBER97b** Oberman, S., and Flynn, M. "Division Algorithms and Implementations." *IEEE Transactions on Computers*, August 1997.
- OKLO08** Oklobdzija, V., ed. *Digital Design and Fabrication*. Boca Raton, FL: CRC Press, 2008.
- OLUK96** Olukotun, K., et al. "The Case for a Single-Chip Multiprocessor." *Proceedings, Seventh International Conference on Architectural Support for Programming Languages and Operating Systems*, 1996.
- OLUK05** Olukotun, K., and Hammond, L. "The Future of Microprocessors." *ACM Queue*, September 2005.
- OLUK07** Olukotun, K.; Hammond, L.; and Laudon, J. *Chip Multiprocessor Architecture: Techniques to Improve Throughput and Latency*. San Rafael, CA: Morgan & Claypool, 2007.
- OMON99** Omondi, A. *The Microarchitecture of Pipelined and Superscalar Computers*. Boston: Kluwer, 1999.

- OSUN11** Osuna, A., et al. *IBM System Storage Tape Library Guide for Open Systems*. IBM Redbook SG24-5946-07, June 2011.
- PADE81** Padeys, A. "System/360 and Beyond." *IBM Journal of Research and Development*, September 1981.
- PADE88** Padeys, A.; Moore, B.; Smith, R.; and Buchholz, W. "The IBM System/370 Vector Architecture: Design Considerations." *IEEE Transactions on Communications*, May 1988.
- PARH10** Parhami, B. *Computer Arithmetic: Algorithms and Hardware Design*. Oxford: Oxford University Press, 2010.
- PATT82a** Patterson, D., and Sequin, C. "A VLSI RISC." *Computer*, September 1982.
- PATT82b** Patterson, D., and Piepho, R. "Assessing RISCs in High-Level Language Support." *IEEE Micro*, November 1982.
- PATT84** Patterson, D. "RISC Watch." *Computer Architecture News*, March 1984.
- PATT85a** Patterson, D. "Reduced Instruction Set Computers." *Communications of the ACM*, January 1985.
- PATT85b** Patterson, D., and Hennessy, J. "Response to 'Computers, Complexity, and Controversy.'" *Computer*, November 1985.
- PATT88** Patterson, D.; Gibson, G.; and Katz, R. "A Case for Redundant Arrays of Inexpensive Disks (RAID)." *Proceedings, ACM SIGMOD Conference of Management of Data*, June 1988.
- PATT01** Patt, Y. "Requirements, Bottlenecks, and Good Fortune: Agents for Microprocessor Evolution." *Proceedings of the IEEE*, November 2001.
- PAVA97** Pavan, P., et al. "Flash Memory Cells—An Overview." *Proceedings of the IEEE*, August 1997.
- PEIR99** Peir, J.; Hsu, W.; and Smith, A. "Functional Implementation Techniques for CPU Cache Memories." *IEEE Transactions on Computers*, February 1999.
- PELE97** Peleg, A.; Wilkie, S.; and Weiser, U. "Intel MMX for Multimedia PCs." *Communications of the ACM*, January 1997.
- PFIS98** Pfister, G. *In Search of Clusters*. Upper Saddle River, NJ: Prentice Hall, 1998.
- POLL99** Pollack, F. "New Microarchitecture Challenges in the Coming Generations of CMOS Process Technologies (keynote address)." *Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture*, 1999.
- POPE91** Popescu, V., et al. "The Metaflow Architecture." *IEEE Micro*, June 1991.
- PRES01** Pressel, D. "Fundamental Limitations on the Use of Prefetching and Stream Buffers for Scientific Applications." *Proceedings, ACM Symposium on Applied Computing*, March 2001.
- PRIN97** Prince, B. *Semiconductor Memories*. New York: Wiley, 1997.
- PRIN02** Prince, B. *Emerging Memories: Technologies and Trends*. Norwell, MA: Kluwer, 2002.
- PROP11** Prophet, G. "Use GPUs to Boost Acceleration." *IDN*, December 2, 2011.
- PRZY88** Przybylski, S.; Horowitz, M.; and Hennessy, J. "Performance Trade-Offs in Cache Design." *Proceedings, Fifteenth Annual International Symposium on Computer Architecture*, June 1988.
- PRZY90** Przybylski, S. "The Performance Impact of Block Size and Fetch Strategies." *Proceedings, 17th Annual International Symposium on Computer Architecture*, May 1990.
- RADI83** Radin, G. "The 801 Minicomputer." *IBM Journal of Research and Development*, May 1983.
- RAGA83** Ragan-Kelley, R., and Clark, R. "Applying RISC Theory to a Large Computer." *Computer Design*, November 1983.
- RAMA77** Ramamoorthy, C. "Pipeline Architecture." *Computing Surveys*, March 1977.
- RECH98** Reches, S., and Weiss, S. "Implementation and Analysis of Path History in Dynamic Branch Prediction Schemes." *IEEE Transactions on Computers*, August 1998.

- REDD76** Reddi, S., and Feustel, E. "A Conceptual Framework for Computer Architecture." *Computing Surveys*, June 1976.
- REIM06** Reimer, J. "Valve Goes Multicore." *ars technica*, November 5, 2006, arstechnica.com/articles/paedia/cpu/valve-multicore.ars
- RICH07** Riches, S., et al. "A Fully Automated High Performance Implementation of ARM Cortex-A8." *IQ Online*, Vol. 6, No. 3, 2007, www.arm.com/iqonline
- SAKA02** Sakai, S. "CMP on SoC: Architect's View." *Proceedings, 15th International Symposium on System Synthesis*, 2002.
- SATY81** Satyanarayanan, M., and Bhandarkar, D. "Design Trade-Offs in VAX-11 Translation Buffer Organization." *Computer*, December 1981.
- SCHA97** Schaller, R. "Moore's Law: Past, Present, and Future." *IEEE Spectrum*, June 1997.
- SCHW99** Schwarz, E., and Krygowski, C. "The S/390 G5 Floating-Point Unit." *IBM Journal of Research and Development*, September/November 1999.
- SEAL00** Seal, D., ed. *ARM Architecture Reference Manual*. Reading, MA: Addison-Wesley, 2000.
- SERL86** Serlin, O. "MIPS, Dhrystones, and Other Tales." *Datamation*, June 1, 1986.
- SHAN38** Shannon, C. "Symbolic Analysis of Relay and Switching Circuits." *AIEE Transactions*, Vol. 57, 1938.
- SHAN03** Shanley, T. *InfinBand Network Architecture*. Reading, MA: Addison-Wesley, 2003.
- SHAN05** Shanley, T. *Unabridged Pentium 4. The: IA32 Processor Genealogy*. Reading, MA: Addison-Wesley, 2005.
- SHAR97** Sharma, A. *Semiconductor Memories: Technology, Testing, and Reliability*. New York: IEEE Press, 1997.
- SHAR03** Sharma, A. *Advanced Semiconductor Memories: Architectures, Designs, and Applications*. New York: IEEE Press, 2003.
- SHEN05** Shen, J., and Lipasti, M. *Modern Processor Design: Fundamentals of Superscalar Processors*. New York: McGraw-Hill, 2005.
- SIEG04** Siegel, T.; Pfeffer, E.; and Magee, A. "The IBM z990 Microprocessor." *IBM Journal of Research and Development*, May/July 2004.
- SIEW82** Siewiorek, D.; Bell, C.; and Newell, A. *Computer Structures: Principles and Examples*. New York: McGraw-Hill, 1982.
- SIMA97** Sima, D. "Superscalar Instruction Issue." *IEEE Micro*, September/October 1997.
- SIMA04** Sima, D. "Decisive Aspects in the Evolution of Microprocessors." *Proceedings of the IEEE*, December 2004.
- SIMO96** Simon, H. *The Sciences of the Artificial*. Cambridge, MA: MIT Press, 1996.
- SING10** Singh, G., et al. "The Feeding of High-Performance Processor Cores—Quickpath Interconnects and the New I/O Hubs." *Intel Technology Journal*, September 2010.
- SING11** Singh, G. "The IBM PC: The Silicon Story." *Computer*, August 2011.
- SLOS04** Sloss, A.; Symes, D.; and Wright, C. *ARM System Developer's Guide*. San Francisco: Morgan Kaufmann, 2004.
- SMIT82** Smith, A. "Cache Memories." *ACM Computing Surveys*, September 1982.
- SMIT87** Smith, A. "Line (Block) Size Choice for CPU Cache Memories." *IEEE Transactions on Communications*, September 1987.
- SMIT88** Smith, J. "Characterizing Computer Performance with a Single Number." *Communications of the ACM*, October 1988.
- SMIT89** Smith, M.; Johnson, M.; and Horowitz, M. "Limits on Multiple Instruction Issue." *Proceedings, Third International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1989.
- SMIT95** Smith, J., and Sohi, G. "The Microarchitecture of Superscalar Processors." *Proceedings of the IEEE*, December 1995.
- SMIT08** Smith, B. "ARM and Intel Battle over the Mobile Chip's Future." *Computer*, May 2008.

- SODE96** Soderquist, P., and Leaser, M. "Area and Performance Tradeoffs in Floating-Point Divide and Square-Root Implementations." *ACM Computing Surveys*, September 1996.
- SOHI90** Sohi, G. "Instruction Issue Logic for High-Performance Interruptable, Multiple Functional Unit, Pipelined Computers." *IEEE Transactions on Computers*, March 1990.
- STAL88** Stallings, W. "Reduced Instruction Set Computer Architecture." *Proceedings of the IEEE*, January 1988.
- STAL11** Stallings, W. *Data and Computer Communications, Ninth Edition*. Upper Saddle River, NJ: Prentice Hall, 2011.
- STAL12** Stallings, W. *Operating Systems, Internals and Design Principles, Seventh Edition*. Upper Saddle River, NJ: Prentice Hall, 2012.
- STEN90** Stenstrom, P. "A Survey of Cache Coherence Schemes of Multiprocessors." *Computer*, June 1990.
- STEV64** Stevens, W. "The Structure of System/360, Part II: System Implementation." *IBM Systems Journal*, Vol. 3, No. 2, 1964. Reprinted in [SIEW82].
- STON93** Stone, H. *High-Performance Computer Architecture*. Reading, MA: Addison-Wesley, 1993.
- STON96** Stonham, T. *Digital Logic Techniques*. London: Chapman & Hall, 1996.
- STRE78** Strecker, W. "VAX-11/780: A Virtual Address Extension to the DEC PDP-11 Family." *Proceedings, National Computer Conference*, 1978.
- STRE83** Strecker, W. "Transient Behavior of Cache Memories." *ACM Transactions on Computer Systems*, November 1983.
- STRI79** Stritter, E., and Gunter, T. "A Microprocessor Architecture for a Changing World: The Motorola 68000." *Computer*, February 1979.
- SWAR90** Swartzlander, E., ed. *Computer Arithmetic, Volumes I and II*. Los Alamitos, CA: IEEE Computer Society Press, 1990.
- TAMI83** Tamir, Y., and Sequin, C. "Strategies for Managing the Register File in RISC." *IEEE Transactions on Computers*, November 1983.
- TANE78** Tanenbaum, A. "Implications of Structured Programming for Machine Architecture." *Communications of the ACM*, March 1978.
- TJAD70** Tjaden, G., and Flynn, M. "Detection and Parallel Execution of Independent Instructions." *IEEE Transactions on Computers*, October 1970.
- TOMA93** Tomasevic, M., and Milutinovic, V. *The Cache Coherence Problem in Shared-Memory Multiprocessors: Hardware Solutions*. Los Alamitos, CA: IEEE Computer Society Press, 1993.
- TOON81** Toong, H., and Gupta, A. "An Architectural Comparison of Contemporary 16-Bit Microprocessors." *IEEE Micro*, May 1981.
- TUCK87** Tucker, S. "The IBM 3090 System Design with Emphasis on the Vector Facility." *Proceedings, COMPCON Spring '87*, February 1987.
- UNGE02** Ungerer, T.; Rubic, B.; and Silc, J. "Multithreaded Processors." *The Computer Journal*, No. 3, 2002.
- UNGE03** Ungerer, T.; Rubic, B.; and Silc, J. "A Survey of Processors with Explicit Multithreading." *ACM Computing Surveys*, March 2003.
- VOGL94** Vogley, B. "800 Megabyte Per Second Systems Via Use of Synchronous DRAM." *Proceedings, COMPCON '94*, March 1994.
- VONN45** Von Neumann, J. *First Draft of a Report on the EDVAC*. Moore School, University of Pennsylvania, 1945. Reprinted in *IEEE Annals on the History of Computing*, No. 4, 1993.
- VRAN80** Vranesic, Z., and Thurber, K. "Teaching Computer Structures." *Computer*, June 1980.
- WALL85** Wallich, P. "Toward Simpler, Faster Computers." *IEEE Spectrum*, August 1985.
- WALL91** Wall, D. "Limits of Instruction-Level Parallelism." *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 1991.

- WANG99** Wang, G., and Tafti, D. "Performance Enhancement on Microprocessors with Hierarchical Memory Systems for Solving Large Sparse Linear Systems." *International Journal of Supercomputing Applications*, Vol. 13, 1999.
- WEIC90** Weicker, R. "An Overview of Common Benchmarks." *Computer*, December 1990.
- WEIN75** Weinberg, G. *An Introduction to General Systems Thinking*. New York: Wiley, 1975.
- WEIS84** Weiss, S., and Smith, J. "Instruction Issue Logic in Pipelined Supercomputers." *IEEE Transactions on Computers*, November 1984.
- WEYG01** Weygant, P. *Clusters for High Availability*. Upper Saddle River, NJ: Prentice Hall, 2001.
- WHIT97** Whitney, S., et al. "The SGI Origin Software Environment and Application Performance." *Proceedings, COMPCON Spring '97*, February 1997.
- WHIT11** White, B., et al. *IBM zEnterprise 196 Technical Guide*. IBM Redbook SG24-5946-07, June 2011.
- WILE03** Wilen, A.; Schade, J.; and Thronburg, R. *Introduction to PCI Express—A Hardware and Software Developers Guide*. Hillsboro, OR: Intel Press, 2003.
- WILK65** Wilkes, M. "Slave Memories and Dynamic Storage Allocation." *IEEE Transactions on Electronic Computers*, April 1965. Reprinted in [HILL00].
- WILL90** Williams, F., and Steven, G. "Address and Data Register Separation on the M68000 Family." *Computer Architecture News*, June 1990.
- YEH91** Yeh, T., and Patt, N. "Two-Level Adapting Training Branch Prediction." *Proceedings, 24th Annual International Symposium on Microarchitecture*, 1991.
- ZHAN01** Zhang, Z.; Zhu, Z.; and Zhang, X. "Cached DRAM for ILP Processor Memory Access Latency Reduction." *IEEE Micro*, July–August 2001.

INDEX

A

- Absolute address, 478, 715
- Access control, 304
- Access time (latency), 115, 193–194
- Accumulator (AC), 20, 71, 411
- Active secondary clustering method, 635–636
- Active standby, 635
- Adders, 384–388
- Addition, 328–331, 350–353
 - floating-point numbers, 349–353
 - two's complement integers, 328–331
- Addressable units, 114
- Address bus, 86
- Addresses, 123–124, 286–288, 294–295,
300–301, 410–412
 - accumulator (AC), 411
- Address generation sequencing, 598
- Address lines, 86
- Address modify instructions, 22–23
- Address registers, 487
- Address space, 294–295
- ARM translation, 300–301
 - base, 287
 - cache memory, 123–124
 - fields, 296
 - I/O memory management, 286–287, 294–295,
300–301
 - logical, 287, 288
 - machine instructions, 407–412
 - number of, 407–412
 - page tables for, 288, 300–301
 - partitioning, 286–287
 - Pentium II translation mechanisms, 294–299
 - physical, 287, 288
 - relative, 288
 - spaces, 294–295
 - virtual memory and, 297, 300–301
- Addressing modes, 452–464, 547–548
- Advanced RISC Machine (ARM), 462–464
 - CPU instruction sets, 451–464
 - direct, 453–455
 - displacement, 453–454, 456–458
 - immediate, 453–454
 - indirect, 453–454
 - Intel x86, 459–461
 - register, 453–456
 - register indirect, 453–454, 456
 - RISC simplicity, 547–548
 - stack, 453–454, 458–459
- Advanced programmable interrupt controller (APIC), 677
- Advanced RISC Machine (ARM), 2, 45–49,
144–146, 299–304, 416–417, 439–440,
462–464, 475–477, 520–526, 595–603,
679–683
 - access control, 304
 - addressing mode, 462–464
 - ARM11 MPCore, 679–683
 - cache memory, 144–146
 - condition codes, 439–440
 - Cortex-A8 processor, 595–603
 - CPU instruction sets, 416–417, 439–440,
462–464, 475–477
 - current program status registers (CPSR),
523–525
 - data types, 416–417
 - embedded systems and, 45–48
 - evolution of, 47–49
 - formats for memory management, 301–304
 - I/O memory management, 299–304
 - instruction format, 475–477
 - instruction-level parallelism and, 595–603
 - interrupt processing, 525–526
 - machine instructions, 416–417, 439–440
 - memory management unit (MMU) for,
300–301
 - memory system organization, 299–300
 - modes, 522–523
 - multicore computers, 679–683
 - operations (opcode), 439–440
 - page tables for, 300–301
 - parameters for memory management, 303
 - processor organization, 520–526
 - register organization, 523–525
 - superscalar processor design, 595–603
 - translation lookaside buffer (TLB), 299–300
 - virtual memory address translation, 300–301
- Allocation, Pentium 4 processor, 594
- Amdahl's law, 56–57, 669
- American Standard Code for Information
Interchange (ASCII), 225
- AND gate, 381
- Antidependency, 584–585
- Arbiter (bus controller), 90
- Arbitration, 90
 - interconnection method of, 90
- Arithmetic and logic unit (ALU), 12, 17,
309–317, 319–358, 654–656
 - addition, 328–331, 349–353
 - computer functions, 12, 320–321
 - development of, 17
 - division, 338–341, 353–356

Arithmetic and logic unit (ALU) (*continued*)
 fixed-point notation, 326–341
 floating-point notation, 341–349
 IBM 3090 vector facility, 654–656
 integers, 322–341
 multiplication, 331–338, 353–356
 precision considerations, 354–356
 in second generation, 25
 semiconductor memory, 33
 subtraction, 328–331, 350–353
 two's complement notation, 323–325, 326–341
 Arithmetic instructions, 22–23, 409
 Arithmetic mean, 54
 Arithmetic operations (opcode), 418, 422
 Arithmetic shift, 336, 424
 Array processor, 644, 649
 Assemblers, 706–710
 Assembly language, 477–478, 695, 698–706
 Asserting, signal, 369
 Associative mapping, 130–132
 Associative memory, 115
 Asynchronous timing, 92–93
 Autoindexing, 458

B

Backward compatibility, 24
 Balanced transmission, 96
 Base, 342
 Base address, 287
 Base-register addressing, 457–458
 Base representation, 673
 Batch operating system (OS), 270, 271–276
 job control language (JCL), 272
 monitor (simple), 271–273
 multiprogramming, 273–276
 Batch system, 270
 Benchmark programs, 53–55
 Biased representation, 342
 Big endian ordering, 447–450
 Binary operator. *See* Binary system
 Binary system, 312
 Bit, 35–37
 Bit allocation, 465–469
 Bit-interleaved parity disk performance
 (RAID level 3), 197–199, 202–203
 Bit length conversion, 325–327
 Bit ordering, endian, 450
 Blade servers, 639
 Block multiplexor, 247
 Blocked multithreading, 628–630
 Block-level distributed parity disk performance
 (RAID level 5), 197–199, 204
 Block-level parity disk performance
 (RAID level 4), 197–199, 203–204
 Blu-ray DVD, 210, 215
 Boolean algebra, 365–367

Boolean (logic) instructions, 409
 Booth's algorithm, 336–338
 Branch target buffer (BTB), 592–593, 598
 Branches, 22–23, 409, 426–427, 462, 498–499,
 504–510, 552–554, 587
 conditional instructions, 22–23, 426–427,
 498–499
 control hazard, 504
 delayed, 510, 552–554
 instructions, 409, 426–427, 462
 loop buffer for, 504–505
 multiple streams for, 504
 pipelining and, 498–499, 504–510,
 552–554
 prediction, 505–510, 587
 prefetch target, 504
 RISC instructions, 552–554
 superscalar processors, 587
 unconditional instructions, 22–23, 426–427
 Buffers, 69
 Bus arbitration technique, I/O, 236
 Bus interconnection, 12, 85–93
 address lines, 86
 control lines, 86
 data lines, 85–86
 design elements, 89–93
 method of arbitration, 90
 multiple hierarchies, 87–89
 structure, 86–87
 timing, 90–93
 width of data bus, 86
 Bus master, 138
 Bus watching approach, 138
 Bus width, 36
 Byte, 106
 Byte multiplexor, 247
 Byte ordering, endian, 447–450

C

Cache, 39
 Cache coherence, 619–625, 683
 directory protocols, 621
 hardware solutions, 621–622
 MESI (modified, exclusive, shared,
 or in valid) protocol, 622–625
 multicore computers, 683
 parallel processing and, 619–625
 problem of, 619
 read hit/miss, 624
 snoopy protocols, 621–622
 software solutions, 620–621
 two-level cache consistency, 625
 write hit/miss, 625
 Cache-coherent nonuniform memory access
 (CC-NUMA), 640

- Cache DRAM (CDRAM), 180
- Cache hit, 122
- Cache line, 120
- Cache memory, 112–158, 532, 541–543, 625, 679, 683
 - addresses, 123–125
 - ARM organization, 144–146, 679, 683
 - design elements, 123–141
 - development of, 532
 - disk performance, 152
 - hierarchy level, 116–117
 - high-level language (HLL)
 - operations, 153–154
 - high-performance computing (HPC), 123–124
 - lines, 120–121, 139
 - locality of reference, 152–154
 - main (physical) performance, 123–125, 152
 - management unit (MMU), 123–125
 - mapping functions, 126–136
 - multicore computers, 679, 683
 - multilevel, 139–141
 - Pentium 4 organization, 141–144
 - register file size increase compared to, 541–543
 - replacement algorithms, 137
 - size, 125, 139
 - SMP shared, 625
 - snoop control unit (SCU), 679, 683
 - split, 141
 - structure, 120–123
 - tags, 121
 - two-level, 152–158, 625
 - unified, 141
 - virtual (logical) performance, 123–125, 152
 - write policy, 137–139
- Cache miss, 122
- Cache set, 132
- Call/return instructions, 433
- CD-ROM, 210
- CD-RW, 210
- Central processing unit (CPU), 12, 25–27, 68, 405–626
 - addressing modes, 451–464
 - arithmetic and logic unit (ALU), 12, 405, 319–404
 - complex instruction set computer (CISC), 534, 536, 568–569
 - component functions of, 68–69
 - data types, 415–418
 - development of, 25–27
 - endian orders, 447–450
 - instruction formats, 464–472
 - instruction-level parallelism, 573–609
 - instruction sets, 405–450
 - machine instruction, 405–415, 533–538, 547
 - operands, 406–407, 413–415, 536–537
 - operations (opcode), 406, 418–431, 535–536
 - processor structure and function, 12, 483–530
 - reduced instruction set computers (RISC), 531–572
 - registers, 12, 486–491, 512–520, 523–526
 - stacks, 441–395
 - superscalar processors, 534, 573–609
- Centralized arbitration, 90
- Chaining, 648
- Character data operands, 414–415
- Chip multiprocessing, 628–633.
 - See also* Multicore computers
 - IBM Power5, 631–633
 - multithreading and, 628–633
 - Pentium 4 processor, 631
- Chip multiprocessor, 665
- Chips, 33–35, 41–43, 164–169
 - computer processing and, 41–43
 - development of, 33–35
 - DRAM for, 174–180
 - enable (CE) pin, 167–168
 - EPROM package, 167–168
 - interleaved (memory bank) memory, 169
 - internal memory and, 164–169
 - logic, 164–166
 - main memory cells, 164–169
 - organization, 41–43, 168–169
 - packaging, 167–168
- Clock (bus) cycle, 50–51, 90–91
- Clocked S–R flip-flop, 389–391
- Clock speed, 50–52
- Clock tick, 50
- Clusters, 633–640
 - active secondary, 635
 - blade servers, 639
 - computer architecture, 637–639
 - configurations, 633–636
 - design requirements, 633
 - OS system design, 636–637
 - passive standby, 634–635
 - shared disk approach, 635–636
 - shared nothing approach, 635
 - SMP compared to, 639–640
- Combinational circuits, 370–388
 - adders, 384–388
 - algebraic simplification, 373
 - Boolean function in, 370–372
 - decoders, 382–384
 - Karnaugh map, 373–376
 - multiplexers, 380–382
 - NAND implementation, 380
 - NOR implementation, 380
 - overview, 370
 - Quine–McCluskey method, 376–379
 - read-only memory (ROM), 384

- Commands, 229–230
 - programmed I/O, 229–230
- Comment, assembly language, 703
- Commercial computers, 23–24
- Committing (retiring)
 - instructions, 588
- Compact disk (CD), 210, 210–214
 - constant linear velocity (CLV), 211
 - digital versatile disk (DVD)
 - compared to, 213–214
 - read-only (CD-ROM), 210–212
 - recordable (CD-R), 210, 212–213
 - rewritable (CD-RW), 210, 213
- Compaction, I/O memory, 286
- Compiler-based register
 - optimization, 543–545
- Complex instruction set computer (CISC), 2, 534, 536, 545–551, 568–569
 - high-level language (HLL) and, 536, 545–547
 - reduced instruction set computer (RISC)
 - architecture compared to, 549–551, 568–569
 - superscalar and RISC systems compared to, 534
- Complex PLD (CPLD), 400
- Compound instructions, IBM 3090 vector facility, 650
- Computer architecture, 7
- Computer arithmetic. *See* Arithmetic and logic unit (ALU)
- Computer instruction, 406
- Computer organization, 7
- Computer performance, 37–59
 - Advanced RISC Machine (ARM), 45–49
 - Amdahl's law, 56–57
 - benchmark programs, 52–55
 - chip organization, 41–43
 - clock speed, 50–52
 - designing for, 38–43
 - embedded systems, 45–49
 - Intel x86 system, 44–45
 - logic-memory balance, 39–41
 - microprocessor speed, 38–39
- Computer systems, 8–13, 65–308
 - cache memory, 65, 112–158
 - external memory, 185–220
 - functions, 8–13, 66–85
 - input/output (I/O), 68, 221–264
 - interconnections, 12, 85–107
 - internal memory, 159–184
 - operating system (OS) support, 265–308
 - peripheral component (PCI), 98–107
 - top-level structure, 13–14, 65–111
- Computers, 1–62
 - Advanced RISC Machine (ARM), 2, 46–49
 - architecture, 1–13
 - components, 10–12
 - data, 10
 - design, 37–43
 - evolution of, 15–37, 43–49
 - function, 8–13
 - instructions, 17
 - integrated circuits, 28–33
 - Intel x86 system, 2, 44–45
 - introduction to, 6–14
 - microprocessors, 35–37
 - organization, 1–14
 - performance, 37–59
 - reader's and instructor's guide, 2–5
 - reasons for study of, 3–4
 - semiconductors, 33–35
 - structure, 8, 10–13
 - transistors, 24–27
 - vacuum tubes, 16–24
 - Web site resources, 5
- Condition codes, 426–427, 434, 439–440, 487–488
 - Advanced RISC Machine (ARM), 439–440
 - branch instructions and, 426–427
 - Intel x86 system, 434
 - program status register, 523
 - registers for, 487–488
- Conditional branch instructions, 22–23, 426–427, 498–499
- Conditional jump, 434
- Constant angular velocity (CAV), 188, 211
- Constant linear velocity (CLV), 211
- Continuous-field simulation, 644
- Control, 224–225, 420, 426–431, 532
 - branch instructions, 426–427
 - bus, 226
 - I/O modules, 224–225
 - logic, 224
 - memory, 534
 - microinstructions, 468, 547
 - operations (opcode), 420, 426–431
 - procedure call instructions, 427–431
 - registers, 515–517
 - skip instructions, 427
 - stack implementation, 430–431
 - storage, 11
 - system operations, 420, 426
 - transfer of, 420, 426–431
- Control registers, 486, 487–490, 515–517
- Control hazards, pipelining, 504
- Control lines, 86
- Control signals, 224
 - input/output (I/O), 224
- Control unit (CU), 12, 532
 - branch instruction, 22
 - IBM 7094, 27
 - in IAS computer, 17–18
 - instruction execution, 23

- microprogrammed, 532
- processor control, 12
- in second generation, 25
- semiconductor memory, 33
- sequencing, 570, 596–602
- Controllers, 90–91, 228, 236–238, 243–246
 - bus arbiter, 90–91
 - channels and, 247
 - input/output (I/O), 228, 236–238, 243–246
 - Intel 8237A DMA, 243–246
 - Intel 82C59A interrupt, 236–238
- Conversion operations (opcode), 420, 425
- Core, magnetization and, 35
- Cortex-A8 processor, 595–603
 - architecture, 595–597
 - decode instruction unit, 598–599
 - fetch instruction unit, 597–598
 - floating-point pipeline, 602–603
 - instruction-level parallelism and, 595–603
 - integer execute unit, 599–602
 - single-instruction multiple-data (SIMD) instructions, 602–603
 - superscalar design, 595–603
- Counters, 394–397
- Current program status registers (CPSR), ARM, 523–525
- Cycle stealing, 240
- Cycle time, pipelining, 500–501
 - instruction execution rate, 51
- Cycles per instruction (CPI), 51–52
- Cylinders, magnetic disks, 191

D

- Daisy chain technique, I/O, 236
- Data, 9–12, 23, 27, 29, 224, 330–331, 412–413
 - ARM types, 416–417
 - channels, 27
 - control, 10
 - input/output (I/O), 224
 - Intel x86 types, 415–416
 - machine instruction types, 320–321, 406–409
 - movement of, 10–11
 - operands, 490–393
 - processing, 9
 - storage, 9–10
 - transfer instructions, 21, 23
- Data buffering, I/O modules, 226
- Data exchanges, 85–86
- Data cache, 144
- Data communication, 10
- Data flow, instruction cycles, 492–493
- Data formatting, magnetic disks, 186–188
- Data hazards, pipelining, 503
- Data (bus) lines, 85–86
- Data processing instruction addressing, ARM, 411

- Data registers, 386
- Data transfer, 89–90, 93, 420
 - bus interconnection, 85–86
 - operations (opcode), 420, 434
 - peripheral component, 98
 - interconnection (PCI), 93
- Database scaling, 671–673
- Decimal system, 310–311
- Decode instruction unit, Cortex-A8 processor, 583–584
- Decoder, 382–384, 590
- Delayed branch, pipelining, 510, 553–554
- Delayed load, pipelining, 554
- Demand paging, 289–290
- Denormalized numbers, IEEE standards, 735
- Device communication, I/O modules, 226
- D flip-flop, 391
- Differential signaling, 96
- Digital Equipment Corporation (DEC), PDP series computers, 25, 32–34, 467–468
- Digital logic
 - Boolean algebra, 365–367
 - combinational circuits, 370–388
 - Gates, 368–370
- Digital versatile disk (DVD), 210, 213–214
- Dijkstra's algorithm, 445–446
- Direct access, 115
- Direct address, 454–455
- Direct addressing mode, 567–568
- Directives, 703–704
- Direct mapping, 126–130
- Direct memory access (DMA), 85, 240–244
 - configurations, 242–243
 - cycle stealing, 242–242
 - direct memory access (DMA), 85, 240–244
 - function, 222, 240–241
 - Intel 8237A controller, 243–246
 - interconnection structure and, 84
 - registers, 244–245
- Directory protocols, 621–622
- Dirty (use) bit, 138
- Disabled interrupt, 80
- Discrete components, 28–29
- Disk cache memory, 152
- Disk drive, I/O, 225
- Diskette, 214
- Disk pack, 190–192
- Disks. *See* Magnetic disks; Optical memory systems
- Disk stripping, 204
- Dispatcher, 278
- Displacement addressing mode, 460
- Distributed arbitration, 90
- Distributed interrupt controller (DIC), 680–681
- Dividend, 338–339

Division, 338–339, 352–353
 floating-point numbers, 342–344
 partial remainder, 338–339
 two's complement restoring algorithm, 340–341
 Divisor, 338
 Double-data-rate DRAM (DDRDRAM), 180–181
 Double-sided disk, 210
 Drive, Pentium 4 processor, 512
 Dual redundancy disk performance (RAID level 6), 192–193
 DVD, 210
 DVD-R, 210
 DVD-ROM, 210
 DVD-RW, 210
 Dynamic linker, 716–718
 Dynamic RAM, 161
 Dynamic random-access memory (DRAM), 38, 161–163, 165–167, 174–175
 cache (CDRAM), 175
 chip logic, 164–166
 double-data-rate (DDR DRAM), 179–180
 high-performance processors, 174–180
 internal main memory, 161–163
 Rambus (RDRAM), 175
 synchronous (SDRAM), 175–176

E

Effective address, 454
 EFLAGS register, Intel x86 processors, 512–513
 Electrically erasable programmable read-only memory (EEPROM), 161, 164
 Electronic Numerical Integrator and Computer (ENIAC), 16–17
 Embedded systems, 46–48
 Emulation (EM), 516
 Enabled interrupt, 80
 Endian byte orders, 447
 Erasable programmable read-only memory (EPROM), 161, 164, 167–168
 chip packaging, 167–168
 internal main memory, 161, 164
 Error control function, 97
 Error correction, 170–174
 code functions, 170
 Hamming code, 171
 hard failure, 170
 internal memory, 170–174
 parity bits, 171
 semiconductor memory, 170–174
 single error-correcting (SEC) code, 174
 soft error, 170
 syndrome words, 171–172
 Error detection, I/O modules, 226

Exceptions, interrupts and, 518, 525–526
 Execute cycle, 21, 69–74, 494
 computer instructions, 20, 69–74
 micro-operations (micro-ops), 589–590
 processor instruction, 491, 494
 Execution, 51–52, 233–234, 475, 484–488, 492, 495–502, 525, 594–596, 606
 control unit (CU), 485, 613–619
 encoding, 556–557
 I/O techniques, 229, 231
 IBM 3033 processor, 505
 instruction rate, 54
 microinstructions, 547
 multithreading, 672
 out-of-order, 581–584, 594–596
 process, 279–280
 RISC machine instructions, 543–547
 superscalar programs, 587–589
 taxonomy of, 613–614
 Exponent overflow, 349
 Exponent underflow, 349
 Exponent value, 347, 349
 External memory, 64, 113–114
 direct-access devices, 217
 magnetic disks, 186–195
 magnetic tape, 215–217
 optical systems, 220
 Redundant Array of Independent Disks (RAID), 186, 196–201
 sequential-access devices, 216

F

Failback, 636
 Failover, 636
 Failure management, clusters, 636
 Family concept, 532
 Fetch cycle, 20, 69–72, 492, 494, 511
 computer instructions, 20, 69–74
 micro-operations (micro-ops), 589–590
 processor instructions, 444
 Fetch instruction unit, Cortex-A8 processor, 598
 Fetch overlap, pipelining, 496
 Field-programmable gate array (FPGA), 398–400
 FireWire serial bus, 250–254
 configurations, 250–252
 cycle master, 254
 link layer, 251–254
 physical layer, 251–252
 transaction layer, 248–252
 Firmware, 97, 208
 First-in first-out (FIFO) algorithm, 137
 Fixed-head disk, 190–191
 Fixed-point notation. *See* Integers
 Fixed-point representation, 326.
 See also Integers

Fixed-size partitions, 284–285
Flag, register organization, 512–513
Flash memory, 161, 164
Flip-flops, 388
Flit, 95
Floating-point formulas, IEEE (Standards) 754-2008, 354–359
Floating-point notation, 341–349, 595, 602–603
 addition, 349–350
 arithmetic and logic unit (ALU) data, 320–322
 arithmetic, 349–357
 biased representation, 343
 Cortex-A8 processor pipelining, 598
 denormalized numbers, 514
 division, 352–353
 exponent value, 342, 347
 guard bits, 353–355
 IEEE standards for, 322, 355–356
 infinity interpretation, 356
 multiplication, 352–355
 NaNs, 356
 normalized numbers, 343–344
 overflow, 345, 348
 Pentium 4 execution unit, 595
 precision considerations, 353–355
 principles, 343–346
 representation, 343–348
 rounding, 354–355
 significand, 349, 359
 subtraction, 349–353
 underflow, 344, 349, 356
Floating-point representation, 345–349.
 See also IEEE (Standards) 754-2008
Floppy (contact) magnetic disks, 190, 192
Flow control function, 97
Flow dependency, 578
Fraction, 313–315
Frames, I/O memory, 287–288
Front end, Pentium 4 processor, 590–593
Fully nested interrupt mode, 237
Functions, 8–13, 18–19, 27, 65–85, 108–112, 226–238, 246–247
 components and, 24, 66–84
 computer operation and, 10–13
 execute cycle, 21, 69–74
 fetch cycle, 21, 69–74
 hardwired programs, 67
 I/O channels, 247–248
 I/O modules, 82–83, 226–227, 246–247
 IAS computer operation, 20–23
 input/output (I/O), 84–85, 226–227, 246–247
 instruction cycle, 20–23, 69–73, 76–80
 interrupts, 74–83
 software components, 67–68
 von Neuman architecture and, 66–68

G

Gaps, magnetic disks, 186
Gates, 368–370
General-purpose computing on GPUs (GPGPU), 43
General-purpose registers, 439, 470, 486
Geometric mean, 55
Global history buffer (GHB), 598
Global variable storage, registers, 461, 541
G Prefix, 35
Gradual underflow, 358
Grant (GNT) signal, PCI, 102–104
Graphical symbol, 370
Graphics processing units (GPUs), 43
Guard bits, 338–339

H

Hamming code, 170
Hard disk, 191
Hard disk drives (HDDs), 205
Hard failure, 169–170
Hardware, 620–621, 665–670
 cache coherence solutions, 619–640
 multicore computers performance, 664–669
 parallelism increase, 664–668
 power consumption, 668–671
Hardware transparency approach, 137
Hardwired programs, 67
Harmonic mean, 54
Hash functions, 290–291
Heads, magnetic disks, 186–187, 189–190
Hexadecimal, 315–317
High-definition optical disks (HD DVD), 214–215
High-level language (HLL), 153, 533–534
 operands, 535–536
 operations, 534–535
 performance characteristics, 151–152
 procedure calls, 536–537
 reduced instruction set computers (RISC), 412
 semantic gap and, 533–534
High-performance computing (HPC), 123
Hit ratio, 118
Host channel adapter (HCA), 253

I

IAS computer, 17–22
IBM. *See* International Business Machines (IBM)
IEEE. *See* Institute of Electrical and Electronics Engineers (IEEE)
IEEE (Standards) 754-2008
 floating-point formulas, 345–349
Immediate addressing mode, 454
Immediate constants, ARM, 476–477
Indexing, 457–458

- Indirect addressing mode, 455–456
- Indirect instruction cycle, 492
- InfiniBand, 253–256
- Infinity, IEEE interpretation, 356
- Infix notation, 445
- Input/Output (I/O), 13, 14, 64, 69, 84–85, 222–260, 420, 425
 - address register (I/OAR), 68
 - buffer register (I/OBR), 68
 - channels, 228, 247–248
 - component functions, 28
 - computer systems, 64, 68, 221–262
 - controllers, 236–238, 243–246, 228
 - data movement and, 10
 - direct memory access (DMA), 85, 222, 240–246
 - disk drive, 225
 - execution techniques, 222, 228–230
 - FireWire serial bus, 250–254
 - function, 246–247
 - high data-transfer capacity, 200
 - high request rate, 200
 - InfiniBand, 256–256
 - Intel 82C55A programmable
 - peripheral interface, 238–240
 - Intel 82C59A interrupt controller, 236–238
 - Intel 8237A DMA controller, 243–246
 - interconnection structure, 84–85
 - interfaces, 222, 238–240, 248–257
 - interrupt-driven, 222, 232–240
 - keyboard/monitor arrangement, 225
 - modules, 83–84, 222–223, 226–228, 246–247
 - multipoint interfaces, 250
 - operations (opcode), 420, 434
 - peripheral (external) devices, 223–225
 - peripheral data devices, 10
 - point-to-point interfaces, 249
 - programmed, 222, 228–232, 238–240
 - RAID 0 performance for, 200–201
- Interactive simulations, 692
- I/O channels, 228, 247–248
- I/O command, 228–229
- I/O controller, 228
- I/O modules, 83–84, 222–223, 226–228, 246–247
 - computer functions and, 83
 - control and timing, 226
 - requirements, 226
 - data buffering, 227
 - device communication, 227
 - error detection, 227
 - evolution of, 246–247
 - function, 83–84, 226–227
 - input/output interfaces and, 222–223
 - interconnection structure, 84–85
 - processor communication, 93, 226–261
 - structure, 227–228
- I/O processor, 247
- Immediate address, 454
- Index register, 457
- Indexed address, 487
- Indexing, 457–458
- Indirect address, 455
- Indirect cycle, 492
- In-order completion, 581
- In-order issue, 581–583
- Input-output control (I/O)
 - IAS Computer structure, 21
- Institute of Electrical and Electronics Engineers (IEEE), 3–4, 345–347, 356–358
 - denormalized number standards, 357
 - floating-point notation standards, 356–358, 354–357
 - infinity interpretation, 348
 - Joint Task Force publications, 3–4
 - NaN standards, 356
 - rounding approaches, 354–355
- Instruction address register, 72–73
- Instruction buffer register (IBR), 21
- Instruction cache, Pentium 4, 142
- Instruction cycle, 20–23, 69–82, 491–485
 - Direct memory access (DMA) code (ICC), 253–254
 - computer functions, 68–82
 - data flow, 492–495
 - execute cycle, 21, 69–77, 494
 - fetch cycle, 20, 69–72, 465, 492
 - IAS computer, 21–22
 - indirect cycle, 491, 494
 - I/O modules, 83
 - interrupt cycle, 77, 494
 - interrupts and, 74–83
 - micro-operations (micro-ops), 589–590
 - multiple interrupts, 80
 - processor, 491–494
 - state diagrams, 73, 80, 493
- Instruction execution rate, 51–52
- Instruction formats, 408, 464–472, 548–549, 558, 566–567
 - Advanced RISC Machine (ARM), 475–477
 - assembly language, 477–479
 - bit allocation, 515
 - Intel x86, 473–475
 - length, 464–465
 - MIPS R4000 microprocessor, 559
 - PDP-8 design, 467–468
 - PDP-11 design, 469–470
 - PDP-10 design, 468–469
 - reduced instruction set computers (RISC), 548–549, 558, 566–568
 - Scalable Processor Architecture (SPARC), 567–568
 - variable-length, 469–476
 - VAX design, 471–472

- Instruction issue, 580
- Instruction-level parallelism, 304, 572–573
 - Advanced RISC Machine (ARM) Cortex-A8 processor, 595–603
 - antidependency, 583–584
 - branch prediction, 587
 - degree of instruction execution and, 577–578
 - execution of superscalar programs, 587–588
 - implementation of superscalar programs, 588
 - instruction issue policy, 580–584
- Instruction pipeline, 500–501
- Instruction prefetch (fetch overlap), 496, 496
- Instruction register (IR), 20, 70, 488
- Instructions, Assembly Language
 - Statements, 703
- Instruction sets, 52, 347–348, 556–557, 563–575
 - addressing modes, 451–479
 - Advanced RISC Machine (ARM), 416–417, 339–341, 462–463, 475–477
 - architecture, 52
 - assembly language, 477–479
 - central processing unit (CPU) functions, 405–449
 - data types, 415–418
 - design, 412–413
 - endian byte orders, 447–449
 - IBM 3090 vector facility ALU, 650, 653
 - instruction formats, 464–472
 - Intel x86, 415–417, 425–433, 459–461, 473–475
 - machine instructions, 405–450
 - MIPS R4000 microprocessor, 556–557
 - operands, 406–407, 372–374
 - operations (opcode), 406, 418–431
 - reduced instruction set computers (RISC), 556–558, 564–566
 - Scalable Processor Architecture (SPARC), 554–556
 - stacks, 447–448
- Instruction window, 583
- Instructions. *See* Machine instructions; Micro-operations (micro-ops)
- Integers, 321–341, 595, 599–603
 - addition, 328–332
 - arithmetic and logic unit (ALU) data, 320–341
 - arithmetic, 326–331
 - converting between bit lengths, 324–326
 - Cortex-A8 processor execute unit, 599–602
 - division, 338–339
 - fixed-point, 326
 - multiplication, 331–338
 - negation, 327–328
 - overflow, 328–329
 - Pentium 4 processor execution unit, 590
 - representation, 321–326
 - sign magnitude, 322
 - subtraction, 328–331
 - twos complement, 322–324, 326
 - unsigned multiplication, 332
- Integrated circuit (IC), 28–33
- Integrated circuits, development of, 28–34
- Intel Pentium 4 processor, 589–595
 - machine parallelism and, 579–580, 586–587
 - output dependency, 581–583
 - procedural dependency, 579
 - register renaming, 584–585, 594
 - resource conflict, 579
 - superscalar processors and, 573, 676
 - true data (flow) dependency, 577–578
- Intel x86 system, 2, 44–45, 236–240, 415, 434–444, 459–464
 - addressing mode, 464–465
 - cache memory, 141–144
 - call/return instructions, 433
 - chip multiprocessing, 702
 - condition codes, 433
 - control register, 515–517
 - Core Duo, 674–676
 - Core i7, 674–676
 - CPU instruction sets, 499
 - data types, 415–418
 - direct memory access (DMA) and, 240, 253
 - EFLAGS register, 512, 514
 - 82C55A programmable peripheral interface, 238–243
 - 82C59A interrupt controller, 236–238
 - 8237A DMA controller, 243–246
 - 8086 microprocessor registers, 490–491
 - 80486 information pipelining, 510–512
 - 80386 microprocessor registers, 489–490
 - evolution of, 44–46
 - I/O memory management, 294–297
 - instruction format, 473–475
 - instruction-level parallelism and, 590–516
 - interrupt-driven I/O and, 232–236
 - interrupt processing, 518–520
 - machine instructions, 351–352, 349–356
 - memory management instructions, 434
 - MMX (multimedia task) instructions, 435–439
 - MMX registers, 517–518
 - multicore computer organization, 676–677
 - operations (opcode), 434–435
 - Pentium 4 processor, 141–144, 589–590
 - Pentium II processor, 290–294
 - processor organization, 512
 - programmable I/O and, 238–240
 - register organization, 488–489
 - single-instruction multiple-data (SIMD) instructions, 435
 - status flags, 434
 - superscalar processor design, 577
- Interactive operating system (OS), 304

- Interconnections, 12–13, 66, 93–98
 - bus, 12, 85–94
 - computer structure and, 14
 - data exchanges, 83–84
 - I/O modules, 82–83
 - memory modules, 82
 - peripheral component (PCI), 98–107
 - processor signals, 83
 - switched, SMP, 516
- Interfaces, 222–223, 238–240, 248–257
 - external I/O, 248–257
 - FireWire serial bus, 250–254
 - InfiniBand, 254–257
 - input/output (I/O), 222–223, 238–240, 248–257
 - I/O modules, 222–223
 - Intel 82C55A programmable peripheral, 238–240
 - multipoint, 249–250
 - parallel I/O, 248–249
 - point-to-point, 249
 - serial I/O, 248–245
- Interleaved memory, 169
- Interleaved multithreading, 627–630
- Intermediate queues, 283–284
- Internal memory, 159–184
 - chips, 165–169
 - dynamic random-access memory (DRAM), 160–162, 166–168, 175–181
 - electrically erasable programmable read-only memory (EEPROM), 162, 164
 - erasable programmable read-only memory (EPROM), 162, 164, 168–169
 - error correction, 170–174
 - flash memory, 161, 164
 - high-level performance, 173–179
 - interleaved, 169
 - main (cell), 160–169
 - programmable read-only memory (PROM), 161, 164
 - random-access memory (RAM), 161–162
 - read-only memory (ROM), 161, 163–164
 - semiconductors, 160–184
 - static random-access memory (SRAM), 163
- International Business Machines (IBM), 25–28, 31–33, 625, 631, 650–657, 684–685
 - address generation sequencing, 598
 - ALU instruction set, 600
 - compound instruction execution, 653
 - Power5 chip multiprocessing, 631–633
 - register-to-register organization, 651
 - 700/7000 series computers, 25–26
 - 360 series computer, 33
 - 3033 processor microinstructions, 505
 - 3090 vector facility, 650
 - z990 SMP mainframes, 659
- International Reference Alphabet (IRA), 225
- Interrecord gaps, 216
- Interrupt, 74–83. *See also* Interrupt-driven I/O
 - in bus structure, 87
 - in control and status registers, 489
 - handling, 76, 680–683
 - in instruction cycle, 491
 - processing, 518–520, 525–526
 - in simple batch systems, 273
- Interrupt cycle, 77, 80, 494–495
 - computer instructions, 76–78, 80
 - micro-operations (micro-ops), 612
 - processor instructions, 444
- Interrupt-driven I/O, 228–226, 232–240
 - bus arbitration technique, 236
 - daisy chain technique, 236
 - design and implementation of, 234–236
 - drawbacks of, 240
 - execution, 233–234
- Intel 82C55A programmable peripheral interface, 238–240
- Intel 82C59A interrupt controller, 236–238
 - multiple interrupt lines, 235
 - interrupt processing, 232–234
 - programmed I/O and, 228–230, 238–240
 - software poll technique, 235, 236
- Interrupt service routine (ISR), 80, 83
- Interrupts, 74–85, 232–238, 273, 281, 518–521, 525–526, 677, 679–683
 - advanced programmable interrupt controller (APIC), 677
 - Advanced RISC Machine (ARM) processing, 525–526
 - ARM11 MPCore, 679–683
 - disabled, 80
 - distributed interrupt controller (DIC), 679–681
 - exceptions and, 518, 525–526
 - fully nested mode, 237
 - handling, 76, 521, 680–683
 - instruction cycle and, 74–85
 - Intel 82C59A modes, 236–238
 - Intel x86 processing, 518–521, 677
 - multicore computers, 677, 679–683
 - multiple, 80–85, 234–236
 - operating system (OS) hardware, 273
 - processing, 232–234
 - program flow of control and, 74–76
 - request signal, 76
 - rotating mode, 238
 - scheduling process, 281
 - special mask mode, 238
 - vector tables, 518–520, 525
 - vectored, 236
- Isolated I/O, 231

J

J-K flip-flop, 391–392
 Job, operating system (OS), 270
 Job control language (JCL), 272
 Jump instruction, 426

K

Kernel (nucleus), 269
 Keyboard arrangement, I/O, 225
 K Prefix, 35
 Karnaugh map, 373–376

L

Label, 701–702
 Lands, compact disks, 211
 Lane, 100
 Large-scale integration (LSI), 33
 Last-in-first-out (LIFO) queue, 458
 L1 cache, 120
 L2 cache, 120
 L3 cache, 120
 Leading edge, 91
 Least-frequently used (LFU)
 algorithm, 137
 Least-recently used (LRU)
 algorithm, 137, 289
 Least significant digit, 310
 Linear tape-open (LTO) system, 217
 Lines, cache memory, 120–121, 139
 Linkage editor, 716
 Linking, 716
 Link layer, 255
 Links, InfiniBand, 253
 Little endian ordering, 415, 447–450
 Load balancing, clusters, 636
 Loading, 710, 713–716
 Load/store addressing, ARM, 462
 Load/store multiple addressing, ARM,
 463–464
 Load-time dynamic linking, 717
 Locality of reference, 117, 152–154
 Local variable, 430–431
 Logical address, 287
 Logical cache, 125
 Logic block, 400–401
 Logic (Boolean) instructions, 409
 Logic-memory performance balance, 39–41
 Logical address, 287, 288
 Logical data operands, 414–415
 Logical operations (opcode), 419,
 422–424
 Logical shift, 423–424
 Long-term scheduling, 277–278
 Lookup table, 400
 Loop buffer, pipelining, 505–506
 Loop unrolling, pipelining, 555–556

M

Machine cycles, 547–549
 Machine instructions, 406–412, 533–538, 547
 addresses, 410–412
 Advanced RISC Machine (ARM), 416–418,
 439–440
 arithmetic, 410
 branch, 409, 426–427
 data types, 409, 414–418
 elements of, 406–407
 high-level languages (HLL) and, 533–535
 instruction set design, 412
 Intel x86, 415–416, 431–438
 logic (Boolean), 409
 memory, 409
 operands, 406–411, 415, 536–537
 operations (opcode), 406–409, 418–431,
 535–536
 procedure calls, 428–431, 433, 537
 reduced instruction set computers (RISC),
 533–538, 547–548
 RISC execution, 533–538
 symbolic representation, 407–408
 test, 409
 Machine parallelism, 579–580, 586–587
 Macro definitions, 704–706
 Magnetic disks, 186–195
 constant angular velocity (CAV), 188–189
 cylinders, 191
 data formatting, 187–190
 floppy (contact), 190, 194
 heads, 186–187, 189–190
 multiple platters, 190
 multiple zone recording, 189
 parameters, 192–195
 read mechanisms, 186–187
 rotational delay (latency), 192–193
 rotational positional sensing (RPS), 192
 seek time, 193
 sequential organization, 194
 single and double sides, 190
 tracks, 187, 191–192
 transfer time, 194
 Winchester format, 189, 192
 write mechanisms, 186–187
 Magnetic tape, 215–217
 Magnetoresistive sensor, 187
 Mainframe computers, 31
 Main memory, 12, 68, 124–125, 152, 160–169,
 267–268
 cache (physical), 124–125, 152
 computer component of, 12, 68
 internal (cell), 160–169
 kernel (nucleus), 269
 OS resource management, 268–270
 Mantissa, 342

- Many integrated core (MIC), 43
- Mapping functions, 125–136
 - associative, 130–132
 - cache memory, 125–136
 - direct, 126–130
 - set-associative, 132–136
- Medium-term scheduling, 278
- Memory address register (MAR), 20, 68, 72, 488
- Memory bank, 169
- Memory buffer register (MBR), 20, 68, 72, 488
- Memory cycle time, 25, 115
- Memory hierarchy, 116
- Memory instructions, 409
- Memory management, 283–304, 434
 - access control, 304
 - addresses, 286–287, 296, 300–302
 - Advanced RISC Machine (ARM), 301–304
 - compaction, 286
 - formats, 296, 301–304
 - input/output (I/O), 276, 301–304
 - Intel x86 machine instructions, 415
 - multiprogramming and, 276, 283
 - operating systems (OS), 267, 276, 283–304
 - paging, 287–288, 296–299
 - parameters, 298, 303
 - partitioning, 284–287
 - segmentation, 293–294, 295–296
 - swapping, 283–284
 - translation lookaside buffer (TLB), 291–293, 299–300
 - virtual memory, 289–290, 300–301
- Memory management unit (MMU), 124, 300–301
- Memory-mapped I/O, 231–232
- Memory modules, 87
- Memory protection, OS, 273
- Memory systems, 112–217
 - access, 118
 - addressable units, 114
 - cache, 112–158
 - capacity, 114
 - external, 185–217
 - hierarchy, 116–119
 - hit, 118
 - internal, 159–184
 - locality of reference, 117, 152–154
 - location, 113
 - miss, 118
 - organization, 116
 - performance, 115–116, 118, 152–158
 - physical characteristics of, 116
 - secondary (auxiliary), 119
 - two-level, 152–158
 - unit of transfer, 114
 - word, 114
- MESI (modified, exclusive, shared, or invalid) protocol, 622–625
- Microcomputer, 8
- Microelectronics, development of, 28–30
- Microinstruction bus (MIB), 468
- Micro-operations (micro-ops), 144, 589–590, 593
 - allocation, 594
 - execute cycle, 69
 - fetch cycle, 69–74
 - front end generation of, 590
 - instruction cycle, 69–74
 - interrupt cycle, 74–83
 - queuing, 590
 - scheduling and dispatching, 595
 - superscalar processors, 574–577
- Microprocessors, 35–37, 38–39, 490–491
 - development of, 35–37
 - Intel 80386 registers, 490–491
 - Intel 8086 registers, 490–491
 - Motorola MC68000 registers, 490
 - register organizations, 490–491
 - speed (performance of), 39–41
- Microprogrammed control units, 532
- Microprogramming language, 469, 612
- Migratory lines, 683
- Millions of floating-point operations per second (MFLOPS) rate, 52
- Millions of instructions per second (MIPS) rate, 51–52
- Minuend, 329
- MIPS rate, 51–52
- MIPS R4000 microprocessor, 556–562
 - instruction format, 566–568
 - instruction set, 564–566
 - pipelining instructions, 559–562
- Mirrored disk performance (RAID level 1), 197–198
- Miss, 118
- MMX (multimedia task), Intel x86 processors, 435–439, 517–518
 - instructions, 435–438
 - registers, 517–518
- Mnemonics, 408, 702
- Monitor (simple batch OS), 271–273
- Monitor arrangement, I/O, 225
- Most significant digit, 310
- Moore's law, 29–31
- Motorola MC68000 microprocessor
 - registers, 490
- Movable-head disk, 190
- M Prefix, 35
- Multicore computers, 631, 664–689. *See also*
 - zEnterprise 196, I/O structure
 - ARM11 MPCore, 679–683
 - chip multiprocessors as, 626–633
 - database application, 671–674

- hardware performance, 665–669
- Intel Core Duo, 676–677
- Intel Core i7, 677–679
- Intel x86 organization, 676–679
- organization, 674–675
- overview, 665
- parallelism increase, 665–668
- power consumption, 668–669
- software performance, 669–674
- speedup time increase, 670
- threading, 671–672
- Multicore processors, 43
- Multicore strategy, 43
- Multilane distribution, 96
- Multilevel cache memory, 139–141
- Multiple zoned recording, 189
- Multiple instruction, multiple data (MIMD)
 - stream, 613
- Multiple instruction, single data (MISD)
 - stream, 613
- Multiple interrupt lines, I/O, 235
- Multiple parallel processing, 649–650
- Multiple platters, magnetic disks, 190
- Multiple streams, pipelining, 505
- Multiplexer, 380–382
- Multiplexor, 27
- Multiplexor channel, 247
- Multiple zone recording, 189
- Multiplicand, 332
- Multiplication, 331–338, 352
 - Booth's algorithm, 335–337
 - floating-point numbers, 349–352
 - twos complement, 333–338
 - unsigned integers, 328–330
- Multiplier quotient (MQ), 20
- Multipoint interfaces, 250
- Multiprocessor OS design, SMP considerations
 - for, 619
- Multiprogramming operating system (OS), 270, 273–276, 283
 - batches, 273–276
 - memory management and, 276
 - uniprogramming compared to, 270, 276
- Multitasking, operating systems (OS), 274
- Multithreading, 626–633
 - chip multiprocessing, 628, 631–633
 - explicit, 627–631
 - implicit, 626–627
 - parallel processing, 626–633, 636–637
 - process, 626–627
 - switches, 627
 - thread, 627

N

- NAND gate, 369
- NaNs, IEEE standards, 356
- Negation, integers, 327–328
- Negative overflow, 344
- Negative underflow, 344
- Network layer, 256
- Nibble, 315
- Noncacheable memory approach, 138–139
- Nonredundant disk performance (RAID level 0), 197–198
- Nonremovable disk, 190
- Nonuniform memory access (NUMA), 613, 639–643
 - advantages and disadvantages of, 643
 - cache-coherent (CC-NUMA), 640
 - motivation, 640–641
 - organizations, 641–643
 - parallel processor architecture, 646–649
 - uniform memory access (UMA), 640
- Nonvolatile memory, 119
- NOR gate, 370
- Normalized numbers, 342–343
- Nucleus. *See* Kernel (nucleus)
- Number system
 - binary system, 312
 - binary vs decimal, 312–313
 - decimal system, 310–311
 - fractions, 313–315
 - hexadecimal notation, 315–317
 - positional number system, 311
- Numerical data operands, 413

O

- Offset addressing, ARM, 462
- One-pass assembler, 709
- Ones complement representation, 347
- Opcode. *See* Operations (opcode)
- Operands, 406–407, 413–415, 536–537
 - characters, 414–415
 - high-level language (HLL), 536–537
 - logical data, 415
 - machine instructions, 406–407
 - numbers, 413–414
 - packed decimal representation, 414
 - reduced instruction set computers (RISC), 536–537
- Operating system (OS), 265–304
 - Advanced RISC Machine (ARM) memory management, 299–304
 - batch, 270, 272–276
 - computer system support, 265–304
 - evolution of, 270–271
 - functions, 266–276
 - Intel Pentium II memory management, 294–299
 - interactive, 270
 - interrupts, 273
 - memory management, 266, 276, 283–304

- Operating system (OS) (*continued*)
 - memory protection, 273
 - multiprogramming, 270, 273–275
 - objectives, 266–267
 - privileged instructions, 273
 - resource management, 268–270, 275–276
 - scheduling, 266, 270, 277–283
 - setup time, 270–271
 - time-sharing, 276–177
 - uniprogramming, 270
 - user/computer interfacing, 266–267
 - utilities, 266–267
 - Operations (opcode), 19, 23, 406, 418–431, 535–536
 - Advanced RISC Machine (ARM), 439–440
 - arithmetic, 418, 422
 - computer instructions, 19, 23
 - conversion, 420, 425–426
 - data transfer, 418, 320–322
 - high-level language (HLL), 535–536
 - input/output (I/O), 420, 425
 - Intel x86, 431–440
 - logical, 419, 422–424
 - machine instructions, 406, 418–440
 - reduced instruction set computers (RISC), 535–536
 - system control, 420, 425
 - transfer of control, 420, 425–430
 - Optical memory systems, 210–215
 - Blu-ray DVD, 210, 215
 - compact disk (CD), 210, 210–214
 - digital versatile disk (DVD), 210, 213–214
 - high-definition optical disks (HD DVD), 214–215
 - types of, 210
 - OR gate, 368
 - Original equipment manufacturers (OEM), 33
 - Orthogonality, 468, 469
 - Out-of-order execution, 581–584, 594–595
 - Out-of-order issue, 583–584
 - Output dependency, parallelism, 581–583
 - Overflow, 328–329, 344, 349
- P**
- Packed decimal representation, 413–415
 - Packets, data, 95
 - Page fault, 289
 - Page frame, 287
 - Pages, 287
 - Page tables, 288, 290–291, 300–301
 - Pages, I/O memory, 287–288
 - Paging, 287–291, 296–299
 - demand, 289–290
 - frame allocation, 287–288
 - I/O memory management, 287–291, 296–299
 - page replacement, 289–290
 - page tables, 288, 290–291
 - Pentium II processor, 296–299
 - virtual memory, 289–291
 - Parallel I/O interfaces, 248–249
 - Parallel organization, 611–756
 - cache coherence, 612, 620–621
 - chip multiprocessing, 612, 628–631
 - clusters, 612, 633–640
 - multicore computers, 664–687
 - multiple processor organizations, 613–614
 - multithreading, 612, 626–633
 - nonuniform memory access (NUMA), 612, 614, 640–643
 - parallel processing, 613–656
 - symmetric multiprocessors (SMP), 612, 614–619, 694
 - vector computation, 644–656
 - Parallel recording, 216
 - Parallel register, 393
 - Parallelism, 693, 573–603, 636, 665–668
 - cluster applications, 636
 - instruction issue policy, 580–584
 - instruction-level, 573–603
 - limitations, 577–579, 581–584
 - machine, 579–580, 586–587
 - multicore computer increase, 665–669
 - Parameters, magnetic disks, 192–195
 - Parametric computing, 637
 - Parity bits, 171
 - Partial product, 332
 - Partial remainder, 338–339
 - Partitioning, I/O memory management, 284–287
 - Passive standby clustering method, 634–635
 - PCI. *See* Peripheral component interconnection (PCI)
 - PCI Express (PCIe)
 - overview, 98
 - physical architecture, 98–100
 - physical layers, 100–102
 - transaction layer, 102–107
 - data link layer, 107–108
 - PDP-8 Bus Structure, main memory, 35
 - PDP-8 instruction format design, 467–468
 - PDP-11 instruction format design, 469–470
 - PDP-10 instruction format design, 468–469
 - Pentium 4 processor, 141–144, 589–595, 631
 - allocation, 594
 - chip multiprocessing, 631
 - drive, 591, 593
 - floating-point execution unit, 595
 - front end, 590–593
 - instruction-level parallelism and, 589–595
 - integer execution unit, 595
 - micro-operations (micro-ops), 589–591, 594–595
 - organization, 141–144
 - out-of-order execution logic, 594–595

- register renaming, 594
- superscalar design, 589–595
- trace cache fetch, 591, 593
- trace cache next instruction pointer, 591–593
- Pentium II processor, 294–299
 - address spaces, 294–295
 - formats for memory management, 297
 - I/O memory management, 294–299
 - paging, 298–299
 - parameters for memory management, 298
 - segmentation, 295–296
 - virtual address fields, 296
- Peripheral component interconnection (PCI), 98–107
 - arbitration, 108
 - bus interconnection structure, 98–101
 - configuration, 98–99
 - data transfers, 100
 - request (REQ) signal, 103
 - signal lines, 98
 - special interest group (SIG), 98
- Peripheral (external) devices, I/O, 223–225
- Phase change, 213
- Phit (physical unit), 94
- Physical address, 287, 288
- Physical cache, 125
- Physical dedication, 90
- Physical layer, 251–252, 255
- Pipelining, 495–511, 532, 551–556, 559–562, 576–577, 602–603, 646–649
 - branch prediction, 506–509
 - branches and, 505–510
 - bubble, 502
 - Cortex-A8 processor, 602–603
 - cycle time, 500
 - delayed branch, 510, 553–555
 - delayed load, 554
 - development of, 532
 - floating-point instructions, 602–603, 647–650
 - hazards, 501–504
 - instruction prefetch (fetch overlap), 496, 505
 - Intel 80486 processor, 510–511
 - loop buffer, 505–506
 - loop unrolling, 555–556
 - MIPS R4000 microprocessor, 559–562
 - multiple streams, 505
 - optimization, 553–556
 - performance, 500–501
 - processor instructions, 495–512
 - RISC instructions, 551–556, 559–562
 - single-instruction multiple-data (SIMD) instructions, 602–603
 - speedup factor, 501–502
 - strategy, 495–500
 - superpipelined approach, 576–577
 - superscalar approach compared to, 576–577
 - vector computations and, 646–649
- Pits, compact disks, 210
- Platters, 186, 190–191
- Point-to-point interconnect, 95–98.
 - See also* Quick Path Interconnect (QPI)
- Point-to-point interfaces, 249
- Pollack's rule, 669
- POP stack operation, 419
- Positional number system, 311
- Positive overflow, 344
- Positive underflow, 344
- Postindexing, 458, 462
- Power consumption, 668–669
- Power density, 41
- Power management logic, 677
- Preindexing, 458, 462
- Privileged instructions, 273
- Procedural dependency, parallelism, 579
- Procedure calls, 428–431, 433, 537
 - control transfer instructions, 427–431
 - high-level language (HLL), 537–538
 - Intel x86 call/return instructions, 433
 - reduced instruction set computers (RISC), 537
 - stack implementation of, 429–430
- Procedure return, 433
- Process, 277–283, 626–627
 - concept of, 277
 - control block, 279
 - data, 10
 - execution, 280–283, 626
 - interrupt, 281
 - multithreading, 626–627
 - resource ownership, 626
 - scheduling, 277–283, 626
 - states, 278–280
 - switch, 627
- Processors, 12–13, 85, 226–227, 484–526
 - Advanced RISC Machine (ARM)
 - organization, 520–526
 - arithmetic and logic unit (ALU), 12, 484–485
 - communication, 85, 226–227
 - control unit (CU), 12
 - cycle time, 51
 - I/O modules, 85, 226–227
 - instruction cycle, 491–495
 - Intel x86 organization, 512–520
 - interrupt processing, 518–520, 525–526
 - modes, ARM, 522–523
 - pipelining instructions, 495–512
 - registers, 12, 486–491, 512–518, 523–525
 - requirements of, 484–486
 - signals, 85
 - structure and function, 483–527
 - system interconnection (bus), 12–13, 85, 485–486

Product of sums (POS), 372
 Program counter (PC), 20, 69–70, 488
 Program status word (PSW), 489
 Programmable array logic (PAL), 398
 Programmable logic array (PLA), 397–401
 Programmable logic devices (PLD), 397–401
 Programmable logic devices, 397–401
 Sequential circuits, 388–397
 field-programmable gate array, 398–401
 programmable logic array, 397–398
 types of, 397
 Programmable read-only memory (PROM), 161, 164
 Programmed I/O, 222, 228–232, 238–240
 commands, 229–230
 drawbacks of, 240
 execution, 228–232
 instructions, 232–234
 Intel 82C55A programmable
 peripheral interface, 238–240
 interrupt-driven I/O and, 228–232, 238–240
 isolated, 231
 memory-mapped, 231
 PUSH stack operation, 431

Q

Queues, 282–284, 594
 I/O, 282–283
 intermediate, 283–284
 long- and short-term, 282
 memory management swapping, 283–284
 micro-operations (micro-ops), 595
 processor scheduling, 282–283
 Quick Path Interconnect (QPI), 679
 characteristics, 93–94
 link layers, 96–97
 physical layers, 95–96
 protocol architecture, 94–95
 protocol layers, 97–98
 routing layers, 97
 Quiet NaN, 357
 Quine-McCluskey method, 376–380
 Quotient, 313

R

Radix point, 321
 RAID. *See* Redundant Array
 of Independent Disks (RAID)
 Rambus DRAM (RDRAM), 178
 Random access, 115
 Random-access memory (RAM), 161–162
 Rate metric measures, 55–56
 Ratio, averaging results, 54–55
 Read hit/miss, 624
 Read mechanisms, magnetic disks, 186–187
 Real memory, 290

Reading/report assignments, 696
 Read-mostly memory, 164
 Read-only memory (ROM), 161, 163–164
 Read-with-intent-to-modify (RWITM), 624–625
 Read-write dependency, 594
 Recommended reading, 718
 Recordable (CD-R), 210
 Reduced instruction set computers (RISC), 2, 531–569
 addressing mode simplicity, 548–549
 architecture, 545–551
 CISC and superscalar systems
 compared to, 534
 compiler-based register optimization, 543–545
 complex instruction set computer (CISC)
 architecture compared to, 549–551, 568–569
 development of, 533
 high-level language (HLL) and, 533–538
 instruction execution, 533–538
 instruction formats, 548–549, 558, 566–567
 instruction sets, 556–559, 564–566
 machine cycle instructions, 547
 MIPS R4000 microprocessor, 556–562
 operands, 536–537
 operations, 535–536
 pipelining instructions, 551–556, 559–562
 procedure calls, 537–538
 register-to-register characteristics, 547–548
 registers, 538–545, 563–564
 Scalable Processor Architecture (SPARC), 562–568
 Redundant Array of Independent Disks (RAID), 186, 195–205
 bit-interleaved parity (level 3), 197, 202–203
 block-level distributed parity (level 5), 197, 204
 block-level parity (level 4), 197, 203
 characteristics of, 196
 dual redundancy (level 6), 197, 204–205
 Hamming code, redundant via (level 2), 197, 202
 levels, 196–198, 204–205
 mirrored (level 1), 197, 201–202
 nonredundant (level 0), 197–198
 redundancy, 3, 202–203
 striping (level 0), 197–201
 Redundant disk performance via Hamming
 code (RAID level 2), 197, 202
 Reentrant procedure, 429
 Register addressing, 455–456
 Register file, instruction pipe line, 562
 Register indirect addressing, 456
 Register renaming, 584–585, 594
 Register-to-register organization, 547–548, 651–627

Registers, 12, 19–20, 452–456, 485–491, 512–518, 523–525, 538–545, 563–564, 651–654
 address, 487
 addressing mode, 452–454
 Advanced RISC Machine (ARM) organization, 523–525
 cache memory compared to, 541–543
 compiler-based optimization, 543–545
 condition codes (flags), 487–488
 control, 486, 487–488, 515–517
 current program status (CPSR), 523–524
 data, 486
 EFLAGS, 512–514
 general-purpose, 486, 523
 global variable storage, 541
 IAS computer memory and, 19–20
 IBM 3090 vector facility, 651–654
 indirect addressing mode, 453–454, 456
 instruction (IR), 20, 489
 instruction buffer (IBR), 20, 489
 Intel 80386 microprocessor, 490–491
 Intel 8086 microprocessor, 490–491
 Intel x86 organization, 512–518
 larger file approaches, 538–543
 memory address (MAR), 20, 488
 memory buffer (MBR), 20, 488
 microprocessor organizations, 490–491
 MMX, 517–518
 Motorola MC68000
 microprocessor, 490–491
 program counter (PC), 20, 488
 program status word (PSW), 489
 reduced instruction set computers (RISC), 538–545, 563–564
 registers, 12, 485–491
 Scalable Processor Architecture (SPARC), 562–564
 status, 486, 488–490
 user-visible, 486–488
 windows, 539–541, 563–564
 Register window, 539–541
 Relative address, 288, 457, 461
 Relocation, 710–713
 Remainder, 313
 Removable disk, 190
 Replacement algorithms, cache memory, 137
 Request (REQ) signal, PCI, 243
 Research projects, 692
 Resident monitor, 271
 Resistive-capacitive (RC) delay, 41
 Resource conflict, parallelism, 579
 Resource hazards, pipelining, 502–503
 Resource management, OS, 268–270, 275–276
 Resource ownership process, 626
 Retire, ARM Cortex-A8, 596
 Ripple counters, 394–395

RISC. *See* Reduced instruction set computers (RISC)
 Root complex, 98
 Rotate (cyclic shift) operation, 424
 Rotating interrupt mode, 238
 Rotational delay (latency), magnetic disks, 193
 Rotational positional sensing (RPS), 193
 Rounding, IEEE standards, 355–356
 Router, InfiniBand, 253
 Run-time dynamic linking, 718

S

Saturation arithmetic, 436
 Scalable Processor Architecture (SPARC), 562–564
 instruction format, 566–567
 instruction set, 564–566
 register set, 563–564
 Scalar values, 447
 Scheduling, 266, 270–271, 277–283, 595, 626
 efficiency of, 270–271
 interrupt process, 281
 long-term, 277–278
 medium-term, 278
 micro-operations (micro-ops), 595
 multithreading, 626
 operating system (OS) function, 266, 270, 277–283
 process, 277–280, 626
 queues, 282–283
 short-term, 278–283
 state of a process, 278–280
 techniques, 280–283
 Secondary (auxiliary) memory, 119
 Sectors, magnetic disks, 188
 Seek time, magnetic disks, 193–194
 Segmentation, Pentium II processor, 293–296
 Selector channel, 247–248
 Semantic gap, 533–534
 Semiconductors, 33–35, 160–169.
 See also Internal memory
 Semiconductor memory, 160–169
 Semiconductor technology, 119
 Sequential circuits, 388–397
 clocked S–R flip-flop, 389–391
 counters, 394–397
 D flip-flop, 391–393
 flip-flops, 388
 registers, 393–394
 S–R latch, 388–389
 Sequencing, 271, 535, 593
 Sequential access, 114
 Sequential organization, magnetic disks, 194–195
 Serial I/O interfaces, 248–249
 Serial recording, 216

- Serpentine recording, 216–217
- Server clustering approaches, 635
- Set-associative mapping, 132–136
- Setup time, operating system (OS) efficiency, 270–271
- Shift register, 393–394
- Short-term scheduling, 278–283
- Sign bit, 322
- Significand overflow, 350
- Significand underflow, 350
- Sign-magnitude representation, 322
- Signal lines, PCI, 84
- Signaling NaN, 356
- Significand, 342, 350
- Simple PLD, 398
- Simulation projects, 694
- Simultaneous multithreading (SMT), 628–631
- Single error-correcting (SEC) code, 174
- Single-error-correcting, double-error-detecting (SEC-DED) code, 174
- Single-instruction multiple-data (SIMD), 435–438, 602, 613–615
 - Intel x86 instructions, 434–438
 - pipelining instructions, 602–603
 - stream, 613–615
- Single instruction, single data (SISD) stream, 613–615
- Single large expensive disk (SLEP), 196
- Single-sided disk, 190
- Single-system image, 637
- Skip instructions, 427
- Small Computer System Interface (SCSI), 89
- Small-scale integration (SSI), 397
- SMP. *See* Symmetric multiprocessors (SMP)
- Snoop control unit (SCU), 679, 683–684
- Snoopy protocols, cache coherence, 621
- Soft error, 170
- Software, 25, 67–68, 620–621, 669–674
 - cache coherence solutions, 620–621
 - database scaling applications, 670–671
 - development of, 25
 - multicore computer performance, 669–674
 - system components, 67–68
 - Valve game threading, 672–673
- Software poll technique, I/O, 236
- Solid-state component, 24
- Solid state drives (SSDs), 24
 - flash memory, 206–207
 - HDD compared, 207
 - organization of, 207–209
 - overview, 205–206
 - practical issues, 209
- Spatial locality, 154
- Special interest group (SIG), PCI, 98
- Special mask interrupt mode, 238
- Speculative execution, 39
- Speed metric measures, 54
- Speedup factor, 501–502
- Split cache memory, 141
- S–R Latch, 388–389
- Stacks, 411, 429–430, 458–459
 - addressing mode, 453, 458–459
 - frames, 430
 - pointer (SP), 523
 - procedure call implementation, 429–430
 - zero-address instructions, 411
- State diagrams, instruction cycles, 73, 81, 493
- State of a process, 278–280
- Static random-access memory (SRAM), 163
- Status flags, 434
- Status registers, 486, 488–490
- Status signals, I/O, 224
- Stored-program concept, 17
- Striped data, 198
- Striped disk performance (RAID level 0), 197–201
- Subnets, InfiniBand, 253
- Subnormal number, 357–358
- Substrate, 186
- Subtraction, 328–331, 349–352
 - floating-point numbers, 349–352
 - twos complement integers, 328–331
- Subtrahend, 329
- Sum of products (SOP), 371
- Superpipelined approach, 576–577
- Superpipelined processor, 576–577
- Superscalar processors, 534, 573–603
 - Advanced RISC Machine (ARM) Cortex-A8, 595–603
 - branch prediction, 587
 - CISC and RISC systems compared to, 534
 - committing (retiring) instructions, 588
 - design issues, 579–588
 - development of, 574
 - execution of programs, 587–588
 - implementation of programs, 588
 - in-order completion, 581
 - instruction issue policy, 580–584
 - instruction-level parallelism and, 573–603
 - Intel Pentium 4, 589–603
 - out-of-order completion, 581–583
 - parallelism limitations, 577–579, 581–583
 - register renaming, 584–585
 - superpipelined approach compared to, 576–577
- Swapping, I/O memory management, 283–284
- Switch, 253, 627
- Symmetric multiprocessors (SMP), 613, 614, 615–619
 - clusters compared to, 615–616
 - organization, 616–619
 - parallel processor architecture, 619

- system characteristics, 615–617
- two-level shared caches, 622
- Synchronous counter, 395–397
- Synchronous DRAM (SDRAM), 175–178
- Synchronous timing, 92–93
- Syndrome words, 171–172
- System bus, 12, 85–86
- System control operations, 425
- System interconnection (bus), 12–13, 85, 485–486
- System Performance Evaluation Corporation (SPEC), 53–55

T

- Tags, cache memory, 121
- Target channel adapter (TCA), 253
- Temporal locality, 154
- Test bank, 696
- Test instructions, 409
- Thermal control units, 676–677
- Thrashing, 130, 289
- Thread, 627
- Threading, multicore computers, 672–673
- Thumb instruction set, ARM, 476–477
- Thunderbolt, 250
- Time multiplexing, 90
- Time-sharing operating systems (OS), 276–277
- Timing, 90–93, 226
 - asynchronous, 92–93
 - bus interconnection, 90–93
 - I/O modules, 226
 - synchronous, 90–91
- Top-level computer structure, 13–14, 66
 - execute cycle, 21, 69–74
 - fetch cycle, 20, 69–74
 - functions, 8–14, 65–83
 - instruction cycle, 20–24, 69–74, 76–83
 - interconnections, 12–13, 84–107
 - timing diagrams, 394
- Trace cache fetch, Pentium 4 processor, 591, 593
- Trace cache net instruction pointer, Pentium 4 processor, 591–593
- Tracks, magnetic disks, 186, 190–191
- Transaction layer, 102
- Transducer, I/O, 224
- Transfer of control operations, 420, 426–431
- Transfer rate, 115–116
- Transfer time, magnetic disks, 193–194
- Transistors, development of, 24–33
- Translation lookaside buffer (TLB), 291–293, 299–300
- Transport layer, 256
- True data (flow) dependency, parallelism, 577–579
- Truth table, 366
- Two-level cache memory, 152–158

- Two-pass assembler, 706, 708
- Twos complement, 322–324, 326–341
 - arithmetic, 326–331
 - division restoring algorithm, 339–341
 - geometric depiction of, 330
 - multiplication, 333–338
 - operation, 327
 - representation, 322–323

U

- Ultra-large-scale integration (ULSI), 33
- Unary operator, 410
- Unconditional branch instructions, 22–23, 427
- Unconditional jump, 432
- Underflow, 344, 349, 357
- Unified cache memory, 141
- Uniform memory access (UMA), 640
- Uniprocessors, 613, 615
- Uniprogramming, operating systems (OS), 270, 274
- Unit of transfer, 113–114
- Universal Automatic Computer (UNIVAC), 24
- Upward compatible, 24
- User/computer interfacing, OS, 266–267
- User-visible registers, 486–488
- Utilities, OS, 267–268
- Utility program, 267

V

- Vacuum tubes, development of, 16–37
- Valve game threading, 672–673
- Variable-length instruction formats, 469–472
- Variable-sized partitions, 285–286
- VAX instruction format design, 471–472
- Vector, 236
- Vector computation, 644–656
 - ALU instruction set, 654–656
 - chaining, 648–649
 - compound instructions, 654
 - IBM 3090 vector facility, 650–656
 - multiple parallel processing, 649–650
 - parallel processing, 646–647
 - pipelining approaches, 646–647
 - register-to-register organization, 651–654
 - vector processing, 644–650
- Vector facility, IBM 3090, 650–651
- Vector floating-point (VFP) unit, 603
- Very-large-scale integration (VLSI), 33
- Very long instruction word (VLIW)
- Virtual address fields, 296
- Virtual cache memory, 124–125, 152
- Virtual lanes, InfiniBand, 254–255
- Virtual memory, 289–291, 300–301
 - ARM address translation, 300–301
 - demand paging, 289–290
 - I/O memory management, 289–291, 300–301

Virtual memory (*continued*)
 inverted page table structure, 290–291
 page replacement, 289–290
 Pentium II address fields, 296
 Virtual storage, 651
 Volatile memory, 116
 Von Neuman machine, 17–24, 66–68

W

Wafer, silicon, 29, 30
 Watchdog, 679
 Web site resources, 5–6, 358
 Winchester disk format, 189
 Windows, register file size increase using,
 539–541, 563–564
 Words, 19, 113, 465
 addressing modes, 447
 in page table structure, 290

Write after read (WAR) dependency, 578–579
 Write after write (WAW) dependency, 581–583
 Write back technique, 138, 620
 Write hit/miss, 625
 Write mechanisms, magnetic disks, 186–187
 Write policy, cache memory, 137–139
 Write through technique, 137–138, 620
 Writing assignments, 696

X

X86 and ARM data types, 431
 XOR gate, 366

Z

zEnterprise196, I/O structure
 cache structure, 685–686
 channel structure, 256–257
 system organization, 258–260, 684–685